Compositional Machine Learning for Time-Verifiable, Safe, and Explainable Cyber-Physical Systems

Sobhan Chatterjee

Supervisors:
Partha Roop
Nitish Patel

Department of Electrical, Computer, and Systems Engineering
The University of Auckland

A thesis submitted in fulfilment of the requirements for the degree of Doctor of Philosophy in Electrical and Electronic Engineering, the University of Auckland, 2025.
This thesis is for examination purposes only and is confidential to the examination process.

Abstract

The integration of Machine Learning (ML) into safety-critical Cyber-Physical Systems (CPSs) presents a fundamental challenge: how to leverage the performance benefits of ML while ensuring the rigorous safety, timing, and explainability requirements of critical applications. Traditional monolithic machine learning approaches create black-box systems that are difficult to verify, implement on hardware for timing analysis, and explain to stakeholders. This thesis addresses these challenges by introducing a novel paradigm of compositional machine learning that builds ML-based CPSs that are safe, time-verifiable, and explainable by construction.

In this thesis, we introduce the systematic decomposition of monolithic models into smaller models. This decomposition preserves functional behaviour while enabling parallel execution and modular verification. We develop novel compilation frameworks that translate Python-based Artificial Neural Network (ANN) models to both VHDL (for Field Programmable Gate Array (FPGA) implementation) and C code (for embedded systems), enabling Worst-Case Execution Time (WCET) analysis. Our compilers specifically support compositional ANN architectures, addressing a critical gap in existing tools. Moreover, we introduce a policy-driven framework for safe-by-construction ML systems that mines safety properties from data and uses them to guide the training of compositional models. Additionally, our runtime enforcement mechanism ensures policy adherence in real-time, providing a safety net for unexpected scenarios. Finally, we develop a novel compositional explainability paradigm combining multiple explainability techniques to provide actionable insights at sub-model and system levels.

We validate our framework through comprehensive case studies in autonomous vehicles and healthcare, demonstrating generalisability, practical applicability and concrete benefits. Through extensive evaluation, we demonstrate that compositional ANN models can achieve up to 85% reduction in WCET, 53% reduction in hardware resources, and 40% reduction in computations while maintaining comparable performance to monolithic approaches. Furthermore, we illustrate how a compositional policy-mining approach helps obtain ML models that are safer than their monolithic counterparts. Finally, we also show how compositional ML models are inherently more explainable than monolithic models. Overall, the results establish that compositionality enables the development of ML-based CPSs that are not only more performant but also more verifiable, implementable, and trustworthy.

Acknowledgements

Ancient wisdom says, “A journey of a thousand miles begins with a single step”, and I could not help but feel the same when I started my PhD during the first wave of the pandemic. I knew I had set out on a path, but I had no idea where it would lead and how long the path lay ahead. During this journey, which interestingly has taken a sixth of my current life (alas! you can compute my age now), I have met some amazing people, and this acknowledgement goes only a short way to express my gratitude.

I want to begin by expressing my deepest gratitude to my supervisors, Prof. Partha Roop and Dr. Nitish Patel, for their invaluable guidance, support, and mentorship throughout this research journey, especially during the uncertain COVID-19 years. Their expertise, patience, and encouragement have been instrumental in shaping my research. To you, Partha, I will fondly remember you as the purveyor and personification of your favourite quote: “In research, we stand on the shoulders of giants”. Thanks for all your help and for believing in me.

A big thanks to all my colleagues in the PRETzel research group, especially to Nathan Allen, Alex Baird and Hammond Pearce, who finished their PhD before me and whose help and research laid the groundwork for my work. I also want to thank Moon Kim, Pengqian Han, Luman Wang, Logan Kenwright, Chuanshang Yin, Henry Liu, Jonathan King, the P4P students and others for their support, advice, collaboration and the light moments we shared. You all kept me going during this journey.

A special thanks to Frederick Sundram for his invaluable clinical insights during the depression case study. I would also like to express my gratitude to Jyoti Mishra for providing the dataset for the case study.

I would be remiss if I did not express my profound thanks to Aron Jeremiah, Ahmed Faiz, Saumya Shankar and Jerry Jacob for being the wonderful combination of a colleague, a friend and a flatmate. Our discussions on esoteric research, debates on abstruse topics, and banter over frivolous ideas made my PhD much more enjoyable and manageable.

Lastly, I am immensely thankful to my family for their continued support and belief in me throughout this journey. To my parents, my brother and grandparents, I could not express enough how much your love and support mean to me. Without our conversations, I would have struggled to make it this far in my life so far from home.

Sobhan Chatterjee

Table of Contents

List of Figures

List of Tables

List of Acronyms

AdaBoost	Adaptive Boosting
Adam	Adaptive Moment Estimation
AI	Artificial Intelligence
ALE	Accumulated Local Effects
ALM	Adaptive Logic Module
ANN	Artificial Neural Network
AV	Autonomous Vehicle
BA	Balanced Accuracy
BFGS	Broyden-Fletcher-Goldfarb-Shanno
BMI	Body Mass Index
BP	Blood Pressure
C-SSRS	Columbia Suicide Severity Rating Scale
CF	Car Following
CMA-ES	Covariance Matrix Adaptation Evolution Strategy
CNN	Convolutional Neural Network
CPS	Cyber-Physical System
CTL	Computational Tree Logic
CV	Cross-Validation
DAcc	Dorsal anterior cingulate cortex
DE	Differential Evolution
DL	Deep Learning
DLC	Discretionary Lane Change
DLPFC	Dorsolateral Prefrontal Cortex
DNN	Deep Neural Network

EEG	Electroencephalography
ELU	Exponential Linear Unit
EMA	Ecological Momentary Assessment
ES	Evolutionary Strategy
FPGA	Field Programmable Gate Array
FSM	Finite State Machine
HANN	Hardware Artificial Neural Network
LC	Lane Change
LCD	Lane Change Decision
LCE	Lane Change Execution
LD	Lucky Door
LLM	Large Language Model
LTL	Linear Temporal Logic
LUT	Look-Up Table
MAE	Mean Absolute Error
MBD	Model-Based Design
MIP	Mixed-Integer Programming
ML	Machine Learning
MLP	Multilayer Perceptron
MSE	Mean Squared Error
NgOpt	Nevergrad Optimiser
OLS	Ordinary Least Squares
OS	Operating System
PHQ	Patient Health Questionnaire
PSO	Particle Swarm Optimization
RE	Runtime Enforcement
ReLU	Rectified Linear Unit
RF	Random Forest
RL	Reinforcement Learning
RNN	Recurrent Neural Network
RTOS	Real-Time Operating System

SHAP	Shapley Additive Explanations
SOS	Structural Operational Semantics
STL	Signal Temporal Logic
SVM	Support Vector Machine
TA	Timed Automata
Tanh	Hyperbolic Tangent
VDTA	Valued Discrete Timed Automata
VHDL	Very High-Speed Integrated Circuit Hardware Description Language
WCET	Worst-Case Execution Time
XAI	Explainable Artificial Intelligence
XGB	Extreme Gradient Boosting

Chapter 1

Introduction

1.1 Overview

Cyber-Physical Systems (CPSs) are at the heart of modern safety-critical applications, from autonomous vehicles to medical devices. As these systems grow in complexity, data-driven approaches using Machine Learning (ML)—particularly Artificial Neural Networks (ANNs)—have become indispensable for tasks such as perception, decision-making, and control, where traditional model-based design methods are no longer feasible [1]. However, the deployment of ML models in safety-critical CPSs introduces a fundamental tension: while ML models excel at learning complex, nonlinear input-output relationships from data, they are typically constructed as monolithic black-box systems that are difficult to verify, implement on hardware for timing analysis, and explain to stakeholders [2].

This monolithic design paradigm creates three interconnected problems. First, the formal verification of large monolithic ANNs is NP-Complete [3], making it computationally intractable for the network sizes required in real-world applications. This is compounded by the difficulty of implementing such networks on dedicated hardware platforms like Field Programmable Gate Arrays (FPGAs) for formal Worst-Case Execution Time (WCET) analysis, where only networks of a few tens of neurons can be feasibly deployed [4, 5]. Second, ensuring that ML-based CPSs adhere to safety specifications remains an open challenge. Existing formal verification approaches, whether based on reachability analysis [6], assume-guarantee reasoning [7], or abstraction techniques [8], struggle with the scale and nonlinearity of monolithic models. However, safety-critical applications demand rigorous guarantees that the system will never enter an unsafe state. Third, the lack of explainability in monolithic ML models undermines trust and limits their adoption in high-stakes domains such as healthcare [9]. While post-hoc explainability methods like Shapley Additive Explanations (SHAP) [10] and LIME [11] exist, their utility diminishes as model complexity and the number of in-

put features grow, making it difficult to extract clinically or operationally meaningful insights.

This thesis addresses these challenges through a unifying principle: *compositionality*. CPSs are inherently compositional, typically comprising distributed controllers and hardware components that are designed, verified, and maintained modularly [12, 13]. We argue that the ML models embedded within these systems should be designed with the same compositional principles. By systematically decomposing monolithic ML models into smaller, functionally equivalent sub-models, we can enable modular verification, facilitate hardware implementation for timing analysis, incorporate formal safety guarantees at every design stage, and provide more granular and actionable explanations.

The key insight of this thesis is that compositionality is not merely a system-level architectural choice but can be applied at the ML model level itself, yielding benefits across verification, safety, timing, and explainability simultaneously. We develop novel methodologies for decomposing monolithic ANNs into compositional architectures for both classification and regression tasks, design compilers that translate these compositional models to hardware (VHDL for FPGAs) and embedded software (C code), introduce a policy-driven framework that mines safety specifications from data and uses them to guide compositional model training with runtime enforcement, and propose a compositional explainability paradigm that combines multiple post-hoc techniques to provide insights at both the sub-model and system levels.

We validate our framework through two comprehensive case studies: autonomous vehicle lane-changing (addressing safety and timing) and personalised mood score prediction for depression (addressing explainability). Through extensive evaluation, we demonstrate that compositional ANN models achieve up to 85% reduction in WCET, 53% reduction in hardware resources, and 40% reduction in computations, while maintaining comparable or better performance relative to monolithic approaches, with enhanced safety guarantees and explainability.

1.2 Research Questions

The overarching goal of this thesis is to develop a compositional ML framework for CPSs that addresses the challenges of verification, timing, safety, and explainability. This goal is pursued through five specific research questions, each targeting a distinct aspect of the problem.

RQ1. **How can we decompose a monolithic ML model into a compositional model such that performance and functional behaviour are preserved?**

The central challenge here is to develop a principled methodology for replacing a single large ANN with multiple smaller sub-models that collectively reproduce the same input-output behaviour. For classification tasks, this means decomposing a multi-class classifier into independent binary classifiers—one per class—and then merging their outputs to recover the original classification. For regression tasks, this involves partitioning the feature space into clusters (e.g., using unsupervised methods such as k-means) and training a separate regression model for each cluster.

Existing work on ANN decomposition has been limited. [14] and [15] decompose trained monolithic networks into smaller independent networks, but their focus is on modularity rather than reducing overall network size or enabling verification; the total number of parameters may even increase after decomposition. Model reduction approaches such as [16] and [17] attempt to simplify networks through abstraction, but do not always produce simpler architectures and lack systematic, quantitative evaluation criteria. The key gap is the absence of a systematic, quantitatively evaluated methodology for decomposing monolithic ANNs that demonstrably reduces computational cost and resource requirements while preserving—or improving—task performance. This thesis addresses this gap for both classification and regression tasks across multiple real-world datasets.

RQ2. **How can we implement the compositional model on hardware for timing analysis using WCET?**

For time-critical CPSs, it is essential to guarantee that every computation completes within its deadline. Implementing ANNs on FPGA hardware enables formal WCET analysis because the absence of caches, pipelines, and branch predictors yields fully deterministic, cycle-accurate behaviour [18]. However, existing FPGA compilation tools for ANNs (e.g., [19, 5, 4]) are tailored to monolithic architectures and support only small networks with limited activation functions. None supports compositional architectures where multiple sub-models execute in parallel under synchronous semantics.

For example, consider an autonomous vehicle lane-change decision system implemented as a monolithic ANN on an FPGA. The entire network must execute sequentially, and the WCET scales with the total number of neurons. In a compositional design, however, independent binary classifiers can execute in parallel on separate FPGA resources, potentially reducing the WCET to that of the largest sub-model plus a small merging overhead. The research gap lies in the lack of general-purpose compilers that can translate compositional ANN architectures—developed in Python using frameworks like TensorFlow or PyTorch—to synthesisable Very High-Speed Integrated Circuit Hardware Description Language

(VHDL) (or C code for embedded systems) with synchronous compositional semantics. This thesis develops such compilers and demonstrates their use for formal timing verification.

RQ3. **How can we ensure that the compositional model is safe by construction?**

Safety-critical CPSs require not just post-hoc verification but safety guarantees that are designed into every stage of the ML pipeline. This question asks how formal safety specifications can be integrated into the compositional model design and training process itself, so that the resulting system is safe by construction rather than by subsequent analysis.

Consider an autonomous vehicle that must satisfy a safety policy such as "do not initiate a lane change when the gap to the nearest vehicle in the target lane is below a threshold". In a monolithic design, such policies are difficult to encode into the training process because the model treats all inputs holistically. In a compositional design, however, we can decompose this high-level policy into sub-policies—each associated with a specific sub-model—and train each sub-model to satisfy its corresponding sub-policy. For instance, one sub-model might handle gap-acceptance decisions while another handles speed-matching, each with its own formally defined safety constraints.

While existing works have explored policy mining [20, 21] and compositional verification [6, 7], no existing methodology combines mined formal policies with compositional ML model design to ensure policy compliance throughout the training and deployment pipeline. This thesis fills this gap by introducing a policy-driven framework that mines safety properties from data, decomposes them into sub-policies aligned with compositional sub-models, and uses these sub-policies to guide training.

RQ4. **How can we ensure that the compositional model adheres to safety specifications?**

Even with safe-by-construction training, learned models may encounter scenarios not represented in the training data, potentially leading to unsafe outputs. This question addresses the need for a runtime safety net that monitors model outputs and enforces compliance with safety specifications in real time.

Runtime Enforcement (Runtime Enforcement (RE)) provides a lightweight, dynamic approach to this problem: a formal monitor observes the system's outputs at runtime and, upon detecting an imminent policy violation, corrects the output before it reaches the actuators. For example, if a compositional lane-change model recommends a lane change but the runtime monitor determines that the

safety gap constraint is violated, the monitor overrides the output to prevent the lane change. Several RE mechanisms have been proposed for general reactive systems [22–25], but none has been applied to compositional ML models. This thesis develops and demonstrates runtime enforcement specifically tailored to compositional ML-based CPSs, using Valued Discrete Timed Automata (VDTA) as enforcement monitors.

RQ5. **How can we build compositional models that are explainable and provide actionable insights based on the model's explanations?**

Explainability is essential for building trust in ML models, particularly in domains like healthcare where practitioners need to understand *why* a model produces a given prediction in order to act on it. Monolithic models that ingest large numbers of disparate features make it difficult to decompose model behaviour into meaningful explanations, because all features are entangled within a single model [26]. When such a model underperforms, it is challenging to isolate which group of features is responsible.

A compositional approach offers a natural solution: each sub-model is associated with a semantically meaningful subset of features (e.g., activity-related features, sleep-related features, dietary features in a mood prediction model), enabling focused explanations at the sub-model level. For example, if a personalised mood prediction model decomposes into sub-models for activity, sleep, diet, and cognition, a clinician can examine the SHAP values and Accumulated Local Effects (ALE) plots for each sub-model independently to understand which lifestyle domain most influences a patient's mood, and what specific changes within that domain might improve outcomes.

While post-hoc explainability methods are well-established [10, 27, 28], their application has been limited to monolithic models, where they extract only the most influential features without providing structured, domain-aligned insights [26]. No existing work has explored how compositional ML design can enhance explainability by enabling multi-level, multi-technique explanations aligned with meaningful feature groupings. This thesis introduces a compositional explainability paradigm that combines SHAP, ALE, and Anchors at both the sub-model and system levels, demonstrating enhanced interpretability over monolithic approaches.

1.3 Contributions

In this work, we address the above questions by proposing a new paradigm for ML-based CPS design that is compositional, explainable, and adheres to safety specifica-

tions. Such a framework not only has applications in the examples considered in this work but can be used in any similar CPSs using ML models.

1. **A method for decomposing a monolithic ML model into a compositional model:** This work develops a novel procedure to replace large ANN monolithic designs with smaller compositional designs and implement them on a Field Programmable Gate Array (FPGA) for timing analysis using synchronous compositional semantics. To illustrate our approach, we develop regression and classification ANN designs for multiple real-life datasets, including transportation and healthcare datasets.

2. **A compiler to compile ANN models in Python to VHDL code for timing verification:** One of the key requirements to test the WCET of an ANN-based CPS through hardware implementation is a compiler that compiles the ANN models developed in Python using Tensorflow or PyTorch to VHDL code for FPGA hardware implementation. While several open-source compilers exist that can compile an ANN model to VHDL code, they cannot work with the compositional architecture proposed in this work due to its synchronous compositional semantics. Hence, we designed a new compiler in Python (as a Python package) to produce VHDL code from PyTorch and Tensorflow-Keras ANN models using synchronous compositional semantics.

3. **A compiler to compile ANN models in Python to C code for embedded system applications:** In tandem with the ANN to VHDL compiler, we also developed an ANN to C code compiler to enable the use of the developed compositional ANN models in embedded system applications. Again, while several open-source compilers exist that can compile an ANN model to C code, they cannot work with the compositional architecture proposed in this work due to its synchronous compositional semantics. Hence, we designed a new compiler in Python (as a Python package) to produce C code from PyTorch and Tensorflow-Keras ANN models. We do not directly use the C code for the embedded system applications in this thesis and present it in Appendix C.

4. **An approach to building safe-by-construction compositional designs:** In this work, we introduce a policy-driven framework for compositional ML-based CPS. Using an Autonomous Vehicle (AV) as a representative case study, we show how to mine meaningful safety policies from data, and use these policies to decompose a monolithic ML model into composable sub-models and guide the training of each sub-model. To address unexpected or unseen scenarios where learned models may fail to comply with safety expectations, we employ a run-

time enforcement mechanism that monitors model outputs and ensures policy adherence in real time.

5. **A method for building models that are explainable, focusing on compositional design:** We propose an explainable ML framework for both monolithic and compositional ML models and explore the approach on a safety-critical personalised mood score (depression scale) prediction case study - predicting the mood score of a patient based on their lifestyle factors. It is safety-critical as the model is used to guide clinical interventions. Our compositional approach builds models from separate sub-models aligned with clinical symptom groups (e.g., activity, diet, cognition) using a parallelised optimisation approach. Also, to explain model decisions, we develop a novel compositional explainability paradigm that applies SHAP, ALE and Anchors rules to provide actionable insights for clinical interventions. A side effect of this work is that MLP models are found to be more performant than other baseline ML models.

1.4 Organisation of Thesis

The rest of this thesis is organised as follows:

1. **Chapter 2:** This chapter provides the technical background necessary to understand the research presented in this thesis. It covers the fundamentals of CPSs, ML (with a focus on ANNs), explainability techniques, and formal modelling. It also presents the principles of compositionality in model-based CPS design and summarises the use of data-driven ANNs in CPSs, including their limitations in verification and timing analysis.

2. **Chapter 3:** This chapter reviews existing work directly related to the technical contributions of this thesis. It begins with compositional modelling for ML-based CPSs, then covers FPGA hardware compilation, compositional verification and safety, explainability in compositional design, and the literature on the case studies considered in this work—autonomous vehicles and personalised mood score prediction for depression.

3. **Chapter 4:** This chapter presents our first major contribution: a systematic methodology for decomposing monolithic ML models into compositional architectures and implementing them on hardware for timing analysis. We develop novel compilers that translate Python-based neural network models to both VHDL (for FPGA implementation) and C code (for embedded systems), enabling formal

WCET analysis. This work addresses Research Questions 1 and 2, demonstrating how compositional models can be decomposed and implemented for timing verification.

4. **Chapter 5:** Building on the compositional framework discussed in Chapter 4, this chapter introduces our policy-driven approach for building safe-by-construction ML-based CPS. Using AV lane-changing as a case study, we demonstrate how safety policies can be mined from data and used to design and train compositional sub-models. We also present runtime enforcement mechanisms that ensure policy adherence in real-time, addressing Research Questions 3 and 4 regarding safety guarantees and verification.

5. **Chapter 6:** This chapter takes inspiration from the compositional framework discussed in Chapter 4 to show how compositionality can aid in building more explainable ML models, addressing Research Question 5. Particularly, we develop a novel compositional explainability paradigm that combines SHAP, ALE, and Anchors techniques to provide actionable insights at both the sub-model and system levels. Using a personalised mood score prediction case study, we demonstrate how compositional models can offer enhanced explainability compared to monolithic approaches.

6. **Chapter 7:** This final chapter summarises the contributions made throughout the thesis. We reflect on the implications of our work for the broader field of ML-based CPSs and outline promising directions for future research, including extensions to other domains and integration with emerging technologies.

Chapter 2

Background

2.1 Overview

This chapter provides the background necessary to understand the research presented in this thesis. We begin by introducing Cyber-Physical Systems (CPSs), which forms the primary application domain for our work. This includes discussions on safety-critical systems, timing analysis and Worst-Case Execution Time (WCET) analysis, and the distinction between real-time and synchronous execution models. We then present the fundamentals of Machine Learning (ML), particularly on Artificial Neural Networks (ANNs), their training processes, evaluation metrics, and hyperparameter tuning techniques.

We also cover explainability in ML, introducing key techniques such as Shapley Additive Explanations (SHAP), Accumulated Local Effects (ALE), and Anchors essential for building explainable models. We then discuss formal modelling techniques, including Finite State Machines (FSMs), Timed Automata (TA), and Runtime Enforcement mechanisms that are important for verifying system safety and correctness.

Following this technical foundation, we present the theoretical background on traditional model-based design and compositionality in CPS, examining how complex systems can be decomposed into manageable components. We then explore the current state of monolithic ML approaches in CPS, focusing on their strengths in handling complex, nonlinear relationships and their limitations in verification, timing analysis, and explainability, and examine existing attempts to use compositional approaches to ML-based CPS design.

2.2 Cyber-physical Systems

CPSs are digital systems that continuously interact with their physical environment. They consist of sensors, actuators and communicating devices that interact with each

other and the physical world in a feedback loop [12]. CPSs consist of the physical process, the *plant* and the embedded digital system, the *controller*. A good example of a cyberphysical system will be an Autonomous Vehicle (AV), where the various sensors in the car are used by multiple connected controllers within the car to manoeuvre the car safely in its environment (the plant).

There are often safety-critical real-time systems, meaning they require *firm* guarantees on their safety and timing behaviour [13]. For example, consider the task of lane changing in an AV, which we use later as a case study. To make a successful lane change, the vehicle must sense the physical environment, such as the vehicles surrounding it, and determine when it is safe for the car to lane change based on learned behaviour. If the conditions are safe, inputs must be given to the digital car controls to manipulate the physical car components to execute a safe lane change. Thus, a functional and safe CPS needs control, computing, and communication to work together synergistically.

2.2.1 Safety Critical System

Embedded systems can be classified as safety-critical [29, 30] if their proper function is important to ensuring the safety of people around them. A malfunction in these systems can lead to injury or even death. These systems require that they never produce an output that puts them in an unsafe state. Continuing the example of lane changing, which is a safety-critical system, it will mean that the car should never try to execute a lane change when there are cars next to it in the adjacent lanes.

It is crucial to verify the correctness of such a system to ensure safety. Verifying safety-critical systems often involves using high-level programming languages, as they allow for specifying the safety policies or specifications in an easy-to-understand and easy-to-code manner, which reduces the risk of errors in code logic. For example, easyRTE[1] is one such tool that allows the safety specifications to be written in a programmer-friendly *rte* language, which can then be invoked to ensure the runtime correctness of a system [25].

2.2.2 Timing Analysis and Worst-Case Execution Time

The physical world is continuous, while the digital controller in a CPS is discrete in nature. This requires that the digital system respond to the changes in the physical environment in a timely fashion. This is known as the system's timing requirement. Real-time systems that require strict timing requirements or deadlines to ensure safe operation are termed as *hard real-time* systems. Missing these timing deadlines can

[1] https://github.com/PRETgroup/easy-rte

have grave consequences. Systems where a few timing requirements can be missed with little consequences are termed as *soft real-time* [18] systems.

Generally, upper bounds on the execution time of such systems are needed to satisfy constraints for real-time systems with strict timing constraints. Timing analysis of real-time systems attempts to determine these bounds when a system is executed on a specific hardware [18]. Assuming that a real-time system comprises a collection of tasks that achieve a standard functionality, the execution times of tasks vary with the input data or different environment behaviour [18]. This variation results from the different paths different inputs may take within a task, depending on different conditions.

Figure 2.1 shows an example variation of execution times depending on input data and environment factors. The blue curve represents the set of all execution times, whereas the orange curve represents the measured set of execution times. Thus, the longest execution time, in reality, is called the WCET. It indicates a system's timing performance under the worst possible input and environment conditions [18].

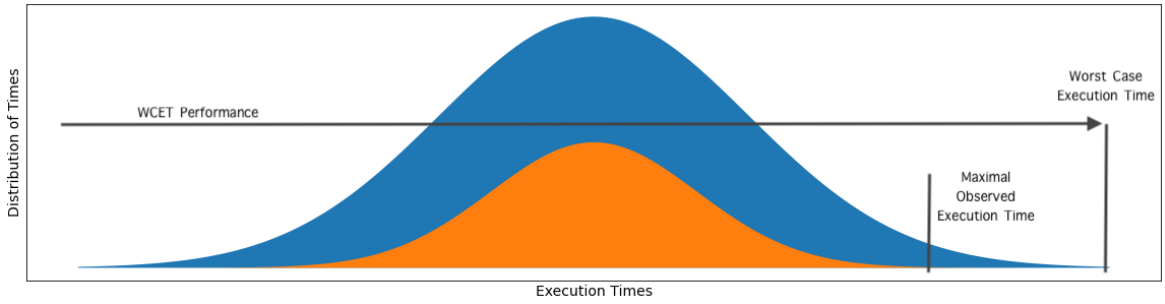


Figure 2.1: Showing the distribution of example variation in execution times for a task as inputs and environment conditions change. The blue shaded region depicts the actual or real execution time variation, whereas the orange region represents the observed execution times.

As the execution time for a particular task depends on the path the control takes during a task and the time that statements and instructions spend on this path, the determination of WCET has to consider the potential control-flow paths and the execution times for this set of paths [18]. There are two main approaches to timing analysis using WCET. First are Measurement-based methods that execute the task or task parts on the given hardware or a simulator for some set of inputs. Then, the maximal observed execution times are derived from the set of observed execution times. The worst maximal execution time is then used as the WCET. As expected, this method is limited by the type of inputs checked and can often provide an underestimation of the WCET [18].

On the other hand, Static methods do not depend on executing code on real hardware or a simulator. They instead take the task code itself, examine the set of possible

control flow paths through the task, combine control flow with a (abstract) model of the hardware architecture, and obtain upper bounds (WCET) for this combination. Static methods emphasise safety by producing bounds on the execution time and guarantee that the execution time will not exceed these bounds.

2.2.3 Real-time and Synchronous Execution

When multiple components in a CPS must interact, they must all advance in lockstep with wall-clock time. In purely software-based subsystems, this can be enforced via synchronisation signals. However, when real physical devices and computational models coexist, no single clock signal can span both domains. Instead, each element must honour *real-time constraints* [31], meaning that its computation or physical action at every moment must complete in precisely the same duration as the corresponding simulated time.

Furthermore, a CPS must be *reactive* to interact predictably with its environment. Conventional CPUs achieve reactivity either by *polling*—continually checking inputs at regular intervals, which wastes cycles and can incur long delays—or by *interrupts*, where a hardware signal forces an immediate change in program flow. While interrupts reduce idle CPU time, they complicate static timing analysis, often causing an explosion in the state space that must be examined. As a result, determining whether such systems meet hard real-time deadlines typically requires statistical simulations (e.g., Monte Carlo methods) that cannot exhaustively cover every path.

By contrast, synchronous execution adopts a discrete model of computation in which program behaviour is divided into atomic reactions separated by logical ticks [32, 33]. At each tick, inputs are sampled instantaneously, internal computations occur atomically, and outputs are committed simultaneously at the tick's end. The implementation enforces a known WCET per reaction so that logical time can be safely mapped to physical time. Unlike hardware interrupts—which may preempt execution arbitrarily and require context-save/restore—external events in the synchronous model are queued until the next tick boundary. At that point, the current reaction terminates cleanly and control transfers to the appropriate handler, without preserving any intermediate state. This provides a provable upper bound on response latency while keeping the state space small and deterministic.

Synchronous execution is ideal for embedded control where deterministic timing is non-negotiable, such as in safety-critical CPS. Its key advantages include:

- **Deterministic Logical Time:** Rather than modelling continuous wall-clock time directly, the synchronous approach provides a logical clock and enforces that each reaction finishes within a known bound [32, 33]. This yields a fully deterministic, formally analysable timing model. In contrast, continuous-time

real-time systems must mirror physical time precisely—introducing hardware jitter, interrupt interference, and integration-solver uncertainties that are typically checked by empirical measurement rather than full formal proof [34, 35].

- **Formal Deadline Guarantees:** As each synchronous reaction has a provable worst-case reaction-time bound (often mapped directly to its WCET on the target hardware), we can perform static schedulability analyses to guarantee deadlines [36]. In continuous-time or Real-Time Operating System (RTOS)-based systems—where tasks may be preempted by interrupts, use diverse clocks, or depend on Operating System (OS) scheduling—formal guarantees are far more complex and often require conservative benchmarks or runtime monitoring to ensure deadlines [34].

2.3 Machine Learning

As model-based design approaches to designing CPSs become unmanageable due to the complexity of the systems, we have seen a shift towards the use of data-driven models using Machine Learning (ML). Here, we discuss the basics of ML, particularly ANNs, and their training and evaluation processes.

Broadly, ML is a field of Artificial Intelligence (AI) that focuses on developing algorithms that allow mathematical models to learn from data and make decisions based on that data by learning patterns or representations in data [37]. ML is used in a range of applications, such as natural language processing (Large Language Models (LLMs)), speech recognition, and image recognition. In this work, we extensively use ML models (Artificial Neural Networks, in particular).

2.3.1 Machine Learning Tasks

The task of using mathematical algorithms to learn representations from data can be categorised into three main kinds: supervised learning, unsupervised learning, and reinforcement learning. Each of these types of learning has its own set of tasks and algorithms, and is used in different scenarios depending on the data and the problem at hand.

Supervised Learning: In Supervised learning, an ML model learns the input-output mapping from a set of labelled data. The model is trained on labelled data, where the input is the feature vector and the output is the label. The model then learns the relationship between the input and the output, and can predict the output for unseen data.

The most commonly used supervised learning tasks are classification and regression. In classification tasks, the model learns to predict the class label of the input

data, while in regression tasks, the model learns to predict a continuous value. The most common supervised learning algorithms are Decision Trees, Ensemble Models like Random Forest (RFs), Support Vector Machines (SVMs), and ANNs. As we have labelled data, we will use ANNs for both classification and regression in this work, as ANNs have better representative power than other ML approaches.

Unsupervised Learning: Unsupervised learning is where the model learns the underlying structure of the data without any labels. The model is trained on an unlabelled dataset, where the input is the feature vector. The model then learns the relationship between the input data points and can group similar data points together. The most commonly used unsupervised learning tasks are clustering and dimensionality reduction. In clustering tasks, the model learns to group similar data points together, while in dimensionality reduction tasks, the model learns to reduce the number of features in the data.

Reinforcement Learning: Reinforcement learning teaches models when to take certain actions in an environment to maximise a reward signal. The reinforcement learning algorithm aims to learn a policy that maps the state of the environment to an action. The model is trained on a set of state-action-reward tuples, where the state is the current state of the environment, the action is the action taken by the model, and the reward is the reward signal received by the model. The model then learns the relationship between the state and the action, and can take actions to maximise the reward signal. The most common reinforcement learning tasks are game playing and robotics.

2.3.2 Artificial Neural Networks

Artificial Neural Networks (ANNs) are, as the name suggests, networks of artificial neurons that attempt to model the behaviour of biological neurons. ANNs use linear mathematical functions and nonlinear functions called activation functions to learn patterns or representations in data [1]. ANNs are composed of layers of neurons, with each layer having a specific number of neurons. The first layer is called the input layer, the layers in between are called hidden layers, and the last layer is called the output layer. The more layers an ANN has, the more *deep* it becomes. Networks with multiple hidden layers are called Deep Neural Networks (DNNs). DNNs are part of a subset of ML called Deep Learning (DL). Similarly, networks with one or fewer hidden layers are called a *shallow* Neural Network.

Most ANNs determine the untimed nonlinear relationships or mappings between a provided set of inputs and their corresponding outputs through a learning process called *training*. This process manipulates the parameters/weights of an ANN to learn an accurate representation of the relationship between the inputs and the outputs.

This ability allows them to perform tasks such as classification of input data, forecasting (regression), and language translation. The most common types of ANNs are Multilayer Perceptrons (MLPs), Convolutional Neural Networks (CNNs), and Recurrent Neural Networks (RNNs). While ANNs can be used for supervised, unsupervised or reinforcement learning, we focus on their use in the supervised learning tasks of classification and regression. Also, we focus on MLPs as they are the most common type of ANNs used in this work.

2.3.2.1 Multilayer Perceptrons

ANNs can be composed of artificial neurons of one or more types. MLPs are the simplest kind of ANNs and are composed of perceptrons. Perceptrons are the simplest form of artificial neurons and are composed of a linear function followed by a nonlinear activation function. Mathematically, the output of a perceptron is given by Equation 2.1.

$$\mathbf{Out} = f(\mathbf{W} \cdot \mathbf{In} + \mathbf{B}) \tag{2.1}$$

where, **Out** is the output of the perceptron; **W** is the weights vector; **In** is the input vector; **B** is the bias vector; and f is the nonlinear activation function. The weights and biases are learned during the training process.

The activation function f is a nonlinear function that introduces nonlinearity into the output of the perceptron. This nonlinearity is crucial for the perceptron to learn complex nonlinear patterns in the data. Without the nonlinearity, the whole model will be reduced to a linear model, and the model will only learn linear relationships between the inputs and the outputs. The most common activation functions are Linear (Equation 2.2), Sigmoid (Equation 2.3), Hyperbolic Tangent (Tanh) (Equation 2.4), ReLU (Rectified Linear Unit, (Equation 2.5)) and Softmax (Equation 2.6). The following are the mathematical expressions for these activation functions, in the same order.

$$f(x) = x \tag{2.2}$$

$$f(x) = \frac{1}{1 + e^{-x}} \tag{2.3}$$

$$f(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}} \tag{2.4}$$

$$f(x) = \max(0, x) \tag{2.5}$$

$$f(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}} \tag{2.6}$$

where x_i is the ith element of the input vector, x_j is the jth element of the input vector, and n is the number of elements in the input vector.

While the choice of activation function rests on the problem at hand, the sigmoid is commonly used in the output layer of binary classification tasks. In contrast, softmax is used in the output layer of multi-class classification tasks. Linear, Tanh, and ReLU are primarily used in hidden layers of MLPs for regression and classification tasks. Linear activation functions are normally used in the output layer of regression tasks.

MLPs are usually fully connected, meaning that all the neurons in a layer are connected to all the neurons in the next layer. Equation 2.7 expresses the mapping of inputs and outputs at layer l of an MLP.

$$\mathbf{Out}^l = f^l(\mathbf{W}^l.\mathbf{Out}^{l-1} + \mathbf{B}^l) \tag{2.7}$$

where, $\mathbf{Out}^l$ is the output vector produced by layer l; $\mathbf{W}^l$ is the weights matrix for layer l; $\mathbf{Out}^{l-1}$ is the input vector for layer l, which is the equal to the output from the previous layer $l-1$; $\mathbf{B}^l$ is the bias vector; and f^l is the nonlinear activation function for layer l. For the first layer $l = 1$, the input to the network becomes the input to the layer. An MLP model is illustrated in Figure 2.2.

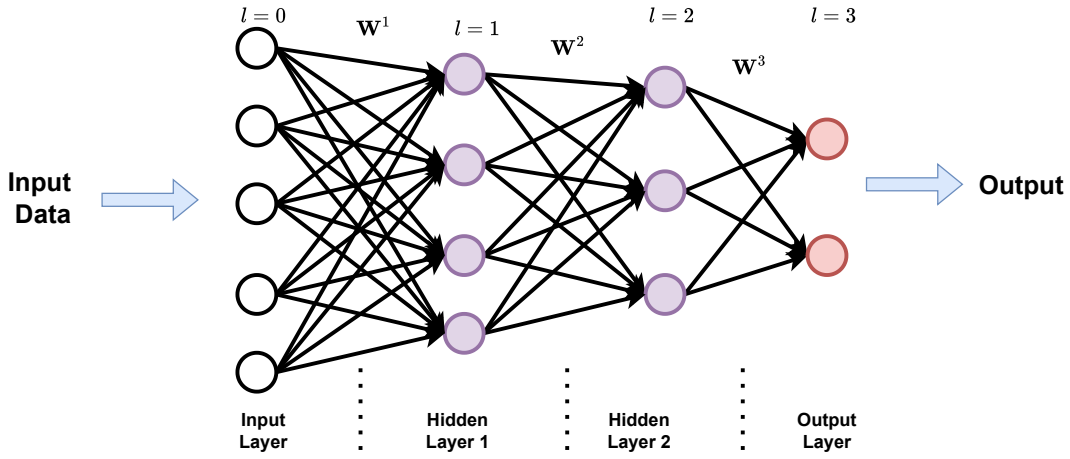


Figure 2.2: A representative MLP model.

2.3.2.2 Training

ANNs *learn* the representation of the input-output mapping through a process called training. During training, the ANN modifies the weights $\mathbf{W}$ of the network such that a particular Cost function (also called Loss function) is optimised given a set of inputs

or features and outputs. The loss function is usually a piecewise differentiable function of the true outputs and the outputs predicted by the network. It measures how much the predicted output deviates from the true output. The most common Loss function for classification is the Cross-entropy loss, defined as in Equation 2.8. This is used in multi-class classification tasks.

$$L_{CrossEntropy} = -\sum_{i=1}^{C} y_i \cdot \log \hat{y}_i \tag{2.8}$$

where, C is the number of classes in the data, y_i is the expected class i label/output, and $\hat{y}_i$ is the predicted output probability for a class i. For binary classification tasks, a special case of the Cross-entropy loss, the Binary Cross-entropy loss is used as shown in Equation 2.9.

$$L_{BinaryCrossEntropy} = -y \cdot \log \hat{y} - (1-y) \cdot \log (1-\hat{y}) \tag{2.9}$$

Moreover, in regression tasks, the Mean Squared Error (MSE) loss is used as shown in Equation 2.10. Here, the goal is to minimise the difference or error between the expected and predicted output values, which are usually continuous values.

$$L_{MSE} = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \tag{2.10}$$

where, y_i is the expected output, and $\hat{y}_i$ is the predicted output. Here, n is the number of samples in the dataset.

The optimisation of the Loss function is performed by an optimiser (such as Adaptive Moment Estimation (Adam) [38]), which computes the ANN gradient in weight space with respect to the loss function. This gradient is then used to update the weights as shown in Equation 2.11.

$$\Delta \mathbf{W} = \eta \frac{\delta \mathbf{L}}{\delta \mathbf{W}} \tag{2.11}$$

where $\mathbf{L}$ is the Loss function and η is the learning rate that controls the magnitude of change in the weights.

This process of weight updation is called back-propagation. While the step of calculating the output across multiple layers using Equation 2.7 is called the forward pass, the weight updation step is called the backward pass. These two steps can be repeated iteratively until the desired Loss value or performance is obtained. If this process is not controlled correctly, the model may not converge to the desired output or overfit the data. Overfitting is a common problem in ANN models, where the model begins to learn the noise in the data rather than the actual input-output representation. This makes the model fail to generalise well to unseen but similar data. Normalisation

of data and regularisation techniques can be used to prevent convergence issues and overfitting, respectively [1].

Regularisation: The problem of overfitting in ANNs is a common problem, and is usually solved using techniques such as dropout, early stopping, and regularisation during training. Dropout is a technique where a subset of neurons is randomly turned off during training, which helps in preventing the model from overfitting. Early stopping is a technique where the training is stopped when the model starts to overfit the data. Regularisation is a technique where a penalty term is added to the Loss function to prevent the model from learning complex patterns in the data. The most common regularisation techniques are L1 and L2 regularisation, which add the absolute and squared sum of the weights to the Loss function, respectively.

Normalisation: Often, the data used for training is normalised before it is sent to the ANN. This technique avoids the slower convergence of ANN models during training due to differences in scaling across features. While different scaling approaches are available, the most commonly used ones are Standard (or Z) scaling (see 2.12) and MinMax scaling (see 2.13).

$$\mathbf{X} = \frac{\mathbf{X} - \boldsymbol{\mu}}{\boldsymbol{\sigma}} \tag{2.12}$$

where $\mathbf{X}$ is the matrix of input samples (excluding the labels), $\boldsymbol{\mu}$ is a vector containing the mean of the samples for each feature, and $\boldsymbol{\sigma}$ is the vector containing the standard deviation for the samples.

$$\mathbf{X} = \frac{\mathbf{X} - \mathbf{X}_{min}}{\mathbf{X}_{max} - \mathbf{X}_{min}} \tag{2.13}$$

where $\mathbf{X}_{min}$ and $\mathbf{X}_{max}$ are the minimum and maximum values of the samples for each feature.

Training Libraries: Python libraries such as TensorFlow [39] and PyTorch [40] automate this process of training, and are often used by designers for ANN construction and training due to the simple API-based interface that these libraries provide. These tools provide means to define the ANN architecture, the Loss function, the optimiser, and the training parameters, and then train the model with a single function call. They also provide tools to control and monitor the training process, such as callbacks, which can be used to save the best model, reduce the learning rate, stop the training process early if the model starts to overfit the data, and regularise the model.

2.3.2.3 Evaluation

Another part of ANN model development is the evaluation of the models developed. The evaluation is performed by measuring the model's performance on selected data

using a set of metrics chosen based on the task at hand. These metrics try to ascertain the extent to which the model has learned the input-output representation in the data. In other words, the metrics measure how well the model output matches the expected (true) output.

For classification tasks, metrics such as accuracy, precision, recall, and F1 score are used. For regression tasks, metrics such as Mean Squared Error (MSE), Mean Absolute Error (MAE), and R-squared are used. The following equations show the calculation of the metrics used in classification tasks.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{2.14}$$

$$Precision = \frac{TP}{TP + FP} \tag{2.15}$$

$$Recall = \frac{TP}{TP + FN} \tag{2.16}$$

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \tag{2.17}$$

where, TP is the number of True Positives, TN is the number of True Negatives, FP is the number of False Positives, and FN is the number of False Negatives. Also, the following equations show the calculation of the metrics typically used in regression tasks.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{2.18}$$

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{2.19}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{2.20}$$

where, y_i is the expected output, $\hat{y}_i$ is the predicted output, and $\bar{y}$ is the mean of the expected output.

The choice of metric is crucial as it determines the performance of the model. For example, accuracy is not a good metric in a classification task when the classes are imbalanced. Take the case where we have 100 data points, of which 90 belong to a particular class. If a model predicts all the datapoints to belong to the majority class, the accuracy of the model will be 90%. However, the model has not learned the input-output representation in the data. In such cases, metrics like the F1 score are preferred. Similarly, for regression tasks, the MSE is a good metric when the data

does not have outliers or is normalised, while the MAE is a good metric when we want to obtain the error in the same range as the data.

Usually, the performance of a model is evaluated using a metric on a separate set of data called the test set, which is not used during the training process. This is done to ensure that the model generalises well to unseen data. However, this approach can lead to data-selection bias if the test set does not represent the data distribution. To avoid this, a technique called cross-validation is used. In cross-validation, the data is divided into k folds, and the model is trained on $k-1$ folds and tested on the remaining fold. This process is repeated k times, with each fold serving as the test set once. The model's performance is then averaged over the k test folds to get a more accurate estimate of the model's performance. 5-fold Cross-Validation (CV) is one of the most commonly used techniques. We can also *stratify* the data into folds to ensure that the folds have nearly the same proportion of the output variable/label. This is useful when the data is imbalanced [1].

2.3.2.4 Hyperparameter Tuning

Hyperparameters are parameters that control the model's learning process and can significantly impact the model's performance. Common hyperparameters include the learning rate, the number of layers in the model, the number of neurons in each layer, the activation function, the optimiser, the batch size, and the number of epochs. The process of finding the best hyperparameters for a model is called hyperparameter tuning.

Tuning is usually performed using techniques such as grid search, random search, Bayesian optimisation or heuristic approaches like Evolutionary Algorithms. While Grid Search and Random Search are the most commonly used techniques, they are computationally expensive and time-consuming. They do not offer a guided search of the hyperparameter space and can be inefficient when the hyperparameter space is large. Hence, we focus on Bayesian Optimisation and Evolutionary Algorithms for hyperparameter tuning.

Bayesian optimisation : Bayesian optimisation is an advanced technique for hyperparameter tuning that is particularly effective when model evaluations are expensive or time-consuming. Instead of exhaustively searching the hyperparameter space, Bayesian optimisation builds a probabilistic surrogate model—commonly a Gaussian Process—to approximate the relationship between hyperparameters and the model's performance metric (such as accuracy or loss). This surrogate model is updated iteratively as new hyperparameter configurations are evaluated. This makes it especially useful for tuning complex models or when computational resources are limited [41].

Evolutionary Algorithms: Evolutionary Algorithms are a class of algorithms inspired by natural selection and evolution [42]. These algorithms start with a random population of candidates and iteratively evolve the population using selection operators to choose the *fittest* or most viable candidates. The process continues until a stopping criterion (usually based on a performance metric) is met. They offer a more guided and efficient search of the hyperparameter space compared to Grid Search and Random Search. Some of the most commonly used Evolutionary Algorithms are Particle Swarm Optimization (PSO) [43], Differential Evolution (DE) [44], and Evolutionary Strategy (ES) [45].

2.4 Explainability in Machine Learning

While there is no mathematical definition for explainability[2] regarding its use in Machine Learning, it is commonly accepted that it is the degree to which a human can understand and consistently predict the outputs of an ML model [46]. This is important for a number of reasons, including the need to understand the model's behaviour, to ensure that it is making decisions in a fair and unbiased way, and to build trust in the model. This is particularly important when using *black-box* models (models for which it is too complex to understand their behaviour) used in high-stakes applications, such as healthcare.

We can broadly categorise explainability techniques into two categories: model-specific and model-agnostic. Model-specific techniques try to explain the decisions made by a specific model, while model-agnostic techniques can be applied to any model. Model-specific techniques include decision trees [47], which are easy to interpret but have limited expressive power, and ANN, which are difficult to interpret but can make more complex associations from multimodal data.

Another way to categorise explainability techniques is by when they are applied: pre-hoc, in-hoc, and post-hoc [9]. Pre-hoc techniques are applied before the model is trained and include techniques such as feature selection and feature engineering [48]. In-hoc techniques are applied during the model training and include techniques such as attention mechanisms and saliency maps.

Model-agnostic and post-hoc techniques, including Shapley Additive Explanations (SHAP) [10], Accumulated Local Effects (ALE) [27], and Anchors [28], are particularly useful to interpret ANN outputs. In this section, we will discuss these techniques in more detail.

[2]We use explainability and interpretability interchangeably in this work, as used in [46].

2.4.1 Shapley Additive Explanations

SHAP provides both local (explaining each prediction) and global (predictions over the entire dataset) explanations [10]. It computes the contribution of each feature to the prediction by linearly approximating the Shapley values. The computation of Shapley values for a feature i is given by:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [f(S \cup \{i\}) - f(S)] \tag{2.21}$$

where N is the set of all features, S is a subset of N, f is the model, and ϕ_i is the SHAP value for feature i.

SHAP simplifies the equation above and specifies an explanation as a linear model [9]. The SHAP value for a feature i is given by:

$$f(x) = \phi_0 + \sum_{i=1}^{M} \phi_i x_i \tag{2.22}$$

where $f(x)$ is the model prediction, ϕ_0 is the average prediction of the model - the prediction average over a background or reference dataset. Also, ϕ_i is the SHAP value for feature i, and x_i is the feature value, and M is the number of features.

As such, SHAP values are computed in relation to the average model prediction. Thus, a SHAP value of -0.4 for a feature in a sample, for instance, means that the model prediction decreases by 0.4 from the average for a change in that feature. SHAP can be used to find the most important features of each participant. We get one SHAP value per data instance per feature and compute the mean of the absolute SHAP values for all instances in the dataset to obtain the overall SHAP value of a feature and the global feature importance for a model and a dataset.

2.4.2 Accumulated Local Effects

ALE plots illustrate how specific features impact a model's predictions, with their values representing the main effect of a feature at a given point relative to the average prediction. Unlike some methods, ALE handles correlated features effectively [27]. We can determine using ALE plots how the most important features, as identified by SHAP, affect the model's predictions. These plots reveal how the effect of a feature on the prediction changes with different feature values, providing insight into whether an increase (or decrease) in the feature leads to a corresponding rise (or fall) in the model's prediction compared to the average.

The ALE values are computed in two steps. First, we compute the local effects. The local effects of Accumulated Local Effects are computed by dividing the feature

into many intervals and computing the differences in the predictions. This procedure approximates the gradients and also works for models without gradients. First, we estimate the uncentered effect:

$$\hat{\tilde{f}}_{j,ALE} = \sum_{k=1}^{k_j(x)} \frac{1}{n_j(k)} \sum_{i:x_j^{(i)} \in N_j(k)} [f(z_{k,j}, x_{\setminus j}^{(i)}) - f(z_{k-1,j}, x_{\setminus j}^{(i)})] \tag{2.23}$$

where f is the model, x is the data instance, j is the feature, k is the interval, $N_j(k)$ is the set of instances in the kth interval, $z_{k,j}$ is the kth interval value, and $n_j(k)$ is the number of instances in the kth interval.

Second, the uncentered effect is centred so that the mean effect is at zero [9]. The centred effect is given by:

$$\hat{f}_{j,ALE} = \hat{\tilde{f}}_{j,ALE} - \frac{1}{n} \sum_{i=1}^{n} \hat{\tilde{f}}_{j,ALE}(x_j^{(i)}) \tag{2.24}$$

where, $\hat{f}_{j,ALE}$ denotes the centred Accumulated Local Effect for feature j at a specific value. The term $\hat{\tilde{f}}_{j,ALE}$ represents the uncentered Accumulated Local Effect, which reflects the sum of local changes in the model's prediction attributable to feature j. The second term, $\frac{1}{n}\sum_{i=1}^{n} \hat{\tilde{f}}_{j,ALE}(x_j^{(i)})$, computes the mean uncentered effect across all n data instances, where $x_j^{(i)}$ is the value of feature j for the i-th instance. Subtracting this mean centres the ALE values so that their average effect over the dataset is zero, making interpretation relative to the average model prediction easier.

The value of the centred ALE can be understood as the main effect of the feature at a specific value compared to the average prediction of the data. For example, an ALE estimate of -4 at $x_j = 6$ means that when the j^{th} feature has a value of 6, then the prediction is lower by four compared to the average prediction.

2.4.3 Anchors

Anchors provide explanations for a prediction made by any black-box classification model by identifying an IF-THEN decision rule that sufficiently *anchors* the prediction [28]. A rule is considered to anchor the prediction if changes in other features do not change the outcome. Additionally, anchors incorporate the concept of coverage, indicating the range of other, potentially unseen, instances to which the explanation applies.

We can define an anchor A formally as follows:

$$\mathbb{E}_{D_x(z|A)}[1_{f(x)=f(z)}] \geq \tau, A(x) = 1 \tag{2.25}$$

where x is the instance being explained, $D_x(z|A)$ is the distribution of perturbations to x that satisfy the anchor A, f is the classification model, τ is the precision threshold, and $A(x)$ is a set of predicates, i.e., the resulting rule or anchor, such that $A(x) = 1$ when all feature predicates defined by A correspond to x's feature values.

The above definition is impossible to solve, so a probabilistic definition is used in practice.

$$P(prec(A) \geq \tau) \geq 1 - \delta, \text{ with } prec(A) = \mathbb{E}_{D_x(z|A)}[1_{f(x)=f(z)}] \tag{2.26}$$

Here, $prec(A)$ denotes the precision of the anchor A, which is the expected proportion of perturbed instances that retain the same prediction as the original instance x. The threshold τ is the minimum required precision for an anchor to be considered valid. The parameter δ represents the statistical confidence level, ensuring that the probability the precision is at least τ is no less than $1 - \delta$.

Moreover, coverage is formally defined as an anchor's probability of applying to its neighbours, i.e., its perturbation space. Coverage is given by:

$$cov(A) = \mathbb{E}_{D_{(z)}}[A(z)] \tag{2.27}$$

In the above equation, $cov(A)$ represents the coverage of the anchor A. The term $\mathbb{E}$ indicates the expected value, and $D_{(z)}$ refers to the distribution over perturbed instances z. The function $A(z)$ is an indicator function that equals 1 if the anchor A applies to the instance z, and 0 otherwise. Thus, the coverage measures the proportion of instances in the local neighbourhood of the explained instance for which the anchor conditions hold true.

Hence, to get the anchors with coverage, we optimise the following:

$$\max_{As.t.P(prec(A)\geq\tau)\geq 1-\delta} cov(A) \tag{2.28}$$

We can use Anchors to show how predictions for classification models can be explained in a rule-based manner. This can be useful for understanding the model's behaviour and building trust in the model.

2.5 Formal Modelling Techniques

Formal modelling techniques are needed to describe the properties of a CPS. These techniques help describe the behaviour of the system and its environment in a mathematically precise manner. These mathematical models can then be verified using verification software with proven correctness guarantees. In this section, we will discuss three formal models: Finite State Machines, Timed Automata, and Valued Discrete

Timed Automata. We will also briefly cover a formal verification approach called Runtime Enforcement.

2.5.1 Finite State Machines

A Finite State machine (FSM) is a mathematical model extensively used in formal modelling and verification, and is also known as a finite-state automaton. At any point in time, the model can be in one of a finite number of states. The FSM execution starts at its *initial* state and progresses to other states based on the inputs to the model - this process is called a *transition*. When the model can only transition to one other state based on a specific set of inputs, the model is said to be a deterministic FSM. On the other hand, if the model can transition to more than one state given the same set of inputs, the model is said to be a non-deterministic FSM. The discrete states in an FSM allow for the control flow of a system or property to be broken down into logical sections, making it easier to understand and verify the system's behaviour.

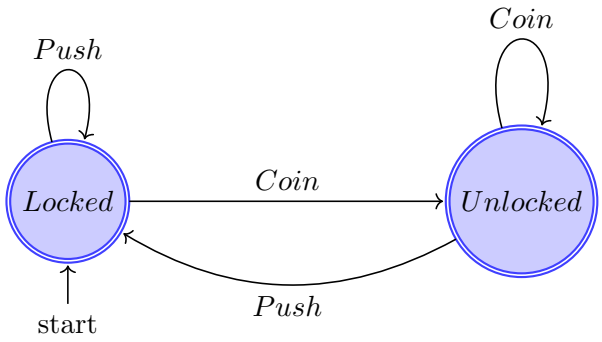


Figure 2.3: An FSM for a coin-operated turnstile.

For example, consider the case of a coin-operated turnstile. Here, the control starts at the *Locked* location. If a coin is inserted, the control moves to the *Unlocked* location. If the turnstile is pushed, the control moves back to the *Locked* location. This can be modelled as an FSM with two states: *Locked* and *Unlocked*. The FSM can be represented as in Figure 2.3. The system remains in an unlocked state as long as the coins keep getting inserted. Also, the control can only move to *Unlocked* state if a coin is inserted; pushing the turnstile without inserting a coin will not change the state of the system.

2.5.2 Timed Automata

There are times when a system or property behaviour may depend on the time taken to perform certain actions. In such cases, a TA can be used to model the system or property. A timed-automaton is constructed by adding a finite set of real-valued clocks to a FSM. Multiple real-valued clocks can be defined, and they can be initialised from

zero and reset to zero to measure the time taken for an action to occur. The clocks can also be used to define time constraints on the transitions between states.

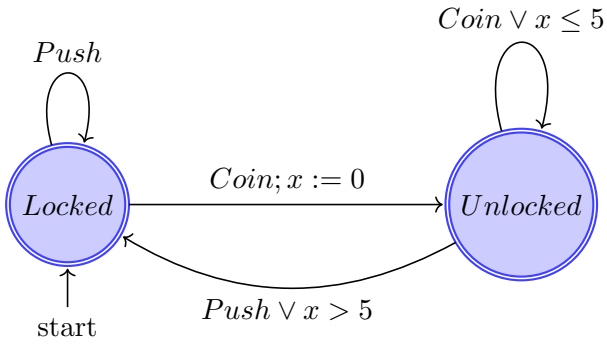


Figure 2.4: A TA for a coin-operated turnstile.

An example of a simple TA is shown in Figure 2.4, which is an extension of the FSM in Figure 2.3. Here, the system remains in the *Unlocked* state for a maximum of 5 time units after a coin is inserted. If the turnstile is pushed within this time, the system moves back to the *Locked* state. However, if the turnstile is not pushed within 5 time units, the system moves to the *Locked* state anyway. The TA can be used to model the time taken for the turnstile to be pushed after a coin is inserted.

2.5.3 Valued Discrete Timed Automata

A Valued Discrete Timed Automata (VDTA) is an automaton characterised by a finite set of locations, a finite collection of discrete clocks to track temporal evolution, and external variables—which correspond to input and output channels—that represent the system's data flows [25]. In addition to these external variables, VDTAs include internal variables to carry out computations, whereas external variables simply capture data exchanged between the monitored system and its environment.

VDTAs operate on an implicit logical tick, much like synchronous programming languages [49]. Every transition is defined with respect to this tick: at the tick's start, all inputs are sampled, and at its end, all outputs are produced. This tick thus embodies one atomic reaction of the reactive system. As the majority of this work uses synchronous semantics, VDTA will be used to model system specifications. Complete semantics of the VDTA can be found in [25].

2.5.4 Runtime Enforcement

It is not always possible to model the entire system behaviour using formal methods due to the complexity of the system. Runtime Enforcement (RE) can be used in such cases to ensure the system behaves as expected. RE monitors the system's behaviour

at runtime and ensures that the system satisfies certain policies. If the system violates the policies, RE takes corrective action to bring the system back to a safe state.

Runtime enforcement can be implemented using a monitor that observes the system's behaviour and checks if the system is adhering to the specified policies. If the system violates the policies, the monitor can take corrective actions to bring the system back to a safe state. The monitor is constructed as a formal model (such as FSMs, TA, and VDTA). The monitor can be used to check the system's behaviour against the specified policies and take corrective actions if the system violates the policies. The monitor can be implemented as a separate module that runs alongside the system and checks the system's behaviour periodically. If the monitor detects a violation, it can send a signal to the system to take corrective actions.

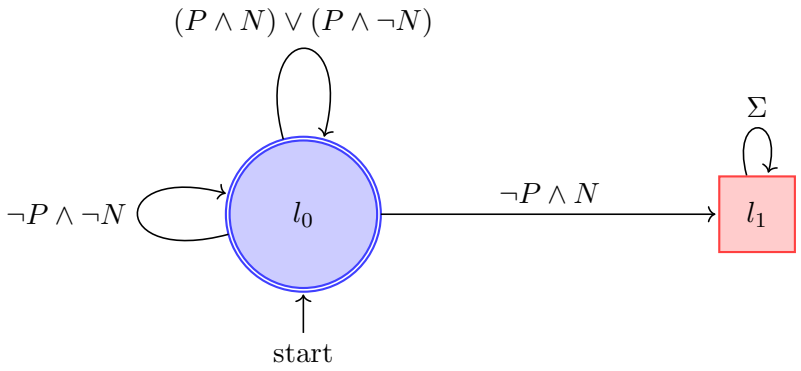


Figure 2.5: Runtime enforcement monitor (VDTA) for lane change system in an AV.

Figure 2.5 shows an example of a runtime enforcement monitor for the lane change system in an AV. The monitor checks that the car should not make a lane change when the safety property P is not satisfied and the lane change model N suggests a lane change. In all other cases, the system stays in the safe state l_0. If the monitor detects an imminent violation (i.e., transition to l_1), it updates the output of the lane change module ($N \rightarrow \neg N$) so that no lane change can occur.

Having established the technical foundations of CPSs, ML, explainability, and formal modelling, we now turn to the principles of compositionality that underpin the design of complex systems. These compositional principles, originally developed in the context of model-based design, provide the theoretical basis for the compositional ML framework developed in this thesis. We also examine how data-driven models have been used in CPSs and the challenges that arise from their monolithic design.

2.6 Cyber-Physical Systems and Compositionality

Designing CPSs by iterative trial and error—building prototypes, testing them, discovering faults, and revising—remains the most common approach [13]. Although this

method is straightforward, it is both expensive and potentially unsafe: flaws detected late in development are costly to remedy, and defects that only surface after deployment can jeopardise human safety. In many industries, post-release fixes are now routine—from over-the-air software patches in consumer electronics to vehicle recalls in the automotive sector and device corrections in medical equipment.

2.6.1 Model-based Design of CPS

To overcome the drawbacks of the trial-and-error approach, Model-Based Design (MBD) emerged as a more disciplined alternative [50]. In this approach, engineers develop abstract models, ranging from simple spreadsheets to fully executable simulations, that emulate key aspects of the target system. These models offer several benefits. They eliminate the risks associated with experimenting on real hardware, reduce costs by requiring fewer resources than building full prototypes, and enable rapid, scalable testing through simulations that can be run in parallel for extensive coverage within tight schedules. Furthermore, formal verification techniques applied to models can exhaustively explore system behaviours, uncovering corner-case errors before implementation. The process of correcting a model is typically simpler and less expensive than modifying physical prototypes.

However, models are abstractions and inevitably introduce simplifications. Ensuring that these approximations do not invalidate analysis results demands careful management of model fidelity—a central scientific challenge in system design.

At its core, MBD addresses three fundamental questions [13]:

1. **Modelling :** How can we precisely and efficiently capture the essential characteristics of the system under design? Creating accurate models is inherently difficult, especially across diverse engineering domains—mechanical, chemical, biological, and beyond. Each field has developed its own specialised modelling languages, modelling algorithms, and tools, but within the context of MBD.

2. **Analysis:** Does the model satisfy the intended requirements—functionally, temporally, and economically? After constructing a model, the next step is to verify that it meets its specifications, covering correctness, performance, reliability, and cost metrics. As systems become complex, closed-form analytic methods often become infeasible, and *computer-aided techniques* such as model checking rise to prominence.

   In model checking, one frames a system model S and a formal policy φ (expressed, for example, in temporal logic) and then automatically determines whether the system satisfies the policy $S \models \varphi$. If not, the tool produces a counterexample illustrating the violation. Temporal logics distinguish between Safety (e.g.,

"always p" means that p never fails) and Liveness policies (e.g., "eventually p" asserts that p will occur at some point).

The principal obstacle in formal verification is the *state-explosion problem*: even modest systems can yield astronomically large state spaces, exacerbated further by continuous or hybrid dynamics. Although undecidability is a problem for rich models, ongoing research continuously extends the reach of practical verification tools through abstraction, compositional techniques, and heuristic optimisations.

3. **Implementation:** By what processes can we transform the verified model into a deployable system, while preserving its certified properties? Ultimately, a verified model must be translated into an executable system. *Automated code generation* frameworks already exist in industry, but they must meet a critical requirement: *semantic preservation*. That is, the generated code should faithfully implement the behaviours that the model was proven to exhibit. Perfect preservation—down to real-number arithmetic—is typically impossible on finite-precision hardware. Nonetheless, one can often identify and guarantee the *essential properties* needed for correct operation, while relaxing or approximating secondary aspects.

2.6.2 Compositionality in Model-Based Design

Contemporary definitions of a "system" that merge state and dynamics into one monolithic entity become unwieldy for large-scale systems, such as modern automobiles, aircraft, or complex biological organisms. Capturing every possible state and transition in a single model (i.e., a *monolithic model*) is impractical. Instead, we decompose a complex system into smaller subsystems, each of which may itself be further subdivided [51].

There are multiple paradigms for how subsystems communicate and synchronise:

1. **Synchronous composition**, where all subsystems advance in lock-step, and every transition occurs simultaneously across components.

2. **Asynchronous (interleaving) composition**, where at most one component transitions at any given moment, leaving the others unchanged.

Beyond timing considerations, subsystems may be connected in series, in parallel, or via feedback loops, as shown in Figure 2.6[3]. Depending on the modelling framework, these "connections" can represent zero-delay wires, message queues (e.g., FIFO buffers in dataflow models), or shared-memory constructs typical of multithreaded software [13].

[3]We consider systems with a static structure. Dynamic systems are useful in several fields, including biology and economics, but are not considered in this work.

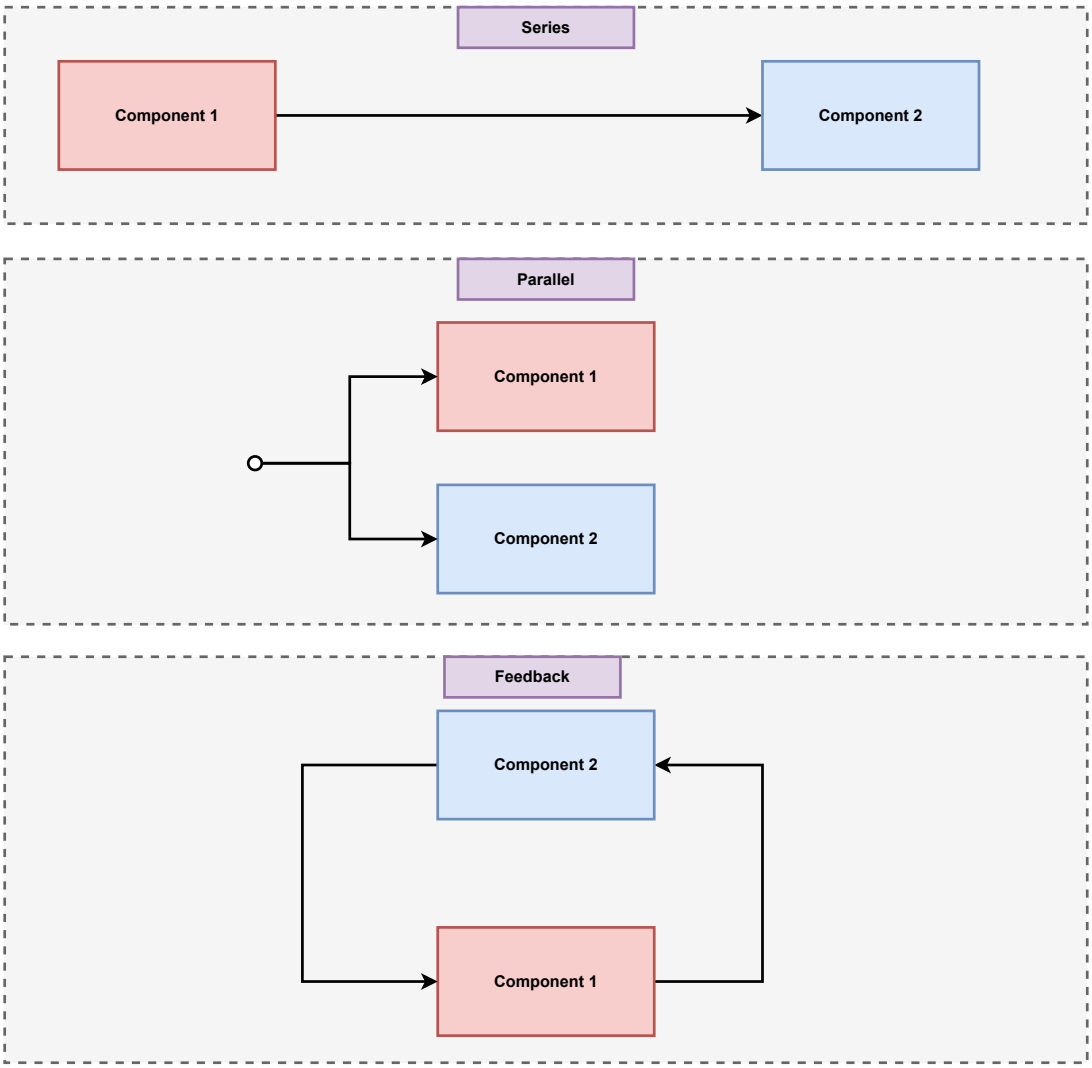


Figure 2.6: Composition of subsystems.

Generally, we see compositionality as serving two main purposes. First, it helps us tame complexity by building sophisticated systems out of simpler components. Second, it addresses heterogeneity by enabling the integration of systems of different kinds [52]. Compositionality in system design thus divides into two main themes:

1. **Scalability of analysis:** Can we infer global system properties by examining individual components? Techniques such as assume-guarantee reasoning and refinement allow us to conclude that the composed system inherits the desired properties if each component satisfies its contract (or refines a specification).

2. **Modularity and reuse:** Can we replace one subsystem with an equivalent one without re-verifying the entire assembly? Well-defined interfaces abstract away internal details, exposing only what is necessary for correct interaction, thereby enabling safe replacement, extension, and reuse of components.

Together, these compositional principles underpin a systematic approach to designing, verifying, and evolving complex engineered systems. The following section summarises how data-driven ML models are used in CPSs and the main barriers to their verification and deployment; Chapter 3 then reviews the related literature in detail.

2.7 Data-driven Models in CPS

2.7.1 ML-based Models in CPS

As the complexity of engineered systems continues to grow, traditional model-driven design approaches that rely on detailed physical process modelling are becoming increasingly impractical and difficult to scale. This is particularly true for CPSs, where the intricate interplay between computational and physical components leads to highly complex and often poorly understood dynamics. In such contexts, constructing accurate, first-principles models of the underlying processes can be prohibitively time-consuming, require significant domain expertise, and may fail to capture all relevant behaviours or uncertainties in real-world environments.

In response to the challenges, data-driven techniques for system design have emerged as a compelling alternative [53–55]. Among these, ML models like ANNs have gained significant prominence due to their remarkable ability to approximate complex, nonlinear input-output relationships directly from data, without the need for explicit knowledge of the system's internal dynamics [1]. ML models are particularly well-suited for tasks where the mapping from inputs to outputs is difficult to express analytically, or the system is subject to variability and noise that are hard to model using traditional methods.

For example, in autonomous vehicles, an ML-based classifier can be trained to recognise potential lane-changing opportunities by learning from large datasets of driving scenarios. Rather than relying on a hand-crafted set of rules or a detailed physical model of vehicle-environment interactions, the ANN can directly infer the relevant patterns and decision boundaries from sensor data, such as camera images, lidar point clouds, or radar signals. This data-driven approach enables the system to adapt to a wide range of conditions and edge cases that might be difficult to anticipate or encode explicitly, thereby enhancing the flexibility and robustness of CPS design.

Among all models, ANN models have found the most applications in CPSs due to their ability to approximate complex nonlinear relationships and their ability to be trained on large datasets. The body of work using ANNs far exceeds the body of work using other models, and we will primarily focus on ANN-based literature. We will now

(a) Example image of a dog correctly classified as a dog.

(b) Example noisy image of a dog that could be incorrectly classified as a cat.

Figure 2.7: An example of a dog correctly classified as a dog and incorrectly classified as a cat by an ANN model. This image is AI-generated using the DALL·E 3 model via ChatGPT on July 30, 2025. Adding noise to the image can cause the ANN to misclassify the image as a cat or some other animal from the dataset. This is a representative example of the errors that can occur in ML-based systems.

look at the errors, explainability, and verification barriers of ML-based (particularly ANN-based) CPSs.

2.7.2 Errors, Explainability, and Verification Barriers

ML-based systems are often prone to a variety of errors [56]. These errors can arise from several sources, including overfitting to training data, poor generalisation to unseen scenarios, and sensitivity to small input perturbations (adversarial examples) [57]. Additionally, data-driven models may suffer from distributional shifts, where the operational environment differs from the training data, leading to unexpected or unsafe behaviours [58].

Consider the figure in Figure 2.7 [59], where adding noise to the image causes the ANN to misclassify the image. If an AV is using a similar ANN to make a decision, it could potentially cause a crash. Errors can also result from incomplete or biased training datasets [60], mislabelled data, or insufficient model capacity to capture the

true system dynamics. Furthermore, implementation issues such as quantisation errors, numerical instability [61], and hardware-induced faults can further degrade the reliability of ANN-based systems, especially in safety- and time-critical applications.

The deployment of ML models like ANNs in real-time, safety-critical, and time-critical application domains imposes stringent requirements for formal guarantees regarding both their functional correctness [62] and their timing behaviour [63]. In these domains, it is not sufficient for an ML model to perform well on average; it must be possible to provide rigorous assurances that the network will always behave as intended, within specified time constraints, under all possible operating conditions. Various formal verification techniques have been developed and proposed to address these needs for ANNs [64]. However, the formal verification of ANNs is known to be an NP-Complete problem [3], making it computationally intractable for medium to large-scale networks.

A significant barrier to the widespread adoption of formal verification methods for ANNs is the prevalent design paradigm in which these networks are constructed and implemented as large, monolithic black-box [2] models. In such monolithic architectures, the entire functionality is encapsulated within a single, often very large, network, with little or no internal modularity or structure that can be exploited for analysis. These monolithic ANN designs are typically characterised by their substantial size, high computational and memory requirements, and slow execution times [2]. They are also resource-intensive, demanding significant hardware resources for training and inference, and are rarely amenable to parallelisation or incremental development and verification [13].

As a result, the formal verification of monolithic ANNs often necessitates the use of substantial approximations [65] or the development of complex abstraction techniques [8] to make the problem tractable. These approximations and abstractions, while sometimes effective, can introduce conservatism or incompleteness, potentially undermining the strength of the guarantees that can be provided.

Furthermore, the sheer size and complexity of monolithic ANN designs impose practical limitations on their implementation, particularly when targeting affordable custom hardware platforms such as Field Programmable Gate Arrays (FPGAs) for timing analysis - due to their deterministic timing behaviour. In practice, this means that only relatively small networks—often comprising just a few tens of neurons—can be feasibly implemented on such hardware, as demonstrated in studies such as [4] and [5]. This hardware constraint further restricts the applicability of monolithic ANN designs in resource-constrained, safety- and time-critical systems, highlighting the need for alternative, more modular approaches.

2.8 Conclusions

This chapter introduced the background needed for the remainder of the thesis. We defined CPSs as safety-critical, real-time systems that require timely and correct interaction between physical plants and digital controllers, and explained why WCET analysis and synchronous execution are relevant when timing guarantees matter. We then summarised core ML concepts, with emphasis on feedforward ANNs, their training and evaluation, and post-hoc explainability techniques (SHAP, ALE, and Anchors). Formal modelling tools—including FSMs, timed automata, and runtime enforcement—were presented as mechanisms for specifying and monitoring system behaviour.

On the design side, we contrasted model-based compositionality in CPS with the growing use of data-driven ANNs in control applications. We discussed typical ML failure modes, the difficulty of verifying monolithic networks, and practical hardware limits on implementing them for timing analysis on resource-constrained platforms.

Chapter 3

Literature Review

3.1 Overview

Building on the background presented in Chapter 2, this chapter reviews literature directly related to the technical contributions of this thesis. We begin with compositional approaches to Machine Learning (ML)-based Cyber-Physical Systems (CPSs), distinguishing model-level decomposition from system-level pipeline design and identifying the research gap that motivates our work. We then survey three strands of related work—Field Programmable Gate Array (FPGA) implementation and timing analysis, compositional safety and verification, and compositional explainability—before reviewing the literature on our two case-study domains (autonomous vehicles and personalised mood score prediction). Throughout, we highlight gaps that the subsequent technical chapters address.

3.2 Compositional Modelling in ML-based CPS

A compositional approach, as elaborated in Section 2.6.2 of Chapter 2, promotes the use of multiple smaller, functionally equivalent subsystems instead of a single, large, monolithic system. This methodology is particularly advantageous for the design of ML-based CPSs, where scalability, modularity, and verifiability are critical concerns.

Recent research in data-driven CPS design has explored various strategies to incorporate compositionality. These efforts span a spectrum of objectives and methodologies. For instance, several studies, including [66–69], leverage compositional architectures specifically to facilitate the formal verification of CPS. By structuring systems as compositions of smaller, verifiable modules, these works aim to make the verification process more tractable and scalable, enabling the analysis of properties that would be infeasible to check in a monolithic setting.

While the works mentioned above use the idea of compositionality, they do so on a system level. They do not tackle the monolithic ML model in a compositional or modular manner, accounting for most of the verification overhead due to its nonlinearity and computational complexity. Although [14] and [15] present approaches to decomposing trained monolithic ANNs to multiple small independent networks, their focus is not on easing verification. Hence, the overall network size remains unchanged or may even increase, which can prove detrimental despite the smaller size of the decomposed models. On the other hand, studies, such as [16] and [17], which use model reduction techniques to simplify ANNs instead of compositionality, do not always produce a simple ANN, and the method used therein is not always intuitive.

It is important to distinguish the model-level decomposition discussed above from system-level pipeline composition, which is already common in industrial practice. The autonomous driving domain illustrates this distinction clearly. Waymo's architecture employs a modular, multi-model pipeline in which separate Artificial Neural Network (ANN) models handle distinct tasks—perception from LiDAR, camera, and radar inputs, behaviour prediction of other road users, and motion planning—each independently trained and connected in a processing pipeline [70, 71]. This represents *system-level composition*: the overall system is decomposed into functional modules, but each module remains a monolithic ANN that is trained and deployed as a single black box. In contrast, Tesla's Full Self-Driving (FSD) system has historically pursued a more end-to-end approach, using large monolithic neural networks to map raw camera inputs more directly to driving outputs [72, 73]. More recent iterations of both platforms have converged towards hybrid architectures, but the fundamental dichotomy between modular pipelines and monolithic end-to-end models persists across the industry [72].

Crucially, the compositionality proposed in this thesis is fundamentally different from this system-level pipeline design. Rather than arranging multiple independently trained models in a processing pipeline, our approach concerns the decomposition of a *single* ANN model—for example, a lane-change classifier—into multiple smaller sub-models (e.g., one binary classifier per class) that collectively replace the original monolithic model. This is *model-level decomposition*. System-level composition, as practised by Waymo and others, does not address the verification, Worst-Case Execution Time (WCET), or explainability challenges inherent in each individual monolithic model within the pipeline: each model in a Waymo-style pipeline is still a large, monolithic black box that is difficult to formally verify, implement on resource-constrained hardware, and explain. Model-level decomposition, as developed in this thesis, directly tackles these challenges by reducing the size and complexity of each individual model.

Overall, there is a lack of research that studies how to systematically, quantitatively, and objectively decompose large monolithic ANN models into smaller networks

at the model level. Moreover, a detailed quantitative comparison between monolithic and compositional designs based on their performance is long overdue. The remainder of this chapter reviews existing work in the three areas—timing analysis, safety, and explainability—where model-level compositionality offers the greatest benefit, and situates our case studies within that context.

3.3 Implementing Compositional ANN Models to FPGA Hardware for Timing Analysis

Table 3.1: Comparison between related works and our hardware compilation method (Mon. for Monolithic design, and Com. for Compositional design. Composed in the Design column stands for designs that combine (not decompose) multiple smaller models.)

Research	Precision	Activation Approx.	Activation Function	Neurons/ Layers	General Method	Design
Himavathi et al.[19]	Fixed	LUT	Sigmoid, Linear	23/4	No	Mon.
Marouf et al. [74]	Floating	LUT	Linear, Sigmoid	7/2	No	Mon.
Nguyen et al. [5]	Fixed	LUT	Relu, Sigmoid	58/2	No	Mon.
Allen et al. [75]	Fixed	LUT	Relu, Sigmoid, Tanh, Linear	35/6	Yes	Composed
Sarić et al. [76]	Fixed	LUT	Sigmoid	15/2	No	Mon.
Zairi et al. [77]	Fixed	LUT	Tanh, Sigmoid	7/2	No	Mon.

Timing analysis for worst-case execution time (WCET) is a critical component in the design of time-critical cyber-physical systems (CPS). By verifying that every computation finishes within its allotted budget, we can guarantee the system meets stringent real-time requirements. Notably, implementing neural networks on dedicated FPGA hardware simplifies this analysis: the absence of caches, pipelines, branch predictors, and out-of-order execution yields fully predictable, cycle-accurate behaviour [18].

To understand how these benefits have been exploited, we survey existing works that deploy artificial neural network (ANN) models on FPGAs. Table 3.1 summarises each approach in terms of numerical precision, activation-function approxima-

tion method, supported activation functions, tested network size (neurons and layers), application specificity, and support for compositional designs.

From this comparison, several trends and limitations emerge. First, most hardware compilation techniques are tailored to particular applications and validated only on small-scale networks. The key obstacle appears to be the heavy resource demands of large ANNs on affordable FPGA platforms, which discourages general-purpose, scalable solutions. Second, nearly all approaches approximate activation functions using lookup tables (LUTs), and typically cover only a few activation functions, e.g. Sigmoid, Tanh, or Rectified Linear Unit (ReLU), while more complex functions like Softmax (which requires exponentials) remain unsupported.

Finally, despite the clear advantages of synchronous hardware semantics for deterministic timing, few works adopt a truly synchronous execution model (see Section 2.2.3 in Chapter 2). [78] and [2] do employ synchronous hardware for multiple smaller ANNs, but neither addresses compositional implementation of a single, larger ANN, nor extends to a broad class of activation functions. In fact, no existing approach both decomposes monolithic ANN models into compositional architectures and implements them on FPGAs with formal timing analysis in mind.

Taken together, the evidence from existing hardware implementations reinforces three interconnected requirements that motivate this thesis. First, FPGAs are not merely an alternative to embedded Graphics Processing Units (GPUs); they are a viable platform for obtaining provable WCET bounds on ANN inference. As Wilhelm et al. [18] establish, static WCET analysis depends on fully deterministic hardware behaviour—a property that GPUs, with their caches, dynamic thread scheduling, and shared-memory contention, fundamentally cannot provide. For safety-critical CPSs that must comply with certification standards, measurement-based timing estimates are insufficient; only the cycle-accurate, cache-free execution model of FPGAs enables the formal timing guarantees required.

Second, the compilation of ANN models to synthesisable VHDL is a necessary step in this verification chain. Python-based training frameworks such as TensorFlow [39] and PyTorch [40] produce software-level model descriptions that cannot be directly executed on FPGA fabric in a timing-analysable manner. A systematic compilation to VHDL bridges this gap: it translates the trained model's weights, biases, and activation functions into a synchronous hardware description whose execution time is bounded by a fixed number of clock cycles. Without such a compiler, practitioners are forced to hand-craft hardware designs for each network topology—a process that, as Table 3.1 shows, has limited existing implementations to at most 58 neurons and a narrow set of activation functions. An automated VHDL compilation pathway is therefore essential for scaling FPGA-based ANN deployment beyond small, application-specific prototypes.

Third, compositional decomposition directly addresses the resource constraints that have restricted existing FPGA implementations to small networks. Monolithic ANNs demand proportionally large amounts of FPGA logic resources, on-chip memory, and routing fabric—requirements that quickly exceed the capacity of affordable FPGA devices [76, 5]. By decomposing a single large model into multiple smaller sub-models, compositionality reduces the peak resource demand of any individual component, enables the sub-models to be mapped to separate hardware resources for true parallel execution, and makes WCET analysis tractable for each sub-model independently. The synchronous frameworks of Allen et al. [78] and Yang et al. [2] demonstrate the feasibility of running multiple smaller ANNs on a single FPGA, but neither addresses the prior step of *deriving* those sub-models from a monolithic original—the gap that this thesis fills.

3.4 Safe-by-Construction Compositional CPS

Formal verification of CPSs is a challenging task, especially for large-scale CPSs. This complexity is exacerbated by the use of ML-based components, such as ANNs, in the system, which have large state spaces and are nonlinear. Table 3.2 shows some of the current works on verifying ANN-based CPSs.

The literature on verification of neural-network-enabled systems can be grouped into three broad areas. First, a core set of works directly embraces compositional falsification or assume-guarantee reasoning by decomposing a complex controller into manageable subunits [6, 7]. [80] extend this theme by using inductive invariants. [79] further uses compositionality to get error-bound guarantees under stochastic conditions. A second area centres on reachability, hybrid-system methods [81, 86] and surrogate verification [82, 83]. Techniques such as abstraction-refinement loops [84] and differential verification via symbolic-interval analysis [85] provide complementary means to bound and compare network behaviours. Finally, informal orthogonal techniques, such as adversarial training [87], have also been used to make ML models safer by increasing their robustness against certain unexpected inputs and scenarios.

However, these compositional and reachability-based verification methods exhibit NP-complete complexity, resulting in state-space explosion and scalability barriers even with heuristics or advanced branching strategies [7, 82, 86]. Decidability is often confined to shallow architectures—e.g., single hidden-layer networks—while over-approximation techniques trade termination guarantees for coarse abstractions and incomplete proofs [81, 83]. Moreover, most approaches are tailored to specific network classes or narrow input domains, such as feed-forward ReLU models or discretised scenarios, which limits their generality and precision [85, 79]. Achieving tighter error bounds invariably increases computation and memory costs, leading to timeouts or

Table 3.2: Studies that perform verification of ANN-based CPSs

Study	**Focus**	**Verification Paradigm**
Ivanov et al. (2021) [6]	Compositional verification of ANN controllers	Reachability analysis
Watson et al. (2025) [79]	Scenario-based verification with neural perception	Probabilistic, scenario-based
Zhou et al. (2023) [80]	Invariant-based verification of ANN-controlled systems	Inductive invariants
Ivanov et al. (2018) [81]	Safety verification of hybrid systems with ANN	Reachability analysis
Păsăreanu et al. (2018) [7]	Compositional verification of ANN components	Assume-guarantee reasoning
Cai et al. (2024) [82]	Surrogate verification for image-based controllers	Reachability analysis
Wang et al. (2022) [83]	Temporal logic objectives for ANN controllers	Automata-based
Elboher et al. (2019) [84]	Abstraction framework for ANN verification	Abstraction-based
Paulsen et al. (2020) [85]	Differential verification of ANNs	Symbolic intervals
Kouvaros et al. (2021) [86]	Scalable ReLU network verification	Mixed-Integer Programming (MIP) reduction

looser probabilistic guarantees when refinements or scenario counts are constrained [80, 79]. Finally, adversarial training does not provide any formal guarantees on model behaviour.

Consequently, there are other bodies of work for verification, which are dynamic and lightweight. For example, Runtime Verification and Runtime Enforcement. We do not need to model the system here. We just take the executions of the (black-box ANN/ML) systems at runtime and monitor them to ensure their compliance with a set of formal requirements. Additionally, in the event of a violation, using an enforcement monitor/enforcer corrects the output. This means an (untrustworthy) input execution (in the form of a sequence of events that does not comply with the policy) is modified into an output sequence that complies with a policy (e.g., a safety requirement).

There are several mechanisms by which a correction is made: framework [22] considers blocking the execution in case of a violation, [23] considers modifying input sequence by suppressing and/or inserting actions, [88] considers buffering input actions until a future time when it could be forwarded, [24, 25] considers instantly correcting the executions suitable for reactive systems, safety-critical systems and cyber-physical

systems. Nevertheless, Runtime Verification and Runtime Enforcement have not been used on compositional ML models.

The need for compositional ML design in the safety domain is further underscored by a fundamental scalability argument. As Katz et al. [3] have shown, verifying even basic properties of ReLU networks is NP-complete, and the verification cost grows exponentially with the number of neurons and layers. Compositional decomposition directly mitigates this: by replacing a single large model with multiple smaller sub-models, the state space that any individual verification tool must explore is reduced, making component-level verification feasible where monolithic verification would time out [7, 86].

Furthermore, compositional design can create natural boundaries for runtime enforcement. Rather than enforcing a single global safety property over the aggregate output of a monolithic model, each sub-model can be paired with a dedicated enforcer [24, 25] that monitors compliance with a local sub-policy. This per-component enforcement reduces the state space of each individual monitor, increases the precision of corrective actions, and ensures that a violation in one component can be isolated and corrected without disrupting the outputs of other components.

The compositional structure should also enable policy mining at the sub-model level: safety properties can be extracted from the training data of each component independently, yielding tighter and more meaningful constraints than those obtainable from a monolithic model whose features span multiple, unrelated domains.

All in all, while there have been attempts to develop safer ML-based systems using compositionality and other approaches, there have been little to no attempts to develop a formal framework that combines formalism with compositional ML model design to make ML-based systems safe by construction, i.e., formal safety is designed into every stage of ML-based system design.

3.5 Compositionality for Designing Explainable Systems

Often, in some systems, especially ones used in the healthcare domain, predicting with high accuracy is insufficient. Most ML approaches are black-box approaches, i.e., they do not show how they reached a prediction [9]. Without explaining why a model predicts, professionals using such models cannot determine what insights the prediction contains [89]. These insights can then be used to check a model's fidelity (whether the model predictions make sense) [46] and suggest solutions based on these explanations. This ensures the transparency and accountability of the models.

Recent advances in Explainable Artificial Intelligence (XAI) offer solutions to the problem of trustworthiness in ML models. Explainable models, such as Decision Trees [9], can be easily processed/simplified to explain their outputs [90]. However, their expressive power is limited by their size, and increasing their expressiveness decreases their interpretability. ANN models can make more complex associations from multimodal data and yield better-performing models [1, 89], but are not inherently explainable [9].

With the availability of post-hoc explainable methods (see Section 2.4 in Chapter 2 for a deeper dive into these methods), such as Shapley Additive Explanations (SHAP) [10] and Local Interpretable Model-agnostic Explanations (LIME) [11], explaining performant black-box DL models has become easier [9]. Studies such as [91–93] use explainability techniques on ML models to obtain insights into the model outputs. Moreover, with the growth of works using ML-based models in healthcare, recent works have begun exploring explainability in the healthcare setting [94–98].

However, the use of explainability has been limited to the extraction of the most influential model features/inputs using SHAP or LIME [26]. Furthermore, the development of ML models and their use of explainability has been limited to the monolithic ML/DL model— where a single model ingests a vast array of disparate features—making it challenging to decompose model behaviour into intuitive, clinically meaningful explanations due to the complexity and interdependence of inputs. Also, when the model underperforms, it is often challenging to isolate which cluster (or set of features) is causing the issue since they are all entangled in one model.

Compositional ML design can offer a principled solution to these explainability limitations. When a monolithic model is decomposed into sub-models, each handling a semantically coherent group of features, explainability methods such as SHAP [10] and LIME [11] can be applied *within* each sub-model, yielding feature attributions that are confined to a single domain and therefore directly interpretable by domain experts. For instance, in a healthcare application, a sub-model responsible for sleep-related features produces SHAP values that a clinician can immediately relate to sleep hygiene recommendations, without the confounding influence of activity or physiological features that would be present in a monolithic model [89, 46].

Furthermore, compositional architectures naturally support multi-level explanations: at the global level, the relative contributions of entire sub-models reveal which domains are most influential for a given prediction; at the local level, within-component feature attributions provide fine-grained detail. This hierarchical structure aligns with the practical needs of different stakeholders—clinicians require domain-level summaries, while data scientists need feature-level diagnostics—and addresses the monolithic model's inability to provide explanations at varying granularities [9, 90].

The compositional approach also simplifies model debugging: when a model underperforms, the modular structure makes it straightforward to identify which sub-model (and therefore which feature domain) is responsible, enabling targeted retraining without disrupting the other components. Despite the possible advantages, a compositional approach to designing and explaining ML models has not been explored in recent studies.

Having reviewed the general literature on model-level compositional design and its implications for timing, safety, and explainability, we now turn to the application domains in which this thesis evaluates the proposed framework.

3.6 Review of Case Study Literature

We use two main case studies in this work - the first is the case study of an Autonomous Vehicle (AV), and the second is the case study of a personalised mood score prediction model for depression. We discuss the existing literature on the case studies, what current research gaps exist, and what compositional design approaches can address.

3.6.1 Developing Models for Autonomous Vehicles

An AV is a vehicle that is capable of sensing its environment and operating with minimal or no human involvement. Autonomous vehicles are also known as self-driving cars, robotic cars, or driverless cars. The technology is a combination of sensors, cameras, radar, and artificial intelligence (AI) to navigate and drive the vehicle.

One of the key challenges in developing autonomous vehicles is designing models that can accurately interpret the data collected by the sensors and make decisions in real-time. These models must detect and classify objects in the environment, predict the movements of other vehicles and pedestrians, and plan a safe and efficient route to the destination while handling unexpected situations, such as road closures, accidents, or adverse weather conditions. Hence, they represent a safety-critical CPS.

To simplify the development of autonomous vehicles, studies often divide the tasks performed by the vehicle into smaller sub-tasks [99]. For example, car-following and lane-changing are two common subtasks that are essential for autonomous driving. In this section, we will discuss each of these subtasks in more detail.

3.6.1.1 Car Following

Car Following (CF) is the task of maintaining a safe distance between the AV and the vehicle in front of it. This is an essential task for autonomous vehicles, as following too closely can lead to accidents, while following too far behind can slow traffic flow. CF algorithms use data from the vehicle's sensors to estimate the speed and position of

the vehicle in front of it, and then adjust the vehicle's speed and position accordingly. These algorithms need to be able to handle variations in speed, road conditions, and traffic density, and respond quickly to changes in the environment.

While various car-following models have been proposed in the literature, the Intelligent Driver Model (IDM) [100, 101] is one of the most widely used models. The IDM is a microscopic car-following model that describes the acceleration of a vehicle based on the distance to the vehicle in front of it, the relative speed, and the desired speed of the driver.

The IDM model is given by the following equation:

$$\frac{dv}{dt} = a\left(1 - \left(\frac{v}{v_0}\right)^{\delta} - \left(\frac{s^*}{s}\right)^2\right) \tag{3.1}$$

where v is the speed of the vehicle, a is the maximum acceleration, v_0 is the desired speed, δ is the acceleration exponent, s is the distance to the vehicle in front, and s^* is the desired minimum distance. The IDM model is used in this work for its simplicity and its ability to include driver preferences in modelling car-following behaviour. More recently, ANN-based CF models have also been proposed in the literature, which use ANNs to learn the relationship between the vehicle's speed and position and the speed and position of the vehicle in front of it [102].

3.6.1.2 Lane Changing

Lane changing is the task of moving the AV from one lane to another. Lane Change (LC) is necessary when the vehicle needs to overtake another vehicle, merge onto a highway, or exit a highway. Lane changing algorithms use data from the vehicle's sensors to detect other vehicles in adjacent lanes, estimate their speed and position, and determine when it is safe to change lanes. A key challenge in lane changing is predicting the movements of other vehicles and anticipating how they will respond to the AV's actions.

Discretionary Lane Change (DLC) is a subtype of lane changing where the driver has the choice to change lanes [103]. In this case, the AV must decide whether it is safe and appropriate to change lanes based on the traffic conditions and the driver's preferences. Usually, a DLC occurs when the preceding car is moving slower than the driver's desired speed. Mandatory lane changing, on the other hand, is when the driver is required to change lanes, such as when a lane is closed or when the vehicle needs to exit the highway. In this case, the AV must change lanes quickly and safely to comply with the traffic rules or destination requirements (see Figure 3.1).

Modern lane changing models are based on Neural Networks, and there are two main variations in the way ANN-based LC models are developed. The models pre-

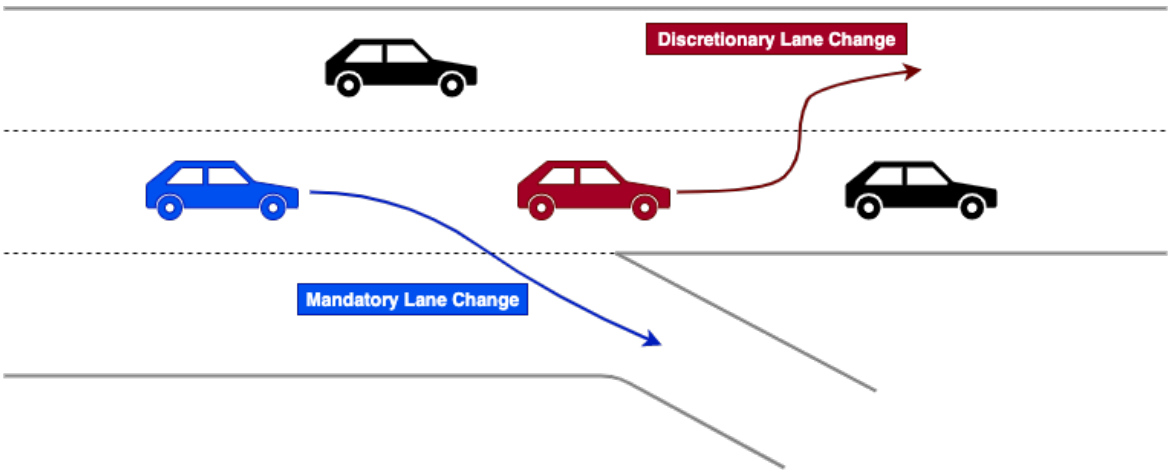


Figure 3.1: Cars performing a discretionary LC (left) and a mandatory LC (right).

sented in [104] and [102] learn the entire LC procedure as a single process, where the models decide when a vehicle can execute a LC by changing the position of the vehicle accordingly. In contrast to this approach, studies such as [105] and [106] model LC as a two-step process: (1) learning to decide when it's safe to change lane; and (2) learning to execute the decision, and build models for both steps or either. We use this approach in our work as it allows for the segregation of the vital decision-making module from the succeeding execution module, enabling greater and singular focus on the decision-making module.

The autonomous driving domain exemplifies why all three pillars of this thesis—FPGA-based timing analysis, compositional safety, and compositional explainability—are needed in concert. The driving subtasks reviewed above are inherently safety-critical and time-critical: a car-following or lane-changing decision must be computed within a strict real-time deadline dictated by vehicle speed and traffic density [99]. Monolithic ANN models for these subtasks cannot be deployed on FPGAs for formal WCET analysis because their size exceeds the resource budgets of affordable devices [76, 5]. Compositional decomposition addresses this directly: by splitting the monolithic model into smaller sub-models—for instance, one binary classifier per lane-change decision class—each sub-model is small enough for FPGA implementation and synchronous WCET analysis [78, 2]. Simultaneously, the decomposition enables per-component safety enforcement [24, 25] and component-level explainability [10], allowing each driving subtask to be independently verified, monitored, and explained.

3.6.2 Personalised Mood Score Prediction for Depression

Depression is a disorder involving loss of pleasure or interest in activities for long periods and is associated with sustained mood deterioration [107]. It can affect several aspects of life, including relationships and work. According to the World Health

Organisation (WHO) 2023 estimates, 5% of adults (approximately 300 million) worldwide experience depression, with women 50% more likely to experience depression than men. It is a significant contributor to the 700,000 suicides every year around the world [108].

Despite this, more than 75% of people in low- and middle-income countries receive no treatment due to a lack of investment in mental health, a lack of healthcare professionals and social stigma associated with mental health disorders [108]. For those who receive treatment, antidepressant medications are often the first line of treatment. However, they have low effectiveness as only one-third of all patients show symptom remission, as evidenced in large clinical trials [109, 110].

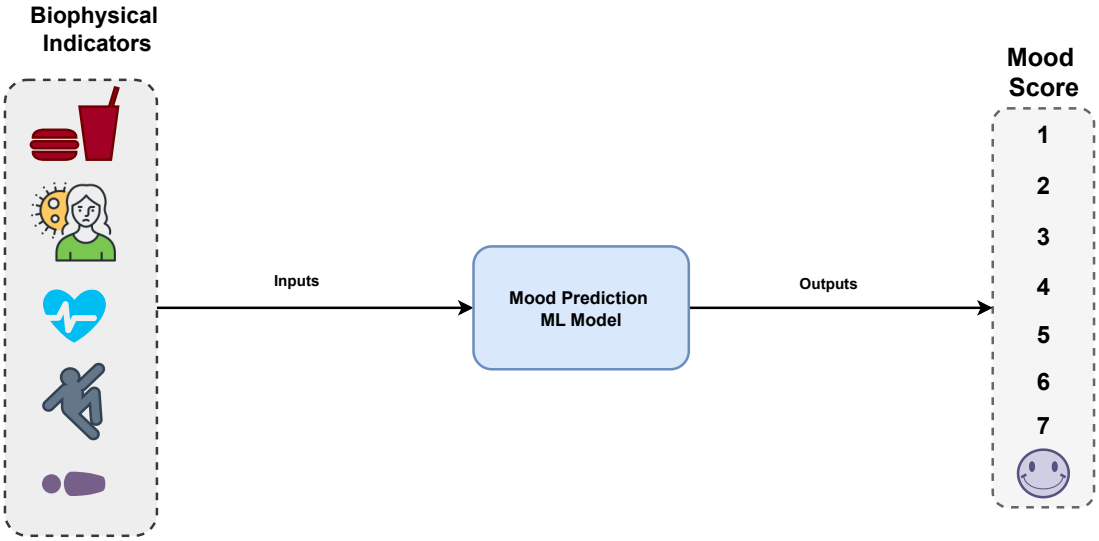


Figure 3.2: The general ML methodology for mood score prediction.

Therefore, interest has grown towards approaches that supplement clinical interventions. Studies have shown that lifestyle interventions, such as better sleep hygiene [111], practising mindfulness [112], physical activity interventions [113], and dietary interventions [114–116], have promise in managing depression [117]. Given the prevalence of devices with sensors that can be used to monitor biophysical data, such as lifestyle activities using smartphones and smartwatches, researchers are proposing using such devices for detecting, monitoring and managing depression [118]. The use of wearable technology to supplement clinical approaches is particularly appealing as it is unobtrusive, real-time, often passive (requiring little or no active input by a depressed individual/patient), of finer granularity (more data in the same time period) and allows assessments to occur in the person's usual environment [119].

As changes in mood and consistently low mood are often associated with depression, studies have tried to use mood as an indicator to monitor and predict the progression of depression. Previous studies have used data from a variety of sensors on these

devices, such as GPS location [120–122], phone and app usage patterns [123–125], voice and ambient noise [126], and motion sensor information [127], to either detect or predict future changes in mood. These studies have focused primarily on using Machine Learning (ML) and its subtype, Deep Learning (DL), models to develop predictive models owing to their excellent ability to learn associations in complex data (see Figure 3.2 for a general methodology). Moreover, other studies categorise the sensor data into activity data, sleep data, heart data or phone usage data and then build ML- and DL-based predictive models using them [128, 127, 129–134].

However, most previous studies using ML- and DL-based predictive models have focused on cross-sectional research, despite the failure of cross-sectional studies to apply to larger, more representative samples [135]. Moreover, cross-sectional works fail to account for the substantial inter-individual variability in clinical response to the same treatment or behavioural recommendations for depression due to genetic, environmental, behavioural, lifestyle, and interpersonal risk factors [136, 137]. Personalised models built on longitudinal data are more suited to account for such variability. Hence, recent works have begun focusing on personalised, predictive models for depression [128, 122, 138, 139, 98].

Nevertheless, these studies have used monolithic ML models, which are not only large but also not properly explainable due to their large number of features and their interdependence. Using post-hoc explainability methods only extracts the most influential features/inputs of the model, which is not enough to explain the mood score, leading to a lack of trust in the model. The use of compositional ANN models can not only help improve model performance by learning better representations from biophysical data, but it can also address issues of explainability by decomposing the model into smaller, more explainable components.

3.7 Conclusions

This chapter reviewed literature on compositional ML for CPSs. We first distinguished model-level decomposition of a single ANN from system-level pipeline composition in autonomous driving, and identified the lack of systematic, quantitative studies comparing monolithic and compositional ANN designs. We then surveyed work on FPGA-based ANN implementation, compositional verification and runtime enforcement, and compositional explainability, noting in each area that existing methods treat models as monolithic black boxes or compose only at the system level. Finally, we reviewed the AV and depression case-study literatures, where monolithic models dominate despite safety, timing, and interpretability requirements that align with a compositional approach. Together, these strands motivate the technical contributions developed in the remainder of the thesis.

Chapter 4

Exploring Compositional Neural Networks for Real-Time Systems

4.1 Overview

Chapters 2 and 3 showed that there was a noted lack of research that systematically, quantitatively, and objectively presents how a large monolithic ANN model used in a Cyber-Physical System (CPS) can be replaced with several smaller networks (a compositional design) to ease their implementation on hardware for timing verification using WCET. This becomes especially pertinent for time-critical real-time CPSs such as autonomous vehicles. Hence, in this chapter, we present a novel compositional design process for Artificial Neural Network (ANN) models used in CPSs. We use several real-life datasets to demonstrate the compositional design process, including a depression dataset to predict depressive mood scores and a sizeable transportation dataset to develop the models that can help cars perform Discretionary Lane Change (DLC). Through these examples, we make the following contributions.

1. We present a detailed methodology of how a Multilayer Perceptron (MLP)-based feedforward monolithic design can be replaced with smaller models that combine to form a compositional design, reducing the hardware footprint of the model with minimal to no performance loss. The method presented is not specific to the datasets and type of Machine Learning (ML) models considered in this chapter and can be used with most feedforward ML models with minor to no changes.

2. We describe how the compositional design can be implemented on hardware for proper timing analysis and develop a compiler to convert the compositional design into a hardware model.

3. We quantitatively examine the monolithic and compositional designs using software models to test their performance, and hardware models to compare the

WCET, hardware logic requirements, and their verification, captured by comparing neuron connections and computations required for inference. We demonstrate that, for the DLC dataset, using a compositional rather than a monolithic design can achieve significant reductions: up to 85% in WCET, around 53% in hardware resources, and around 40% in computations and neuron connections, for only a minor reduction in performance.

The chapter is structured as follows. Firstly, in Section 4.2, we provide the details on the datasets considered to build the models in this work. In Sections 4.3 and 4.4, we introduce the compositional designs considered in this chapter. Subsequently, Section 4.5 describes the model development procedure. Later, in Section 4.6, we describe the hardware implementation and the compiler used to convert the compositional design into a hardware model. Next, we present the comparison reports and discussions in Section 4.7 to show how the compositional models compare to the monolithic models. Finally, Section 4.8 provides a discussion of the results—including their intuition, dataset dependence, and generalisability—and concluding remarks.

4.2 Datasets

Here, we look at example datasets and how they can be modelled using a compositional approach. We will look at classification datasets. We begin with small toy datasets that are used for illustrative purposes only. Specifically, we will look at the Iris, Wine, and Diabetes datasets. We then describe the real-life depression dataset [98] and the massive NGSIM transportation dataset [140].

4.2.1 Iris Dataset

The Iris dataset is a small dataset that contains 150 samples of iris flowers [141]. The dataset has four features: sepal length, sepal width, petal length, and petal width. The dataset has three classes: Setosa, Versicolor, and Virginica. The dataset is used to classify the iris flowers into one of the three classes based on the features.

4.2.2 Wine Dataset

The Wine dataset is another small dataset that contains 178 samples of wine [142]. The dataset has 13 features: Alcohol, Malic acid, Ash, Alcalinity of ash, Magnesium, Total phenols, Flavanoids, Nonflavanoid phenols, Proanthocyanins, Colour intensity, Hue, OD280/OD315 of diluted wines, and Proline. The dataset has three classes: Class 1, Class 2, and Class 3. The dataset is used to classify the wine samples into one of the three classes based on the features.

4.2.3 Diabetes Dataset

The Diabetes dataset is a small dataset that contains 442 samples of diabetes patients [143] and ten features. The dataset has ten features: age, sex, Body Mass Index (BMI), average Blood Pressure (BP), s1, s2, s3, s4, s5, and s6. The meaning of each feature might be unclear (especially for s5) as the documentation of the original dataset is not explicit. Although the Diabetes dataset is a regression problem, we transform it into a classification problem by clustering it into three groups to form a new target/output feature.

4.2.4 Depression Dataset

We use a dataset previously published in [98] to build regression-based personalised (one model per subject) mood score prediction models. This dataset was collected during a one-month study of 14 mild-moderately depressed adult human subjects (with a mean age of 21.6 $\pm$ 2.8 years and ten females) before the onset of the COVID-19 pandemic. Although the dataset originally includes data collected from 14 individual subjects, in this work, we focus on a subset of seven subjects for our experiments and analysis. This decision was made to keep the scope of the results manageable and the presentation concise, allowing for a more detailed and focused discussion of the compositional modelling process and results. As we develop personalised models, the number of subjects is not a limiting factor.

The dataset includes data collected from smartwatches, smartphones and clinical EEG-based neuro-cognitive evaluations. The collected mood score values lie between 1 and 7, with 1 for feeling not depressed and 7 for feeling severely depressed. We pre-process the raw dataset containing 48 features (or predictors) using an imputation method described in [144] and obtain 43 input features (i.e. inputs to a model), one output feature (mood scores) and one feature to preserve timing information. The amount of data per human subject lies between 34 and 120 data points owing to data contamination and missing values. More information on the features can be found in [144].

4.2.5 NGSIM Dataset

This data was taken from a segment of the I-80 Freeway in Emeryville, California and a segment of US Highway 101 in Los Angeles, California [140]. Although the dataset is substantive, with over six million data points collected for over 10,000 vehicles, it contains some noise [145], which we filter using the method proposed in [146]. Samples are then extracted from the filtered dataset, as justified in [147]. For this extraction, we first find the six vehicles (cars or not, see Figure 4.2) surrounding a lane changing

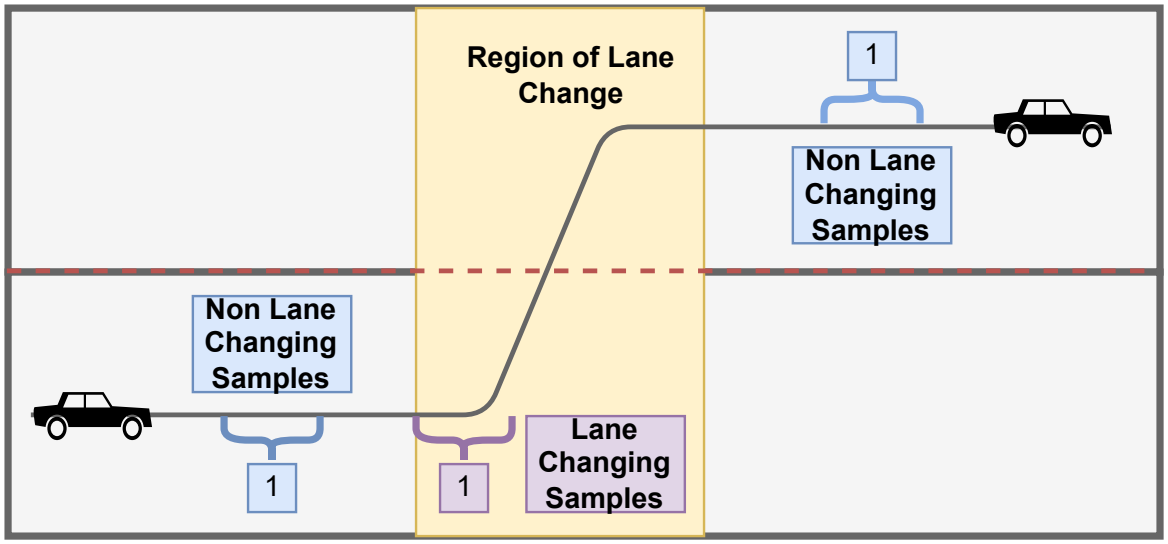


Figure 4.1: Sample extraction from the NGSIM dataset.

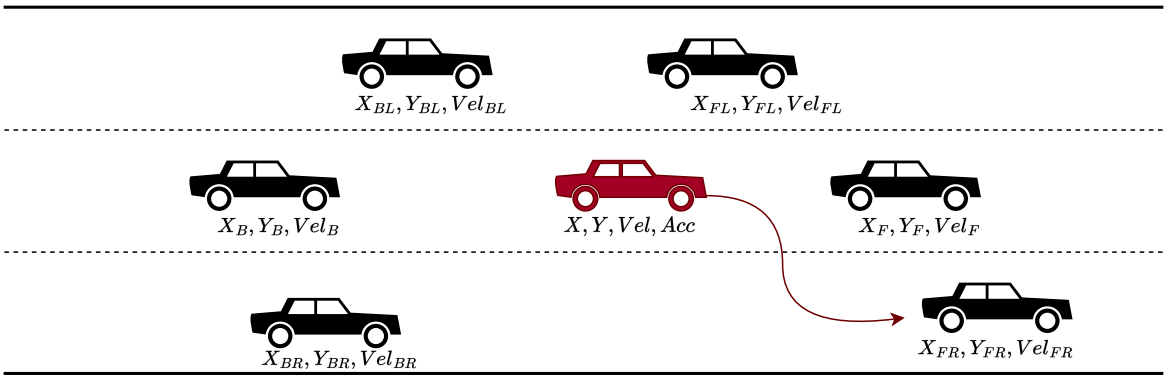


Figure 4.2: Arrangement of cars and their important parameters before the red car can change lanes. The subscripts F, B, L and R refer to cars in the front, the back, the left and the right of the red subject vehicle, respectively. Variables X, Y, Vel, and Acc refer to the position, velocity, and acceleration of the cars.

(*ego*) car and extract 1-second regions (10 data points) from around the time an ego begins to lane change and also 1-second regions from before and after the lane change to include samples that correspond to conditions that are feasible or as infeasible for a lane change respectively (Figure 4.1).

The extracted dataset contains 16,676 sample points from 559 lane changing (LC) cars, each sample containing 22 parameters or features. There are 11,006 No LC (lane keep) samples (labelled 1), 4,380 Left LC samples (labelled 0) and 1,290 Right LC samples (labelled 2). The features that act as inputs to the models include the position of the ego car (2 features), its velocity (1 feature), its acceleration (1 feature), and the relative positions (12 features) and velocities (6 features) of surrounding vehicles as input parameters to the model. Finally, we label the dataset according to the number of classes.

4.3 2-Stage Compositional Design for Classification

Our method relies on the divisibility of a monolithic classification model on the classes that the model predicts. We decompose or divide a monolithic design with C output classes into $C-1$ compositional models, where each model corresponds to a particular class. The motivation for this approach comes from the DLC case study, where the problem is to classify whether the current state of the subject vehicle is safe for a left, right or no lane change. When working on this problem's compositional idea, we discovered that we could work with ANN models for left and right lane changes and omit the one for no lane change. This omission is natural for humans, as we often arrive at a no lane change decision after failing to decide on left or right. Further, this is implementable because we can deduce the omitted class based on the outputs of the two models.

The method we developed generalises this idea to classifications with more than three classes. The $C-1$ ANNs are developed such that a model corresponding to class C will output a High (value $\geq$0.5) when the input to the model has the same class label; otherwise, the model will output a Low (value $<$0.5). Also, the models are constructed such that the sum of neurons in all $C-1$ models equals that of the monolithic design to ensure a fair comparison. This may mean that the individual compositional models may have a slightly different number of total neurons if the number of neurons in the monolithic case is not a multiple of the number of compositional models. However, the total number of neurons in a design will always remain the same in both designs. These ANNs run in parallel at the first stage, and their outputs are then passed onto a *merge* model (in the second stage) that interprets these outputs and generates a single output value corresponding to any one of the C class labels.

While it is also possible to add a model similar to the merge model at the end of the monolithic design to perform an *argmax* operation (i.e. find the argument with the highest element) to produce a class with the highest probability, such an exercise is unnecessary. In contrast to the argmax operation, which is an integral part of the prediction step in a monolithic design, the merge model in the compositional design is added after the prediction step of individual compositional models. It performs the crucial task of producing a single output through conflict resolution among the outputs of the compositional models. Thus, while the monolithic design is just a single-stage design (with C output nodes per model), the compositional design is a 2-stage pipeline (see Figure 4.3).

The outputs from the compositional models are passed onto a merge model that interprets these outputs and generates a single output value corresponding to any one of the C class labels. We build the compositional models separately from the monolithic models and then combine the trained models using the merge model.

This design method cannot be used for monolithic designs with fewer than three classes because, for two classes, both monolithic and compositional designs have a single model and hence become equivalent. While using C models instead of the $C-1$ we use can eliminate this limitation, it can be expected to significantly affect the classification performance of the compositional design by reducing the number of neurons in the compositional models, since we try to maintain the same number of neurons between the monolithic and compositional designs. Also, an additional model can be expected to make the compositional design bulkier and harder to implement on hardware.

This premise for developing compositional models is not specific to the case studies considered in this work. It applies to any feedforward ANNs used for classification problems with three or more classes. This includes a wide range of multidisciplinary classification problems, such as the classification of medical conditions, street signals, driving situations, and others, where this approach to compositional classification can be used.

4.3.1 Design Decisions

The notion of compositionality for the classification models we use presents the following two main design-related questions.

1. Which $C-1$ classes should be chosen to build the compositional models?

2. How do we divide the neurons among the compositional models to ensure equal total neurons in both designs and the lowest performance loss?

3. How to combine the outputs of the compositional models, i.e., how to design the merge model?

4.3.1.1 Choosing Classes

The division of the monolithic model into $C-1$ classes can be based either on domain (data) knowledge or on using qualitative and quantitative methods. For a more quantitative approach, we recommend using the $confusion-matrix$ we obtain from the monolithic design evaluation. Then, the diagonal of the matrix can be sorted to find the top $C-1$ easily classifiable classes and build models for them. A higher diagonal value implies higher ease in classifying the corresponding class. Building models for the $C-1$ easily classifiable classes should ensure that the remaining C^{th} class is properly deduced by reducing the number of false positives from the trained models.

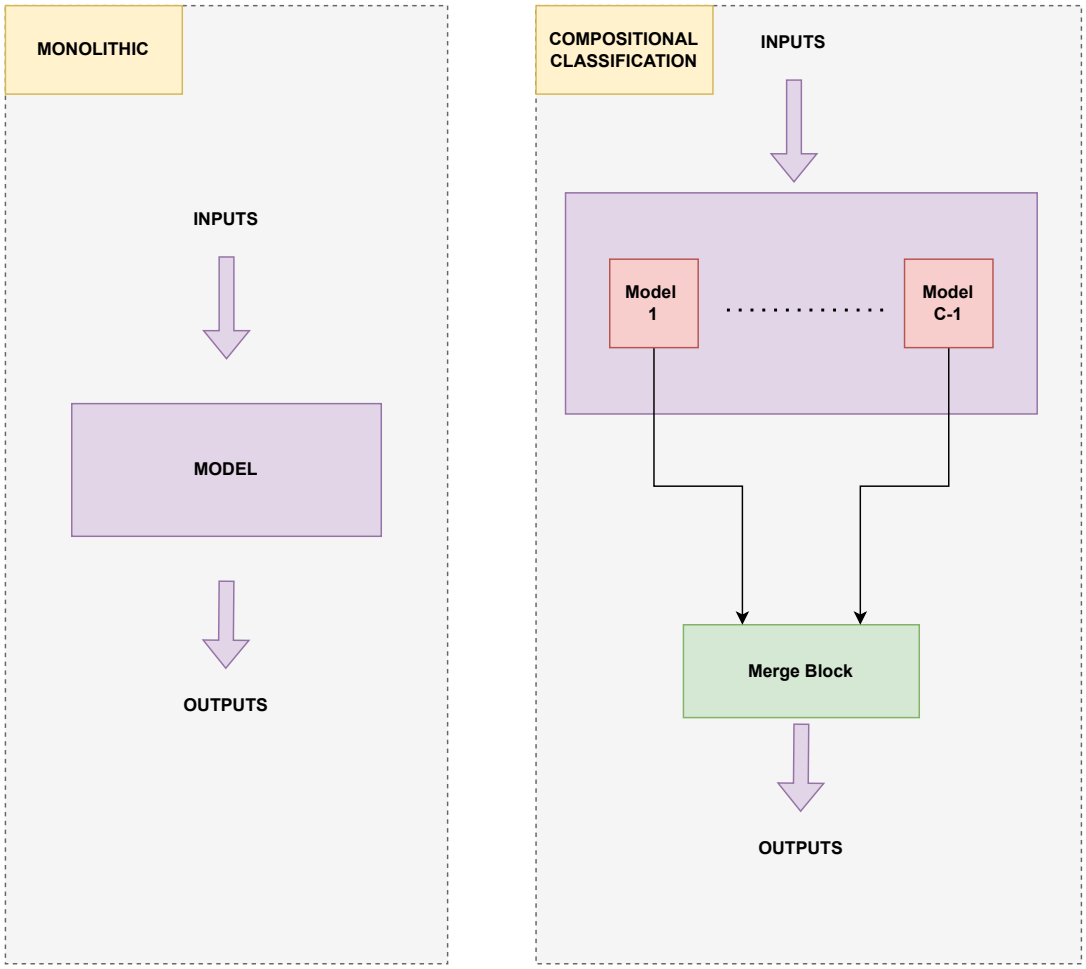


Figure 4.3: Two-stage monolithic design and a 2-stage compositional design. We assume that we build models for the classes labelled $1, 2, \cdots, C-1$, and the C^{th} class is deduced as shown.

$$confusion - matrix = \begin{bmatrix} TP_1 & E_{1,2} & \cdots & E_{1,C} \\ E_{2,1} & TP_2 & \cdots & E_{2,C} \\ \vdots & \vdots & \ddots & \vdots \\ E_{C,1} & E_{C,2} & \cdots & TP_C \end{bmatrix} \tag{4.1}$$

In Equation 4.1, TP_c represents the number of true positives, i.e. the proportion of samples from class c that were correctly classified as c, and $E_{c1,c2}$ represents the proportion of samples of class $c1$ that were incorrectly classified as $c2$.

Similarly, the division of neurons among the $C-1$ models can be based on intuition/domain knowledge or a quantitative approach. We propose beginning with dividing the neurons equally between the chosen models and then deciding whether to optimise the neuron division among the models based on the performance of the compositional design vis-à-vis the monolithic design. A designer may choose a thresh-

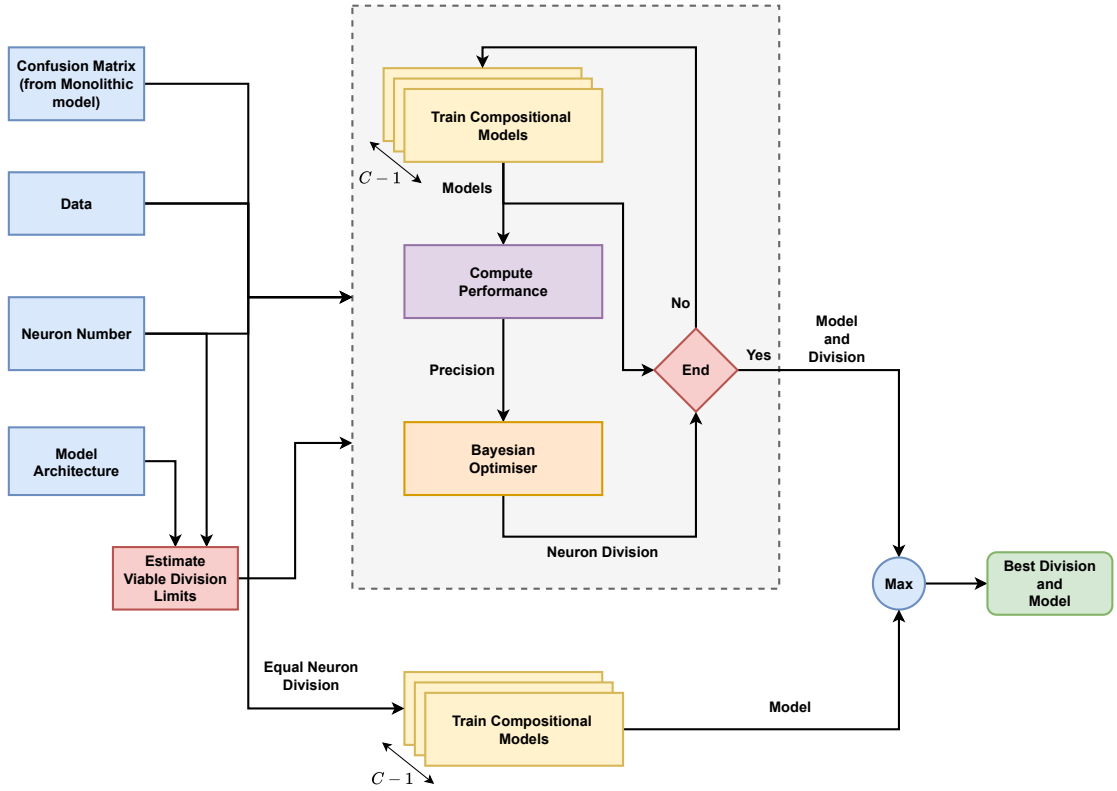


Figure 4.4: Diagram showing how using the confusion-matrix, the compositional models are chosen for training and optimised in a loop to maximise *BalancedAccuracy* and find the best division of neurons between them. The neuron number (N) determines the total number of neurons in the design. The optimisation ends after 50 trials. Viable division limits are also computed to ensure model architecture constraints.

old for performance loss observed (if any) when migrating to a compositional design before opting for the optimisation route. This ensures that the compositional design is not over-engineered and that the performance loss is minimal.

We use the optimisation approach if the equally divided compositional design has a Precision loss of more than one per cent. We optimise the compositional design Precision using Bayesian optimisation with Gaussian processes [148] while ensuring that the total number of neurons in the compositional models remains the same. The optimisation can be stopped after a fixed number of trials or after the loss has attained a specific value. Also, any other suitable optimisation method, such as evolutionary algorithms, can be used in place of the one proposed, but the goal of optimising the model performance remains the same.

In the general case, all the compositional models will share the same set of input features, i.e., those used by the monolithic design. However, if data domain knowledge can be used to remove a few redundant features in a few compositional models, it should be used to reduce the number of neurons in the compositional models, which can be beneficial for hardware implementation.

4.3.1.2 Dividing Neurons between models

We either divide the neurons equally between the compositional models, or we optimise the top $C-1$ models such that the division of neurons between them maximises the Precision [149] of the overall compositional system based on certain conditions.

For optimisation, we use Bayesian optimisation with Gaussian processes [148] using the Latin Hypercube Sampling (LHS) [150] to generate random points for consideration. We use a function to provide architecturally viable limits for the division of neurons to the optimiser. This function is necessary to ensure that no model has any layers with zero neurons (which basically eliminates the layer). Although the optimisation procedure should eventually consider the case where we have an equal division of neurons (if this division corresponds to a nearby local or global minimum), the stochastic nature of the procedure makes it difficult to guarantee that this division will be considered in the finite number of optimisation steps/trials the procedure takes. Hence, we separately develop compositional models with equal division of neurons and take the compositional design with the maximum *Precision* between the optimised and equally divided designs. The procedure is shown in Figure 4.4.

We can stop the optimisation after a fixed number of trials or after the performance (*Precision*) has attained a specific value. In this work, we stop the optimisation after 50 trials. Also, any other suitable optimisation method, such as evolutionary algorithms, can be used instead of the one proposed, but the goal here remains the same. Moreover, we use optimisation only when the difference between the Precision of the monolithic design and the compositional design with equally divided neurons is more than 1%. This is ensured by training the equally divided models first, which saves computational resources and time.

4.3.1.3 Designing the Merge Model

The merge model can be designed to suit individual specifications. We illustrate the two ways used for two situations: (1) when choosing a model with decision conflict (more than one model predicting a High) is unsafe, and (2) when choosing a model with decision conflict is safe. For both ways, when only one of the $C-1$ compositional models produces a High, the merge module passes on the label for that model's class. Moreover, when none of them passes a High, i.e. none of the models is confident about its prediction, the label of the remaining C^{th} class is passed as the output.

$$Out = \begin{cases} \mathrm{C}_{safe} & \text{two or more Models output} > 0.5 \\ 1 & \text{if } \mathit{Model}_1 \text{ output} > 0.5 \text{ only} \\ 2 & \text{if } \mathit{Model}_2 \text{ output} > 0.5 \text{ only} \\ \vdots & \\ \text{C-1} & \text{if } \mathit{Model}_{C-1} \text{ output} > 0.5 \text{ only} \\ \mathrm{C} & \text{if all Model output} \leq 0.5 \end{cases} \tag{4.2}$$

However, when more than one model passes a High, the merge model in the safety-critical case (first case) outputs the label C_{safe} of the class that can be used safely in a decision conflict (see Equation 4.2). For the second case, where the output can be chosen between the conflicting models, the merge model outputs the label of the model with the maximum output value to allow the label of the most confident model to pass its output (see Equation 4.3). This method enables the compositional classification of datasets with C classes with just $C - 1$ models.

$$Out = \begin{cases} \text{argmax(outputs)} & \text{two or more Models output} > 0.5 \\ 1 & \text{if } \mathit{Model}_1 \text{ output} > 0.5 \text{ only} \\ 2 & \text{if } \mathit{Model}_2 \text{ output} > 0.5 \text{ only} \\ \vdots & \\ \text{C-1} & \text{if } \mathit{Model}_{C-1} \text{ output} > 0.5 \text{ only} \\ \mathrm{C} & \text{if all Model output} \leq 0.5 \end{cases} \tag{4.3}$$

We can also use other mathematical functions, including ANNs, to combine the outputs of the compositional models. The ANN can be trained to learn the best combination of the outputs from the compositional models. It can be trained using the outputs from the compositional models as input and the actual output as the target using the backpropagation algorithm. However, we propose that we begin with the conditional merge model and then move to more complex merge blocks if the conditional merge model does not meet the performance requirements.

4.3.2 Example

Consider the 3-class classification problem from Figure 4.5 where we have a monolithic model that classifies the classes $0, 1, 2$. We can divide the monolithic model into two compositional models that classify the classes 0 and 1. Class 2 is inferred based on the output of the two models. The merge model is designed as mentioned in 4.3. The compositional model outputs the class label based on the output of the models. If both models output a value greater than 0.5 (i.e., probability of more than 0.5), the

merge model outputs the class (either $C0$ or $C1$) label with the highest output value. If only one model outputs a value greater than 0.5, the merge model outputs the class label of that model. If none of the models outputs a value greater than 0.5, the merge model outputs the class label class $C2$.

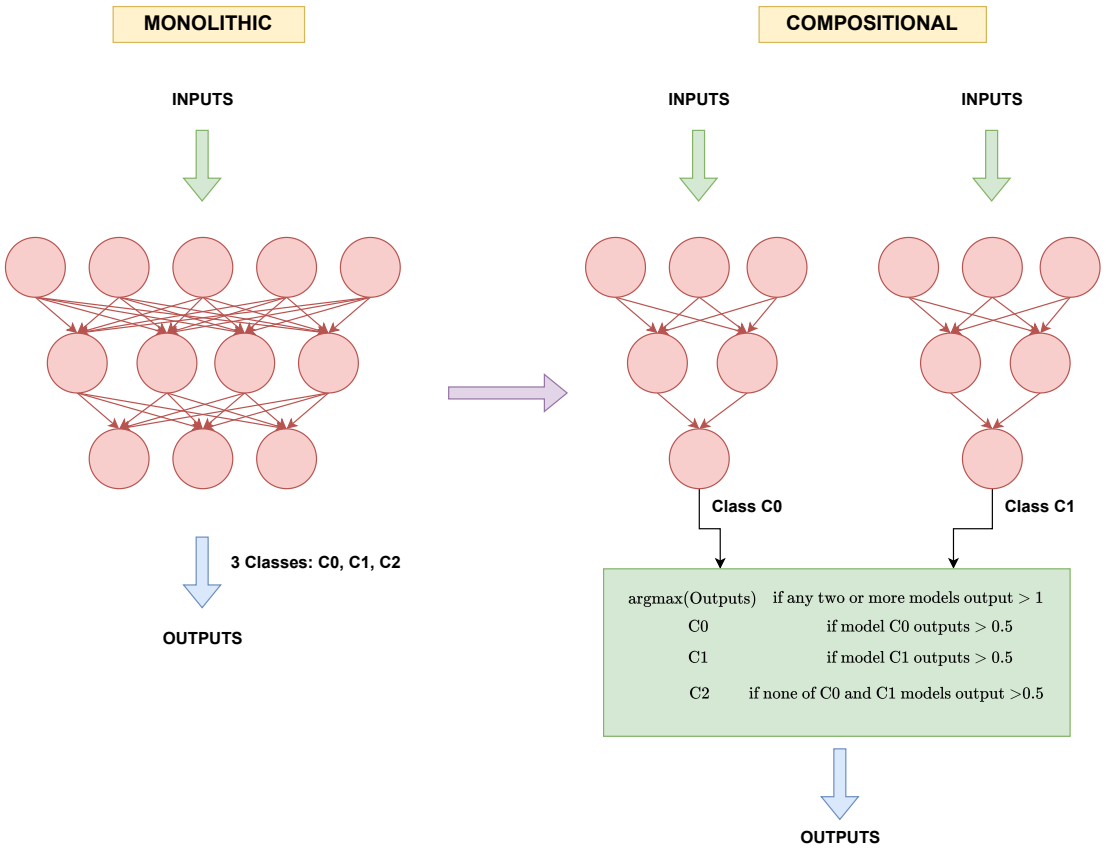


Figure 4.5: Two-stage monolithic design and a 2-stage compositional design with three classes.

Compositional and monolithic models have the same number of neurons (12 neurons). Also, as can be seen from the figure, the monolithic model has three output nodes for the three output classes, while the compositional model has one output node each for the single class they learn to predict. Here, in this example, the input features are divided between the two models, but they can also be shared between the models.

4.4 2-Stage Compositional Design for Regression

The compositional design for regression problems is similar to the classification problem, with a few differences. Since there are no classes in a regression problem, we do not divide the monolithic MLP design based on the values of the output feature. Here, we exploit the divisibility of the input feature set into individual compositional

models. If the feature can be divided into N clusters, we can build N compositional models.

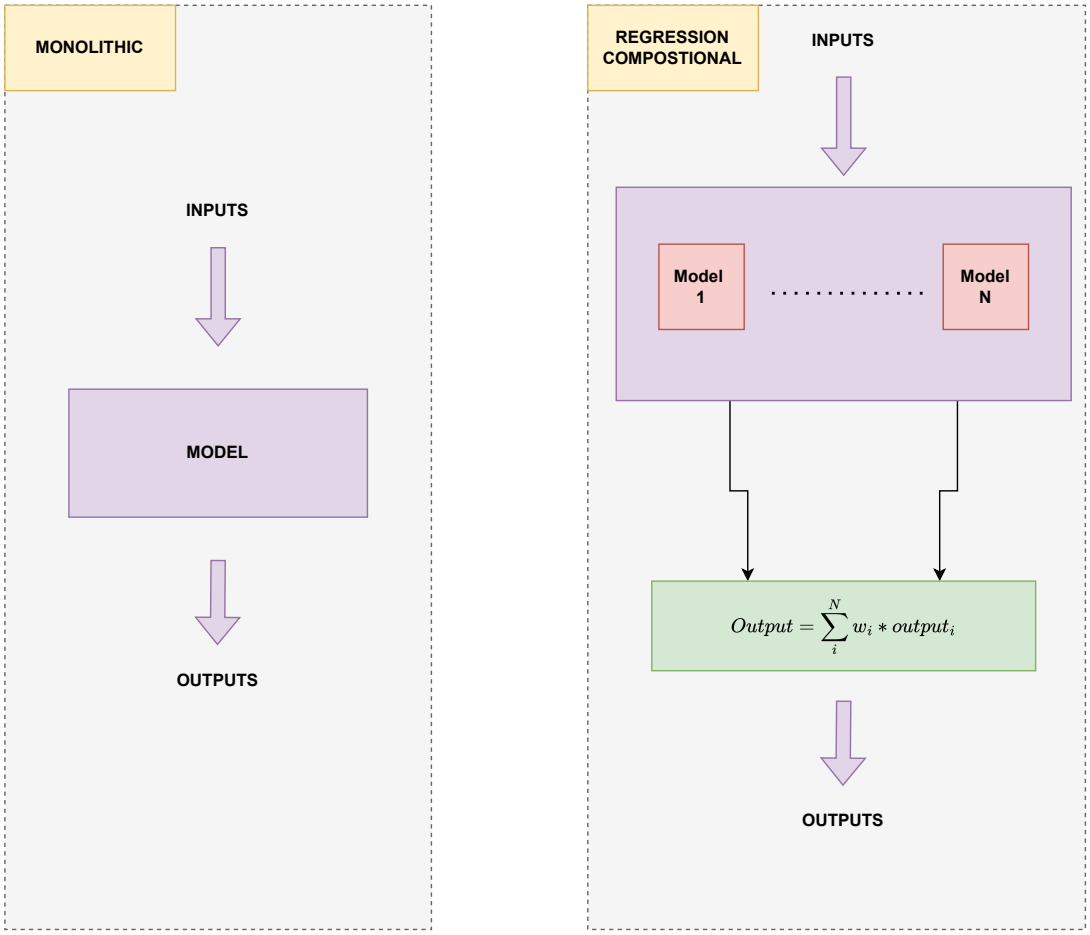


Figure 4.6: Single-stage monolithic design and a 2-stage compositional design. We assume that we build models for the clusters labelled $1, 2, \cdots, N$.

Similar to the classification problem, the compositional design for the regression problem is also a two-stage design, with the first stage containing the N compositional models (running in parallel) and the second merge stage combining the several model outputs into one (see Figure 4.6). The individual compositional models output a single continuous value. These are then combined using a mathematical function to output a single continuous value.

4.4.1 Design Decisions

The notion of compositionality for regression models we use presents the following two main design-related questions.

1. Which N clusters should be chosen to build the compositional models?

2. How do we divide the neurons among the compositional models to ensure equal total neurons in both monolithic and compositional designs, and the lowest performance loss?

3. How do we combine the outputs of the compositional models?

4.4.1.1 Choosing the Feature Clusters

We propose using data domain knowledge first to divide the input features into N clusters. If domain knowledge is not available or does not achieve the same level of performance as the monolithic design, we recommend using a quantitative approach. We propose using the K-means clustering algorithm [151] to divide the input features into N clusters. The K-means algorithm is a simple and efficient algorithm that can be used to partition the input features into N clusters. The algorithm works by iteratively assigning each input feature to the nearest cluster and then recalculating the cluster centroids. The algorithm converges when the cluster assignments do not change or if the algorithm reaches a specified number of iterations.

Another way to cluster the input features is to use Hierarchical clustering with correlation as the distance metric. This method is useful when the number of clusters is not known beforehand. The algorithm works by iteratively merging the two closest clusters until only one cluster remains. The algorithm can be stopped at a specific number of clusters or when the distance between the clusters exceeds a certain threshold.

Any other suitable clustering algorithm can also replace the clustering algorithms discussed. We can even use multiple algorithms to find the best set of features to keep in each cluster. We can begin by finding the best number of clusters, and then choose the best features to keep in each cluster by observing which features are most repeated in the clusters. Overall, the goal is to divide the input features into N clusters that can be used to build the compositional models.

As we are dividing based on the features, the models in the compositional design will have a different number of features than the monolithic model, but the total number of features entering both designs will remain the same. The divided feature sets can be mutually exclusive or have repeated features. However, we advise using a set of mutually exclusive features to prevent feature redundancy, if that is desired, in the compositional models when we combine the models in the second step of the compositional design.

4.4.1.2 Dividing the Neurons

We either divide the neurons equally between the compositional models, or we optimise the top N models such that the division of neurons between them minimises the

Mean Absolute Error (MAE). We propose beginning with dividing the neurons equally between the chosen models and then deciding whether to optimise the neuron division among the models based on the performance of the compositional design vis-à-vis the monolithic design. A designer may choose a threshold for performance loss observed (if any) when migrating to a compositional design before opting for the optimisation route. This ensures that the compositional design is not over-engineered and that the performance loss is minimal.

We can use a similar methodology for performance optimisation as discussed in the classification section. We simply replace the performance metric Precision with MAE in the algorithm shown in Figure 4.4.

4.4.1.3 Combining the Outputs

We propose a weighted combination of the outputs from the compositional models (as a merge block) as shown in Equation 4.4 for a simpler hardware implementation. The weights of the weighted average are learned using backpropagation. We could also use a plain average of the outputs from the compositional models to further simplify the merge block.

$$output\_value = \sum_{i=1}^{n} w_i \times output_i \tag{4.4}$$

An ANN can also be used to replace the weighted combination of the outputs. The ANN can be trained to learn the best combination of the outputs from the compositional models. It can be trained using the outputs from the compositional models as input and the actual output as the target using the backpropagation algorithm. However, we propose that we begin with the weighted average merge block and then move to more complex merge blocks if the conditional merge block does not meet the performance requirements.

4.5 Model Development

We follow the following steps for MLP-based ANN design development of the datasets described. First, we pre-process the data with standard normalisation/scaling if it is not normalised, which modifies a dataset as shown in Equation 2.12. Second, we train the monolithic design model for different ANN architectures, as we do not have existing monolithic models for any of the example datasets considered. Third, we build the compositional design architecture.

When building the compositional design models, we can choose to have the same number of total neurons or different numbers in the two designs. While the for-

mer ensures a fairer performance comparison, we experiment with both approaches to comprehensively evaluate the two-stage compositional design. Finally, we train the compositional design with corresponding datasets using a 5-fold Cross-Validation (CV) method (from Section 2.3.2.3).

Moreover, we use the merge block in Equation 4.3 for the Small dataset models, as conflicts in these examples are not safety-critical. We use the merge block in Equation 4.2 for the DLC case study models, as choosing either the left LC or right LC model where conflict occurs can be unsafe. Here, we use the No Lane Change class as the C_{safe} class (see Equation 4.2). For the depression dataset regression models, we use the merge block in Equation 4.4.

Additionally, we adopt random seed selection, best-model saving, and learning rate reduction practices to alleviate the undesirable effect of stochasticity (variability in model representations due to randomness) in training. The training of both the compositional and monolithic models is performed in Python using the well-known TensorFlow-Keras Library on a 64-bit Windows 10 system with Intel Core i7-9700CPU @ 3.00 GHz and 32 GB RAM. The following subsections discuss the specifics of developing models for the various datasets.

4.5.1 Small Datasets

We use a four-layer classification architecture for all monolithic and compositional designs for all three datasets. We ensure that the monolithic and compositional designs have the same total number of neurons. The Wine and Diabetes dataset monolithic designs have 24, 12, 6, and 3 neurons, while the Iris dataset monolithic model has 20, 10, 4 and 3 neurons in the four layers from input to output. Next, we find the classes to build the compositional models using the quantitative method mentioned in Section 4.3.1.

We build compositional models for classes 0 and 1 for the Wine and Diabetes classes and classes 0 and 2 for the Iris dataset. We do not optimise the neuron division; instead, we divide the neurons equally between the two compositional models, as there was no performance loss. Further, we use Hyperbolic Tangent (Tanh) for all layers except for the last layer, which uses Softmax for the monolithic model and Sigmoid for the compositional models. Also, both designs are trained for 200 epochs and a batch size of 8 for the Wine and Diabetes datasets and 4 for the Iris dataset.

4.5.2 Depression Dataset Models

We build personalised regression models for the Depression dataset, which contains data from seven human subjects. As it is usually challenging to build models on data collected from human biophysical signals due to their complexity, models often need

Table 4.1: Parameter Values for the DLC Case Study

Layers	Features (S) Mon./Com.	Total Neurons Varying N			
1	22/16	80	118	157	195
3	22/16	80	118	157	195
6	22/16	80	118	157	195
10	22/16	80	115	157	192

Table 4.2: Search values (or limits) for parameters during hyperparameter optimisation of the Depression Dataset. (N.A. means Not Applicable)

Monolithic			
Hyperparameter	**Lower Limit**	**Upper Limit**	**Values**
Number of layers	2	4	N.A
Neurons in input layer	50	90	N.A
Activation	N.A	N.A	tanh, relu, elu, linear
Batch size	N.A	N.A	4, 6, 8
Compositional			
Hyperparameter	**Lower Limit**	**Upper Limit**	**Values**
Number of layers	2	4	N.A
Neurons in input layer	10	15	N.A
Activation	N.A	N.A	tanh, relu, elu, linear
Batch size	N.A	N.A	4, 6, 8

to be optimised for their performance. Instead of manually choosing and tweaking a few parameters to obtain better performance, we choose Evolutionary Algorithm (EA) based algorithms to optimise the number of hidden layers in the model, the number of neurons in the input layer, the activation of the hidden layers, and the training batch size for both monolithic and compositional models. The designs are optimised for 50 iterations using MAE as the performance metric.

Once we have optimised the monolithic models, we divide the input feature into six mutually exclusive sets. We divide the features based on whether they refer to food, activity, sleep, physiological, psychological or neurocognitive feature sets and build compositional models for the sets. A compositional design contains these six models, which are trained and optimised separately and then combined using the merge block (from Equation 4.4), the weights of which are learned using a gradient descent-based algorithm. The search parameters for the optimisation can be found in Table 4.2. Regardless of the optimisation procedure, as we are making models for a regression problem, the final layer in the models for both designs always uses the *linear* activation function.

Furthermore, we do not ensure that the compositional and monolithic designs have the same total neurons. However, we ensure that the neuron ranges for the compositional models are nearly a sixth of the monolithic model. This difference helps ascertain

how well the compositional approach works when the total number of neurons in the compositional design differs from the monolithic design.

4.5.3 Discretionary Lane Change Case Study Models

We built different monolithic and compositional classification designs for the DLC dataset, each with a different architecture. The Left and Right Lane Change (LC) classes are chosen for the compositional design based on observed DLC behaviour, and the No lane change class is left out. Moreover, we experiment with three values of total neurons, i.e. {118, 157, 195}, and different numbers of layers, i.e. {1, 3, 6, 10}, in the monolithic and compositional designs (see Table 4.1).

Furthermore, we optimise the neuron division as the equal division of neurons between the models did not yield designs with good compositional performance. We optimise the designs using Bayesian optimisation based on the Precision of the compositional designs. We end the optimisation after 50 trials and use 20 initial points with LHS. We also experiment with a design with three models (C models) and neurons equally divided among them for a more thorough comparison.

Moreover, while we have 22 features for the monolithic examples, we remove the features corresponding to cars on the right lane for the Left LC Model, and for the Right LC Model, we do the opposite. This feature reduction is justified as the vehicles travelling in the right lane have a limited impact on a driver's decision to change lanes to the left and vice versa. This reduction results in each compositional model having just 16 features. We should, however, note that the compositional design, on the whole, has the same number of inputs as the monolithic model.

As each compositional model makes decisions for a single class, the model can have two labels. Each label corresponds to the probability of that model producing a High (corresponding to label 1) or Low (corresponding to label 0). For instance, we relabel the developed datasets when training a compositional model corresponding to class C. The samples corresponding to class C are labelled 1, and the rest are labelled 0. This method contrasts with the labelling procedure described to train the monolithic model, where we have labels from 1 to C for a dataset with C classes.

4.6 Hardware Implementation

The implementation feasibility of the monolithic and compositional designs for all datasets on hardware is also essential when quantitatively evaluating their pros and cons. Therefore, we develop a compiler to compile the ANN designs (TensorFlow-Keras [39] and PyTorch [152] ANN designs) to Very High-Speed Integrated Circuit Hardware

Description Language (VHDL) for implementation on a Field Programmable Gate Array (FPGA) through a process adapted from [75].

We develop the compiler (compiling TensorFlow-Keras ANN models to VHDL) from scratch using techniques adapted from [75], particularly the pipelined multicyclic approach to ANN execution[1]. This approach enables the reuse of neuron instances mapped into hardware to reduce the number of total hardware components needed for mapping and enables us to fit larger networks than would usually be possible. Furthermore, we extend the work in [75] by adding support for compositional architectures and secondary ANN layers (like the Batch Normalisation layer) and more activation functions (such as *softmax*, *Exponential Linear Unit (ELU)*) which are implemented through discontinuous piecewise linear functions and Look-Up Tables (LUTs). We also make the compiler generic enough to work with arbitrary model compositions, including the compositional paradigm discussed in this work.

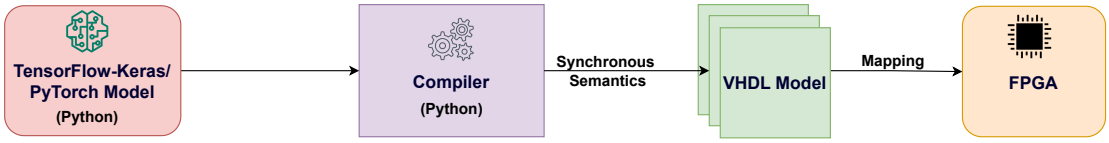


Figure 4.7: Compilation pipeline for converting ANN designs to VHDL.

The compilation pipeline can be seen in Figure 4.7. In contrast to the work in [75], we develop the compiler in Python. This allows the compiler to be *imported* (as a module) into existing Python-based ANN projects for easy compilation of ANN models. The compiler is widely applicable because Python is the most commonly used platform to develop ANN designs. It accepts both Tensorflow-Keras and Pytorch models or the weights and architectures of the models in an ANN design as its input. It compiles the design using synchronous semantics (explained in the next section) to a set of VHDL files that execute the ANN design in a multicyclic manner. These files can be easily mapped onto any FPGA that accepts VHDL designs. We call the hardware implementations of ANNs Hardware Artificial Neural Networks (HANNs).

4.6.1 Semantics of Hardware ANNs

4.6.1.1 General Semantics

To define the execution semantics of HANN, we borrow the structural semantic definitions used in [75]. We begin by defining the concepts of delayed inputs and $\leftarrow$ or assignment. We denote the delayed semantics by the superscript ^{P} — a process that introduces a slight delay but creates a deterministic operation. To further denote a cumulation of delays for t time steps, the superscript notation $^{P^t}$ is used, with $t = 1$

[1]The compiler can be imported in Python from https://test.pypi.org/simple/nn2vhdl==2.0.3

for a unit delay. We provide a mechanism for capturing the structural transformation from ANNs to these synchronous semantics used in HANN construction.

Firstly, at the lowest level, we provide the semantics for executing a single neuron. Each neuron takes some vector of inputs $\boldsymbol{I}$ and produces a single output O. Internally, the neuron has a set of weights $\boldsymbol{W}$ (where $|\boldsymbol{W}| = |\boldsymbol{I}|$) and a single bias b, and an arbitrary activation function a. The operation of a neuron with these properties is shown in Equation 4.5, with the set of input values being from the previous tick, denoted by $\boldsymbol{I}^P$, as described earlier.

$$O \leftarrow a(\boldsymbol{W} \cdot \boldsymbol{I}^P + b) \tag{4.5}$$

The synchronous execution semantics of a HANN are such that data dependencies are broken at layer boundaries, meaning that individual neurons of a specific layer are executed each tick. The formal semantics of the operation of a HANN are presented using Structural Operational Semantics (SOS) rules [153] of the following form:

$$D \rightarrow D' \tag{4.6}$$

where D is the set of data variables before the transition, and D' is the set of data variables after the transition.

Additionally, a further form of these semantics is used, which includes the option of including a predicate p that must be satisfied for the rule to hold, as shown below.

$$\frac{p}{D \rightarrow D'} \tag{4.7}$$

These transitions are composed together using synchronous semantics, allowing all neurons in a layer to be executed in lockstep with each other at the same rate, as determined by some global tick. Each neuron in the layer will execute a single computation transition per tick before synchronisation occurs, and the process repeats. During a tick, a neuron reads the inputs from the previous tick and executes them to generate the output. Also, delayed semantics provide a mechanism for dealing with data dependencies, as described before. This means that all variable reads of the system refer to values from the last tick (denoted by ^{P}).

The semantics of a neural network is split into two parts: one which captures the mapping of values between neurons through weights and activation (Equation 4.8) and one which captures the parallel execution of transitions within a network (Equation 4.9). For Equation 4.8, which captures the mapping of values between neurons, $d_{i,l}$ is the output of the i^{th} neuron in layer l which is being assigned to, $d^P_{0,l-1}$ through $d^P_{n-1,l-1}$ are the output values from all n neurons in the previous layer $l-1$ and the previous tick. Finally, f is a function that uses weights, biases, and activation to create

the final sole output value. Indeed, Equation 4.8 represents the operation presented in Equation 4.5 in the SOS format.

$$D \rightarrow D[d_{i,l} \leftarrow f(d^P_{0,l-1}, d^P_{1,l-1}, \ldots, d^P_{n-1,l-1})] \tag{4.8}$$

Equation 4.9 captures the parallel execution semantics for each of the previously described transitions for neuron evolution and its mapping function. This parallel execution is used for all transitions within the same synchronous tick. Here, $d_{i,l}$ and $d_{j,l}$ represent two different data variables from two different neurons i and j at layer l that are being modified through functions f and g respectively at the same instant. Here, all instances of d in Equation 4.9 can either be an input or an output data variable, depending on the transition that is taking place. Due to the delayed semantics mentioned earlier, the two transitions can be combined into a single transition that performs two data-variable updates, regardless of their dependencies. For a multi-layer network, the semantics expressed by Equations 4.8 and 4.9 run across all layers until the final output is obtained.

$$\frac{D \rightarrow D[d_{i,l} \leftarrow f(\ldots)], D \rightarrow D[d_{j,l} \leftarrow g(\ldots)], i \neq j}{D \rightarrow D[d_{i,l} \leftarrow f(\ldots), d_{j,l} \leftarrow g(\ldots)]} \tag{4.9}$$

Additionally, it is essential to note that this rule is commutative and associative. Any arbitrary number of transitions can be executed at the same instant in time while all rules still hold. Further, the commutation property means that these transitions may be specified in any order, and the overall system's output will be identical.

4.6.1.2 Compositional Semantics

While these semantics can completely describe a monolithic design operation with a single HANN, the compositional design needs novel semantics for a multi-HANN two-level pipeline. For the first level, which consists of multiple HANNs, the process is similar to a single large HANN. Each HANN is executed in parallel with synchronisation at each tick using the associativity rule, which semantically results in each neuron in a tick being executed in parallel as shown in Equation 4.9, where f and g can be neuron mapping functions from two different HANNs executing within the same layer.

$$\frac{D \rightarrow D[d_1 \leftarrow H_1(\cdots)], \ldots, D \rightarrow D[d_{C-1} \leftarrow H_{C-1}(\cdots)]}{D \rightarrow D[O \leftarrow r(d_1^{P^{L-L_{1}+1}}, \cdots, d_{C-1}^{P^{L-L_{C-1}+1}})]} \tag{4.10}$$

The semantics of the second level in the design can be obtained by combining the outputs from the compositional HANNs using a functional module that only executes when all preceding HANNs have finished executing. This requires delays, introduced using the superscript ^{P}, to compensate for the different numbers of layers in the HANNs. We define the semantics as shown in Equation 4.10, where C is the

number of classes; $H_1, \cdots, H_{C-1}$ represent the $C-1$ HANNs that map the network inputs to their respective outputs $d_1, \cdots, d_{C-1}$; $L_1, \cdots, L_{C-1}$ represent the number of layers in the HANNs; L is the number of layers in the HANN that has the maximum number of layers; $^{P^{L-L_1+1}}, \cdots, ^{P^{L-L_{C-1}+1}}$ denote the delays we introduce to the outputs of individual HANNs to ensure deterministic operation with the additional 1 ensuring that the merge block reads the outputs from the previous tick once all of them have been produced, and; r represents the function that executes the merge block. r maps the delayed outputs of HANNs to the single output variable O, and O is the output of the entire compositional design. Thus, the semantics here ensure that all inputs to the merge function are delayed and read simultaneously, irrespective of the number of layers in the individual HANNs.

4.6.2 Multi-cycle Execution

We use the semantics previously explained to explain the multi-cycle approach to HANN execution used in this work. In doing so, we extend the multi-cycle execution presented in [75] with the compositional case. In multi-cycle execution, fewer neurons are instantiated in hardware, and multiple clock cycles are used for each execution tick. The set of internal variables for each neuron is stored in single-cycle memory, with each hardware instance of a neuron being fed one such set of variables each clock cycle. The neuron will read these variables, perform computation, and then write the updated output back to memory within one cycle.

Subsequently, a counter will be incremented, which causes the neuron to read the next set of variables from memory in the next clock cycle. Once all required computations have been performed, the execution is complete, signifying the end of the tick. This will cause variables to be mapped to their subsequent neurons in the next layer and allow execution for the next tick to begin. Moreover, the number of cycles needed for a single execution is proportional to the maximum number of neurons of a particular type (activation).

This approach only changes how an HANN is operationally executed between ticks, from a single-cycle to a multi-cycle operation. This preserves the tick-based semantics of the execution presented in the sections before and, in turn, assures determinism in the HANN execution as before. Moreover, the reuse of neuron instances across multiple clock cycles in the multi-cycle approach results in an implementation with smaller logic requirements at the cost of increased computation time. Since modern FPGAs have reasonably high clock frequencies, the increase in computation time should be minimal. With this approach, we can synthesise larger networks in hardware while still allowing for the features that the semantics provide.

4.6.3 Compilation of ANN design to VHDL

The algorithms presented in Algorithms 1, 2, 3, and 4 show the high-level pseudocode for the compiler. This compiler converts an Artificial Neural Network (ANN) design, which may consist of one or multiple ANN models, from popular deep learning frameworks (TensorFlow/Keras and PyTorch) into synthesisable VHDL code using the multicyclic synchronous semantics described in the last two sections. To achieve this, an intermediate JSON representation is used to represent both the hierarchical inter-model (connections between models) and intra-model (connections within a model, i.e., between layers) dependencies and connections[2].

The compiler operates through two key phases, as illustrated in Algorithm 1:

1. **Model Parsing** (Algorithm 2): Extracts architecture and weights from neural network models. This builds a JSON representation of the network topology.
2. **VHDL Code Generation** (Algorithms 3 and 4): Produces synthesizable VHDL code with fixed-point arithmetic

Algorithm 1 Compile TensorFlow/PyTorch Models to VHDL

```
1: function NN2VHDL(models, conn, aParts, bSize)
2:     jModel ← PARSE(models, conn, aParts, bSize)
3:     vhdlFiles ← WRITEVHDL(jModel)
4:     return vhdlFiles
5: end function
```

4.6.3.1 Interface Algorithm

The interface algorithm (Algorithm 1) orchestrates the entire neural network to VHDL conversion process. The algorithm consists of three main steps:

1. **Line 2**: Calls the `parse` function to convert the input models into a structured JSON representation
2. **Line 3**: Calls the `writeVHDL` function to generate VHDL files from the JSON representation
3. **Line 4**: Returns the complete set of generated VHDL files

The `nn2VHDL` function serves as the entry point for the compiler. It accepts the following inputs:

[2]The compiler only works for feedforward ANN designs using activations of type linear, Tanh, sigmoid, exponential, ELU, and Rectified Linear Unit (ReLU) in MLP architectures

- `models` (*models*): A list of ANN models or the weights and architecture of the models in an ANN design.
- `conn` (*conn*): The interconnections between the models.
- `aParts` (*aParts*): The number of pieces/parts to use for approximating piecewise linear activation functions.
- `bSize` (*bSize*): The bit size for the fixed-point representation used in computations.

Algorithm 2 Parse Models to JSON Representation

function PARSE(*models*, *conn*, *aParts*, *bSize*)
  $jModel \leftarrow \{\}$
  for $model, model\_name$ **in** *models* **do**
    $mA \leftarrow$ GETMODELARCHITECTURE(*model*)
    $mW \leftarrow$ GETMODELWEIGHTS(*model*)
    $mAc \leftarrow$ GETPIECEWISEACT(*model*, *aParts*)
    $mLc \leftarrow$ GETLAYERCONNECTION(*model*)
    $mJ \leftarrow$ {"architecture" : mA, "weights" : mW, "act_parts" : mAc, "layer_conn" : mLc}
    $jModel[model\_name] \leftarrow mJ$
  end for
  $jModel$["conn"] $\leftarrow$ *conn*
  $jModel$["fp_size"] $\leftarrow$ *bSize*
  return $jModel$
end function

Algorithm 3 Write VHDL Files from JSON Representation

function WRITEVHDL($jModel$)
  $modelVHDL \leftarrow []$
  $aFile \leftarrow []$
  for *conn* **in** $jModel$["conn"] **do**
    for $model\_names$ **in** *conn* **do**
      $mJ \leftarrow jModel[model\_names]$
      $mVHDL \leftarrow$ GENERATEVHDL(mJ)
      $actVHDL \leftarrow$ GENACTVHDL(mJ["act_parts"])
      append $actVHDL$ **to** $aFile$
      append $mVHDL$ **to** $modelVHDL$
    end for
  end for
  $mFile \leftarrow$ GENMODPIPELINEMAP($modelVHDL$)
  $lFile \leftarrow$ GENLIBRARY($jModel$["fp_size"])
  return $[mFile, aFile, lFile]$
end function

Algorithm 4 Generate VHDL Code Components

1: **function** GENERATEVHDL(*model*) ▷ Convert a single model's JSON representation to VHDL code.
2: **return** *vhdlCode*
3: **end function**
4: **function** GENACTVHDL(*activationParts*) ▷ Generate VHDL for piecewise linear activation functions.
5: **return** *activationVHDLCode*
6: **end function**
7: **function** GENMODPIPELINEMAP(*modelVHDL*) ▷ Generate a pipeline map for connecting models in VHDL.
8: **return** *pipelineMapCode*
9: **end function**
10: **function** GENLIBRARY(*fixedPointSize*) ▷ Generate library files for fixed-point arithmetic.
11: **return** *libraryCode*
12: **end function**

4.6.3.2 Model Parsing Algorithm

The model parsing algorithm (Algorithm 2) is responsible for extracting the neural network architecture from different deep learning frameworks. The algorithm operates as follows:

1. **Line 3**: Initializes an empty JSON model structure
2. **Lines 4-12**: Iterates through each model in the input list and extracts:
   - **Line 5**: Model architecture via `getModelArchitecture`
   - **Line 6**: Model weights via `getModelWeights`
   - **Line 7**: Piecewise activation function definitions via `getPiecewiseAct`
   - **Line 8**: Layer connections via `getLayerConnection`
3. **Line 9-10**: Constructs and stores a unified JSON representation for each model
4. **Lines 13-15**: Adds global configuration including inter-model connections and fixed-point bit size

The algorithm begins by detecting the framework type (PyTorch or TensorFlow/Keras) and routes to the appropriate parsing function. For PyTorch models, it traverses the computational graph to extract layer information, while for TensorFlow models, it analyses the model's layer structure directly. The algorithm then constructs a unified representation of the model architecture, including connections between layers and activation functions. It generates piecewise linear approximations for non-linear

activation functions using Look-Up Tables (LUTs), which provide a resource-efficient mechanism for activation computation in hardware. For each activation function type, it calculates the appropriate derivative and constant terms needed for the linear approximation. The algorithm divides the function domain into a specified number of segments and computes the slope and intercept for each segment using the derivative at the midpoint.

4.6.3.3 Compositional Model Parsing

The compositional model parsing is handled within the main parsing algorithm (Algorithm 2). The algorithm handles complex neural network architectures composed of multiple interconnected models by:

1. **Lines 4-12**: Processing each individual model independently to extract its architecture and components
2. **Line 13**: Storing inter-model connections in the global JSON structure
3. **Line 14**: Maintaining fixed-point configuration across all models

This approach enables the conversion of sophisticated neural network systems that cannot be represented as a single sequential model by creating a unified representation that includes all individual models and their interconnections.

4.6.3.4 VHDL Code Generation Algorithm

The VHDL code generation algorithm (Algorithm 3) transforms the JSON representation into synthesisable VHDL code. The VHDL mapping (from JSON) adopts a synchronous, multicyclic, and pipelined approach as discussed in Section 4.6.1. This method reduces the hardware resources required for mapping by reusing neuron instances within the hardware. The algorithm operates as follows:

1. **Lines 3-4**: Initialises lists to store model VHDL files and activation function files
2. **Lines 5-13**: Iterates through all connections and models:
   - **Line 7**: Retrieves the JSON representation for each model
   - **Line 8**: Generates VHDL code for the model via `generateVHDL`
   - **Line 9**: Generates activation function VHDL via `genActVHDL`
   - **Lines 10-11**: Appends generated files to existing model files
3. **Line 14**: Generates pipeline mapping for all models via `genModPipelineMap`

4. **Line 15**: Generates fixed-point arithmetic library via `genLibrary`

5. **Line 16**: Returns the complete set of VHDL files

The VHDL code generation process is carried out by several key functions, as described in Algorithm 4. The `generateVHDL` function converts a single model's JSON representation into VHDL code, while `genActVHDL` produces VHDL implementations for piecewise linear activation functions. The `genModPipelineMap` function is responsible for generating a pipeline map that connects the various models within the VHDL design, and `genLibrary` creates the necessary library files to support fixed-point arithmetic operations.

Thus, the compilation system generates a complete VHDL project structure that includes several key components: a main VHDL file serving as the top-level entity with the full neural network implementation; a component library containing fixed-point arithmetic functions and utilities; individual VHDL files for each activation function; and, optionally, a result processing block for classification tasks. Listing 4.1 shows a basic usage example of the tool in Python.

```
# Import necessary libraries
from nn2vhdl import ToVHDL
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense

# Create a simple neural network
def create_model():
    inp = Input(shape = (16,), name = 'Input')
    x = Dense(int(16), activation = 'relu')(inp)
    x = Dense(int(8), activation  = 'relu')(x)
    x = Dense(int(4), activation  = 'relu')(x)
    out = Dense(1, activation = 'sigmoid')(x)

    model = Model(inp, out)
    # Compile model
    model.compile(optimizer = "adam", loss =
       'binary_crossentropy')
    return model

# Create two models
modelL = create_model()
modelR = create_model()

# Convert Compositional ANN model to VHDL
```

```
ToVHDL(models=[modelL, modelR],
    \textsl{Model}_names = ["\textsl{Model}_L",
       "\textsl{Model}_R"],
    connections = ["input[0:16] -> \textsl{Model}_L",
       "input[6:22] -> \textsl{Model}_R"], #Specify the
       connections between the models
    classes = [0,1,2], #Specify the labels of the classes
    max_inputs=22, #Specify the number of inputs
    result_block=True) #Specify if a merge block is required
```

Listing 4.1: Basic Usage Example of the compiler in Python

4.7 Results

This section presents the results of the monolithic and compositional designs. We quantify the benefits and limitations of a compositional design against a monolithic design by comparing the predictive performance using F1-score for classification models and MAE for regression models, the hardware usage in terms of Adaptive Logic Modules (ALMs) and the Worst-Case Execution Time (WCET) for the hardware implementations, and the number of computations and connections in the design.

F1-score is the harmonic mean of the Recall and Precision, and produces a real number between 0 and 1. The MAE, as expected, is a real number over the entire real number domain. The performance of a classification (or regression) design is better if a design has a higher (or lower) F1-score (or MAE) value. Equations 2.17, 2.15, and 2.16 define the F1-score, Precision, and Recall metrics, respectively. Since we use a 5-fold CV approach, we average the performance metrics across the five folds before the F1-score and MAE calculation.

Also, we found that using a 32-part piecewise linear approximation of the activation function and a 32-bit fixed-point representation of all numbers in the design, with 16 bits reserved for the fraction part, results in minimal performance loss due to loss in Precision when compiling the designs to VHDL. Furthermore, the ALMs are computed by fitting the developed VHDL designs onto the hardware of the Stratix-V FPGA using Quartus Prime Standard Version 19.1. We discontinue design fitting and call the design unsynthesisable if Quartus takes longer than 24 hours or if the design logic requirements are higher than the device can provide. In such instances, we report the ALM estimate as produced by Quartus but do not report the WCET, as it cannot be calculated.

Additionally, the WCET is calculated from the maximum cycle frequency of the slow hardware synthesis model of Quartus for each design as shown in Equation 4.11,

where f_{max} is the maximum cycle frequency of the design, and N_{cycles} is the number of cycles required for a single execution of the design.

$$WCET = \frac{N_{cycles}}{f_{max}} \tag{4.11}$$

Finally, we compare the number of ANN computations, i.e. the number of additions and multiplications, and the number of connections encountered during one pass of a set of inputs. We report the percentage reduction in calculations and connections for the compositional design against the monolithic design.

4.7.1 Best Neuron Divisions

Table 4.3: Best Neuron Divisions (DLC Dataset). (TN refers to the total number of neurons in the model. Left and Right refer to the compositional models for the Left and Right lane change classes. The values are the fraction of neurons in each model after optimisation.)

	1 Layer			3 Layers		
Models ↓ / TN →	118	157	195	118	157	195
Left	0.69	0.71	0.74	0.62	0.54	0.51
Right	0.31	0.29	0.26	0.38	0.46	0.49
	6 Layers			10 Layers		
Models ↓ / TN →	118	157	195	115	157	192
Left	0.58	0.60	0.64	0.70	0.58	0.58
Right	0.42	0.40	0.36	0.30	0.42	0.42

Table 4.3 presents the results of the neuron division optimisation for the compositional models in the DLC dataset. In compositional design, the overall model is split into two sub-models, each responsible for predicting a specific class: the Left and Right lane change classes. The table shows, for a range of total neuron counts (TN) and different network depths (1, 3, 6, and 10 layers), the fraction of the total neurons allocated to each sub-model after the optimisation process. For example, with a single hidden layer and 118 total neurons, 69% of the neurons are allocated to the Left model and 31% to the Right model.

Based on the values in Table 4.3, the optimal division of neurons between the Left and Right compositional models for the DLC dataset is not equal and varies depending on both the total number of neurons and the network depth. Across all configurations, the Left model consistently receives a higher fraction of neurons than the Right model, suggesting that the Left lane change class is more complex or requires greater model capacity to achieve optimal performance. Furthermore, we see a balanced division of neurons between the Left and Right models for higher layer counts and higher

total neuron counts. This indicates that both the complexity of the classes and the architecture of the model influence the best allocation strategy.

4.7.2 Computations and Connection Reduction

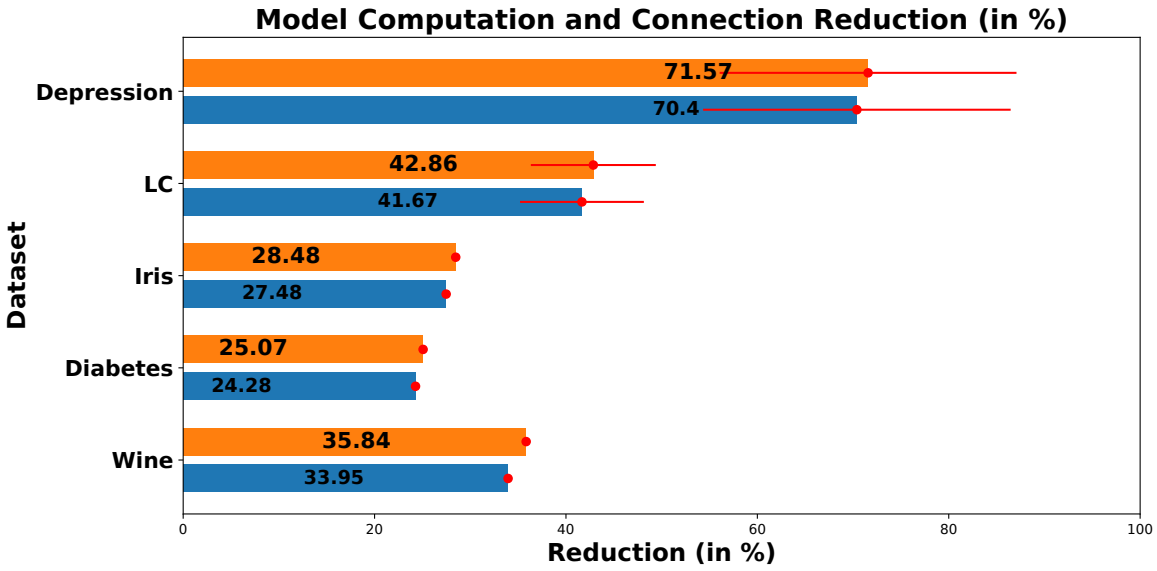


Figure 4.8: ANN computation and connection reduction for the hardware ANNs for all datasets. The blue bars represent the percentage reduction in computations, whereas the orange bars represent the percentage reduction in connections when using a compositional design instead of a monolithic design. The Red bars show the variation in the values.

The percentage reduction in computations and connections for the compositional designs against the monolithic designs for all datasets is shown in Figure 4.8. For brevity, we average the values across cases for both the DLC and Depression datasets. We observe that a compositional design results in an average reduction of computations and connections of 40% over all datasets. This reduction is higher for bigger datasets and larger models, at around 40-70%, and at least 24% for smaller datasets.

The Depression dataset has the highest reduction in computations and connections of around 70% with a standard deviation of around 16%, which can be attributed to a larger number of models per design and unequal neuron numbers between the monolithic and compositional designs. The DLC dataset has a reduction of around 40% with a standard deviation of around 6%. The Diabetes dataset has the lowest reduction in computations and connections, which can be attributed to its small design size.

Also, the reduction in connections and computations is similar across all datasets, as reducing the connections will also reduce the computations, and vice versa. This re-

duction in connections and computations corroborates the excellent hardware resource gains design we saw when using compositional in the previous subsections.

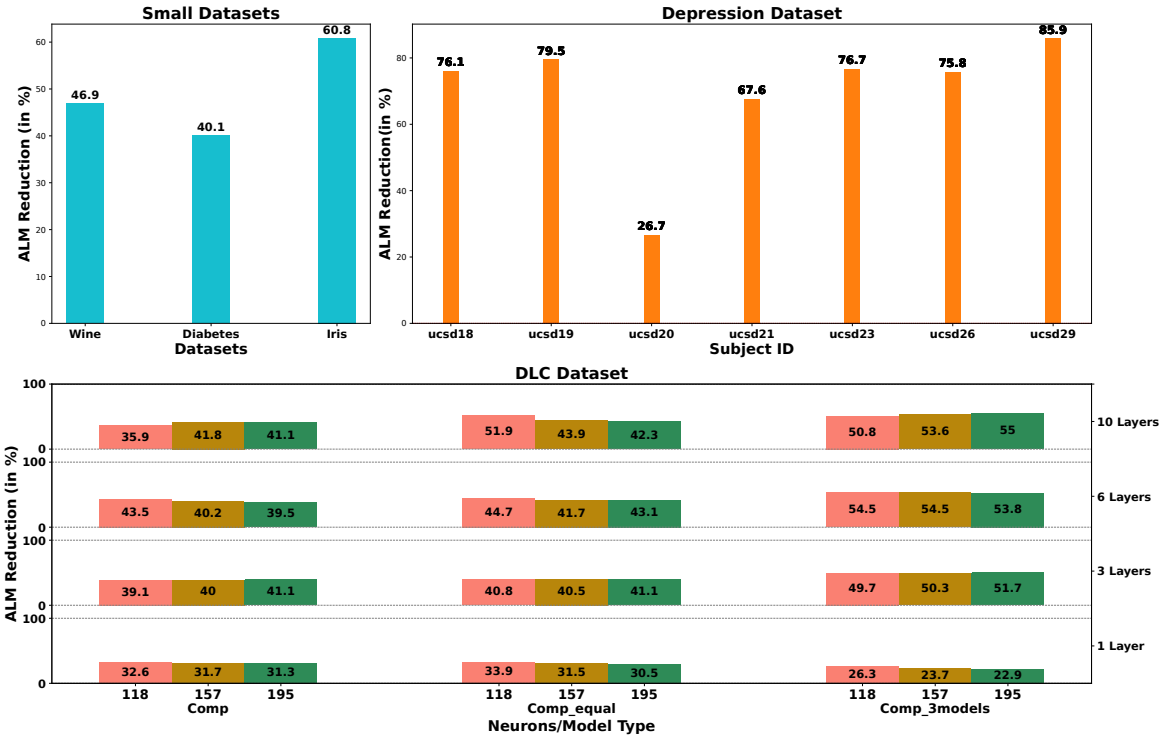


Figure 4.9: ALM reduction for all datasets. The blue bars represent the percentage reduction in ALM usage when using a compositional design instead of a monolithic design. The red bars show the variation in the values.

4.7.3 Hardware Performance

Figure 4.9 shows the ALM reductions for all datasets considered in this work. As we can see, the compositional design results in a hardware resource usage (measured using ALM usage) reduction of between 26 and 85 per cent, with an average reduction of 53.51 per cent across all datasets. Within the DLC dataset designs, the optimised compositional model and the equally divided compositional model seem to have similar ALM reductions and outperform the three-model compositional model.

Furthermore, we see more variation in the ALM reduction in the Depression dataset, as the monolithic and compositional designs do not have the same number of total neurons. Especially for subject ucsd20, the decrease in ALM usage is only 26.7%. This outlier is explained by the massive increase of 187% in the total number of neurons used in the compositional design vis-à-vis the monolithic design, reducing the resource gain. Notably, despite this significant increase in the number of neurons between the designs, the compositional design still needs fewer hardware resources.

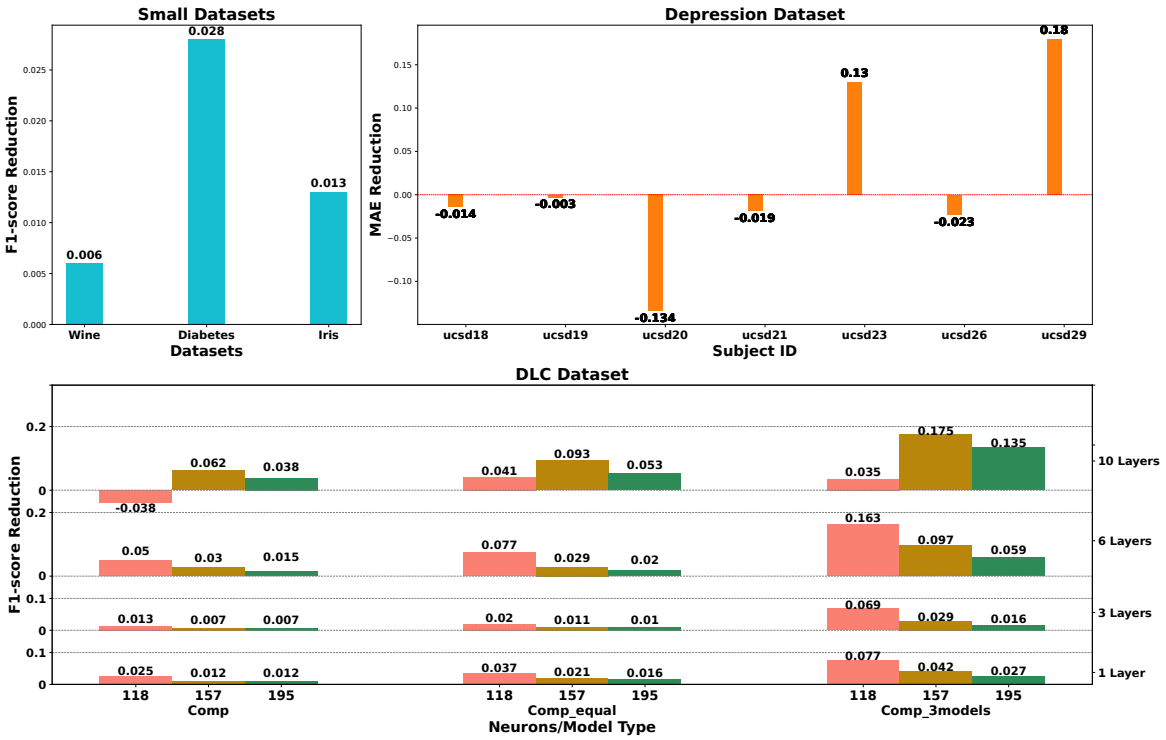


Figure 4.10: Predictive performance reduction (in absolute terms) for all datasets. Each bar plot refers to each small dataset in the Small Datasets plot. The bars in the Depression Dataset plot correspond to the seven human subjects (with anonymised names) considered. The DLC dataset plot is shown for various total neuron numbers, layer counts, and model combinations. "Comp" stands for the optimised compositional design, "Comp-equal" stands for the compositional design with an equal number of neurons between the compositional models, and "Comp-3models" stands for when the compositional design has three models with an equal number of neurons.

4.7.4 Predictive Performance

Using a compositional design results in a minor reduction in designs' predictive performance for most designs and datasets, as shown in Figure 4.10. The reduction in F1-score for classification models is less than 0.1 for most designs, with the exception of the 3-model compositional design, where the reduction is around 0.2 for some cases. Within the DLC dataset designs, the optimised compositional model seems to perform the best.

Also, as F1-score (which lies between 0 and 1) is generally less than 0.1, we can safely say that the reduction is less than 10% for 92% and is less than 5% for 76% of all classification designs. Moreover, the reduction in MAE for regression designs is less than 0.1 for 85% of the subjects. As mood scores are integer values that range between 1 and 7, an absolute error/difference of 0.1 can be considered minor. Interestingly, designs for two subjects (ucsd23 and ucsd29) outperform the monolithic design.

Further, the F1-score reduction shows a general decreasing trend for most designs for the DLC dataset, with the reduction decreasing as the number of neurons in-

creases. As the number of neurons increases, the compositional design is able to learn the complex DLC dataset better and begins to catch up with the monolithic design in performance. One exception is the 10-layer design with 118 total neurons, where the compositional design marginally outperforms the F1-score of the monolithic design. Since this is an outlier, we attribute this performance to artefacts arising from randomness in the ANN architecture parameters.

4.7.5 WCET Performance

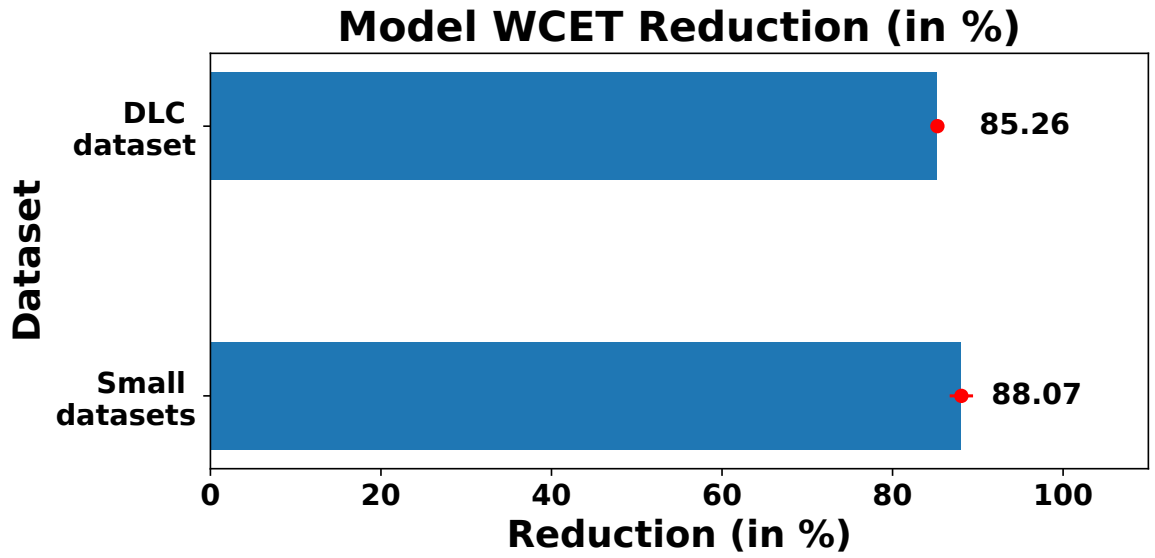


Figure 4.11: Percentage WCET Reduction for the DLC dataset and the Small Datasets. The DLC and Small dataset reductions are averaged over their respective cases. The red error bars at the end show the variation in the values that have been averaged.

The percentage reduction in WCET for the compositional designs against the monolithic designs for the DLC and Small datasets is shown in Figure 4.11. We averaged the percentage of WCET reduction across cases for both the DLC and small datasets, as they had minimal variation. We compute and average the WCET reduction for designs that are synthesisable. Overall, 47.22% of the compositional and 25% of the monolithic designs are synthesisable for the DLC dataset designs. All compositional and monolithic designs developed for the Small datasets were synthesisable.

We can see from Figure 4.11 that the WCET reduction for both the DLC and Small datasets is around 85%. This indicates a significant increase in the network speed for the compositional designs. As the compositional design has a much smaller fan-out of addition and multiplication components on hardware, compositional designs are significantly better in timing performance than monolithic designs.

Furthermore, we found that none of the Depression dataset monolithic designs could be synthesised on hardware due to their high resource requirements. Hence, we could not compute the WCET reduction metric, and the results were not presented. Nevertheless, as all compositional models were synthesisable on hardware, we could compute the WCET of the compositional designs, which was around $1.4\mu s$ for all subjects.

4.7.6 Summary

Overall, we make the following findings for the MLP-based ANN designs.

- The compositional designs result in a significant reduction in hardware usage vis-à-vis monolithic designs with an average reduction of 53.51% over all datasets at a minor reduction (<5% for most designs) in performance. The reduction in performance decreases as the number of neurons increases.
- The compositional designs result in a massive reduction in WCET vis-à-vis monolithic designs with an average reduction of around 86% over all datasets.
- The compositional designs significantly increase implementable designs vis-à-vis monolithic designs with an average increase of around 58% over all datasets.
- The compositional designs significantly reduce design computations and neuron connections vis-à-vis monolithic designs with an average reduction of around 40% over all datasets.
- The $C-1$ model compositional design performs better at ALM reduction and predictive performance reduction than the C model compositional design for classification problems.

4.7.7 Discussion

We now discuss the intuition behind the observed results and comment on how far the conclusions extend beyond the datasets considered in this chapter.

The reductions in ALMs, WCET, and neuron connections shown in Figures 4.8, 4.9, and 4.11 arise mainly from the structure of the compositional design rather than from properties unique to one application. A monolithic MLP must implement all hidden neurons and connections on a single hardware datapath, which increases fan-out and critical-path length on the FPGA. In contrast, a compositional design runs several smaller ANNs in parallel (for classification) or on separate feature clusters (for regression) before applying a merge block. As we noted in Section 4.7, the compositional design has a much smaller fan-out of addition and multiplication components on

hardware, which explains the large WCET reductions for the DLC and small datasets. Similarly, the average reduction of around 40% in computations and connections is higher for bigger datasets and larger models, and lower when the monolithic baseline is already small, as in the Diabetes dataset.

We have also observed that using a compositional design results in only a minor reduction in predictive performance for most designs, as shown in Figure 4.10. This is expected because we maintain nearly the same total number of neurons between the monolithic and compositional designs while allowing each compositional model to specialise on a narrower sub-problem. For the DLC dataset, the neuron division in Table 4.3 allocates more neurons to the Left lane change model than to the Right model, which is consistent with the class imbalance described in Section 4.2. Furthermore, as the number of neurons increases, the compositional design learns the complex DLC dataset better, and the performance gap narrows. The Depression dataset shows the largest reductions in computations and connections, which can be attributed to the larger number of models per design and the fact that the number of neurons between the monolithic and compositional designs is not equal.

We do not claim that every numerical result in this chapter will transfer unchanged to other datasets. Nevertheless, the qualitative trends—lower hardware usage, shorter WCET, fewer connections, and near-monolithic predictive performance—should hold for other feedforward ANN models in time-critical CPSs whenever the monolithic model is large enough to benefit from decomposition and the compositional split aligns with the structure of the problem (e.g., distinct classes or feature clusters). The DLC dataset is representative of a high-dimensional, imbalanced classification problem in transportation, whereas the small UCI datasets mainly illustrate the method at a modest scale. The Depression dataset highlights personalised regression models with limited data per subject, where compositionality improves implementability on hardware. The magnitude of the reported gains will depend on dataset size, class balance, model depth, and the decomposition strategy chosen.

4.8 Conclusion

In this chapter, we present a novel application-agnostic approach to replacing single monolithic feedforward ANN designs with compositional designs composed of several smaller ANNs and use several examples and tests to examine the capabilities of each design, both on software and hardware. We illustrate the approach through a two-stage method to designing compositional ANNs and examine it against monolithic ANNs through multiple example datasets, including a depression dataset and a DLC dataset. To aid in proper timing analysis using WCET, we develop a compiler that uses synchronous execution semantics to implement ANN designs on hardware. We

further use a multicyclic approach that enables the reuse of hardware components and reduces the number of hardware resources required.

After a comprehensive comparison, we show that the very nature of compositionality makes it possible to significantly reduce the number of hardware resources (ALMs) required to synthesise the MLP-based ANN models into hardware, specifically the Stratix-V FPGA board. Despite the minor performance loss, we discuss how a compositional design helps significantly reduce neuron connections, hardware resources and the WCET. The significant decrease in neuron connections and the overall size of individual models should also make verification easier. However, one of the key limitations of the approach is the need for an existing Monolithic model, which is a feedforward ANN and whose internal structure is known. Nevertheless, the method presented is not specific to the datasets presented and the type of ML model considered in this chapter. We have only shown the 2-stage approach with MLP models, where the merge block can be any relevant, arbitrary mathematical function; however, the compositional design allows for arbitrary combinations of different feedforward ML models as long as the input-output compatibility of the stages is maintained. We will explore more compositional design variants in the next two chapters.

Chapter 5

Compositionality for Safe-by-construction ML-based Cyber-Physical Systems

5.1 Overview

In light of the problems with the verification of ML-based CPS, as highlighted in Chapters 2 and 3, in this chapter, we present a framework that extends the compositional framework discussed in Chapter 4 to design an ML-based CPS that is safe by construction. We propose learning formally defined policies from realistic data and using them to guide training and deployment. While some recent works have explored policy mining in the context of ML [20, 21], to the best of our knowledge, no existing methodology leverages these mined policies to train compositional ML models to ensure policy compliance and system-level safety. With this work, we make the following contributions.

- We present a novel policy mining framework that derives safety rules or policies from real-world data using defined policy templates.
- We introduce a policy-guided compositional model development and training methodology for ML-based CPS for the first time. We extend the work in the previous chapter by exploring a more complex model architecture and different types of ML models.
- We develop runtime enforcers to detect and mitigate policy violations in real-time, providing an additional safety net for unmodeled conditions.
- We validate our approach on a comprehensive AV case study, demonstrating improved safety guarantees and competitive performance.

The remainder of this chapter is organised as follows. Section 5.2 motivates our approach with a driving scenario that challenges monolithic models. Section 5.3 details our compositional training method. Section 5.4 presents our runtime enforcement strategies. In Section 5.5, we report on our AV experiments. Section 5.6 presents training and simulation outcomes and includes a discussion of their implications. Finally, Section 5.7 summarises the chapter.

5.2 Motivating example

Consider the safety-critical example of an AV. We can design a simple AV comprised of different modules that work on different tasks (as shown in Figure 5.1). This compositional architecture comprises multiple modules implemented using different mathematical models, such as ANN models or others. The following is a brief description of the modules in the AV design.

- Lane Change Decision (LCD) module, which is responsible for deciding when it is safe to change lanes,
- Car Following (CF) module that controls the car following behaviour; it is responsible for maintaining a safe distance from the car in front;
- Lane Change Execution (LCE) module, which is responsible for executing the lane change;
- Controller module, which assists in changing between the three aforementioned modes;
- Runtime enforcers [25, 154] after each module to guarantee its behaviour. The enforcer checks the output of the module and ensures that it satisfies the required policies. Their addition ensures that only correct outputs are sent to the controller module.
- The AV is also equipped with a perception module that uses sensors to detect the environment and other vehicles and provide these inputs to the modules.

Intuition on Compositionality of modules: If we take the example of the LCD module, all three lanes are typically considered during a lane change: the lane the vehicle is currently in and the lanes to its left and right, if available. A human driver would first observe the car ahead to decide if a lane change is necessary, then check the left and right lanes to ensure that there is a safe space to perform the manoeuvre. Thus, the monolithic LCD module can be further decomposed into submodules: one for the "*Opportunity model*" to assess whether a lane change is advisable or necessary, one

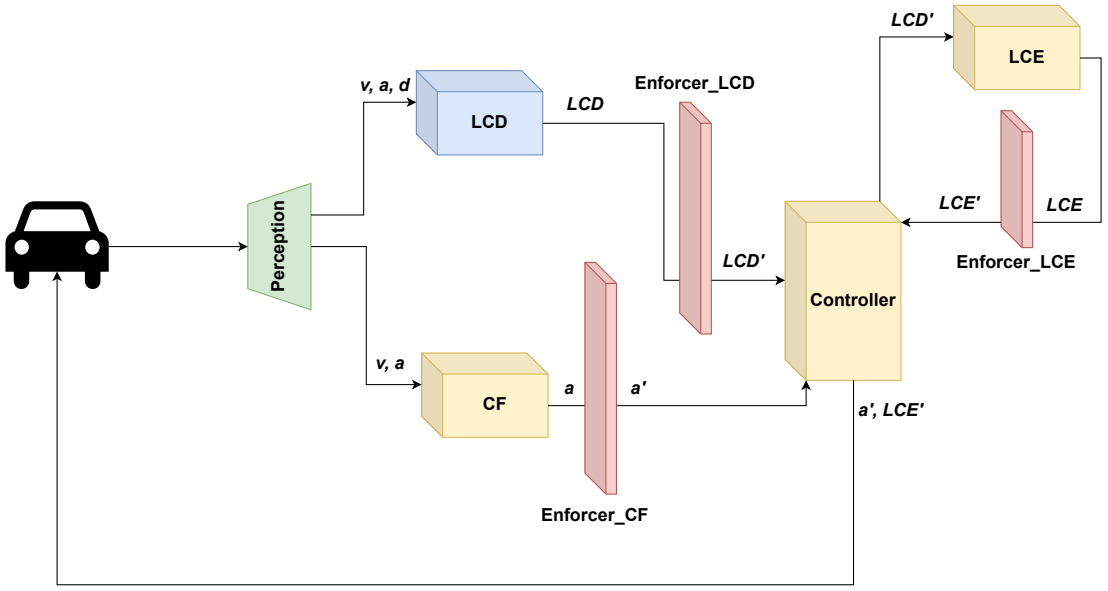


Figure 5.1: AV case study

for the "*Left Lane Change (LC)*" model to evaluate the safety of a left lane change, and one for the "*Right LC*" model to evaluate the safety of a right lane change. This decomposition demonstrates the significance of modularisation for better functionality and analysis.

Intuition on Compositionality of policies: We need to guarantee certain system-level behaviours/policies for the AV, and designing these policies compositionally can be beneficial as well. System-level policies can be defined and decomposed across modules to simplify verification. For example, an AV with three modules can have a system-level "*No collision*" policy decomposed into sub-policies such as "*No collision due to LCD* ", "*No collision due to CF*", and "*No collision due to LCE* ". The policy for the LCD module can be further decomposed and divided across the submodules as discussed previously. This way, the overarching policy, expressed as maintaining sufficient gaps between the AV and adjacent vehicles, can be distributed across various modules for more manageable analysis and enforcement.

In the following sections, we discuss how we can design compositional modules and policies and train such modules with their policies for the AV case study.

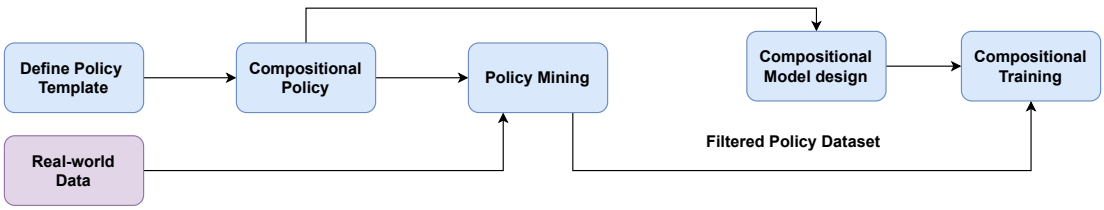


Figure 5.2: The whole compositional policy-driven training methodology.

5.3 Policy-driven Model Development

5.3.1 Overview

Our methodology begins by defining a compositional policy template that specifies the desired structure of policies to be mined (see Figure 5.2). The policies are expressed as predicate-logic formulas. A compositional policy mining process extracts modular policy components from the data using this template. Real-world data is used to guide this process, ensuring that the mined policies reflect actual conditions. A corresponding compositional model is then designed from the mined compositional policy. The model undergoes compositional training using a filtered dataset that conforms to the policy template and is suitable for deployment or further refinement.

To motivate our approach, consider an autonomous driving scenario where an ego/-subject vehicle must decide whether to change lanes when it is surrounded by six surrounding vehicles (two in front and back, and two each on either lane). This decision depends on the dynamic configuration of surrounding vehicles, such as nearby cars' relative distances, velocities, and accelerations.

As an example, a safety policy might state:

> *The ego vehicle may initiate a left lane change only if the gap to the left-front vehicle is sufficiently large, given their relative speed and acceleration.*

This policy can be formalised using predicate logic, with parameters to be learned from data. Let y denote the configuration of the environment, and let $\varphi_{LF}(y)$ encode the safety condition for the left-front vehicle. Then:

$$\varphi_{LF}(y) \equiv lfX > (b_0 lfV + b_1 lfA + b_2), \tag{5.1}$$

where:

- $y = \{lfX, lfV, lfA, \ldots, rpX, rpV, rpA, \ldots, bX, bV, bA\}$ encodes the real-time relative distances, velocities, and accelerations between the ego vehicle and each of the six surrounding vehicles.
- lfX, lfV, lfA are extracted from y
- b_0, b_1, b_2 are parameters to be learned from data.

This policy expresses that the left-front gap lfX must exceed a learned threshold computed as a linear function of the velocity and acceleration. The values of AV configurations (y) are obtained from the highD dataset [155], which contains trajectory data of multiple vehicles recorded on German highways, providing examples of real

driving interactions from which we can learn safety and gap-keeping rules. We now describe the formal structure used to define such policies.

5.3.2 Compositional Policies

We recall standard predicate-logic definitions in this section, derived from [156]. We need to define terms and atoms, which are the basic building blocks of predicate logic.

Term: A term is defined inductively as follows:

1. A variable is a term.
2. A constant is a term.
3. If f is a function symbol and $t_1, \ldots, t_n$ are terms, then $f(t_1, \ldots, t_n)$ is a term.
4. Nothing else is a term.

Atom: An atom (or atomic formula) is defined as follows: If P is an n-place predicate symbol and $t_1, \ldots, t_n$ are terms, then $P(t_1, \ldots, t_n)$ is an atomic formula (or atom).

Well-formed formula (wff): A wff (or just a formula) is defined inductively as follows:

1. An atom is a wff.
2. If φ_1 and φ_2 are wffs, then the following are also wffs:

$$\neg\varphi_1, \quad \varphi_1 \wedge \varphi_2, \quad \varphi_1 \vee \varphi_2, \quad \varphi_1 \rightarrow \varphi_2$$

3. If φ is a wff and x is a variable, then the following are wffs:

$$(\forall x)\, \varphi, \quad (\exists x)\, \varphi$$

4. Nothing else is a wff.

The above can also be seen in the Backus-Naur Form (BNF) for predicate logic:

$$\varphi ::= P(t_1, t_2, \ldots, t_n) \mid \neg\varphi \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \rightarrow \varphi) \mid (\forall x\, \varphi) \mid (\exists x\, \varphi)$$

Semantics: Interpretation of a formula in a non-empty domain D is achieved by assigning values to constant symbols (to each constant assign an element of D), function symbols (to each n-place function symbol assign a mapping $D^n \rightarrow D$) and predicate symbols (to each n-place predicate symbol assign a mapping $D^n \rightarrow \{\mathsf{T}, \mathsf{F}\}$).

Satisfiability: A formula φ is satisfiable if and only if there exists an interpretation I such that φ evaluates to true under I:

$$\varphi \text{ is satisfiable} \quad \Longleftrightarrow \quad \exists I \text{ such that } I \models \varphi.$$

Example 1 (Interpreting an AV policy as a logical formula). Reusing the policy stated in Equation 5.1, we can explain it in light of the current formalism as follows.

$$\underbrace{\varphi_{LF}(y)}_{\substack{\text{Predicate over} \\ \text{configuration } y}} \equiv \underbrace{lfX}_{\text{Term: gap to left-front}} > \underbrace{b_0\, lfV \;+\; b_1\, lfA \;+\; b_2}_{\text{Term: learned linear function}}, \tag{5.2}$$

where:

- y is the state vector including $\{lfX, lfV, lfA, \dots\}$,
- lfX, lfV, lfA are *terms* extracted from y (gap, relative velocity, relative acceleration),
- b_0, b_1, b_2 are constant coefficients *learned* from the highD dataset,
- $>$ is a binary predicate symbol forming an *atomic formula*.

According to the syntax of predicate logic:

- Each of lfX, lfV, lfA is a *term*,
- The expression $b_0\, lfV + b_1\, lfA + b_2$ is a *term*,
- The full formula $lfX > (b_0 lfV + b_1 lfA + b_2)$ is an *atomic formula* $P(t_1, t_2)$.

Thus, $\varphi_{LF}(y)$ is a *well-formed formula (wff)*.
$\varphi_{LF}(y)$ holds under an interpretation I if and only if

$$I \models \varphi_{LF}(y) \quad \Longleftrightarrow \quad I(lfX) > I(b_0\, lfV + b_1\, lfA + b_2).$$

That is, the actual left-front gap exceeds the learned threshold. □

Definition 1 (Compositional Policy). A compositional policy φ is a wff that combines simpler policies (or rules) $\varphi_1, \dots, \varphi_n$ using logical operators and/or quantifiers. Each constituent must itself be a wff.

Formally, let $\varphi_1, \dots, \varphi_n$ be wffs. Then:

$$\varphi ::= \varphi_i \mid \neg\varphi \mid (\varphi_1 \wedge \varphi_2) \mid (\varphi_1 \vee \varphi_2) \mid (\varphi_1 \rightarrow \varphi_2).$$

Example 2 (Lane-change decision policy: a compositional policy). Continuing the autonomous driving scenario from Section 5.3.1, we define the compositional policy for a lane change. At each decision step, the ego vehicle observes its dynamic *configuration* y, which includes the relative distances, velocities, and accelerations of these six neighbours.

We begin by defining six *atomic* policies, each of which checks a "safe-gap" condition against one of the six neighbours:

$$\varphi_{LF}(y),\ \varphi_{LP}(y),\ \varphi_{RF}(y),\ \varphi_{RP}(y),\ \varphi_{F}(y),\ \varphi_{B}(y)$$

Here, for example, $\varphi_{LF}(y)$ ($y = \{lfX, lfV, lfA\}$) asserts that the gap to the left-front vehicle exceeds a learned threshold, while $\varphi_{RP}(y)$ ($y = \{rfX, rfV, rfA\}$) asserts that the gap to the right-rear vehicle is safe, and so on.

Intuitively, the ego may change lanes to the left if both left-front and left-rear gaps are safe (safe left lane change) or to the right if both right-front and right-rear gaps are safe (safe right lane change)—and in either case, there must also be a sufficient gap in the current front and rear directions (opportunity) to ensure safe manoeuvre:

$$\varphi_{LCD}(y) \;=\; [\big(\varphi_{LF}(y) \wedge \varphi_{LP}(y)\big) \;\vee\; \big(\varphi_{RF}(y) \wedge \varphi_{RP}(y)\big)] \;\wedge\; \big(\varphi_{F}(y) \wedge \varphi_{B}(y)\big) \quad (5.3)$$

By Definition 1, this composite policy $\varphi_{LCD}(y)$ is a well-formed formula and is *satisfiable* precisely when there exists some configuration y (and an interpretation of any learned parameters) such that

$$I \models \varphi_{LCD}(y),$$

i.e. all of the required safe-gap conditions hold simultaneously for that state. The next section explains how the parameters for the linear functions used in the policy predicates can be mined from data. □

5.3.3 Policy Mining from Data

The policy mining methodology is shown in Algorithm 5. Here, the algorithm assumes as input the following:

- a wff compositional policy (φ) with constituent policies φ_l, which are wff. We assume that all wff policies have a fixed template of a linear inequality (with some parameters to be estimated). It is an inequality of the form:

$$\varphi_l : \mathcal{A}_i - (\mathbf{X}_i\mathbf{B}) \geq 0, \quad (5.4)$$

Algorithm 5 Policy-mining approach

1: **Input:** Dataset $\mathbf{D} = \{(\mathbf{X}_i, \mathcal{A}_i)\}_{i=1}^{N}$ based on the policy template φ_l, where $\mathbf{X}_i$ are features and $\mathcal{A}_i$ are label values. Target satisfaction percentage $\mathcal{T}$, maximum iterations $\mathcal{I}$.

2: **Output:** Optimized parameters $\tilde{\mathbf{B}}$, satisfaction percentage $\mathcal{S}$, and indices $\mathcal{I}_{\text{sat}}$ of satisfying datapoints.

3: **Step 1: Initial parameter Estimation via Ordinary Least Squares (OLS)**

4: Fit an glsols model using $\mathbf{D}$ to obtain initial parameters $\hat{\mathbf{B}}$:

$$\hat{\mathbf{B}} = \arg\min_{\mathbf{B}} \sum_{i=1}^{N} (\mathcal{A}_i - \mathbf{X}_i\mathbf{B})^2$$

5: **Step 2: Optimization**

6: Set the initial threshold $\mathcal{A}_{\text{fixed}}$ as:

$$\mathcal{A}_{\text{fixed}} = \sum_{j=1}^{k} \hat{B}_j \cdot \text{mean}(\mathbf{X}_j) - \min(\mathcal{A})$$

7: Define the objective function $\mathcal{F}(\mathbf{B})$:

$$\mathcal{F}(\mathbf{B}) = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\left(P \text{ is satisfied for } (\mathbf{X}_i, \mathcal{A}_i)\right)$$

8: Perform optimisation using Broyden-Fletcher-Goldfarb-Shanno (BFGS):

$$\tilde{\mathbf{B}} = \arg\min_{\mathbf{B}} \left(\sum_{i=1}^{N} \left(\mathcal{A}_i - (\mathbf{X}_i\mathbf{B} \pm \mathcal{A}_{\text{fixed}})\right)^2 \right)$$

Stop early if $\mathcal{F}(\tilde{\mathbf{B}}) \geq \mathcal{T}$.

9: Return $\tilde{\mathbf{B}}$, $\mathcal{F}(\tilde{\mathbf{B}})$, and satisfying indices $\mathcal{I}_{\text{sat}}$.

where $\mathcal{A}_i \in \mathbb{R}$ and $\mathbf{X}_i \in \mathbb{R}^k$ are both obtained from a dataset, $\mathbf{B} \in \mathbb{R}^{(k+1)\times 1}$ (where k is the number of features or predictors) is the set of parameters to be optimised, and i refers to the i^{th} datapoint;

- a dataset $\mathbf{D}$ consisting of features ($\mathbf{X} \in \mathbb{R}^{N\times(k+1)}$, where $k+1$ is the number of features and intercept, and N is the number of datapoints) and the corresponding label values ($\mathcal{A}$), extracted based on the policy template;

- a target satisfaction percentage $\mathcal{T}$, e.g. 98%, used both to measure how often the policy holds on the training set and to trigger early stopping once reached.

  Note: Choose $\mathcal{T} < 100\%$ to allow for modelling noise and ensure enough positive and negative examples, balancing high satisfaction with generalisation;

- a maximum number of iterations $\mathcal{I}$ (we need a maximum number of iterations even with a target satisfaction percentage because the target might never be reached).

The algorithm outputs the learned template parameters $\tilde{\mathbf{B}} = \{\tilde{b}_1, \ldots, \tilde{b}_k\}$, the empirical satisfaction rate $\mathcal{S}$—which may fall below or exceed the target $\mathcal{T}$ depending on data granularity—and the index set $\mathcal{I}_{\text{sat}}$ of all datapoints that satisfy the mined policy.

Now, once we get the initial estimation, the algorithm sets a fixed threshold $\mathcal{A}_{\text{fixed}}$. Essentially, $\mathcal{A}_{\text{fixed}}$ sets a baseline value that adjusts the inequality $\mathcal{A}_i - (\mathbf{X}_i\mathbf{B}) \geq 0$ so that the decision boundary reflects both the central tendency of the feature data and the lower bound of the labels. This calibration ensures that the subsequent optimisation is feasible and that the model is not overly restrictive.

Also, to guide the optimisation process, the objective function $\mathcal{F}(\mathbf{B})$ is defined as in line 7 of the algorithm, where the indicator function $\mathbf{1}$ determines whether the policy condition holds for a given data point. The goal is to maximise this function while refining the parameters. The algorithm employs the BFGS optimisation algorithm [157] to minimise the Mean Squared Error (MSE) between the predicted and actual value of $\mathcal{A}$. The optimisation continues until the satisfaction function $\mathcal{F}(\tilde{\mathbf{B}})$ reaches or exceeds the target percentage $\mathcal{T}$, at which point the process terminates early. This step is repeated for each wff in the overall policy (φ).

Note (Repeated Optimisation for Each wff): Algorithm 5 is repeated for each φ_l in the compositional policy φ, ensuring that each policy component is optimised to maximise satisfaction.

Example 3 (Policy Mining for the AV example). Continuing Example 2, we assume that Equation 5.3 defines the compositional policy. The constituent policy formulae (as shown in Equation 5.2) can be seen as a learned instantiation of the classical kinematic inequality:

The classical kinematic gap condition can be written as:

$$\text{gap} \;>\; v\,t \;+\; \tfrac{1}{2}a\,t^2, \tag{5.5}$$

where *gap* is the required safe distance between vehicles, v is the relative velocity, a is the relative acceleration, and t is the reaction time. This formula ensures that the following vehicle has enough space to react safely to changes in traffic.

In this interpretation, we identify gap $= lcfX$, $v = lfV$ and $a = lfA$. All other gap conditions, such as $\varphi_{LP}(y)$, $\varphi_{RF}(y)$, etc., are defined using analogous templates with corresponding feature subsets from y.

Thus, for the AV example, the algorithm 5 uses the dataset $\mathbf{D}$ obtained by extracting features $X_i = \{lcfV, lcfA, ..., bV, bA\}$, which contain the velocities and accelerations of cars, and labels $\mathcal{A}_i = \{lcfX, rcfX, ..., fX, bX\}$, which contains the relative gaps of cars, from the highD dataset from cars that lane change. The algorithm is then run with a certain target satisfaction percentage $\mathcal{T}$ and satisfaction percentage $\mathcal{S}$ for each $\varphi_i \equiv \varphi_{LF}, ..., \varphi_B$ to obtain the optimised values of the unknown coefficients (b_0, b_1, b_2).

□

The policy predicates in Equation 5.2 are not just learned from data. The *structure* of each constituent policy—which variables appear, the linear inequality template, and its connection to domain knowledge such as the kinematic gap condition in Equation 5.5—is fixed by the system designer before mining. Data is used to estimate the unknown coefficients (b_0, b_1, b_2, etc.) and to calibrate how often the predicate holds on real trajectories. We set the target satisfaction percentage $\mathcal{T}$ below 100% because highway recordings contain sensor noise, labelling uncertainty, and borderline manoeuvres; a strict 100% target would overfit rare outliers and leave too few counterexamples for training. The mined parameters therefore *augment* domain knowledge by fitting the template to empirical conditions (e.g. highD), rather than replacing first-principles constraints. In our AV case study, the template is grounded in gap–velocity–acceleration relationships. Coefficients that produce systematically unsafe gaps can be rejected by inspection, by holding out validation data. Ultimately, the runtime enforcer, as we discuss in Section 5.4, guarantees adherence to the mined policy at deployment regardless of ML prediction errors.

5.3.4 Compositional Model Design

Often, compositional policies are defined using a monolithic (a single large) ML model. In such instances, it is prudent to decompose the monolithic model into a compositional model. We propose a novel approach that allows decomposing a monolithic model based on the compositional policy that the monolithic model must satisfy. This approach is model-agnostic and can be used with any ML model.

Definition 2 (Compositional Multilevel Decomposition). Let $\mathcal{X}$ and $\mathcal{Y}$ denote the input and output spaces, respectively. A monolithic ML model is a function

$$M \colon \mathcal{X} \to \mathcal{Y}.$$

Given a compositional policy φ as defined earlier in Definition 1, expressed as a wff over a set of simpler policies $\{\varphi_1, \varphi_2, \ldots, \varphi_n\}$, the decomposition of M into

$\{M_1, M_2, \ldots, M_k\}$ is structured as a tree with leaf nodes and internal nodes. It is defined as follows.

Leaf Nodes of the Compositional Architecture: Each leaf node l corresponds to a well-formed formula (wff) φ_l. This wff may be atomic or itself a composite formula constructed using logical connectives (e.g., $\wedge$, $\vee$) over a finite set of sub-formulae $\{\varphi_i, \ldots, \varphi_j\}$. The node is associated with a sub-model

$$M_l \colon \mathcal{X} \to \mathcal{Y}_l,$$

which maps input configurations to a semantic domain $\mathcal{Y}_l$ over which the formula φ_l is interpreted.

We say that the sub-model M_l satisfies φ_l under interpretation I with respect to a set of inputs $\mathcal{X}_{\text{val}} \subset \mathcal{X}$ if

$$\forall x \in \mathcal{X}_{\text{val}}, \quad I \models \varphi_l\big(M_l(x)\big).$$

In practice, this is relaxed to approximate satisfaction, where we compute the empirical proportion of inputs for which the interpretation satisfies the formula:

$$\frac{|\{x \in \mathcal{X}_{\text{val}} \mid I \models \varphi_l(M_l(x))\}|}{|\mathcal{X}_{\text{val}}|} \geq P_l,$$

for some target percentage $P_l \in [0, 1]$.

The leaf nodes are the sub-models that will be trained to satisfy their respective formulae φ_l under this semantic framework.

Internal Nodes of the Compositional architecture: Each internal node v of the compositional architecture represents a *Merge* block. The *Merge* block combines two wffs using a specified connective $op \in \{\wedge, \vee\}$, based on the given policy.

$$(X, Y, op) = X \ op \ Y$$

where X, Y are wffs and $op \in \{\wedge, \vee\}$ is determined by the policy. □

The structure of the compositional policy guides the architecture of the decomposed system. Leaf nodes represent sub-models satisfying wff, while internal nodes combine the outputs of the leaf nodes or other internal nodes.

Example 4. (Compositional model: AV Case Study Example.) Let us decompose a single monolithic LCD model (considered in Example 2) into a compositional model. In the example, we had $\varphi_{LCD} : [(\varphi_{LF} \wedge \varphi_{LP}) \vee (\varphi_{RF} \wedge \varphi_{RP})] \wedge (\varphi_F \wedge \varphi_P)$ (with interpretation y). Thus, we can design the model with three leaf nodes, each with a separate model, as shown in Figure 5.3. Each leaf node is a wff. The models we obtain

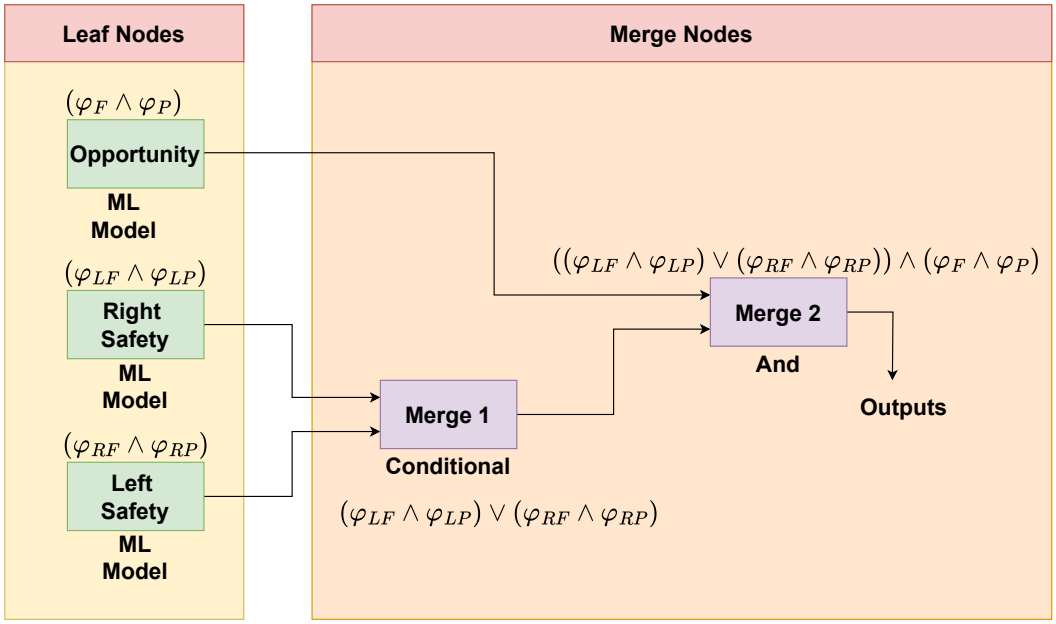


Figure 5.3: 3-stage compositional architecture of the LCD module.

are 1) the Left LC model to determine whether it is safe to execute a left lane change, 2) the Right LC model to determine whether it is safe to execute a right lane change, and 3) the Opportunity model to determine whether it is advisable/needed to perform a lane change. The leaf nodes are shown in green in the decomposed architecture in Figure 5.3.

Now, there will be two *Merge* blocks (shown in the colour purple in Figure 5.3) used to combine the outputs of the models at each stage: The first *Merge* block combines the outputs of the Left LC and Right LC models (using a disjunction operation; disjunction of the outputs $(\varphi_{LF} \wedge \varphi_{LP})$ and $(\varphi_{RF} \wedge \varphi_{RP})$, and the second *Merge* block combines the output of the Opportunity model and the output of the first *Merge* block (using a conjunction operation; conjunction of the outputs $(\varphi_{LF} \wedge \varphi_{LP}) \vee (\varphi_{RF} \wedge \varphi_{RP})$ and $(\varphi_F \wedge \varphi_P)$). The final output of the compositional architecture is the output of the second *Merge* block.

We could have also gone for a six-leaf node model, where each leaf node corresponds to wffs. However, we chose to keep it simple with three leaf nodes. □

We are aware that clean decomposition is not always possible, and this is an important practical limitation. Our framework assumes that the compositional policy φ can be written as a wff over constituent predicates, each associated with a leaf model whose outputs are checked for (approximate) satisfaction of that predicate. This applies directly when sub-policies concern distinct aspects of the same configuration y, as in the LCD example, where gap conditions for individual neighbours are composed logically.

When a policy genuinely spans several ML modules in a pipeline—for example, if a downstream module's safe operation depends on the *decision* produced by an upstream module—the designer should either (i) assign a separate compositional policy to each pipeline stage (as we do for the LCD, lane-change execution, and car-following modules in Section 5.5), with a system-level enforcer on the final control output, or (ii) retain a monolithic policy at the pipeline level and decompose only within a stage where the wff structure is clear. Cross-module dependencies are then captured by the serial composition of modules, not by splitting a single predicate across unrelated models.

5.3.5 Training of Compositional Models

Once we have the compositional model architecture and the model policies, we can train the ML models to emphasise satisfying their policies. One way to approach this problem is to check policy adherence during training and penalise the model when it does not adhere to the policy. However, this approach can be computationally expensive and may not be feasible for large datasets. Thus, we present an approach in Algorithm 6, where we do not do policy satisfaction checks inside the training loop and instead do it at the data preprocessing stage. This approach may reduce the number of overall training data points, but it requires less time and resources to train the models.

Training algorithm: The algorithm takes as input a training dataset $\mathbf{D}$, optimized parameters $\tilde{\mathbf{B}}$, satisfying indices $\mathcal{I}_{\text{sat}}$ (from Algorithm 5), and a compositional policy φ as defined in Section 5.3.3. It also assumes the model designer has provided a decomposition of φ so that leaf models and merge blocks can be correctly trained. The outputs are the trained compositional model $M_{\text{compositional}}$ and its accuracy $\mathcal{P}$.

Step 1: Data Filtering

The dataset $\mathbf{D}$ is split using $\mathcal{I}_{\text{sat}}$ into satisfying ($\mathbf{D}_{\text{sat}}$) and non-satisfying ($\mathbf{D}_{\text{unsat}}$) subsets. Their union forms $\mathbf{D}_{\text{combined}}$, which serves as input for model training.

Step 2: Leaf Model Training

A separate model M_l is trained for each leaf node based on the wffs in φ, using $\mathbf{D}_{\text{combined}}$. Parameters θ_l are optimised to minimise a loss function $\mathcal{L}$ via mini-batch gradient descent. A 5-fold cross-validation scheme is used for generalisation, with optional hyperparameter tuning to improve performance.

Step 3: Compositional Model Construction

The trained leaf models are composed using the logical structure of φ to form the full compositional model $M_{\text{compositional}}$. More complex policies (i.e., more wffs and connectives) yield larger models.

Step 4: Accuracy Evaluation

Algorithm 6 Compositional Policy-Based Training

Input: Dataset $\mathbf{D}$, optimized parameters $\tilde{\mathbf{B}}$, satisfying indices $\mathcal{I}_{\text{sat}}$, compositional policy φ.
Output: Trained compositional model $M_{\text{compositional}}$ and final Accuracy $\mathcal{P}$.
Step 1: Data Filtering
Partition $\mathbf{D}$ into subsets based on satisfaction:

$$\mathbf{D}_{\text{sat}} = \{(\mathbf{X}_i, y_i) \in \mathbf{D} \mid \varphi \text{ is satisfied}\}$$

$$\mathbf{D}_{\text{unsat}} = \{(\mathbf{X}_i, y_i) \in \mathbf{D} \mid \neg\varphi \text{ is satisfied}\}$$

Form the combined dataset:

$$\mathbf{D}_{\text{combined}} = \mathbf{D}_{\text{sat}} \cup \mathbf{D}_{\text{unsat}}.$$

Step 2: Leaf Model Training
for each leaf node l corresponding to wff φ_l **do**
    Construct model $M_l \colon \mathcal{X} \to \mathcal{Y}_l$.
    Train M_l on $\mathbf{D}_{\text{combined}}$:

$$M_l = \arg\min_{\theta_l} \sum_{(\mathbf{X}_i, y_i) \in \mathbf{D}_{\text{combined}}} \mathcal{L}(M_l(\mathbf{X}_i; \theta_l), y_i),$$

    where $\mathcal{L}$ is the loss function.
end for
Step 3: Compositional Model Construction
Assemble the overall compositional model $M_{\text{compositional}}$ by connecting the trained leaf models $\{M_l\}$ using the merge connectives as defined in the compositional multilevel decomposition.
Step 4: Accuracy Evaluation and Update
Evaluate the accuracy $\mathcal{P}$ of $M_{\text{compositional}}$ on $\mathbf{D}_{\text{combined}}$:

$$\mathcal{P} = \frac{|\{(\mathbf{X}_i, y_i) \in \mathbf{D}_{\text{combined}} \mid \varphi\big(M_{\text{compositional}}(\mathbf{X}_i)\big) \text{ is satisfied}\}|}{|\mathbf{D}_{\text{combined}}|}$$

Update the parameters of the leaf models and merge blocks to maximise $\mathcal{P}$ until convergence.
Return $M_{\text{compositional}}$ and $\mathcal{P}$.

Model performance is assessed using accuracy $\mathcal{P}$—the proportion of correctly classified datapoints after inference. A 5-fold cross-validation is performed using the same splits as in Step 2 for consistency.

Please note that approximate leaf-level satisfaction does not compose into an exact system-level violation probability without additional assumptions (e.g. independence of sub-model errors). In this thesis, we derive decomposed guarantees methodically in three steps: mine and validate each φ_l to a chosen $\mathcal{T}$; train and compose sub-models according to the Boolean structure of φ; and measure end-to-end satisfaction of φ on

held-out data and in simulation, where runtime enforcement further bounds the impact of residual violations. The empirical performance of the whole compositional model on validation data is used as a proxy for the exact system-level violation probability, instead of propagating per-leaf satisfaction rates analytically.

5.3.5.1 Monolithic Modelling

The training process described in Section 5.3.5 can also be used to train monolithic models. The only difference is that a single ML model is trained on the filtered dataset $\mathbf{D}_{\text{combined}}$. The trained monolithic model is then evaluated on the dataset using 5-fold cross-validation to obtain its accuracy.

Proposition 1 (Compositional Policy Satisfaction). Let φ be a compositional policy (a well-formed formula as in Definition 1) over a domain D with interpretation I defined over D, and let $M_{\text{compositional}} : \mathcal{X} \to D$ be a model constructed from trained sub-models $\{M_i\}$ and *Merge* blocks (boolean operators $\wedge$ and $\vee$) corresponding to the syntactic structure of φ.

Suppose that for a finite input dataset $\mathcal{X}_{\text{val}} \subset \mathcal{X}$ of size $N = |\mathcal{X}_{\text{val}}|$, the formula φ evaluates to T under interpretation I for at least PN elements of the image of $M_{\text{compositional}}$, where $P \in [0, 1]$.

Then we say that $M_{\text{compositional}}$ satisfies φ with empirical satisfaction level P:

$$\frac{|\{x \in \mathcal{X}_{\text{val}} \mid I \models \varphi(M_{\text{compositional}}(x))\}|}{|\mathcal{X}_{\text{val}}|} \geq P.$$

Appendix B.1 contains the informal proof of the proposition.

5.4 Runtime Enforcement for Robustness to Unseen Scenarios

As most ML models are black-box in nature, we cannot guarantee that the model will adhere to the policy when it encounters new but similar data. Hence, we employ runtime enforcement mechanisms to ensure that the model always adheres to the policy in untrained and unexpected scenarios.

Using formal runtime monitoring approaches and tools, verification/enforcement monitors (enforcers) can be synthesised from the formal requirements of desired policies, which can be integrated with the system. The monitor is fed with the current execution of the system. It verifies the current execution with respect to the specified policies and enforces (corrects) the current execution in the case of violation when dealing with the enforcement of the desired safety policies at runtime (see Figure 5.4).

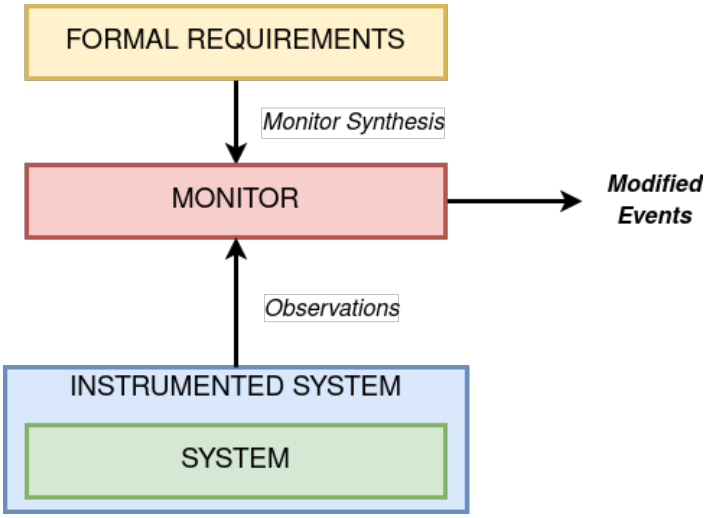


Figure 5.4: An overview of the runtime enforcement process.

5.4.1 Enforcer Synthesis

In our work, we use the enforcer synthesis tools that are based on the approaches proposed in [25, 154] that are suitable for safety-critical and cyber-physical systems. The framework in [25] supports RE monitor synthesis for timed policies. Valued Discrete Timed Automata (VDTA) [25] is used for the formal specification of policies, where valued signals, internal variables, and complex guard conditions are supported, ensuring compatibility with real-world CPS and industrial systems and enforcement monitors are synthesised from these VDTA to enforce a set of desired policies.

Prelimineries to RE. A (finite) word is a finite sequence of elements of a finite alphabet Σ. The length of w is the number of elements in w and is denoted as $|w|$. The sets of all words are denoted by Σ^*. A language (property) over Σ is any subset of Σ^* and is denoted by φ.

Let $\mathbb{R}$ denote the set of real numbers. $M : \mathbb{R}^n \to \mathbb{R}^m$ is a function which takes as input a vector of n real valued numbers and gives as output a vector of m real valued numbers. This function represents an ANN model.

Enforcer for φ. An enforcer (from [25]) for a property φ is a function $E_\varphi : \Sigma^* \to \Sigma^*$ that takes as input a word Σ^* and outputs a word that satisfies φ or can be extended to satisfy φ in the future. It can only edit an input event when necessary, and it cannot block, delay or suppress events.

Some constraints are required on how an enforcer transforms words. These constraints describe the expected behaviour of an enforcer and are recalled below (Def. 3.1 from [154]).

Definition 3. Given property φ, an enforcer for φ satisfies the following constraints:

Soundness:

$$\forall \sigma \in \Sigma^*, \exists \sigma_{extn} \in \Sigma^* : E_\varphi(\sigma) \cdot \sigma_{extn} \models \varphi$$

Monotonicity:

$$\forall \sigma, \sigma_{extn} \in \Sigma^* : \sigma \preccurlyeq \sigma_{extn} \implies E_\varphi(\sigma) \preccurlyeq E_\varphi(\sigma_{extn}).$$

where $\sigma \preccurlyeq \sigma_{extn}$ means that σ is a prefix of σ_{extn}.

Transparency:

$$\forall \sigma, a, \sigma_{extn} \in \Sigma^* : E_\varphi(\sigma) \cdot a \cdot \sigma_{extn} \models \varphi \implies E_\varphi(\sigma \cdot a) = E_\varphi(\sigma) \cdot a.$$

Soundness means that for any input word, the output of the enforcer can be extended to a sequence that satisfies φ. *Monotonicity* means that the output is a continuously growing word: the output of the enforcer for an extended input word σ_{extn} of an input word σ, extends the output produced by the enforcer for σ. *Transparency* [1] means that the enforcer will not unnecessarily edit any event, i.e., for any given input sequence σ and any input event a, if the output of the enforcer for σ (i.e., $E_\varphi(\sigma)$) followed by the input event a has an extension σ_{extn} such that $E_\varphi(\sigma) \cdot a \cdot \sigma_{extn}$ satisfies the property φ, then the output that the enforcer produces for input $\sigma \cdot a$ will be $E_\varphi(\sigma) \cdot a$.

Some other constraints are required, specifically to respect synchronous execution in reactive systems, such as *instantaneity* and are recalled below (Def. 3.1 from [154]).

Definition 4. Given property φ, an enforcer for φ satisfies the following constraint:

Instantaneity:

$$\forall \sigma \in \Sigma^* : |\sigma| \;=\; |E_\varphi(\sigma)|$$

Instantaneity means that for any given input sequence, the length of the output sequence of the enforcer should be the same as the input sequence. This means that the enforcer cannot delay, insert and suppress events. Whenever the enforcer receives a new input event, it must react instantaneously and produce an output event immediately. This requirement is essential for the enforcement of CPSs, which are reactive in nature.

Edit Functions. Given property φ, we introduce $edit_\varphi$, which the enforcer uses for editing input events (whenever necessary) to satisfy property φ. This definition is an adapted version of the edit function presented in Section 2.2 of [154]. The adaptation eliminates bidirectionality, resulting in a purely unidirectional enforcer.

[1] [25] presents a bidirectional enforcer, whereas our approach focuses on a unidirectional enforcer. Our enforcer corrects inputs from the system in a single direction and is synthesised using the monitor synthesis approach outlined in [25]. Consequently, the transparency constraint defined in Def. 3 of our work is slightly modified compared to those in Def. 3.1 of [154].

Definition 5 (Edit function). Given an input word $\sigma \in \Sigma^*$ and an input event $x \in \Sigma$, $edit_\varphi(\sigma, x)$ is the set of events $x' \in \Sigma$ such that the word obtained by extending σ with x' can be extended to a sequence that satisfies the property φ (i.e., $\exists \sigma_{extn} \in \Sigma^*$ such that $\sigma \cdot x' \cdot \sigma_{extn} \models \varphi$). Formally,

$$edit_\varphi(\sigma, x) = \{x' \in \Sigma : \ \exists \sigma_{extn} \in \Sigma^*, \sigma \cdot x' \cdot \sigma_{extn} \models \varphi\}$$

Two proposed methods for editing non-accepting input values are given in [154]. These are either a random edit, where a random but accepting value would be chosen from a list of accepting values, or a minimum distance edit, where an accepting value would be chosen with minimum binary distance compared to the current non-accepting value.

Remark (Soundness, Monotonicity, Transparency and Instantaneity). It has been proved that the enforcer synthesised using the approach outlined in [154] satisfies soundness, monotonicity, transparency and instantaneity constraints given in Def. 3 and 4.

Since our system is composed of (ML) models connected sequentially or in parallel, with an enforcer connected after the entire system, we define properties, such as *inter-model consistency* and *robustness*.

The concept of *inter-model consistency* arises from the observation that, in a decomposed ML system, multiple smaller models $M_1, \cdots M_k$ are interconnected. The output of one or more models often serves as the input to the next, forming a sequence of dependencies. The enforcer operates after the entire system, ensuring that the final output satisfies a global property φ. The intuition behind *inter-model consistency* is that the relationships and dependencies between sub-models $M_1, \cdots M_k$, are conserved by the enforcer E_φ even after corrections and do not contradict each other after correction.

The concept of *robustness* arises from the observation that an ML model may fail to handle adversarial or unexpected inputs, potentially resulting in unsafe or unreliable behaviour. By placing an enforcer after the (decomposed) model, the system ensures operation within predefined bounds, even in scenarios that were not covered during training. In essence, an enforcer acts as a runtime safeguard in managing edge cases and adversarial inputs effectively.

Definition 6. Given n decomposed models $M_1, \cdots M_n$, given input vector $x_1, \cdots, x_n$ to each model and given an enforcer E_φ for property φ, placed after the entire decomposed model, whose output is obtained by E_φ^{out}. The system satisfies the following constraints:

Inter-model consistency:

$$E_{\varphi}^{out} \models \varphi \implies \Big((\forall M_j : M_i \to M_j) \implies M_i \sim M_j\Big)$$

where

- $M_i, M_j \in \{M_1, \cdots, M_n\}$
- $M_1 \sim M_2 \iff \big((O(M_1) \wedge I(M_2) \neq \emptyset) \wedge\ O(M_1) \in \text{ValidOutput}(M_1) \wedge\ I(M_2) \in \text{ValidInput}(M_2)\big)$

Robustness:

$$\forall M_i \in \{M_1, \cdots, M_n\}, \forall \sigma_{adv} \in \Sigma^*, \Big(M_i(\sigma_{adv}) \implies E_{\varphi}^{out} \models \varphi\Big)$$

Inter-model consistency expresses that Given a model M_j whose input depends on outputs of some model M_i (there can be more such models), then the relationships and dependencies between the dependent model M_j and model M_i, are preserved by the enforcer if φ is satisfied by the final output of the enforcer ($E_{\varphi}^{out} \models \varphi$).

Here, $\sim$ denotes compatibility. Two models M_1 and M_2 are compatible ($M_1 \sim M_2$) if the following conditions are met: (1) M_1's outputs are relevant for M_2's processing ($O(M_1) \wedge I(M_2) \neq \emptyset$); (2) M_1's outputs are valid ($O(M_1) \in ValidOutput(M_1)$); (3) M_2's inputs are valid ($I(M_2) \in ValidInput(M_2)$). By valid, we mean that the domain and range (the structural expectations) of the input and output data remain unchanged.

In summary, if the enforcer guarantees that the property φ is satisfied, then the compatibility relationship $M_i \sim M_j$ is maintained between the dependent model M_j and the model M_i on which it relies.

Robustness implies that for any model M_i and any adversarial input σ_{adv}, the enforcer ensures that its output satisfies the property φ (i.e., $E_{\varphi}^{out} \models \varphi$) even when M_i processes σ_{adv}.

This establishes that the enforcer effectively mitigates the impact of any adversarial inputs on the overall system behaviour, ensuring compliance with φ.

Proposition 2 (Inter-model consistency, Robustness). Given a property φ, its enforcer E_{φ} satisfies Inter-model consistency and Robustness constraints as per Def. 6.

Appendix B.2 contains the informal proof of Proposition 2.

5.5 Case Study

In the following subsections, we briefly explain the various modules we consider in the AV case study and define the policies we aim to enforce on these modules. Moreover,

we use different models for different modules to truly represent a real-world scenario and demonstrate the flexibility of our approach.

5.5.1 Case Study Modules

5.5.1.1 Lane Change Decision Module

The LCD is an ML model that decides when it is opportune and safe to change lanes. We experiment with both monolithic and compositional models with various kinds of ML models. In the compositional ML model with three stages, as shown in Figure 5.3, built as discussed in section 5.3.4, each model takes its corresponding car states and performs necessary computations to generate a decision on the lane change.

All three models output a binary variable (0 or 1) as output, indicating whether the car should lane change or not. While the first merge block also outputs a binary decision (0 or 1), the final merge block outputs -1, 1, or 0, indicating whether the car should lane change to the left, right, or stay in the existing lane, respectively.

5.5.1.2 Car Following Module

We use the Intelligent Driver Model (IDM) model [100, 101] for Car Following (CF). The IDM model is a car-following model that describes the dynamics of a vehicle following another vehicle. The model is based on the assumption that the vehicle follows the vehicle in front by maintaining a safe distance. The model provides the car's acceleration based on its current velocity, the distance between the vehicle and the vehicle in front, and the desired velocity. The IDM model is used in this work for its simplicity and ability to include driver preferences when modelling car-following behaviour.

5.5.1.3 Lane Change Execution Model

The LCE model is a simple polynomial model that describes the vehicle's lateral position during a lane change. The model is based on the assumption that the vehicle changes lanes by following a polynomial trajectory. The lateral velocity is computed by taking the derivative of the polynomial that describes the lateral position as a third-degree polynomial function of time.

5.5.1.4 Controller Module

The Autonomous Vehicle (AV) controller operates through three primary states: Car Following (CF), Waiting (Wait), and LCE, with transitions governed by input signals, time constraints, and internal counters.

Initially, the system remains in the Car Following (CF) state unless a LCD signal is received, indicating a left or right lane change intent. Upon receiving such a signal, the controller transitions to the Waiting (Wait) state and initialises a timer. This state is an intermediate phase to prevent abrupt transitions by ensuring the LCD signal persists. If the signal remains active for over one second, the controller proceeds to LCE; otherwise, it reverts to Car Following (CF) if the LCD signal returns to zero.

In the LCE state, the controller executes the lane change manoeuvre. The system remains in this state as long as the LCE signal is active. Upon completion of the lane change, when the LCE signal switches to zero, the controller transitions back to Car Following (CF), restoring its baseline operation.

5.5.2 Policies

5.5.2.1 System-Level Policy

The system-level policy for the AV case study we consider is simple. We want to ensure the AV runs safely on a freeway/motorway while executing human-like lane changes. The policy can be decomposed into two overarching sub-policies.

- $\tilde{\varphi}_{LC}$: Safe human-like lane changes
  - $\tilde{\varphi}_{LCD}$: Safe human-like lane change decision
    - φ_{LF}: Safe human-like lane change decision for left front car.
    - φ_{LP}: Safe human-like lane change decision for left rear car.
    - φ_{RF}: Safe human-like lane change decision for right front car.
    - φ_{RP}: Safe human-like lane change decision for right rear car.
    - φ_{F}: Safe human-like lane change decision for front car.
    - φ_{P}: Safe human-like lane change decision for back car.
  - $\tilde{\varphi}_{LCE}$: Safe human-like lane change execution.
- $\tilde{\varphi}_{CF}$: Safe Car following

The composition of these policies ensures that the AV does not collide with any other vehicle on the road.

$$\begin{aligned} \tilde{\varphi}_{LC} &= \tilde{\varphi}_{LCD} \wedge \tilde{\varphi}_{LCE} \\ \tilde{\varphi} &= \tilde{\varphi}_{LC} \wedge \tilde{\varphi}_{CF} \end{aligned} \tag{5.6}$$

As the goal here is to show the compositional policy mining, training, and enforcement, we only focus on three safety policies ($\tilde{\varphi}_{LCE}, \tilde{\varphi}_{LCD}, \tilde{\varphi}_{CF}$), one for each module, for illustration. More policies can be defined, but the composition will be similar. The following subsections define the policies in detail as a VDTA.

5.5.2.2 Safety Policy for Discretionary Lane Change ($\tilde{\varphi}_{lcd}$)

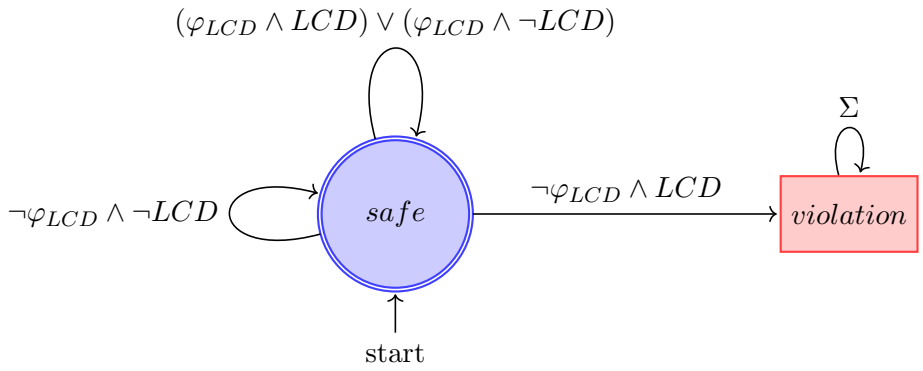


Figure 5.5: Safety policy for discretionary lane change module.

Figure 5.5 shows the policy for the LCD module. Here, $\varphi_{LCD} = ((\varphi_{LF} \wedge \varphi_{LB}) \vee (\varphi_{RF} \wedge \varphi_{RB})) \wedge (\varphi_F \wedge \varphi_{CB})$, where φ_i are policies defined on car gaps and mined using Algorithm 5. If $\varphi_{LCD} = 1$, it is safe to lane change, if $\varphi_{LCD} = 0$, it is not. $LCD = 1$ for lane change and $LCD = 0$ for *No lane change* are the possible outputs from the LCD module. The policy says that we move to a violation state if φ_{LCD} is false and LCD is true, i.e. when the LCD module's output suggests a lane change, but the policy does not think it is safe to lane change, we enter the violation state. Once we enter the violation state, we can recover by setting the LCD to 0, i.e. by making the LCD module output a *no lane change*.

5.5.2.3 Safety Policy for Car Following ($\tilde{\varphi}_{CF}$)

Figure 5.6 presents a policy for the car following module, depicting three distinct states: **acc** (acceleration), **brake**, and **violation**. From either the **acc** or **brake** states, a transition to the **violation** state is triggered if φ_{CF} is false while the acceleration is non-negative in the **acc** state or if, in the **brake** state, the acceleration is greater than the critical threshold ($a > -\frac{u^2}{2} \times gap$). Once we enter the violation state, we can recover by setting the acceleration setpoint (a) to brake and equal to the $-u^2/2 * gap$, which should allow the car to come to a complete stop. Here, u is the velocity of the ego vehicle, $\varphi_{CF} = gap > 2m$ and gap is the distance between the ego vehicle and the front vehicle.

5.5.2.4 Safety Policy for Lane Change Execution ($\tilde{\varphi}_{lce}$)

Figure 5.7 illustrates the safety policy for the LCE module. In this diagram, LCE represents the Lane Change Execution signal, where $LCE = 1$ indicates an ongoing lane change and $LCE = 0$ indicates no active lane change. The variable x tracks the duration of the lane change. The system's behaviour is described as follows:

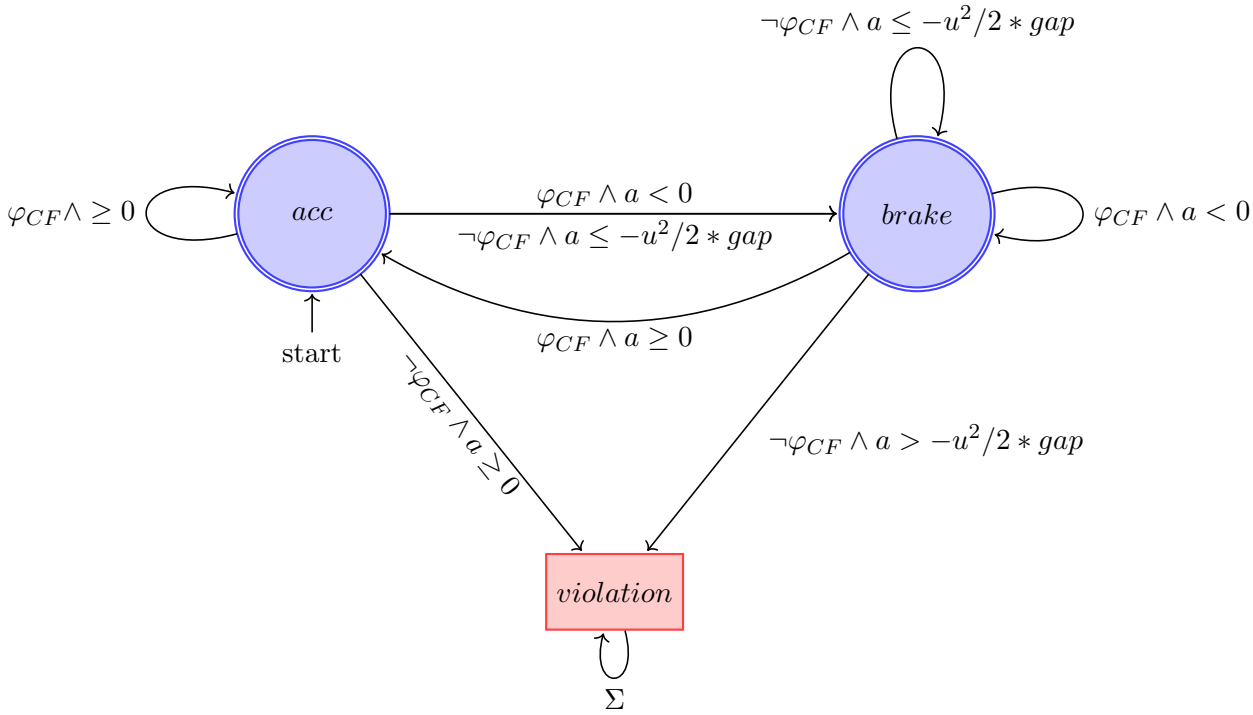


Figure 5.6: Safety policy for car following module.

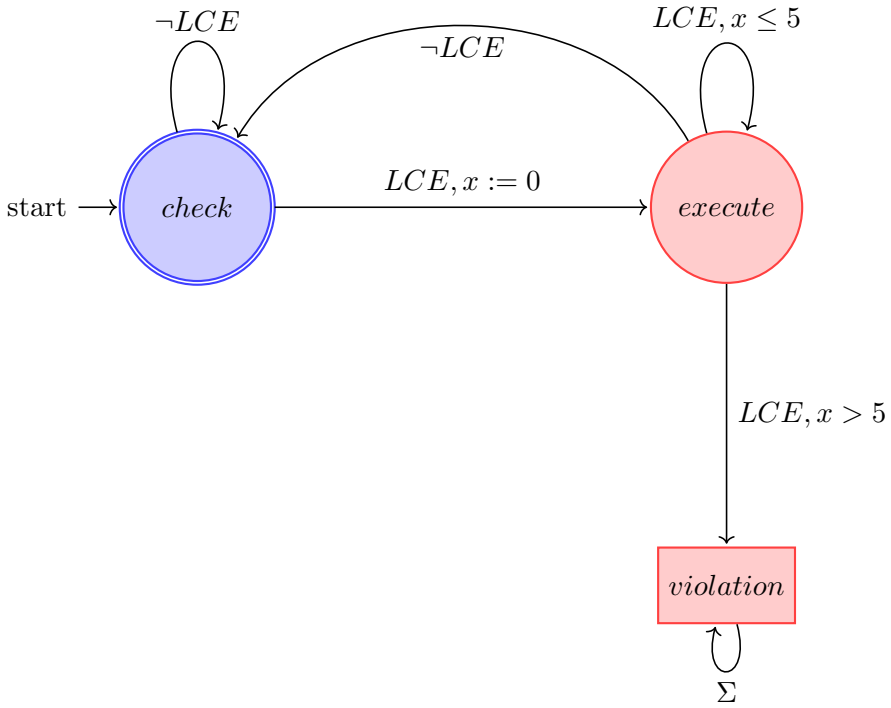


Figure 5.7: Safety policy for the lane change execution module.

The policy states that if it takes longer than 5 seconds to complete the lane change, we enter the violation state, as something may be wrong with the execution. This time duration for a lane change is chosen based on the average lane change duration found in the real-life dataset used to train the LCD module. If we enter the violation state, the

enforcer makes the LCE signal inactive, thereby stopping the lane change execution and moving it to the Car Following state.

5.6 Results

Once we obtain the coefficients for our mined policies and have the trained models and enforcers, we implement them in a Python simulation that mimics the ego/subject AV driving on a freeway. The results of the compositional training and the simulation are presented in this section.

5.6.1 Compositional Policy-Based Training

We use the policy mining technique discussed in Section 5.3 to obtain the policy φ_{LCD} used in the LCD enforcer. The policies are obtained using data from the well-known German highway dataset - highD [155]. We train the LCD ML models to classify between safe (φ_{LCD} is satisfied) and unsafe lane changes (φ_{LCD} is not satisfied).

We experiment with three kinds of classification models in the compositional design: ANN (Artificial Neural Networks), Extreme Gradient Boosting (XGB) [158], and a combination of Adaptive Boosting (AdaBoost) [159] and Random Forest [160] (AdaBoost+Random Forest (RF)) models to observe the effect of model choice. Moreover, we try four target satisfaction percentage levels (No Policy, 70%, 80% and 90%) to observe their effects (from Algorithm 5). *No Policy* refers to a case with no policy-guided training, and a subset of the whole dataset is randomly chosen for training.

Also, we built monolithic models with similar configurations to compare the performance of the compositional models. As our goal here is to show the effect of policies on compositional model development and enforcement, we do not tune the models to obtain optimal performance. The chosen model parameters are available in Appendix B.4.

5.6.1.1 Learnt Policies

Equation 5.7 presents the mined policies for the model built on a 70% satisfaction target percentage in the simulations (i.e. target percentage with the least number of faults).

$$
\begin{aligned}
\varphi_{LF} &: lfX > 80.77 + 11.23 * lfV + 9.72 * lfA \\
\varphi_{LB} &: lpX < -18.13 - 2.82 * lpV - 2.65 * lpA \\
\varphi_{RF} &: rfX > 16.00 - 3.21 * rfV - 1.51 * rfA \\
\varphi_{RB} &: rpX < -35.91 + 4.20 * rpV - 1.89 * rpA \\
\varphi_{F} &: fX > 0.45 + 31.14 * fV - 12.59 * fA \\
\varphi_{B} &: pX < -22.73 - 1.01 * pV + 4.52 * pA
\end{aligned}
\tag{5.7}
$$

We can observe in Equation 5.7 that half the policies have a greater than sign and half have a less than sign. This is because the relative positions of the cars are considered in the policies, and the sign of the gap changes based on the car's position relative to the subject vehicle (i.e., car-subject car). This also explains why the front car policies have positive intercepts (the first coefficient term), and the rear/back car policies have negative intercepts. Also, we can glean from the policies that the cars, on average, need more space on the front left owing to the large positive coefficients of φ_{LF}. Other policies also show a similar behaviour.

Table 5.1: Mean 5-fold Cross-validation Accuracy (%) of Models

Model	**Type**	**70**	**80**	**90**	**No Policy**
ANN	Mono	92.073	91.641	87.060	71.501
	Comp	89.500	89.325	83.168	66.311
AdaBoost+RF	Mono	91.560	90.645	87.834	61.061
	Comp	90.991	91.201	87.306	65.821
XGB	Mono	95.398	95.422	92.408	74.561
	Comp	93.098	93.104	88.458	70.223

Table 5.1 shows the mean accuracy of the models over the 5-fold cross-validation for all models. We can see that the accuracy of the models is generally higher for the monolithic models, except for a few cases in the AdaBoost+RF models. Statistical T-tests on the accuracies for both monolithic (Mono) and compositional (Comp) models also reveal that the monolithic models have a statistically higher accuracy (the p-value is 0.016 at an α of 5%). It seems that the compositional models (which are architecturally more complex) do not learn the relationships in data as well as the monolithic models. Model hyperparameter tuning may help to obtain more performant compositional models.

5.6.2 Simulation Results

The simulation setup used to evaluate the trained models and enforcers is described in Section B.3. We inject faults at regular intervals in the simulation to observe

Table 5.2: Mean and standard deviation of runtime faults for the LCD module (10 runs).

Model	Type	Statistic	70	80	90	No Policy
ANN	Mono	mean	43.150	107.275	307.800	43.150
		std	16.158	62.251	195.716	16.158
	Comp	mean	7.200	2.800	8.950	19.875
		std	0.000	0.000	0.141	8.896
AdaBoost+RF	Mono	mean	3.600	3.200	738.487	743.550
		std	0.000	0.000	472.865	470.663
	Comp	mean	9.500	3.200	626.112	585.400
		std	0.000	0.000	374.425	366.411
XGB	Mono	mean	10.100	4.900	293.988	957.200
		std	0.000	0.000	183.176	610.441
	Comp	mean	9.200	3.400	392.100	466.350
		std	0.000	0.000	221.362	299.387

the behaviour of the AV under simulated unsafe conditions. The modules may also produce unsafe behaviour on their own, which we call runtime faults. During both unsafe behaviours (recorded over 10 random iterations), we expect the enforcer to act and take corrective action. We report these observations as follows.

We can see from Table 5.2 that the LCD module, which is composed of ML models, produced several runtime faults (total faults-injected faults) that activated the LCD enforcer. The number of faults varied across the different configurations of the compositional training. However, the number of faults is generally lower for the compositional models. This observation suggests a trade-off between the number of enforcer activations and the policy satisfaction target percentage. In particular, it can be seen that too high a target percentage and too low a target percentage (No Policy dataset) can lead to more enforcer activations, indicating that the model overfits the training data and cannot generalise well if the target percentage is too high or too low.

We also perform a statistical T-test on the LCD model faults to determine if there is a statistical difference between the values of the monolithic and compositional models. The T-test results show that the compositional models have a statistically significantly lower number of faults (with a p-value of 0.034 at an α of 5%) than the monolithic models. We also test the difference between faults seen in the policy and the No Policy models. The T-test results show that the N Policy models have a statistically significantly higher number of faults (with a p-value of 0.034 at an α of 5%). We also find that the statistical effect sizes measured using Cohen's d are moderate to high, suggesting that the differences in faults are meaningful. This result suggests that the compositional models (resp. policy trained models) are better at generalising

to unseen data than the monolithic (resp. No Policy trained) models and ensuring safer behaviour.

Additionally, the CF module, which is a stateful mathematical model, produced far fewer runtime faults compared to the ML-based LCD module. The mean number of faults over the ten random simulation runs is 0.187. The simple deterministic nature of the CF module, which is governed by strict mathematical rules, does not have the same generalisation issues as the ML models. Similarly, the mathematical LCE polynomial model produced no runtime faults in the simulation run. The number of injected faults is the same for all modules.

We also observe from the simulation logs (which we do not present here for brevity) that the gaps between cars do not become zero at any point for all cases considered, indicating that the enforcers' corrective actions are sufficient to maintain safe behaviour. The logs also show that the subject vehicle executes lane changes where appropriate and does not get stuck in deadlock situations where the vehicle does not know what to do. We do not report these results numerically for brevity.

5.6.3 Discussion

The training and simulation results show a clear split between predictive metrics on held-out cross-validation and safety-related behaviour under fault injection. Table 5.1 indicates that monolithic models achieve higher classification accuracy than compositional models for ANN and XGB ($p = 0.016$). Table 5.2, however, shows fewer runtime faults for compositional models in most configurations ($p = 0.034$), and policy-guided training yields fewer faults than the No Policy baseline. ML-based modules are therefore prone to unsafe runtime behaviour when policies are not used to shape training, even when accuracy appears strong. By contrast, the mathematical car-following and lane-change execution modules produce far fewer faults (mean 0.187 and zero runtime faults, respectively), reflecting their deterministic structure, but they depend on assumptions about module behaviour and offer less flexibility to learn variability from data.

For the LCD module specifically, the lower fault counts of compositional, policy-trained models suggest that classification accuracy alone is not a reliable indicator of how the system will behave when disturbances are injected. Compositional architectures together with policy-filtered datasets emphasise adherence to mined gap conditions; this appears to generalise better under simulation stress than maximising accuracy on the cross-validation split. The trade-off between target satisfaction percentage and faults is also visible in the results: very high or very low targets (90% or No Policy) tend to coincide with more enforcer activations, whereas the 70% set-

ting used for the policies in Equation 5.7 balances fit to highD and stable runtime behaviour.

The mined coefficients in Equation 5.7 illustrate how empirical data refines, rather than replaces, the policy structure fixed in Section 5.3. Each predicate keeps a kinematic template; highD trajectories determine the numerical thresholds. Because leaf models are trained for approximate satisfaction of sub-formulae and coefficients are themselves empirical estimates, system-level safety cannot be obtained by composing per-leaf satisfaction rates in closed form. The simulation outcomes provide the practical assessment instead: policy-trained compositional models reduce faults, simulation logs maintain non-zero gaps between vehicles, and the LCD enforcer corrects residual violations at runtime. Together with the workflow in Section 5.3.4—mine constituents, compose models, evaluate with enforcement—these results show how approximate local policies can still support dependable end-to-end behaviour when validation moves from static accuracy to dynamic simulation.

The structure of the LCD decomposition also explains why the framework is applicable in some pipeline settings but not all. Within the decision module, φ_{LCD} is split into left, right, and opportunity sub-formulae (Figure 5.3) because each predicate refers to a distinct aspect of the same traffic configuration y. Where safety depends on the output of an upstream module, the case study assigns separate policies to LCD, LCE, and car following and applies enforcement on the final control output in the serial simulation described in Section B.3. The comparatively low fault rate of the non-ML modules indicates that compositional policy training and runtime enforcement add the most value alongside learning-based components, which dominate the residual risk observed in Table 5.2.

The three-leaf LCD layout follows driving semantics (lateral direction and opportunity) rather than an exhaustive search over alternative partitions; a six-leaf design per atomic gap condition was not evaluated here. The simulation supports the chosen decomposition but does not rule out other architectures. Overall, the results indicate that a lightweight runtime enforcement layer, combined with compositional policy-based training, can improve safety relative to monolithic and unguided training while accepting a modest reduction in cross-validation accuracy. Policy mining remains limited to linear templates in this chapter, and broader validation on additional datasets would strengthen claims about generalisability.

5.7 Summary

In this chapter, we introduced a model-agnostic policy-driven approach to compositional ML-based safety-critical Cyber-Physical Systems (CPSs). Given policy templates and data, we mine compositional properties, train compositional ML models

(Artificial Neural Network (ANN), XGB, AdaBoost+RF) under satisfaction tolerances, and deploy runtime enforcers. We validated the framework on an AV simulation with lane-change decision, lane-change execution, and car-following modules.

The methodology extracts meaningful linear policies from highD, and simulation shows that policy-based compositional training reduces unsafe runtime behaviour compared with monolithic and No Policy training, with enforcers maintaining safe gaps. Limitations include linear policy templates, manual decomposition choices, and the scope of the single case study. Future work includes richer policy classes, tighter integration of mining and training, and broader evaluation on additional datasets and domains.

Chapter 6

Compositionality for Explainable Personalised Mood Score Prediction

6.1 Overview

In Chapter 3, we discussed how there is a lack of performant mood score prediction models that are performant and explainable. We also suggested that the compositional framework could be used to improve explainability without sacrificing predictive performance. Here, in this chapter, we introduce a novel paradigm for designing monolithic and compositional ML models (taking inspiration from the compositional framework discussed in Chapter 4) tailored to personalised mood score prediction of depressed individuals, which improves explainability without sacrificing predictive performance. The models can be used to predict mood scores and explain how patients' activities affect their mood, suggesting possible indicators upon which to intervene for healthcare professionals and patients (for self-management).

Although our modelling and explainability framework is model agnostic, we illustrate our approach by building ANN (MLP) models on an existing multimodal dataset (from [98]) containing longitudinal Ecological Momentary Assessment (EMA) of depression, data from wearables, and neurocognitive data synchronised with Electroencephalography (EEG) for 14 mild-moderately depressed participants over one month. Through this work, we make the following contributions.

- We develop a parallelised ML modelling and optimisation framework (both monolithic and compositional) that helps train multiple Multilayer Perceptron DL models to predict participants' mood scores - a discrete score used to assess the severity of patients' depressive symptoms.
- We approach the problem of mood score prediction through both regression and classification methodologies, using three different kinds of data preprocessing

methodologies and eight Evolutionary Algorithm and statistical optimisation approaches.

- We show how the monolithic MLP models are better than several commonly used ML models, which we use as baseline models, in terms of performance.
- We develop explainability pipelines for monolithic and compositional models, at personal and population levels, using post-hoc explainability techniques (e.g., SHAP), yielding comprehensive yet focused insights into which biophysical and behavioural indicators drive each participant's mood predictions.
- We compare the performance and explainability of the monolithic and compositional models.

The chapter is structured as follows. Firstly, in Section 6.2, we introduce the dataset used in this work. Later, in Sections 6.3 and 6.4, we introduce the monolithic and compositional models developed in this work, respectively. Subsequently, Section 6.5 presents the results of the models and a discussion of their implications. Finally, Section 6.6 summarises the chapter.

6.2 Data

The dataset used in this work has been published previously [98]. This dataset was gathered following a one-month study of 14 adult human participants (with a mean age of 21.6 $\pm$ 2.8 years and ten females) before the onset of the COVID-19 pandemic.

6.2.1 Study Summary

Human Participants were recruited to the study from the University of California, San Diego College Mental Health Program [161]. The study included participants experiencing moderate depression symptoms assessed using the Patient Health Questionnaire, Patient Health Questionnaire (PHQ)-9 scale (score $>$ 9; participant score range 10-17) [162]. While no structured interview was conducted for the study, suicidal behaviours were screened using the Columbia Suicide Severity Rating Scale (C-SSRS) [163]. Any participants on psychotropic medications maintained a stable dose throughout the one-month study, and no participants demonstrated suicidal behaviours during the study. The study protocol was approved by the University of California, San Diego institutional review board, UCSD IRB# 180140.

The data was collected through two data acquisition modes. First, lifestyle and physiological data were collected using a Samsung Galaxy wristwatch (wearable) that all participants wore throughout the study, except while charging the watch for a few

hours once every 2-3 days. Participants also used an application named BrainE on their iOS/Android smartphone [164] to register their daily EMAs four times a day for 30 days. During each EMA, participants rated their depression and anxiety on 7-point Likert scales, participated in a 30-second stress assessment, and reported their diet (e.g. fatty and sugary food items consumed from a list provided and servings of coffee). Finally, data were collected using neurocognitive assessments in a lab. These assessments were done on days 1, 15, and 30 of the one-month study. Participants completed six cognitive assessment games to assess inhibitory control, interference processing, working memory, emotion bias, internal attention, and reward processing. These assessments were synchronised with EEG. x

6.2.2 Dataset

For each participant, the raw dataset from the study consists of 48 features. Three features were not chosen from the complete dataset as they were speed-based features, which were computed from noisy distance features. Of the remaining 45 features, we chose 43 input features, one output feature, and one feature containing the datestamp for the samples. The input features included both smartwatch and neurocognitive assessment data. Sixteen features were obtained from the smartwatch, and the rest were obtained from the neurocognitive assessment. The wearable and EMA features collected from the smartphone are presented in Table 6.1. Supplementary Table 1 of the supplementary information section of [98] contains a complete description of each input feature.

Moreover, the feature *depressed* with a value between 1 and 7 is used as the output feature. The severity of depressed mood increases from 1 to 7. This feature is used as the label/output when training the deep-learning classification models. The *datestamp* feature is used to order the dataset chronologically before any data preprocessing is performed. Table 6.2 contains sample information for each participant, and Figure 6.1 shows the label distribution per participant.

As seen from Table 6.2, 9 out of 14 participants have features where some values are missing. This could be due to device error or participant behaviour (e.g. a participant may forget to wear the smartwatch for a few hours). The *Missing Values* column in the table represents the total number of missing data points, not the number of missing samples. For instance, there could be a sample where 4 of the feature data points are missing. The *Features* column contains the number of features that have missing data points. A feature could be missing more than one data point across samples. Thus, the *Missing Values* column shows the number of missing data points out of all the data points for the participant, which is 43 (number of features) times the total number of samples. However, there are no samples where all the feature values/data points are

Table 6.1: Summary of features acquired using a smartwatch

#	**Feature**	**Description**
1	distracted	EMA-based 1-7 ratings of "How distracted do you feel right now?" acquired 4 times per day alongside the depressed mood ratings
2	anxious	EMA-based 1-7 ratings of "How relaxed versus anxious do you feel right now?" acquired 4 times per day alongside the depressed mood ratings
3	Mean Breathing Time	Mean breathing time of the 30-sec active stress assessment acquired 4x per day alongside the depressed mood ratings
4	Consistency	Consistency of breathing (1 - coefficient of variation) of the 30-sec active stress assessment acquired 4x per day alongside the depressed mood ratings
5	past day fats	Total fatty items consumed in the 24 hours prior to each depressed mood rating
6	past day sugars	Total sugary items consumed in the 24 hours prior to each depressed mood rating
7	past day caffeine	Total cups of caffeine consumed in the 24 hours prior to each depressed mood rating
8	heart rate	Smartwatch-based heart rate as the mean heart rate in the $\pm$30 minute window around each depressed mood EMA
9	ppg_std	Heart Rate Variability from the Tizen Photoplethysmography data as the standard deviation within the $\pm$15 minute window around each depressed mood EMA
10	cumm_step_count	Cumulative step count taken as the mean value from the past 12 hours of each depressed mood rating
11	cumm_step_calories	Cumulative step calories burnt taken as the mean value from the past 12 hours of each depressed mood rating
12	cumm_step_distance	Cumulative step distance taken as the mean value from the past 12 hours of each depressed mood rating
13	cumm_exercise_calories	Cumulative exercise calories burnt taken as the mean value from the past 24 hours of each depressed mood rating
14	cumm_exercise_duration	Cumulative exercise duration taken as the mean value from the past 24 hours of each depressed mood rating
15	prev_night_sleep	Number of hours of sleep the previous night of each depressed mood rating
16	time_of_day	Time of the day when a particular depressed mood rating was taken: (6:00, 10:00], (10:00, 14:00], (14:00, 18:00], (18:00, 23:59]

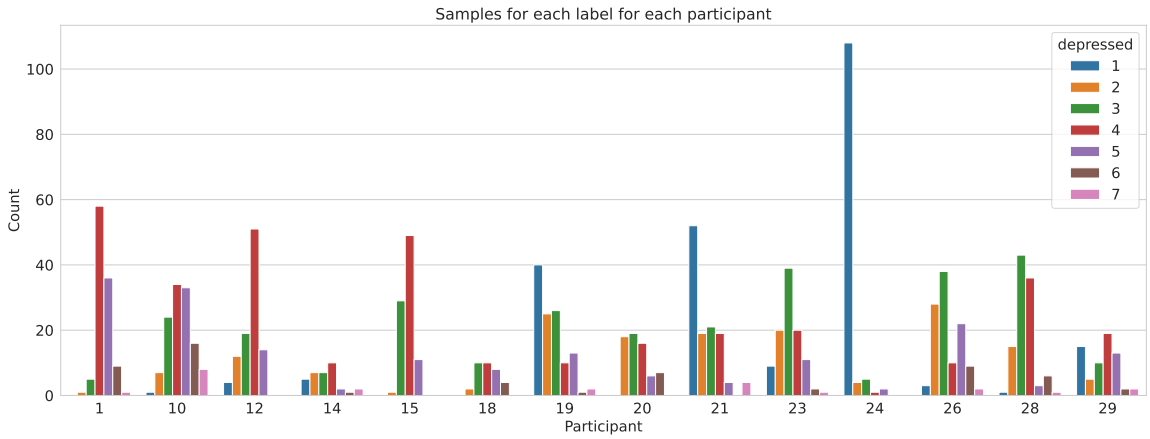


Figure 6.1: Number of label classes (depressed state values) per participant.

missing. Another information of interest is the total number of samples. Participants 14, 18, 21 and 29 have fewer samples and can be expected to perform poorly on deep learning models [1].

Table 6.2: Summary of Samples for each participant

Paricipant	**Total Samples**	**Missing Values (Out of 43 x Total Samples)**	**Features w/ missing values**
1	110	28	1
10	123	108	27
12	100	116	31
14	34	1	1
15	90	0	0
18	34	0	0
19	117	18	1
20	66	11	1
21	119	0	0
23	102	0	0
24	120	21	1
26	112	10	4
28	105	0	0
29	66	9	1

Also, we noticed that some features for some participants are constant values, i.e., the same value repeats for each sample. This may make sense for neurocognitive assess-

ment features (where a participant may perform consistently on the tests), but not for features acquired through the smartwatch. For instance, a participant would be highly unlikely to have the same non-zero value for *exercise calorie* for 30 days. For instance, for some participants (participants 10, 15, 18, 21, and 23), the HRV feature titled *ppg_std* contained what appeared to be a missing or invalid value. The HRV feature indicates the standard deviation of PPG values captured from the Samsung Galaxy smartwatch's sensor within a ± 15-minute window around each depressed mood EMA. The value shown was consistently 999, which we deemed the default value recorded if an error or missing value scenario occurred. Similarly, for participants 18 and 24, 2 physical activity and sleep-based features titled *exercise_calorie*, *exercise_duration*, and *prev_night_sleep* all recorded a value of 999 for every entry.

Moreover, we can see from Figure 6.1 that the label classes (depressed state values) across participants are not balanced. This is expected as the participants are mild-moderately depressed, and the higher ends of the depressed mood scale (which corresponds to severe depression) will be rarely represented. This imbalance, however, requires that special metrics be used when evaluating the model performance [165].

6.2.3 Data Preprocessing

The previous section showed how the dataset contained not only missing data points but also columns with a single unique value. The dataset in its present state cannot be used for training, as most Machine Learning models do not accept datasets with missing values. Furthermore, training a model on a dataset where certain features have uncharacteristically unique values can confound the model and produce erroneous models. Hence, we preprocess the data. As different data preprocessing methods have advantages and disadvantages, we try three data preprocessing methods and build models for each to compare which method suits the dataset.

For the first method, we use *Deletion* as follows:

- Identify participants for whom we have constant feature values for features acquired from the smartwatch - call it set C.
- Remove participants in C.
- Identify (from remaining participants) those participants for whom we have missing data points - call it set M.
- For participants in M, remove the samples containing the missing data points.

The aim in this method is to ensure that each participant has all 43 features with no missing data points. For the first portion of the goal, we remove the participants with constant smartwatch feature values. This step eliminates participants 10, 15, 18,

21, 23 and 24. For the second portion of the goal, we remove the samples/rows with any missing data. This step will reduce the number of samples for some participants. However, this deletion method is the most straightforward data preprocessing method and provides a good baseline against sophisticated data preprocessing methods.

For the second method, we use a *Manual Imputation* technique using a set of differentiated data preprocessing steps as follows:

- Identify participants for whom we have constant feature values for features acquired from the smartwatch - call it set C.
- Remove the features with constant values for participants in C
- Identify participants for whom we have missing data points - call it set M.
- For participants in M, identify the features with missing data points - call it set F.
- Identify if the features in set F contain discrete, continuous or neurocognitive values.
- For discrete features, the missing values are filled with the most frequent value.
- For continuous features, the missing values are imputed using an Iterative Imputer (see Table 6.3).
- For neurocognitive features, we fill in the missing values with zero.

This method chooses the methods used to impute/fill data by the data type. It imputes the missing values with the most frequent value for discrete values. In contrast, for continuous values, it imputes the missing values using an iterative method that computes the missing values in each feature by considering it as a function of all other features in a round-robin manner. We use the iterative imputer for continuous variables as it tries to preserve the influence of other features on the feature with missing values [166]. Finally, we impute the missing values in the neurocognitive features with zero. The neurocognitive features are assessment-based, and imputing missing values with zero seems acceptable, as a zero in an assessment typically implies an empty/void assessment. This method is similar to what is presented in [98].

For the third method, we employ *Automatic Imputation* using a set of sophisticated data preprocessing steps as follows:

- Identify participants for whom we have constant feature values for features acquired from the smartwatch - call it set C.
- Remove the features with constant values for participants in C

- Identify participants for whom we have missing data points - call it set M.
- For participants in M, identify the features with missing data points - call it set F.
- For each feature in F, fill in the missing values with different data-filling methods, one at a time.
- Choose the data-filling method that gives a data distribution that best matches the data distribution before any preprocessing.

This method aims to remove any confounding features while preserving as many samples as possible by filling in missing data. For the first part, we remove the features with constant feature values. Next, we handle missing data by choosing a data-filling method that changes the data distribution the least. Instead of manually choosing an appropriate method, we automate the process and use seven different data-filling methods for each feature with missing values. The chosen methods are summarised in Table 6.3.

Table 6.3: Summary of methods used for handling missing data in a feature column

Method	**Description**
Mean	Fill missing data with the mean of the available data
Median	Fill missing data with the median of the available data
Forward Fill	Fill missing data by continuing the last available data
Backward Fill	Fill missing data by continuing the next available data
Linear Interpolation	Fill missing data through linear interpolation using the data points before and after the missing data
Iterative Imputation [167]	A strategy for imputing missing values by modelling each feature with missing values as a function of other features in a round-robin fashion. This method only uses samples with no missing data as input (in case multiple features in a dataset have missing values)
KNN Imputation [168]	Each sample's missing values are imputed using the mean value from some nearest neighbours found in the training set. Two samples are close if the features that are not missing in either are close.

Finally, we compare the methods using the distribution of the filled-in feature and the original feature vectors. For this, we use the Two-sample Kolmogorov-Smirnov (KS) test, which compares two distributions by finding the maximum difference between the Cumulative Distribution Functions (CDFs) of the two distributions [169]. In general, the KS test is used to compare the underlying continuous distributions F(x) and G(x) of two independent samples. This statistical test uses the null hypothesis

that the two distributions are identical, i.e. F(x)=G(x) for all x. The alternative hypothesis is that they are not identical. The test produces a *statistic* and a *p-value* [170]. The *statistic* obtained from the test is the maximum absolute difference between the empirical distribution functions of the samples.

We choose a confidence level of 95%; i.e. we reject the null hypothesis in favour of the alternative if the *p-value* is less than 0.05. The higher the *p-value*, the more probable the fact that the two distributions (i.e., before and after data filling) are similar and not just due to chance. Thus, we choose a method with the highest *p-value* and low *statistic*. As different methods are chosen for different features for every participant, we decided against reporting them here to maintain the succinctness of the chapter.

6.3 Monolithic Model Development

Since the depression scale is ordinal, i.e. there is an order in the value of the depression/mood score, and it ranges between 1 and 7, we can consider the mood score prediction problem as either a regression or classification problem. As a regression problem, the model will be concerned with developing a model that predicts values that are close to the actual mood scores. A classification model, on the other hand, considers the mood scores as seven classes and tries to predict a class based on the input.

First, we use ten common regression and classification classical ML models to build baseline models against which to compare the performance of MLP models. Then, we use MLP models to build the regression and classification models. Moreover, we build models for the MLP regression and classification models for the different types of data imputation schemes (see Section 6.2.3). The whole model development framework[1] is shown in Figure 6.2.

6.3.1 Base Models

We built a set of base models to act as a baseline for the predictive performance of MLP models on the dataset. We trained ten common classical ML algorithms (eight of which were used in [98]) on the three preprocessed datasets for each participant: Adaboost Regressor, Adaboost Classifier, Elasticnet Regressor, Gradient Boosting Classifier, Gradient Boosting Regressor, Poisson Regressor, Random Forest Classifier, Random Forest Regressor, Support Vector Classifier and Support Vector Regressor. Also, we used a simple grid search (as used in [98]) to tune the hyperparameters of the

[1] Although we experiment with only MLP models in this work, the methodology is model agnostic and can be applied to other ML models too by replacing the MLP model with another ML model.

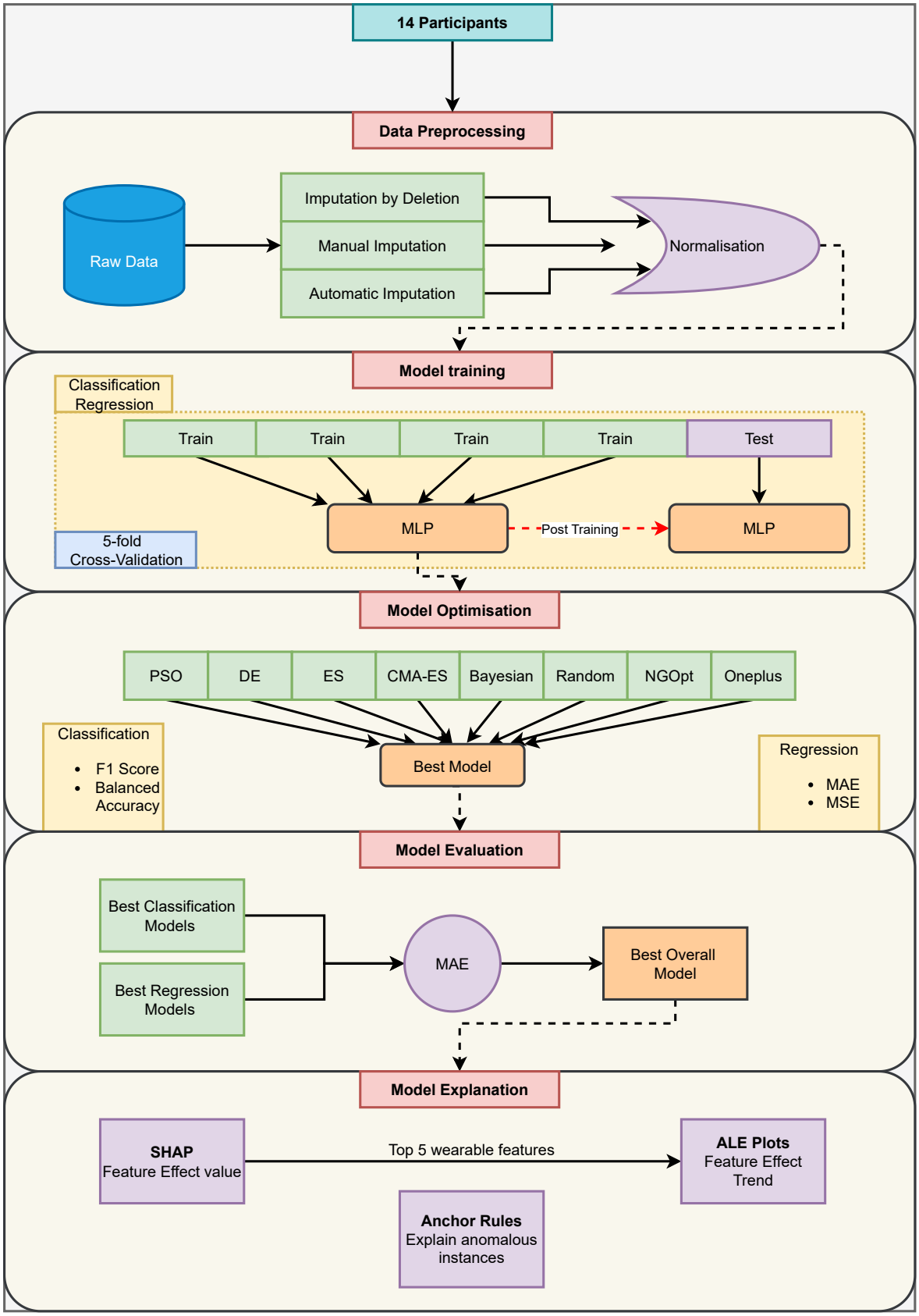


Figure 6.2: The Personalised Mood Score Prediction Framework for monolithic models. This framework is repeated for all 14 participants. We begin at the top with data preprocessing. The preprocessed data is then used for model training of classification- and regression-based MLP models. The Best models from the classification and regression training and optimisation are compared to find the best overall models with minimum Mean Absolute Error (MAE) (MAE). This best overall model is then used to get model explanations using SHAP, ALE plots and Anchor rules.

models. The grid search is a brute-force method that tries all possible combinations of hyperparameters and chooses the combination that provides the best prediction performance.

Furthermore, a Stratified 5-fold Cross-Validation (CV) scheme was used to validate the model performance during and after training. This scheme divides the normalised dataset into five parts, trains a model on the four parts (the training dataset) and tests on the remaining part (the testing dataset). It does so in a round-robin fashion. The division is stratified, meaning each fold contains the same proportion of the different output labels. So, for a 5-fold CV, we built five separate models (with the same architecture) on the five training and test datasets. The overall performance was obtained by taking the mean of the training and test performance values over the five sets. Also, the test datasets do not overlap between the folds. This method ensures that the evaluation of the model is free of data-selection bias, which may arise when using a simple train–test split, as the performance depends on the particular split of the train and test set.

For each participant, the model (out of the ten) with the lowest Mean Absolute Error (MAE) after hyperparameter tuning, irrespective of classification or regression, was chosen as the base model. More details on the grid search and the hyperparameters used for each model are provided in Appendix A.3. Note that the base models were only used for performance comparison with the MLP models and were not used for a model-explanation comparison, as the explainability of such models in a mood-prediction setting has been explored in [98].

6.3.2 MLP Model Architecture

The model architecture differs between the regression and the classification models. As mentioned in the previous section, we use Deep Neural Networks (DNNs) (Multilayer Perceptrons (MLPs)) to build the model. While both models have an input layer, a few hidden layers, and an output layer, the number of neurons in the output layer differs between a regression model and a classification model. As a regression model predicts a single value for each input, all regression models use only one neuron in the output layer with *no* activation. On the other hand, the classification models have seven neurons corresponding to the seven classes (mood scores). Outputs from the neurons are normalised (squashed) using a *softmax* activation. These squashed values (for each neuron) lie between 0 and 1 and represent the probability of an input belonging to that class. The class corresponding to the highest output value is taken as the output. The MLP models (both classification and regression) comprise several MLP layers with some neurons in each layer. The actual number of layers, the number of neurons in each

layer and the activation for each layer are determined by an optimisation algorithm described in Section 6.3.4.

6.3.3 Model Training

All models are built and trained in Python using the TensorFlow-Keras library, which uses a loss function and an optimiser. The loss function is a function that evaluates the model prediction against the actual output value and produces a numeric value based on how different or similar the prediction and the actual values are. Moreover, the optimiser optimises/modifies the weights/parameters of the DNNs so that the loss is minimised. For the classification models, we use a version of the cross-entropy loss (see Equation 2.8) called the *Sparse Categorical Crossentropy*. The regression models use either the Mean Squared Error (MSE) or the MAE between the predicted and actual values as the loss function. We use a version of stochastic gradient descent called *Adam* optimiser to optimise the loss function of all models.

Also, the preprocessed dataset is time-sorted and normalised before it is fed into the models for training. This ensures smoother convergence of the loss function. We use the standard normalisation procedure. It centres the data around zero and gives the dataset's unit standard deviation. This is achieved by subtracting the feature means from each feature and dividing the result by the standard deviation of the feature (see Equation 2.12).

We use a Stratified 5-fold Cross-Validation (CV) scheme to validate the model performance during and after training. The samples in the normalised folds are then randomised and fed into the MLP models for training, i.e. the MLP models take an input of shape $N \times F$, where N is the number of input samples (data points) and F is the number of features. Moreover, we follow these steps for all MLP models built for both regression and classification, irrespective of the data imputation method. We train each model using batches of training data for 100 epochs, i.e. for 100 iterations of the entire training data (divided into batches). We also only save the best model across the epochs and use the early-stopping strategy, which stops the training before the epochs finish if the model's performance does not improve for a certain number of epochs.

6.3.4 Model Optimisation

It is usually challenging to infer the architecture of the DNN that gives the best possible performance, and there are often multiple parameters that influence the performance of a DNN. Hence, we decided to further optimise the model hyperparameters (i.e. the extrinsic model parameters that influence performance, such as the model architec-

ture and training) using multiple Evolutionary Algorithm (EA) based schemes and stochastic schemes to obtain better model performance.

Table 6.4: Summary of optimisation algorithms/methods used in hyperparameter optimisation

Algorithm	**Summary**	**Parameters**
Particle Swarm Optimization (PSO) [171]	Particle Swarm Optimisation of the loss on hyperparameters based on a set of particles (candidate solutions) with their inertia	Population Size: 10
Differential Evolution (DE) [172]	Differential Evolution-based optimisation. It uses differences between points in the population (candidate solution) for doing mutations in fruitful directions	Population Size: 10
Evolutionary Strategy (ES) [45]	Evolution Strategy-based optimisation of hyperparameters is based on the ideas of evolution	Population Size: 10
Covariance Matrix Adaptation Evolution Strategy (CMA-ES) [45]	Covariance Matrix Adaptation Evolutionary Strategy is a variation of ES where the covariance matrix of the distribution is incrementally updated to increase the likelihood of previously successful search steps	Population Size: 10
Random	Random search of the hyperparameter space	N.A
Bayesian [173]	Bayesian Optimisation of the loss on hyperparameters. Optimisation on search space depends on the initialisation type	Initialisation: Latin Hypercube Sampling
Nevergrad Optimiser (NgOpt) [174]	Nevergrad Library's own optimiser	N.A
Oneplusone [174]	1+1 optimisation of the loss on the hyperparameter space	N.A

We use eight different EAs and statistical methods to optimise the number of hidden layers in the model, the maximum number of neurons, the activation of the hidden layers, and the batch size of the training. We optimise the maximum number of neurons and not the number of neurons in each layer, as that would have increased the number of optimisation variables. Increasing the number of optimisation variables increases the optimisation space, making the optimisation problem more difficult. We linearly interpolate the neurons in the hidden layers (whose number is also optimised) using the maximum number of neurons and the number of neurons in the output layer (which depends on whether the model is classification or regression). The eight EA methods we use, and their parameters, are mentioned in Table 6.4. Table A.1 also contains upper and lower limits for each hyperparameter used during the optimisation.

Table 6.5: Search values (or limits) for parameters during hyperparameter optimisation

Hyperparameter	**Lower Limit**	**Upper Limit**	**Values**
Number of layers	2	4	N.A
Maximum number of neurons	30	60	N.A
Activation	N.A	N.A	Hyperbolic Tangent (Tanh), Rectified Linear Unit (ReLU), Exponential Linear Unit (ELU), linear
Batch size	N.A	N.A	4, 6, 8

Figure 6.3 shows the optimisation schematic. Hyperparameter optimisation exists as an outer loop in the inner loop of model training (which optimises the model weights). The optimisation repeats over several iterations, during which the model is trained using the 5-fold CV procedure. We average the model performance over the five folds and use that as the performance metric to optimise the hyperparameters. We use the metrics F1-score and *Balanced Accuracy (BA)* to optimise the classification models and use MSE and MAE to optimise the regression models. We optimise the classification models to reduce the margin between the perfect performance (100% accuracy, for instance) and the performance obtained from the model. For the regression models, we minimise the metrics as they are error-based.

We run the optimisation for 100 iterations for all methods. At the end of the 100 iterations, we take the best models, i.e. models with the best mean 5-fold performance, from each method and find the best among the eight best (one for each optimisation method) models as well. We use the same metrics to optimise the hyperparameters to find the best models. The performance of these best models is then taken as the best for a particular combination of optimisation metric, problem type and data imputation method.

The optimisation procedure is entirely parallelised, and the number of parallel processes is determined by the number of cores in the system used for optimisation and training. We run the optimisation (and training) in a Docker Ubuntu container containing all the required Python libraries, such as TensorFlow-Keras (for training the models) [39] and Nevergrad (for the optimisation) [174]. Parallelisation reduces the optimisation time significantly, making the procedure scalable to a high number of optimisation iterations.

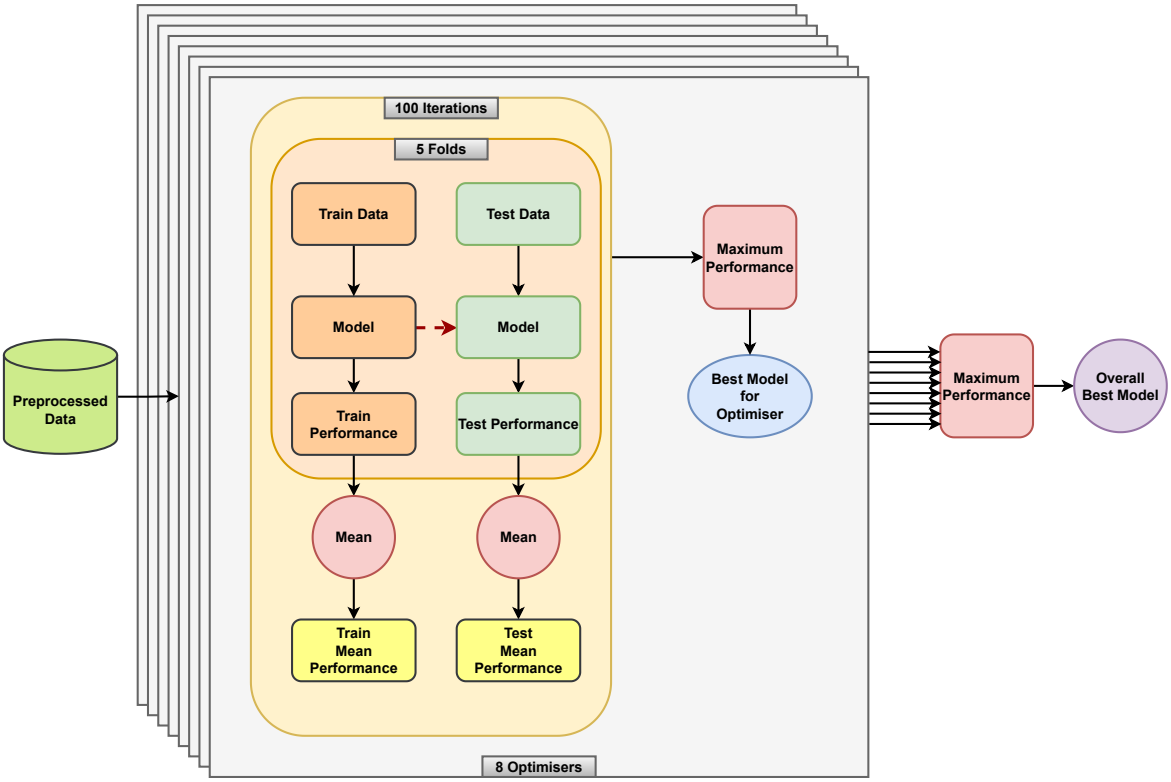


Figure 6.3: The Optimisation Procedure.

6.3.5 Model Evaluation and Explanation

We gather the best model for each combination of problem type (classification or regression), data preprocessing methodology and metric used for hyperparameter optimisation (i.e. 12 combinations). As both regression and classification problems use different performance metrics except for MAE, to find the best overall method, we use MAE as it corresponds to the absolute error. Hence, for each participant, we collect the best models from the 12 combinations, find the model with the lowest test MAE, and use it as the overall best-optimised model.

This overall best model is the final model for the participant, and we use this to extract important indicators/features and explain how those indicators affect mood. To this end, we use three post-hoc explainability methods from the Explainable Artificial Intelligence (XAI) literature [9]. We use Shapley Additive Explanations (SHAP) [10], Accumulated Local Effects (ALE) plots [27] and Anchors [28]. All explanation methods take the unnormalised data as input (which is internally normalised before being fed into the models). Using unnormalised data ensures that the explanations are produced in the actual data range, which makes it easier to interpret.

Here, we used SHAP to find the top five important features of each participant. We got one SHAP value per data instance per feature, and to compute global feature importance for a model and a dataset, we took the mean of the absolute SHAP values

for all instances in the dataset to obtain the overall SHAP value of a feature. Also, as we used a 5-fold CV approach, we found the SHAP values for each fold and averaged them. As with SHAP, we used the test dataset to compute the ALE value for each fold and found the overall ALE value by taking the mean of the ALE values obtained for the five folds. Finally, we used Anchors to show how specific predictions for models could be explained in a rule-based manner.

6.4 Compositional Model Development

6.4.1 Compositional Modelling Overview

In the following sections, we present the design of compositional models. As the name suggests, a compositional model comprises multiple sub-models whose outputs are combined to produce a single output. It is a modular modelling paradigm that promotes using multiple smaller models that should be easier to explain, especially for complex ML models.

Typically, sub-models in a compositional model are built on data obtained by segregating the complete dataset with its complete input feature/predictor set into subsets of features. We call this subset a cluster. Each sub-model reasons on its cluster. The outputs from the sub-models can be combined/composed hierarchically using *Merge* models to form the complete compositional model. This hierarchical multi-staged design gives more control over the flow of data and makes it more transparent. The compositional model and the sub-models can be trained to reason on the same set of outputs or different outputs based on design requirements.

In the personalised mood score prediction case, the compositional model consists of sub-models that are built on clusters obtained from the dataset described in Section 6.2. The goal of both the compositional model and the sub-models is to predict the mood state of a depressed participant, given a set of input features. Each compositional model learns to predict the mood state given a subset of features (cluster) from the complete dataset. It, therefore, learns the relationships between the features contained in its cluster. We combine the outputs of all sub-models, which have learned representations for their cluster, in a specific hierarchical architecture to form the compositional model. Thus, the compositional designs learn separate representations and the relationships between those representations, and so on, based on the compositional architecture.

For example, consider Equation 6.1, where the compositional model architecture consists of four initial models ($Models_1, \ldots, Models_4$) that receive inputs from their feature clusters ($cluster_1, \ldots, cluster_4$). These models are combined hierarchically using three Merge models – M_1^{stage1}, M_2^{stage1}, and M_1^{stage2} in two stages. First, M_1^{stage1}

and M_2^{stage1} combine the outputs of the four initial models. The model outputs from the two Merge models are then combined to produce the final output M_1^{stage2}.

$$\begin{aligned} Output = M_1^{stage2}(M_1^{stage1}(Model_1(cluster_1) + Model_2(cluster_2)) + \\ (M_2^{stage1}(Model_3(cluster_3) + Model_4(cluster_4))) \end{aligned} \tag{6.1}$$

Additionally, similar to the monolithic model, the mood scores may be formulated either as a regression task or as a classification task [37], and all models are evaluated under multiple data-imputation strategies (see Section 6.2.3). The overall MLP development pipeline[2] is illustrated in Figure 6.4.

6.4.2 Compositional Model Architecture

The compositional model (regardless of regression or classification) will always be a funnel-like design with multiple stages, and the number of models per stage decreases as we approach the output. We call the models in the first stage Cluster models, and the models in subsequent stages Merge models. At each stage (except for Stage 1, where the biophysical signals are used as inputs), the outputs from models in the previous stage are used as inputs.

The number of clusters, the features in those clusters, and the subsequent number of combination models and stages are all parameters that have to be based on either expert knowledge or experimentation. In this work, we consider multiple possible approaches to form the clusters and the model architecture to explore which approach offers a design with the best performance and explanation. We choose these approaches based on differences in data sources and expert opinion.

Specifically, we consider three different architectures: (1) a design with six cluster models, one Merge model and two stages; (2) a design with three cluster models, two Merge models and three stages; and (3) a design with ten cluster models, three Merge models and three stages. We cannot consider all combinations and only consider a few decompositions to explore the possible advantages and disadvantages of the compositional model.

6.4.2.1 Approach 1: Compositional Model with 2 Stages and six cluster models

The first architecture we consider consists of six MLP cluster models. The design is shown in Figure 6.5. Here, we divide the clusters based on the source of data - Activity,

[2]Although we experiment with only MLP models in this work, the methodology is model agnostic and can be applied to other ML models too by replacing the MLP model with another ML model.

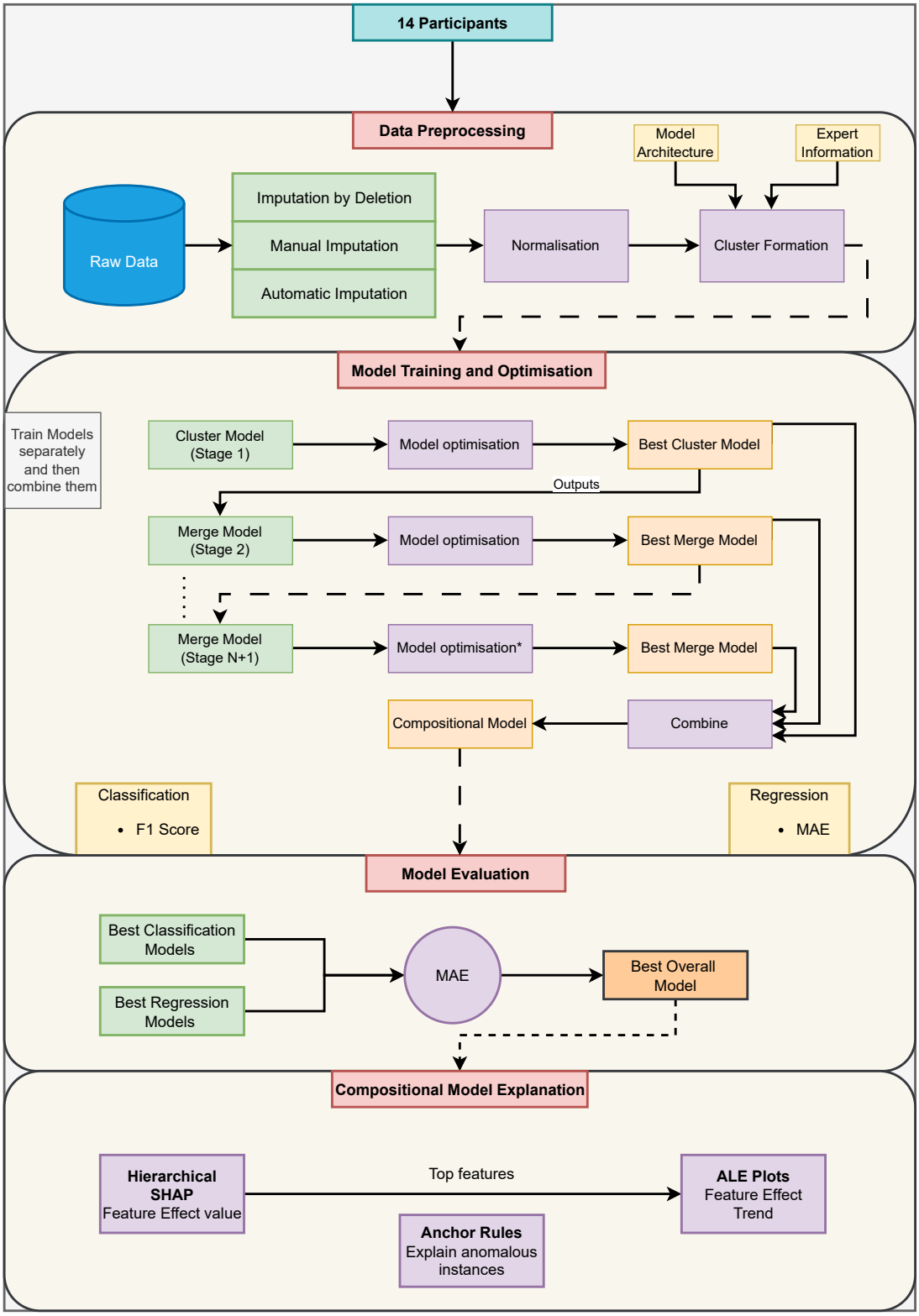


Figure 6.4: The proposed Personalised Mood Score Prediction Framework for the compositional models. This framework is repeated for all 14 participants. We begin at the top with data preprocessing. The preprocessed data is then used to train classification- and regression-based MLP compositional models. The Best compositional models from the classification and regression training (using 5-fold Cross-Validation) and optimisation are compared to find the best overall models with minimum MAE (MAE). This best overall model is then used to get model explanations using SHAP, ALE plots and Anchor rules.

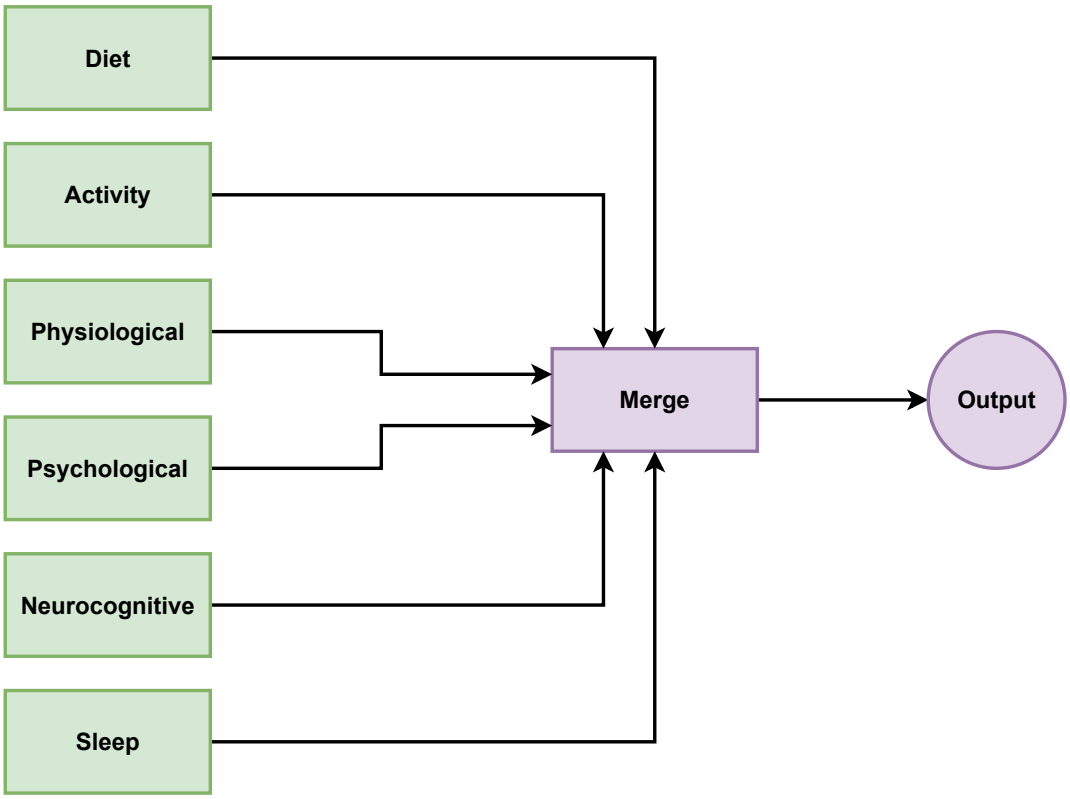


Figure 6.5: Compositional model with six clusters and two stages.

Diet, Physiological, Psychological, Sleep and Neurocognitive. The Activity cluster contains data from wearables that record the activity (exercise) of the participants. The Diet cluster contains data from smartwatches where individuals record their food consumption (particularly sugar, caffeine, and fat consumption).

$$\hat{y} = \sum_{i=1}^{6} w_i \hat{y}_i \tag{6.2}$$

Moreover, the physiological cluster contains data from wearables that record cardiovascular data, e.g. heart rate, breathing rate, etc. The psychological cluster contains data from the smartwatch where individuals record features that reflect the psychological factors affecting mood, such as anxiety and distraction. The sleep cluster contains data from wearables that record the sleep patterns of the participants. Currently, we only have one sleep feature - the previous night's sleep. The Neurocognitive cluster contains data from Neurocognitive tests conducted in the clinic. The Merge model combines the outputs from the cluster models using a white-box weighted average function (see Equation 6.2), whose weights the model learns. All feature names corresponding to the cluster models are given in Appendix A.4.1.

6.4.2.2 Approach 2: Compositional Model with 3 Stages and three cluster models

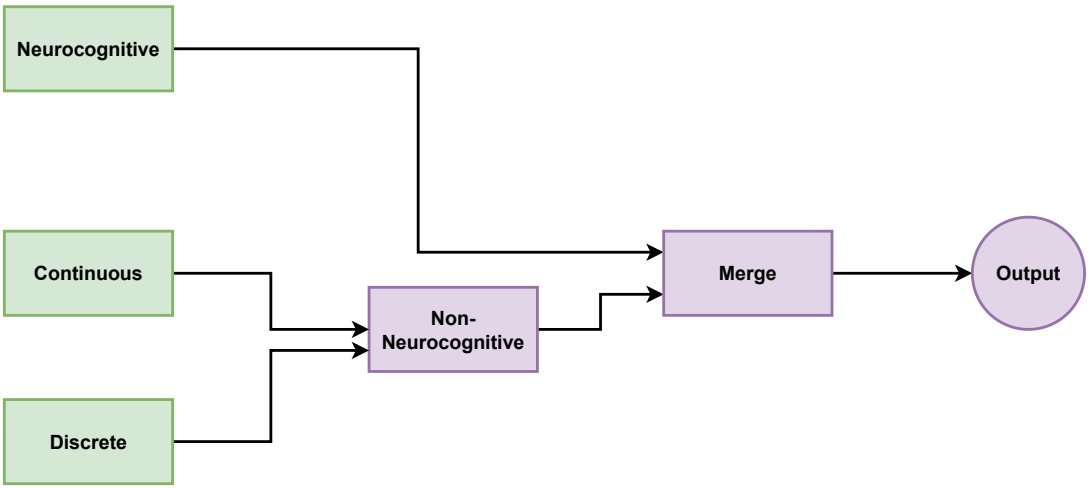


Figure 6.6: Compositional model with three clusters and two stages.

The second architecture we consider consists of three MLP cluster models and three stages. The design is shown in Figure 6.6. Here, we divide the clusters based on the kind of data: Continuous, Discrete, and Neurocognitive. The Continuous cluster contains continuous data from wearables that record the activity (exercise), heart rate, and sleep data of the participants. The Discrete cluster contains data from the EMA recordings, such as the anxiety level, number of cups of coffee, etc. The Neurocognitive cluster contains data from Neurocognitive tests conducted in the clinic.

Here, we have two Merge models. The first Merge model, called Non-Neurocognitive, combines the outputs of the Continuous and Discrete cluster models. The second Merge model combines the outputs from the Non-Neurocognitive Merge models and the Neurocognitive cluster model. Both Merge models are MLP models. The output of the final merge block is the final prediction of the model. All feature names corresponding to the cluster models are given in Appendix A.4.1.

6.4.2.3 Approach 3: Compositional Model with 3 Stages and 10 cluster models

The third architecture we consider is a three-stage architecture with three cluster models. It is shown in Figure 6.7. Here, we begin by dividing the features into three super-clusters based on the source of data: Smartwatch, EMA, and Neurocognitive. Then, we subdivide the super-clusters into smaller clusters. The Smartwatch super-cluster contains the Activity, Sleep, and Heart-based cluster models. The EMA super-cluster contains the Diet, Ema, and Breathing cluster models. The Neurocognitive super-cluster contains the Neurocognitive cluster models.

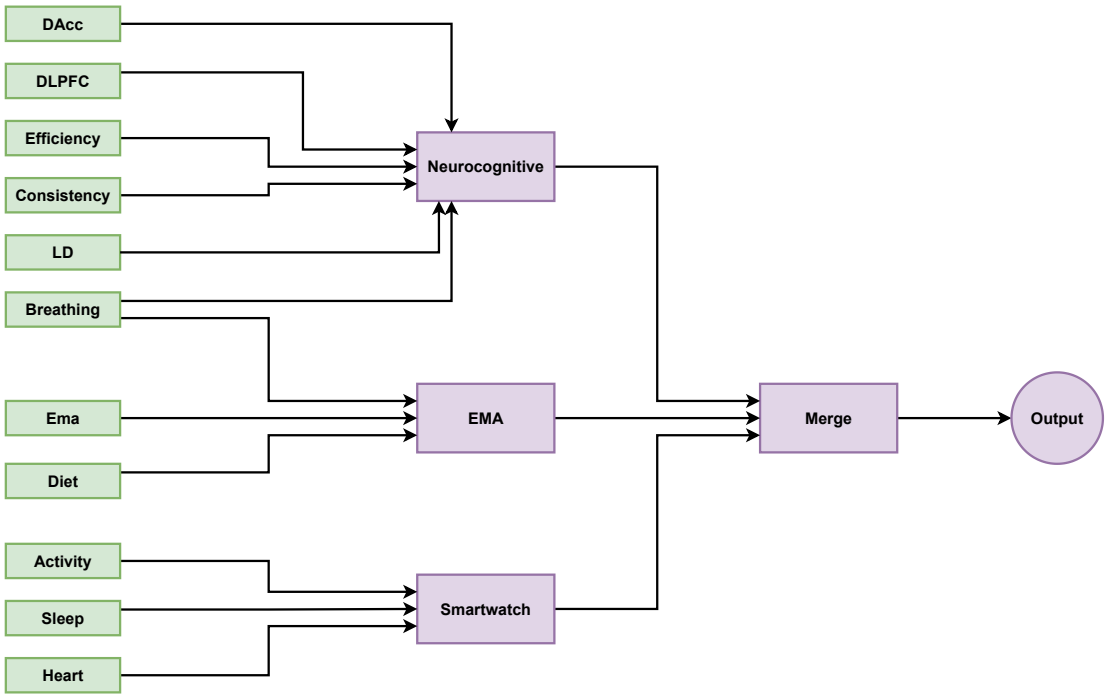


Figure 6.7: Compositional model with three stages and 3 cluster models.

Overall, the first stage contains 10 MLP cluster models. These models in each super-cluster are combined into super-cluster-level MLP Merge models called Smartwatch, EMA, and Neurocognitive. This produces three Merge models in the second stage. These three Merge models are later combined into a final Merge model in the third stage. The output of the final merge block is the final prediction of the model.

To elaborate, the models feeding into the Neurocognitive Merge model correspond to the neurocognitive assessments performed on participants. Dorsolateral Prefrontal Cortex (DLPFC), Dorsal anterior cingulate cortex (DAcc) models correspond to features from recorded neural activity in these brain regions. Lucky Door (LD) contains the reward metrics for the Lucky-Door game assessment. Breathing, Consistency and Efficiency contain features that measure the breathing, consistency and efficiency of participants in the assessments. All feature names corresponding to the cluster models are given in Appendix A.4.1.

6.4.2.4 Regression and Classification Modelling

Regression and Classification compositional models were constructed using the architectures mentioned thus far. In a classification (or regression) compositional model, every sub-model is also a classification (or regression) model. The regression models featured a single output neuron—without any activation function—to generate a continuous prediction for each input. In contrast, the classification models utilised seven output neurons, one per mood-score category, each followed by a *softmax* activation.

The resulting normalised outputs, constrained to the interval $[0, 1]$, represent class-membership probabilities, and the class associated with the maximal probability is selected as the model's prediction. The precise network depth, the neuron counts per layer, and the choice of activation functions were all established via the hyperparameter optimisation procedure detailed in Section 6.4.3.

6.4.3 Compositional Model Training

In a compositional model, each sub-model (cluster and Merge models) is trained separately and sequentially, starting from the cluster models and ending at the last Merge model. As there are no connections between the models in a specific stage, it is also possible to parallelise the training of models in a stage. The training sequence follows the flow of data from the cluster models to the intermediate Merge models and ends at the final Merge model.

These models were developed, trained, and optimised[3] in Python using the same loss function as presented in Section 6.3.4. We used the metrics F1-score to optimise the classification models and used MAE to optimise the regression models. We do not use BA and MSE as performance metrics for the compositional models to optimise the time taken to train the models, which is reduced if fewer metrics are used. Also, BA is less sensitive to the class imbalance and MSE (as in the monolithic models) penalises wrong predictions too much due to its squared term. Table A.14 also contains upper and lower limits for each hyperparameter used during the optimisation.

The other difference is that the compositional models are optimised sequentially, starting from the cluster models and ending at the final Merge model. Model performance was assessed using stratified 5-fold cross-validation (CV). Within each fold, samples were shuffled and presented to the MLP in batches of shape $N \times F$, where N is the batch size and F is the number of features. Figure 6.4 illustrates the overall training and evaluation pipeline.

6.4.4 Compositional Model Explanation

We select the top-performing model for each of the six combinations arising from two problem types (classification and regression) and three preprocessing pipelines for each compositional architecture. Since most metrics differ between regression and classification—aside from MAE, which applies to both—we chose MAE to identify the overall best model, given its focus on absolute rather than relative error.

[3]We do not retrain if an optimisation fails due to an error in the training process to reduce the time taken to train the models, so not all models are optimised for the same number of iterations and optimisers.

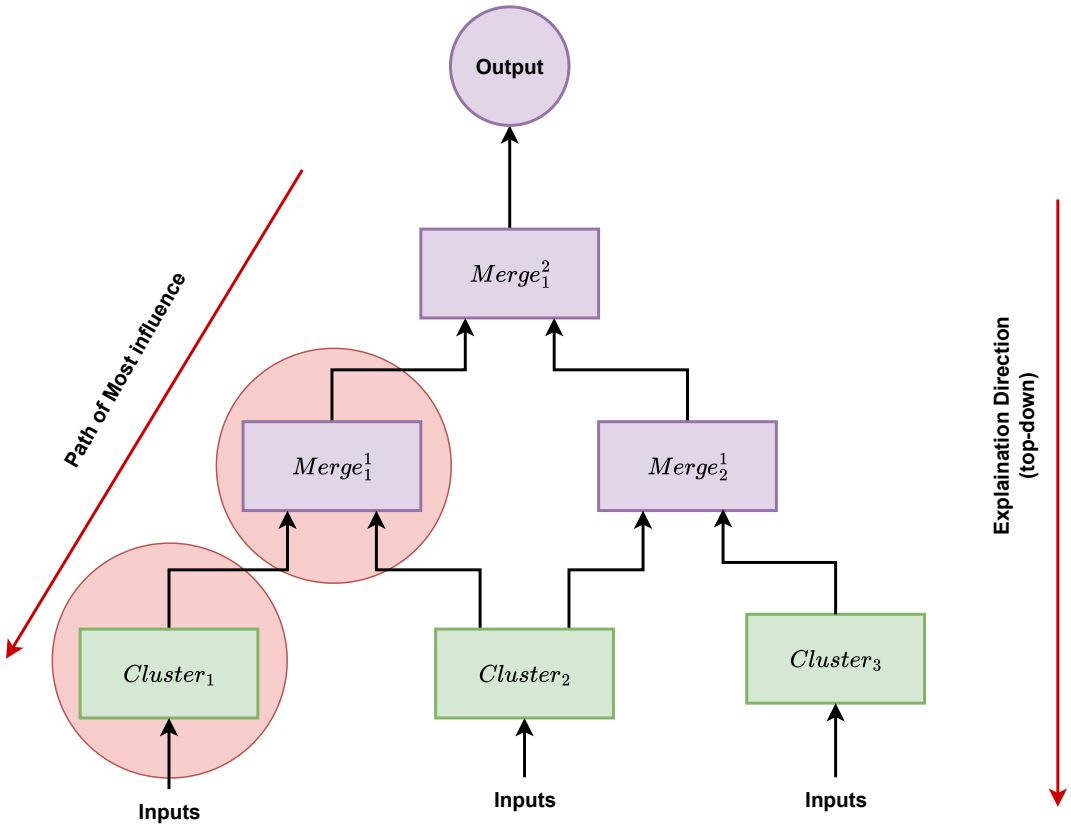


Figure 6.8: Compositional Explainability Methodology

For each participant and compositional model architecture, we gathered the best model from each of the six configurations, determined which one yielded the lowest test MAE, and selected it as that participant's optimally tuned model. This final model then served as the basis for extracting influential predictors and explaining their effects on mood. To achieve this, we employed three post-hoc explainability techniques from the XAI literature [9]: Shapley Additive Explanations (SHAP) [10], Accumulated Local Effects (ALE) plots [27], and Anchors [28].

Using the aforementioned explainability methods is trivial for monolithic/single models. However, the use of compositional models, where we have several sub-models, makes the process of explaining model predictions difficult. While there are various ways of interpreting all possible combinations of cluster models, merge models and combinations of cluster models, we present an intuitive top-down approach.

In our approach, we begin with the Merge model closest to the output and use SHAP values to find the most influential model in the previous stage. We continue this until we reach a cluster model. Using SHAP analysis on the cluster model will provide us with the best input features. The calculation of SHAP differs between classification and regression models, as the number of outputs differs. For a regression model, we simply use the SHAP value we obtain for an instance, as it only produces

one output. However, for a classification model which produces C outputs (where C is the number of classes, which is seven in the personalised mood score prediction dataset), we find the mean of the absolute value of the SHAP for the classes and use this value as the output SHAP value.

For the model in Figure 6.8, we start with $Merge_1^2$. We compute using SHAP that the output from the model $Merge_1^1$ is the most influential. We discover that the output from the model $Cluster_1$ is the most influential for $Merge_1^1$. Then, we proceed to apply SHAP on this cluster model to obtain the most influential features. ALE plots and Anchor rules[4] can also be computed in a similar fashion after the SHAP values and the most influential output/feature have been computed. This approach is beneficial in the case where the number of input features is significant, as it helps focus directly on the most important feature cluster.

The benefit of a compositional model and the explainability method presented is that it allows us to reason about mood score prediction in a hierarchical fashion. We can reason about the outputs of the cluster models and the various merge models. We can also reason about the outputs of combinations of cluster models. For the mood score prediction problem, where we have 43 features as input, the compositional model and the top-down explainability approach help converge on a narrower set of important clusters/models and features faster while ensuring that the effect of feature correlations is mitigated when using SHAP and other explainability methods.

Furthermore, we can compute the best clusters and models at the population level by following the same top-down approach. For each participant, we hierarchically find the best merge or cluster model in the preceding stage, starting from the output using SHAP. Once found, we sort these models based on their frequency of appearance over the population to find the best overall models at each stage. While most influential features can also be obtained from a monolithic model, these feature influence values will be affected by the interactions between all features considered. A compositional architecture removes some of these interactions through its hierarchical and separated model architecture.

Note that the architecture of the compositional model allows a user to apply explainability methods on any cluster/merge model and analyse the results. Also, we do not compute the SHAP and other explanations on the complete compositional model, as the influence of input feature perturbations or gradient (which is used by most post-hoc explainability methods) will be reduced (almost negligible) due to the hierarchical structure of the compositional model, and hence, such an approach will produce spurious feature importances.

[4]As Anchors only work with classification models, we used the algorithm defined in Appendix A.1 to extend anchors and explain the predictions of the regression models.

6.5 Results

In this section, we report the performance and explainability of monolithic and compositional models. We begin with monolithic results and continue with compositional results and comparisons, where applicable. A discussion subsection at the end interprets these findings. Overall, the monolithic results show how MLP models outperform common baseline ML models for several participants, and the compositional results show where decomposition further improves predictive performance and explainability.

6.5.1 Monolithic Model Results

In this section, we report the results of the best base models and the overall best MLP models, computed as discussed in Section 6.3.5. As we build personalised models, we report each participant's results separately. We present the mean and standard deviation of the MAE for the best models. Finally, we show the SHAP values, the ALE plots, and the Anchor rules for the participant MLP models.

6.5.1.1 Model performance

The best MLP model obtained after optimisation is compared to the best base model for each participant. Table 6.6 shows the average MAE values and their standard deviations (MAE STD) for the best base and MLP models evaluated on the 5-fold cross-validation test sets. All models in Table 6.6 have an MAE of less than or equal to 1, which implies that the difference between the actual mood score and the one predicted is, on average, around 1.

Moreover, the performance (MAE) of the MLP models is better (lower MAE) than the base models for 10 out of 14 participants, as evidenced by the bold values in Table 6.6. Although the model hyperparameter search methodology differs between the MLP and the base models, the comparison indicates how powerful MLP models (a comparatively simple DL method) are at learning meaningful representations for mood scores from digital data. Also, the disparity in model type and performance can be attributed to the differences in the participant datasets used to build the personalised models. Most participant datasets have missing data and high data imbalance (a higher proportion of a specific mood score), and depending on the type of imbalance and the amount of good data available, it can make it difficult or easier for certain models to learn associations from them. Overall, MLP models seem to be better able to learn the representations between the input features and the mood score.

Table 6.6: Metrics for the best base and MLP models for each participant. The values in bold indicate lower MAE. (MAE: Mean Absolute Error; STD: standard deviation of MAE; P-1 to P-29 refer to participants 1 to 29.)

Participant ID	Model	MAE	
		Mean	STD
P-1	Support Vector Classifier	0.336	0.145
	MLP Classifier	**0.295**	**0.151**
P-10	Random Forest Classifier	**0.697**	0.113
	MLP Classifier	0.715	**0.089**
P-12	Support Vector Classifier	**0.575**	0.077
	MLP Regressor	0.582	**0.063**
P-14	Support Vector Classifier	0.904	**0.146**
	MLP Regressor	**0.829**	0.327
P-15	Gradient Boosting Regressor	**0.467**	0.115
	MLP Regressor	0.494	**0.050**
P-18	Random Forest Classifier	0.823	**0.185**
	MLP Classifier	**0.638**	0.472
P-19	Gradient Boosting Regressor	1.072	0.171
	MLP Classifier	**0.989**	**0.096**
P-20	Random Forest Classifier	0.636	**0.181**
	MLP Classifier	**0.455**	0.249
P-21	Random Forest Classifier	1.152	0.298
	MLP Regressor	**1.103**	**0.216**
P-23	Gradient Boosting Regressor	**0.883**	**0.307**
	MLP Classifier	0.936	0.550
P-24	Support Vector Classifier	0.158	**0.074**
	MLP Classifier	**0.125**	0.088
P-26	Gradient Boosting Regressor	1.098	0.213
	MLP Regressor	**1.013**	**0.149**
P-28	Random Forest Classifier	0.609	0.173
	MLP Classifier	**0.562**	**0.163**
P-29	Adaptive Boosting (AdaBoost) Regressor	1.188	0.310
	MLP Classifier	**1.030**	**0.253**

Furthermore, Tables A.2 and A.3 show hyperparameter combinations and model parameters for the best MLP models reported in Table 6.6. We find that Deletion and Manual Imputation are the best methods for handling missing data for the participants used in the study. Also, classification models seem to outperform regression models for both the base and MLP models. Of the 14 best MLP models, 9 are classification models, and 5 are regression models. Moreover, among the optimisers used to optimise the hyperparameters, Bayesian, DE, and PSO are the best methods. There is no visible correlation between the model type, choice of hyperparameters and architecture among the participants, and it seems to be dependent on the participant and the type of model.

Appendices A.2 and A.3 contain more information on the model hyperparameters and architectures used for the best models.

6.5.1.2 MLP model explanation

SHAP explanation:

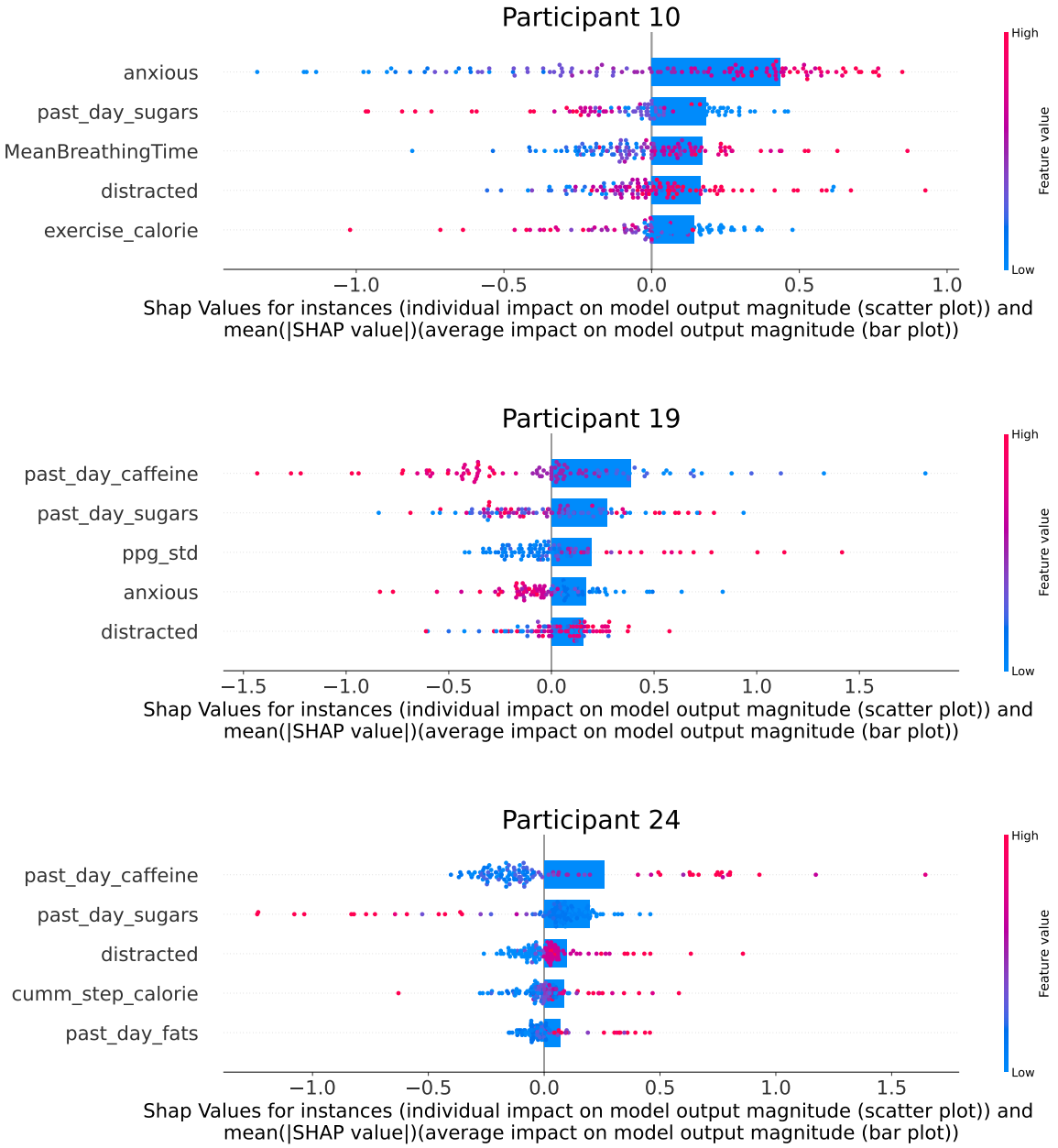
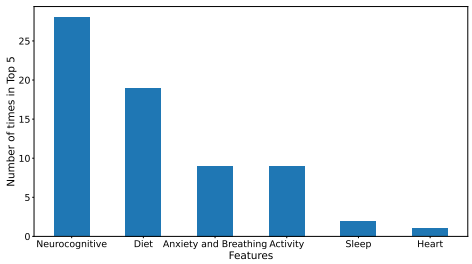
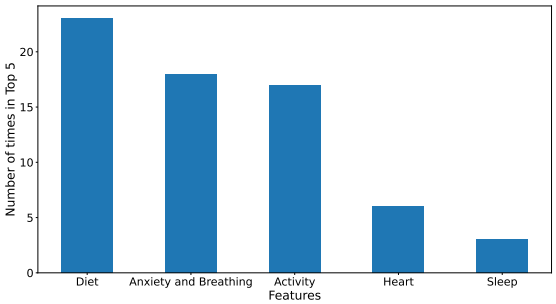


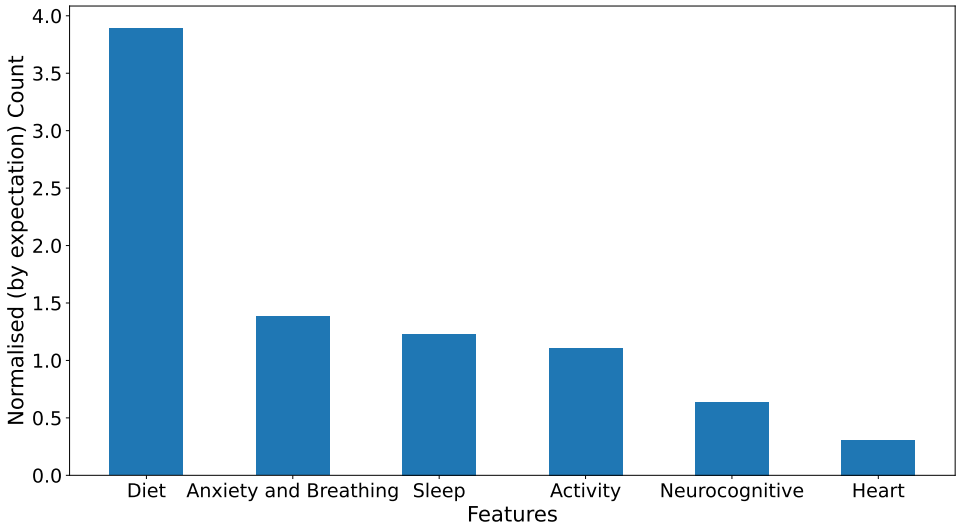
Figure 6.9: The figure shows the SHAP value effects for the top 5 features in the overall best models for Participants 10, 19 and 24. The scatter plots depict the SHAP values for individual samples, with the colour of the points denoting their magnitude. The bar plots superimposed on top show the mean of the absolute value of the SHAP values over all data points. The features are arranged based on the magnitude of the average SHAP values. Plots for all participants can be found in the appendices.



(a) Top feature groups in the overall best models for all participants with neurocognitive features.



(b) Top feature groups in the overall best models for all participants without neurocognitive features.



(c) Top feature groups in the overall best models for all participants with neurocognitive features and corrected for the number of features in each group.

Figure 6.10: The figure shows the top-5 feature groups in the overall best models for all participants. The features are arranged based on the frequency of their appearance in the top-5 features. The groups contain the following features. Diet: *past-day-fats, past-day-sugars, past-day-caffeine*; Anxiety and Breathing: *anxious, distracted, MeanBreathingTime, Consistency*; Activity: *cumm-step-count, cumm-step-calorie, cumm-step-distance, exercise-calorie, exercise-duration*; Heart: *heart-rate,ppg-std*; Sleep: *prev-night-sleep*; and Neurocognitive: all neurocognitive features.

Figure 6.9 shows the five features from the wearable- and EMA-acquired features that affect the model prediction most based on their SHAP values for three example participants. Plots for all participants can be found in Appendix A.6. SHAP explains a prediction (a single prediction) of a data instance by computing the contribution of each feature to the prediction. We take the mean (average) of the absolute SHAP values for all instances in the dataset and sort them to find the top 5 features in the figure.

Figure 6.10 further shows the overall (population-level or for all participants) top feature groups ranked by the number of times they appear in the top-5 features, i.e. by their frequency, with and without the neurocognitive features and corrected for the number of features in each group. The figure 6.10c shows that diet-related features, such as *past-day-sugars*, *past-day-fats*, *past-day-caffeine*, have the highest effect on mood score prediction. This is followed by anxiety-based features (measured through features like *anxious*, *distracted*, and *MeanBreathingTime*). As the neurocognitive group contains a large number of features, the uncorrected top feature groups are dominated by neurocognitive features (see Figure 6.10a). This analysis shows that the normalisation method and the way we group the features can have a significant impact on the top feature groups.

Furthermore, the scatter plot in the figure shows the SHAP value (effect on mood score) for different feature values. There is variability in how certain features affect mood score prediction. For instance, for Participant 10, low values for the feature *anxious* (see blue dots for Participant 10 in Figure 6.9) lead to a decrease in model prediction by 1 in some instances. Therefore, ensuring lower anxiety for this participant could be a suitable intervention. For instance, an increase in *past-day-caffeine* decreases mood score prediction for Participant 19, whereas the opposite is seen for Participant 24.

ALE plot explanation:

Although the scatter plots in the SHAP plots show how high and low values of features affect the model output, they are not good at showing trends. We use ALE plots to see feature trends. Figure 6.11 shows the ALE plots for the top-5 non-neurocognitive features obtained from the overall best models using SHAP value analysis. These plots can help a physician understand how the top predictors influence the model prediction (mood score) with their values. While the plots for some features in some participants are quite simple, others are more complex, denoting a heterogeneity in depressive severity and symptoms. The feature effect values (the ALE values) for participants will differ from SHAP, as ALE plots show only the feature effects separated from any correlation effects between the features (which SHAP does not remove). Also, a positive value (or negative value) on the plot signifies an increase (or decrease) in mood score prediction. The ALE value in the plot shows the magnitude of such effects.

Taking the example of Participant 10, the ALE value of high *anxious* and *distracted* is high. This implies that a high value of both features leads to an increase, i.e. an overall increase in mood score (or depression). The effect of features is not consistent across participants and may seem counterintuitive. For instance, for Participant 19, an increase in *past-day-caffeine* (amount of caffeinated items consumed last day) and *past-day-sugars* (amount of sugary items consumed last day) leads to a decrease in mood score (less depressed). This may seem counterintuitive, as having too many sugary

food items is generally unhealthy. In such instances, other explainable methods, such as SHAP and Anchors (as discussed later), could be used to increase trust in the plots. Comparing these results to the SHAP plot (see the scatter plots in Figure 6.9) for Participant 19, we see that high values of *past-day-caffeine* and *past-day-sugars* decrease the SHAP value or the model prediction of mood score. As the SHAP explanation aligns with the findings of the ALE plots, it increases our trust in the ALE findings.

Figure 6.11 also shows that for some participants, such as participant 24, the feature effect of *distracted* is nearly zero. This does not necessarily mean the feature is unimportant or has no trends. It may happen if one half of the dataset exerts a positive effect and the other half a negative effect, one half cancelling the effect of the other half during averaging. Not much helpful information can be gathered from the plots in such instances. Nevertheless, ALE plots are one of the best for visualising any feature's true effect trends in the presence of interactions/correlation between features.

Anchors explanation:

Finally, we show how Anchors can explain unusual mood scores by finding the best decision rules (predicated on the features) that apply to a prediction. For instance, a participant's mood score may increase by three points (e.g. from 3 to 6), signifying a sudden increase in a depressive mood. By comparing the IF-THEN-based rules obtained from Anchors before and after such changes, we can surmise what may have changed in the observed features to elicit such a mood change.

The Anchors procedure takes in a sample corresponding to a mood score prediction, perturbs the sample to create artificial samples in the neighbourhood of the original sample, and finds the region (rules) in the perturbed neighbourhood where the decision rules do not change the prediction. Since this method can only be used for classification models, we use it on participants where the best model is a classification model. Also, we only use it for instances where the prediction was correct to ensure the rules have fidelity.

Also, Anchors allow the user to specify the maximum number of features to use when finding the rules. This allows the user to choose between concise or comprehensive rules. To show how Anchor rules can be constructed and interpreted, we use anomalous instances from Participants 1, 10 and 24. We begin with the case where the mood score for Participant 1 increases from 3 to 5 within 19 hours, and use a maximum of five features to construct the rules.

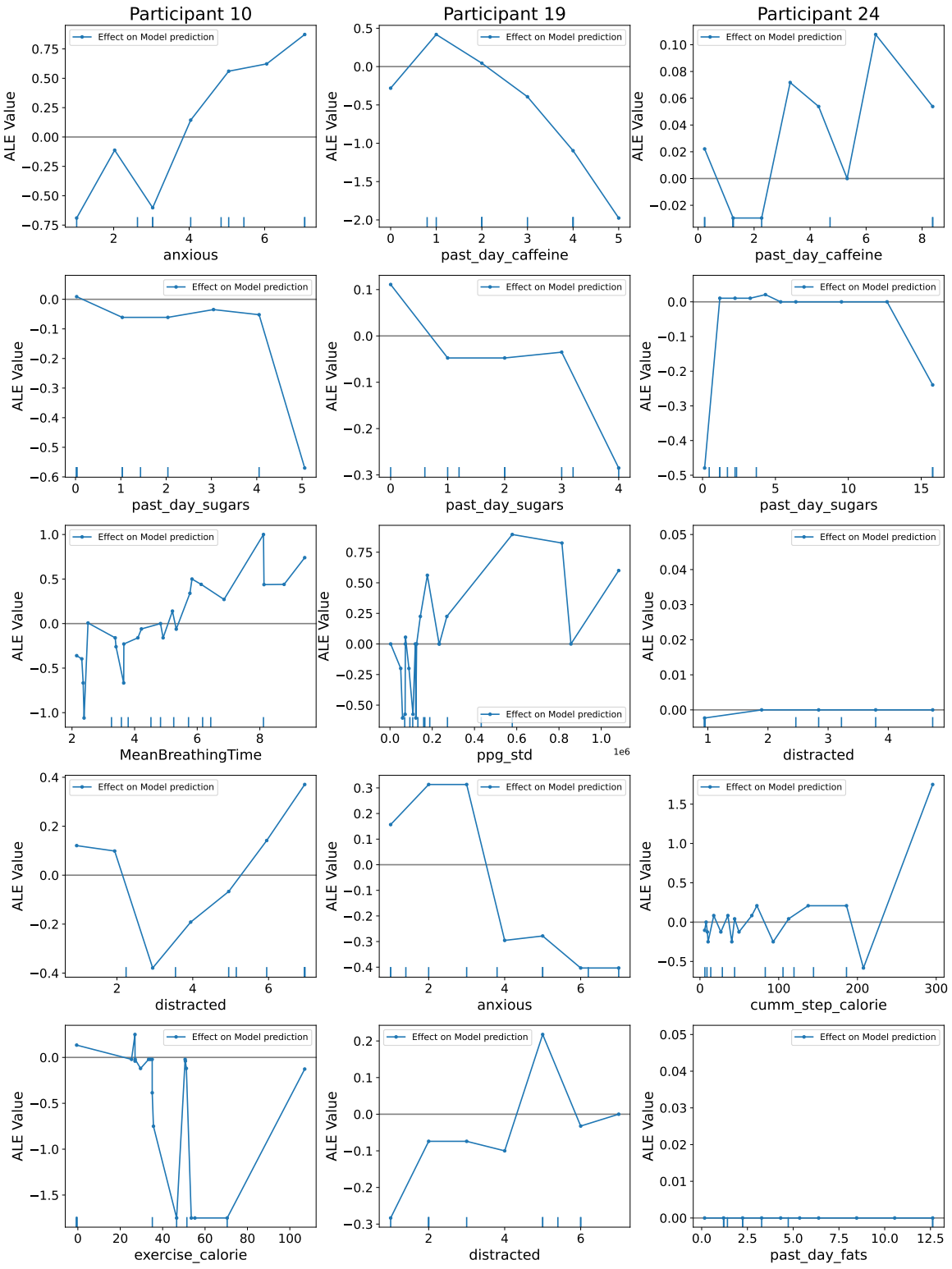


Figure 6.11: The figure shows the Accumulated Local Effects (ALE) plots for the top-5 features in the overall best models for Participants 10, 19 and 24. The x-axis contains the feature values, and the y-axis contains the ALE values. The ALE values denote the magnitude of the average effect of a feature value on the model output, i.e., the mood score. Plots for all participants can be found in the appendices.

$$
\begin{gathered}
IF\ [(past-day-fats \leq 11.40\ items) \wedge (GLbias-dACC > 0.00)]\ THEN; \\
Depressed = 3 \\
Precision : 1 \\
Coverage : 0.20 \\
\\
IF\ [(past{-}day{-}sugars > 25.00\ items) \wedge (distracted > 5.00) \wedge (cumm{-}step{-}calorie \leq 285.62)] \\
THEN; Depressed = 5 \\
Precision : 0.95 \\
Coverage : 0.03
\end{gathered}
\tag{6.3}
$$

Equation 6.3 shows the conditions/rules on the features needed to ensure that the mood score prediction (*Depressed* in the equation) changes from 3 to 5, where $GLbias - dACC$ is the neural activity in the dACC brain region corresponding to bias for frequent gains (see the appendices section in [98]). The first set of conditions pertains to the case when the mood score is 3, and the second set is when the score is 5.

Furthermore, the Precision and Coverage values in the Anchors in Equation 6.3 say that the anchors applied to 20% (3% (in the second set of conditions)) of the perturbation space instances and 100% (95%) of those instances conformed to the rule, i.e. had a prediction of 3 (5). Higher precision and coverage imply greater fidelity of anchor rules to the model behaviour. If we compare the Anchors, we observe that less physical activity, being more distracted and consuming significant amounts of sugary food items (more than 25) are associated with a high mood score/depressive mood. Also, it appears that having fewer fatty food items helps keep the mood score (depression) moderately low (at 3).

Although Anchor rules provide good human-readable explanations for instances, it may be difficult to find rules if the maximum number of features chosen to construct the rules is insufficient. Also, finding more Anchors around the change and future changes should provide a good idea of what feature variations lead to a change in mood score. A similar strategy can also be adopted for other participants to explain normal instances. We could also extend the use of Anchors to describe a period of unusual mood scores by explaining all instances within the period and finding the common rules within the instances. A similar analysis can be obtained for participants not presented here.

6.5.2 Compositional Results

In this section, we present the performance comparison results between the monolithic and compositional personalised mood score prediction models, their performance obtained as discussed in Section 6.3.3. We follow this with a deep dive into how compositional designs can aid explanation using SHAP, ALE plots and Anchor rules, and a comparison of clinical interpretability with monolithic models.

6.5.2.1 Model Performance

Table 6.7: Table containing the metrics for the best monolithic and compositional MLP models for each participant and the compositional approach. The values in bold indicate lower values. (MAE: Mean Absolute Error; STD: standard deviation of MAE; P-1 to P-29 refer to participants 1 to 29.)

ID	Model	MAE		ID	Model	MAE	
		Mean	STD			Mean	STD
P-1	Monolithic	**0.295**	0.151	P-10	Monolithic	0.715	**0.089**
	Approach 1	0.327	0.087		Approach 1	0.778	0.161
	Approach 2	0.345	0.1		Approach 2	**0.673**	0.098
	Approach 3	0.361	**0.086**		Approach 3	0.695	0.139
P-12	Monolithic	0.582	0.063	P-14	Monolithic	0.829	0.327
	Approach 1	**0.565**	0.072		Approach 1	0.876	0.349
	Approach 2	0.577	0.053		Approach 2	0.83	0.426
	Approach 3	0.617	**0.044**		Approach 3	**0.814**	**0.179**
P-15	Monolithic	0.494	**0.05**	P-18	Monolithic	**0.638**	0.472
	Approach 1	**0.4**	0.154		Approach 1	0.686	0.409
	Approach 2	0.441	0.058		Approach 2	0.698	**0.043**
	Approach 3	0.433	0.072		Approach 3	0.67	0.167
P-19	Monolithic	0.989	**0.096**	P-20	Monolithic	**0.455**	0.249
	Approach 1	1.005	0.157		Approach 1	0.586	0.161
	Approach 2	0.912	0.103		Approach 2	0.495	**0.094**
	Approach 3	**0.909**	0.104		Approach 3	0.492	0.126
P-21	Monolithic	1.103	0.216	P-23	Monolithic	0.936	0.55
	Approach 1	1.114	0.218		Approach 1	0.872	0.291
	Approach 2	**1.012**	**0.2**		Approach 2	0.865	0.319
	Approach 3	1.075	0.21		Approach 3	**0.836**	**0.263**
P-24	Monolithic	0.125	0.088	P-26	Monolithic	1.013	0.149
	Approach 1	0.175	0.119		Approach 1	1.037	0.222
	Approach 2	0.158	0.126		Approach 2	**0.96**	**0.118**
	Approach 3	**0.108**	**0.07**		Approach 3	0.992	0.151
P-28	Monolithic	0.563	0.163	P-29	Monolithic	1.03	0.253
	Approach 1	0.59	**0.156**		Approach 1	1.146	**0.141**
	Approach 2	**0.524**	0.178		Approach 2	**0.935**	0.221
	Approach 3	0.619	0.276		Approach 3	0.957	0.22

We perform a performance comparison between the best models for monolithic and the three compositional MLP-based models obtained after optimisation. Table 6.7 shows the average Mean Absolute Error (MAE) and its standard deviation (MAE STD) of the best MLP models calculated over the five folds of the cross-validation procedure.

We can see from the Table that most models have MAEs less than or equal to 1. Furthermore, the compositional models outperform (have the lowest MAE) the monolithic models[5] in 11 out of 14 participant models, as evidenced by the bold values in Table 6.7. Although the optimisation methodology is different for monolithic and compositional models (due to differences in architecture), the results show the strengths of the compositional approach in learning useful representations from mood score data.

We also observe that the best compositional modelling approach varies between participants. This observation can be attributed to the differences in the participant datasets, the inherent representations, and the choice of clusters used to build the compositional models. Most participant datasets have missing values and significant class imbalance—where one mood score predominates—which, depending on the imbalance type and the quantity of reliable data, can either restrict or facilitate a given model's ability to learn meaningful associations. Also, as previously stated, we did not adequately explore all possible combinations of model architectures and cluster combinations, as our goal is to illustrate the benefits of the compositional approach as a whole, and it may be possible to obtain compositional models that exceed the performance of models reported here.

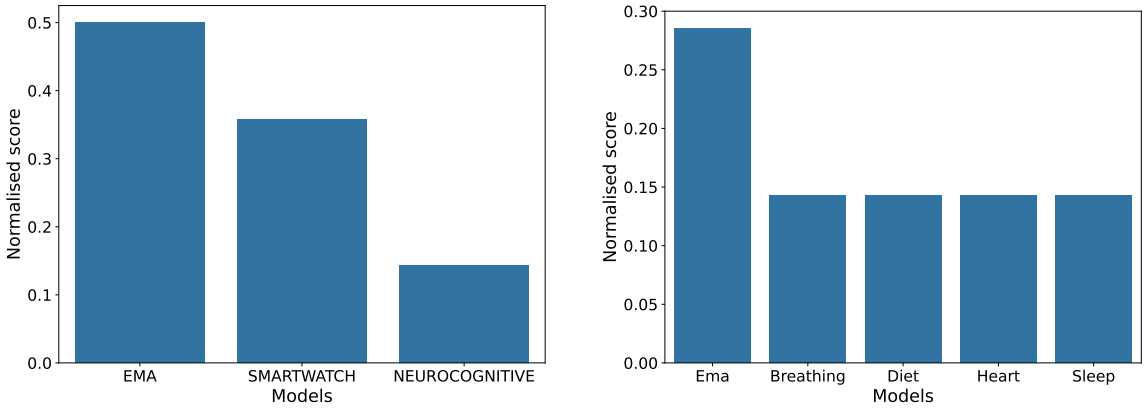
Moreover, we find that Deletion and Manual Imputation are the best methods for handling missing data for the participants used in the study. Also, regression models seem to outperform classification models for the compositional models, in contrast to the monolithic models. There is also no visible correlation between the model type, choice of hyperparameters and MLP architecture among the participants, and it seems to be dependent on the participant and the type of model. The code used to train the models with all the parameters and hyperparameters will be provided upon request.

6.5.2.2 Population-level Importances

We can calculate the best population-level[6] (among 14 participants) cluster and merge models for Participant 19 and Approach 3 using the method mentioned in Section 6.4.4, providing a general overview of the trends. Figure 6.12 shows the bar plots for the best models. We can see that EMA is the best model for Stage 2 Merge models, followed

[5]We only compare with the monolithic models for the best MLP models, as the MLP models were the best models at the population level.

[6]By population, we mean the population sample considered in this study and not the general population.



(a) Best Stage 2 Merge models at the population level

(b) Best Stage 1 Merge models at the population level

Figure 6.12: Normalised Frequency of Best models at the population level for Approach 3.

by Smartwatch and Neurocognitive Merge models. Moreover, Ema, Breathing, and Diet (models that feed into the EMA Merge Model) are the best 3 cluster models at the population level. Also, Breathing, Heart, Diet, and Sleep cluster models have the same importance at the population level.

Thus, we can say that features in the EMA cluster have the most direct and consistent association with mood change. Unfortunately, we cannot perform a direct comparison with the population-level results of [98] as the input features in the clusters are different between their study and ours. To see the best models for other approaches, please see Tables A.15, A.16, and A.17.

Unlike the top feature groups shown in Figure 6.10 for the monolithic models, the best models for the population level are not affected by the normalisation method. As the compositional models are already built on groups of features (i.e. explicit clustering of features), there is no need to group the features again. This reduces the impact of post-processing steps on the top feature groups, allowing us to obtain population-level insights easily. The only exception is the choice and the number of clusters (during the model design phase), which can affect the top feature groups.

6.5.2.3 Compositional Explainability

Although we can present the explainability results (evaluated using the method in Section 6.4.4) for all participants and their three compositional models, to ensure conciseness, we only present and explain the results when using Approach 3 (the most complex architecture we consider) with Participant 19 to demonstrate the explainability pipeline. A similar analysis can be obtained for other compositional architecture approaches and participants whose results are not presented here. Tables A.15, A.16,

and A.17 contain the best models for each stage and for all participants when using Approach 1, 2 and 3, respectively.

We use SHAP to find the top influential/important input features for mood score prediction, use ALE plots to find how the effect of these top input feature values changes with their values, and use anchors to explain individual instances. While we can also look into the top-k, where k is a natural number, input features at each stage of the explanation, we did not present this to ensure brevity.

Approach 3 (presented in Section 6.4.2) uses a model with three stages. We start from the Merge model (in stage 3) in Approach 3 and find the most important model feeding into the Merge model (from stage 2) through SHAP, ALE plots and Anchors. We then find the most important cluster model (Stage 1) for the best Stage 2 model. Finally, the best input features for the cluster model are computed.

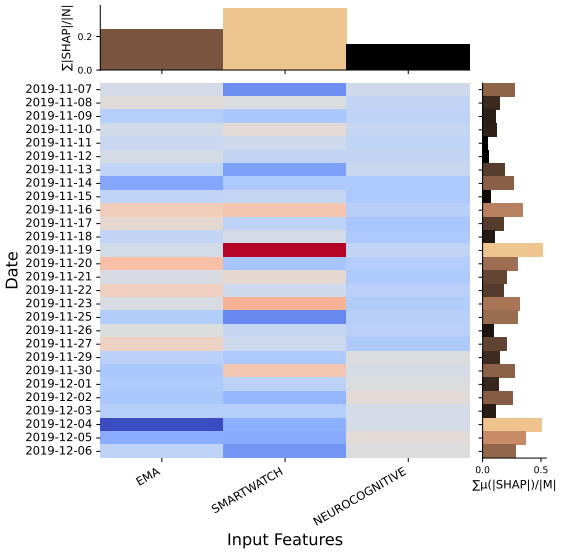
Feature Importances using SHAP:

Figure 6.13 shows the average SHAP values for Participant 19 when using a compositional model developed using Approach 1. Figures 6.13a, 6.13c, and 6.13e show the heatmap plots of the SHAP values averaged over a day. The top and right axes show the average of the absolute SHAP values over all recorded days and the sum of the average SHAP values over the input features (the inputs going into a Cluster/Merge model), respectively. Similarly, figures 6.13b, 6.13d, and 6.13f show the SHAP scatter plots showing how the top-5 feature values affect the output mood score prediction.

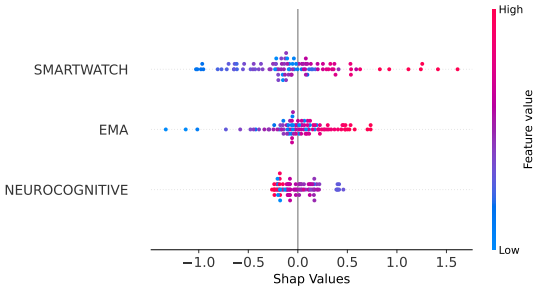
Figure 6.13 is obtained using the top-down approach. First, we compute the top model (from Stage 2) feeding into the Merge model using SHAP values, which is *Smartwatch model*. We then compute the top features for the *Smartwatch model* merge model and find that the *Heart* cluster model is the most influential. Continuing this exercise, we find that *ppg_std* or heart-rate-variability is the most influential feature. This observation indicates that for Participant 19, the input Smartwatch super-cluster is the most influential, particularly the heart-rate-variability feature from its Heart cluster.

Furthermore, the heatmap over the days shows that the effect of *ppg_std* on the participant mood score prediction was mostly negative. This indicates that this feature mainly contributes to reducing mood score prediction and is associated with improving the participant's mood. If needed, other cluster models can also be similarly analysed for their best input features.

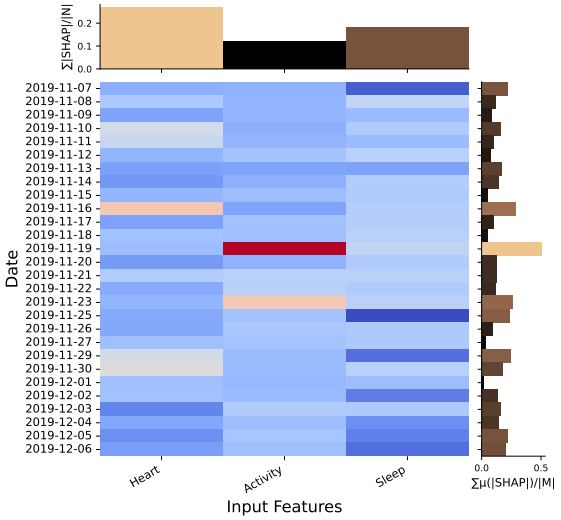
Moreover, the SHAP summary plots from figure 6.13b show that, on the whole, a higher value of the three Stage 2 merge models' outputs increases the value of the mood score prediction (more depressed). Smartwatch is the most influential cluster, followed by EMA and Neurocognitive. Similarly (from figure 6.13d), on the whole, a higher value of the three Stage 1 cluster models that feed into the Smartwatch model



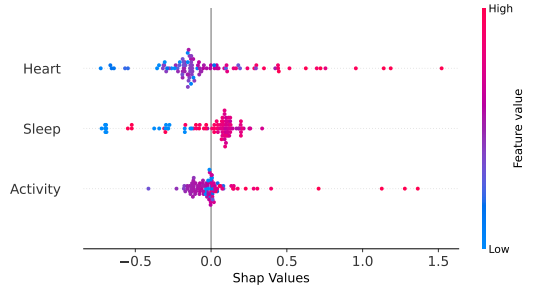
(a) SHAP heatmap for the Merge model



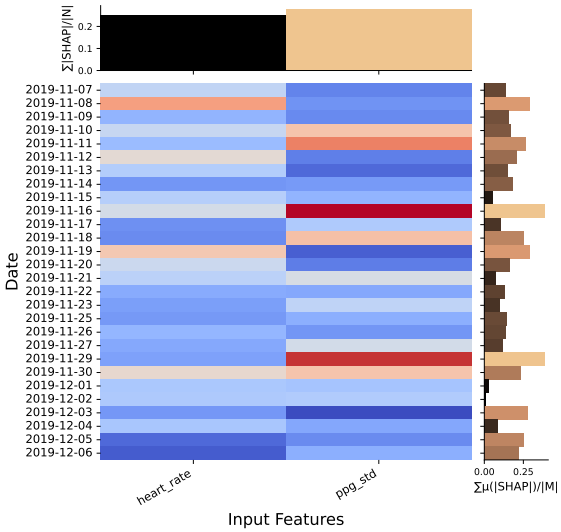
(b) SHAP summary for the Merge model



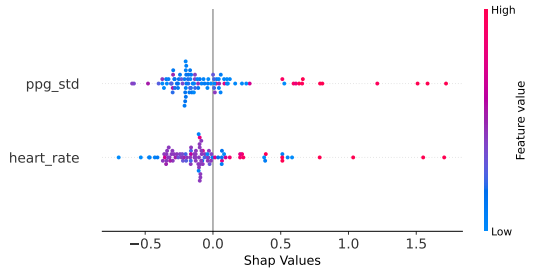
(c) SHAP heatmap for the Smartwatch model



(d) SHAP summary for the Smartwatch model



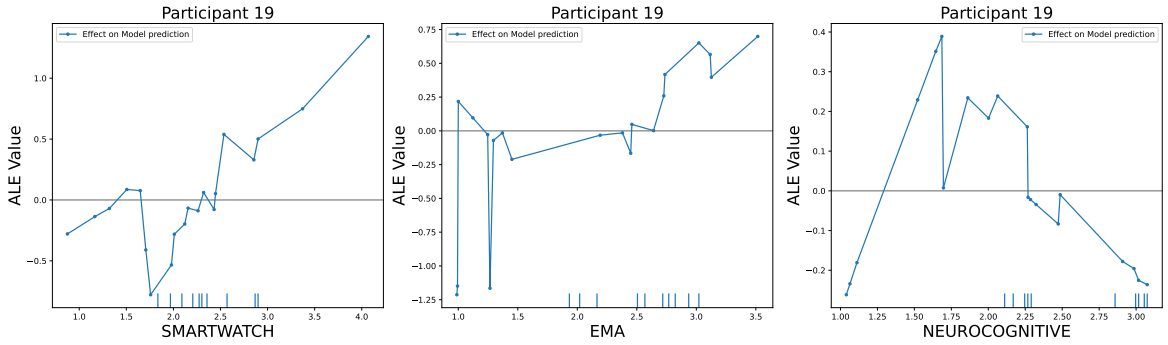
(e) SHAP heatmap for the Heart model



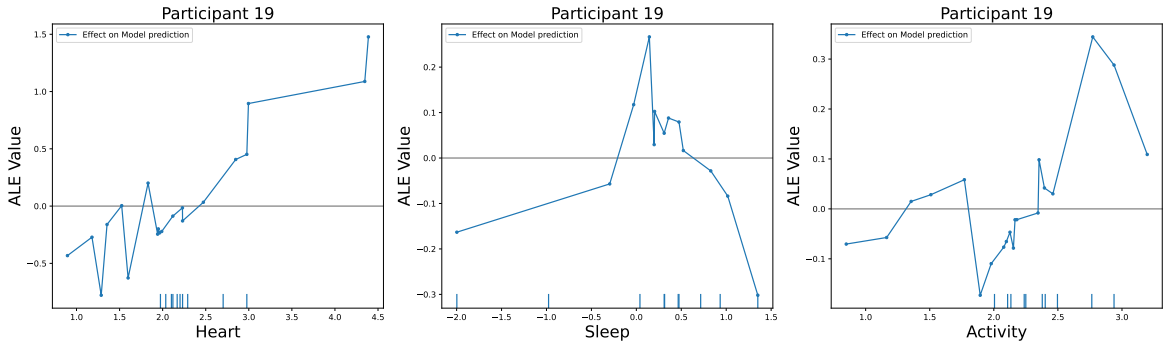
(f) SHAP summary for the Heart model

Figure 6.13: SHAP heatmaps and summary plots for Stage2 (top row), Smartwatch (middle row), and Heart (bottom row) clusters for participant UCSD19 and Approach 3.

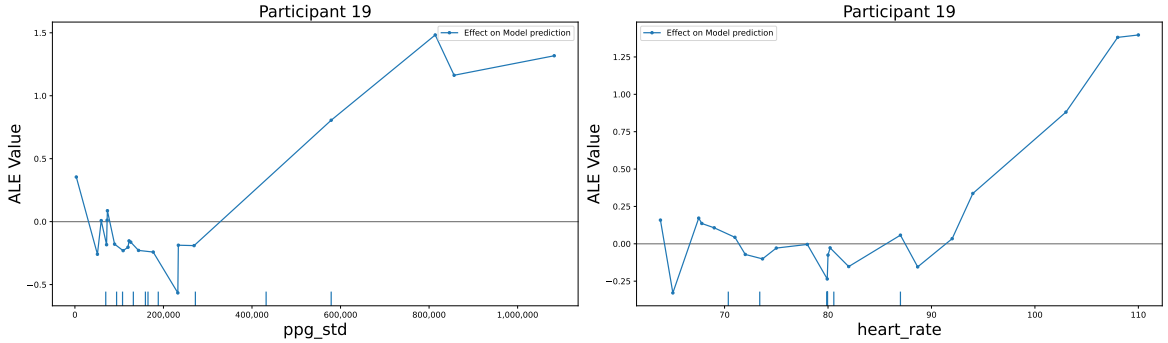
increases the value of the mood score prediction (more depressed). Heart is the most influential cluster, followed by Sleep and Activity. Finally (From figure(6.13f), we also find that lower heart-rate-variability and generally lower heart rate are associated with a lower mood score prediction for this participant. These insights can then be used to tailor personalised interventions.



(a) ALE plot for the Merge model



(b) ALE plot for the Smartwatch model



(c) ALE plot for Heart

Figure 6.14: ALE plots for Participant 19 and Approach 3

ALE Plots for top features:

Figure 6.14 shows the ALE plots for the top input features for the Merge and Diet models obtained using the SHAP analysis discussed in the previous section. A positive value (or negative value) in the plot implies an increase (or decrease) in mood-score prediction.

Figure 6.14 shows the ALE plots for the top input features for the Merge, Smartwatch and Heart models obtained using the SHAP analysis discussed in the previous section. We can see from the figure that the Smartwatch and EMA models, which feed into the Merge model, have an overall increasing trend, indicating that a higher value will most likely increase the mood score (which makes depression worse). However, the Neurocognitive model has an increasing trend for lower values and a decreasing trend for higher values, indicating that Neurocognitive input features that ensure that the model outputs a low or high value can help reduce mood scores.

Also, the ALE plots for the Heart and Activity, which feed into the best Stage 2 model (Smartwatch) model, have an increasing trend. However, the Sleep model does not, indicating that more sleep than baseline sleep can help improve mood. Finally, we see that both *ppg_std* and *heart_rate* from the Heart model contribute to a lower mood score only if their values are lower. For higher values, they increase the mood score. Thus, it is prudent to make sure that activities that keep the heart rate below 90 and the *ppg_std* below 300,000 are used to manage the mood of this participant.

Anchors for Explanation:

Lastly, we demonstrate how Anchors can explain changes in mood scores by identifying the most relevant decision rules—based on the underlying features—that apply to each prediction. For example, suppose a participant's mood rating jumps by 2 points (say, from 2 to 4), indicating a sudden worsening of depressive symptoms. In that case, we can explain the prediction at the moment of change and just before it. By examining the IF-THEN rules generated by Anchors for the periods preceding and following the prediction, we can find which feature alterations likely drove the mood shift. Please note that anchors only explain individual mood predictions, which cannot be generalised over the whole dataset.

$$\begin{aligned}
&\text{IF } \big((\text{Smartwatch} \leq 1.90) \ \wedge\ (\text{EMA} \leq 2.31) \ \wedge\ (\text{Neurocognitive} > 2.11)\big),\\
&\quad \text{THEN } \mathit{Depressed} = 2,\ \ \mathit{Precision} = 0.96,\ \ \mathit{Coverage} = 0.01;\\
&\text{IF } \big((1.90 < \text{Smartwatch} \leq 2.80) \ \wedge\ (\text{EMA} \leq 2.58)\big),\\
&\quad \text{THEN } \mathit{Depressed} = 3,\ \ \mathit{Precision} = 0.96,\ \ \mathit{Coverage} = 0.45.
\end{aligned} \tag{6.4}$$

$$\begin{aligned}
&\text{IF } \big((\text{Sleep} \leq -0.30) \ \wedge\ (\text{Heart} \leq 1.98) \ \wedge\ (\text{Activity} \leq 2.20)\big),\\
&\quad \text{THEN } \mathit{Depressed} = 2,\ \ \mathit{Precision} = 0.94,\ \ \mathit{Coverage} = 0.03;\\
&\text{IF } \big((\text{Activity} > 2.05) \ \wedge\ (1.98 < \text{Heart} \leq 2.83) \ \wedge\ (\text{Sleep} \leq 0.83)\big),\\
&\quad \text{THEN } \mathit{Depressed} = 3,\ \ \mathit{Precision} = 0.96,\ \ \mathit{Coverage} = 0.44.
\end{aligned} \tag{6.5}$$

$$\begin{aligned}
&\text{IF } \big((117501.12 < \text{ppg_std} \leq 270472.29) \ \wedge \ (74.97 < \text{heart_rate} \leq 80.00)\big),\\
&\quad \text{THEN } \mathit{Depressed} = 2, \ \ \mathit{Precision} = 0.50, \ \ \mathit{Coverage} = 0.17;\\
&\text{IF } \big((\text{ppg_std} \leq 140671.59) \ \wedge \ (\text{heart_rate} \leq 92.00)\big),\\
&\quad \text{THEN } \mathit{Depressed} = 3, \ \ \mathit{Precision} = 0.99, \ \ \mathit{Coverage} = 0.55.
\end{aligned} \tag{6.6}$$

Let us consider the case/data point where the mood score for Participant 19 increases from two to three in 4 hrs. We first find the anchor rules for the Merge model, followed by those for the Smartwatch model (the best model of the Merge model). Finally, we compute the anchor rules for the best model for the Smartwatch model, which is the Heart cluster model. We can also find the anchor rules for other cluster models (if needed), but we avoid them here for conciseness.

In equations 6.4, 6.5, and 6.6, we present the anchor rules for the Merge model, Smartwatch model and the Heart model, respectively, where the first set of conditions pertains to the case when the mood score is 2, and the second set is when the score is 3. If we compare the anchor rules, we can see that, in this instance, a higher mood score/depressive mood is associated with a higher value of the Smartwatch model output, a higher value of the Heart and Activity models, and a lower value of feature *ppg_std* and a slightly higher heart rate. Thus, ensuring a lower heart rate variability and heart rate through lower activity and monitoring using a smartwatch shows potential in managing the mood of this participant. These rules can be monitored over a few days to extract patterns in the rules and decide on the best intervention.

6.5.2.4 Clinical Significance and Comparison with Monolithic Models

Overall, the compositional model explainability approach allows a clinician to focus on the most important feature groups and features of a model in a guided hierarchical manner. For instance, in the case of Participant 19, if the clinician wants to obtain the most influential features for the compositional model, they can focus on the SMARTWATCH model, followed by the Heart model and finally the *ppg_std* and *heart_rate* features due to the separation of features and the sub-models used to form the compositional model. This analysis reveals that reducing the heart rate variability and heart rate, both monitored using a smartwatch, shows potential in managing the mood of this participant. A clinician can then use anchor rules to extract the exact range of these features associated with a lower mood score.

Similarly, a clinician may focus on other models, such as the EMA model, followed by its best input models and features. As presented in the previous sections, additional hierarchical analyses of mood changes can be performed to obtain more insights. In

fact, the very design of compositional models allows a clinician to choose a cluster model instead of the whole compositional model if the cluster model is found expressive enough or if a smaller model is needed.

Such analyses are impossible with a monolithic model, where the clinician must focus on the large monolithic model and all its input features, as there is no separation of features and the MLP neurons used to form the model. All neurons are interconnected, and this makes individual analysis of features difficult. Even though SHAP and other approaches can be used to analyse the monolithic model, they may fail to disentangle the feature importances when features are multicollinear.

This limitation with monolithic models could explain the differences in the top features between the monolithic (Diet) and compositional (Heart) models for Participant 19. However, the effect of differences in model architecture and the explainability techniques (which are model dependent) used to analyse the models could also explain the differences in the top features. Overall, monolithic models require that the clinician have a deeper understanding of the model and the explainability techniques to obtain a correct analysis and interpretation of the model behaviour.

6.5.3 Discussion

The monolithic and compositional results show that personalised mood score prediction benefits from MLP models over the baseline classifiers reported in Section 6.5, and that decomposition can improve both error metrics and interpretability for a subset of participants. Table 6.7 shows compositional MLPs matching or beating monolithic MAE for several participants (e.g. P-10, P-14, P-18, P-20, P-23, P-24, P-26), while monolithic models remain stronger for others (e.g. P-1, P-15). There was no single compositional approach (1, 2, or 3) that was uniformly best across the cohort; hyperparameter optimisation was applied per model, but the experiments did not search automatically over the number of stages or cluster groupings, so the reported architectures are illustrative rather than globally optimal.

The explainability results help explain why compositional designs were pursued alongside raw accuracy. For Participant 19, hierarchical SHAP, ALE, and Anchor analyses route attention through the merge model to the smartwatch branch and then to heart-related features, rather than spreading importance across all inputs in one monolithic MLP. That pattern is difficult to obtain when every feature competes within a single network, even with post-hoc methods, and it matches the clinical reading in Section 6.5.2: lower heart-rate variability and activity may associate with higher mood scores for this individual. Monolithic explanations for the same participant emphasise diet-related inputs instead, which is consistent with entangled features in one model rather than with compositional structure alone being wrong.

Compositionality also changes how explanations should be used in practice. Cluster-level sub-models allow SHAP, ALE, and Anchors to be applied within sleep, activity, or heart domains, supporting focused intervention advice and easier identification of which branch underperforms. Merge models remain less transparent than leaf clusters; clusters may not capture cross-domain interactions unless the merge stage learns them, and training several models increases computational cost. Results depend on imputation, missing wearable readings, and short longitudinal records per participant; explainability here describes associations learned by the model, not established causes, and should be checked across SHAP, ALE, and Anchors when predictive performance is modest.

Relative to the autonomous driving case study in Chapter 5, this application does not close a real-time control loop over a physical plant. It still fits the broader Cyber-Physical System (CPS) setting described in Chapter 2: wearable and phone sensors measure behavioural and physiological state, software processes those signals, and outputs (mood scores and explanations) inform monitoring and self-management. Processing is offline on stored trajectories, which precludes the same Worst-Case Execution Time (WCET) and enforcement guarantees as the Autonomous Vehicle (AV) chapter, but it reflects how many health analytics systems are deployed. In this context, compositional modelling mainly advances personalised prediction and explainability rather than hard real-time safety bounds.

6.6 Summary

In this chapter, we introduced monolithic and compositional modelling and explainability frameworks for personalised mood score prediction from multimodal data. Using a one-month dataset of 14 mild to moderately depressed participants, we imputed missing values, trained and optimised MLP models with eight evolutionary and statistical algorithms, validated performance with 5-fold cross-validation, and compared SHAP, ALE, and Anchors explanations at personal and population levels.

Compositional MLPs improved or matched monolithic performance for many participants while providing hierarchical, cluster-aligned explanations that monolithic models do not support directly. The framework is model-agnostic and can be extended to other architectures and modalities. Future work includes explainability for temporal dynamics, automated selection of compositional architectures, and evaluation on larger longitudinal cohorts.

Chapter 7

Conclusion

7.1 Summary

The widespread adoption of Machine Learning (ML) in safety-critical Cyber-Physical Systems (CPSs) has reshaped the way we create and deploy intelligent control systems. Whether it is autonomous vehicles navigating busy city streets or medical devices tracking patient vitals, ML-driven solutions are now making high-stakes decisions that directly affect human lives. At the same time, this shift has exposed new hurdles—traditional, monolithic ML models were not built with rigorous verification, precise timing analysis, strong safety guarantees, or clear explainability in mind. Balancing the intricate demands of these systems against the "black-box" nature of deep neural networks presents a significant challenge: we want the speed and accuracy ML offers, but we cannot sacrifice the safety that critical applications require.

In this thesis, we tackle that challenge head-on by proposing a compositional ML paradigm tailored for CPS. Instead of using an ML model as one large block, we break it down into smaller sub-models, with similar functional behaviour, that can be independently analysed and then composed back together. This modular approach preserves the original network's behaviour while giving us the tools to verify timing properties, construct safer models by design, and offer better explanations for each component's decisions. By weaving compositionality into the very fabric of ML-based CPSs, we lay the groundwork for systems that are not only powerful but also provably safe, time-predictable, and inherently interpretable. The contributions of this thesis are summarised as follows:

1. **Compositional Model Development and Hardware Implementation (Chapter 3):** We introduced the formation of compositional ANN models using a systematic decomposition of monolithic ANN models into smaller, functionally equivalent sub-models. We demonstrated this methodology across multiple real-world datasets, including transportation data for autonomous vehicle lane-

changing and depressive mood score prediction, showing that compositional designs can achieve significant reductions in hardware footprint (up to 53% reduction in resources) and computational complexity (up to 40% reduction in computations) while maintaining comparable performance to monolithic approaches.

It is also here that we developed novel compilation frameworks that translate Python-based ANN models to both Very High-Speed Integrated Circuit Hardware Description Language (VHDL) (for Field Programmable Gate Array (FPGA) implementation) and C code (for embedded systems), enabling formal Worst-Case Execution Time (WCET) analysis. Our compilers specifically support the compositional architecture proposed in this work, addressing a critical gap in existing tools that cannot handle compositional ML designs with ANNs. We demonstrated that compositional models can achieve up to 85% reduction in WCET compared to monolithic implementations, making them suitable for real-time applications with strict timing constraints.

2. **Safe-by-Construction Design (Chapter 4):** We introduced a policy-driven framework for building safe-by-construction ML-based CPSs that addresses the fundamental challenge of ensuring safety in data-driven approaches. Using an autonomous vehicle as a case study, we demonstrated how safety properties (described as Valued Discrete Timed Automata (VDTA)) can be mined from real-world data and used to guide the design and training of compositional models of various kinds (e.g. Artificial Neural Networks (ANNs), Support Vector Machines (SVMs), etc.). We show how our training approach reduces the number of unsafe scenarios. Our Runtime Enforcement (RE) mechanism monitors policy-trained model outputs in real-time and ensures policy adherence, providing a fail-safe for unexpected scenarios where learned models may fail to comply with safety expectations. This approach bridges the gap between formal verification and ML, enabling the development of systems that are both performant and provably safe.

3. **Explainable ML Models (Chapter 5):** We developed a novel explainability paradigm for both monolithic and compositional ML models that addresses the critical need for explainability in safety-critical applications (e.g. medical devices) using ML models. Our approach combines multiple explainability techniques—Shapley Additive Explanations (SHAPs), Accumulated Local Effects (ALEs), and Anchors—to provide actionable insights at both the sub-model and system levels. Using a personalised depression prediction case study, we demonstrated how compositional models can offer enhanced explainability by providing more detailed explanations compared to monolithic approaches, enabling healthcare professionals to better understand how different factors contribute to patient outcomes and make informed intervention decisions.

Throughout the thesis, we validated our compositional framework through comprehensive case studies in two critical domains: autonomous vehicles and healthcare. These case studies demonstrate the practical applicability of our approach and provide concrete evidence of its benefits in terms of performance, safety, and explainability. The results show that compositional ML can address the fundamental limitations of current approaches while maintaining (and in some cases enhancing) the performance benefits that make ML attractive for CPS applications.

The implications of this work extend beyond the specific applications considered in this thesis. The compositional framework we have developed provides a general methodology that can be applied to any domain where ML systems must meet rigorous safety, timing, and explainability requirements. As ML continues to permeate safety-critical applications—from medical devices and autonomous vehicles to industrial control systems and aerospace applications—the need for compositional approaches will only grow more urgent.

7.2 Limitations and Future Directions

Despite the significant progress made in this thesis, several limitations remain that open up promising directions for future research in compositional ML for cyber-physical systems:

1. **Advanced Compositional Models and Architectures:**

    - We have only considered the use of MLP-based models in a feedforward connection in our case studies. Future work could explore the use of more sophisticated compositional patterns beyond the simple parallel and sequential compositions considered in this thesis. This includes feedback loops and dynamic compositions that can adapt their structure based on runtime conditions (see Figure 7.1). Furthermore, the use of more sophisticated models (e.g. Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), etc.) is an important direction for future work as it will allow us to handle more complex data and tasks.
    - We would also like to test the applicability of our compositional framework to more case studies, like the one shown in Figure 7.2, where we could use the compositional framework to predict the economic impact of the Covid-19 pandemic. We use a model to predict the number of cases and deaths and pass this into Stock and Currency models (with other inputs) to predict the economic impact. This compositional design separates concerns and makes the decision flow more apparent.

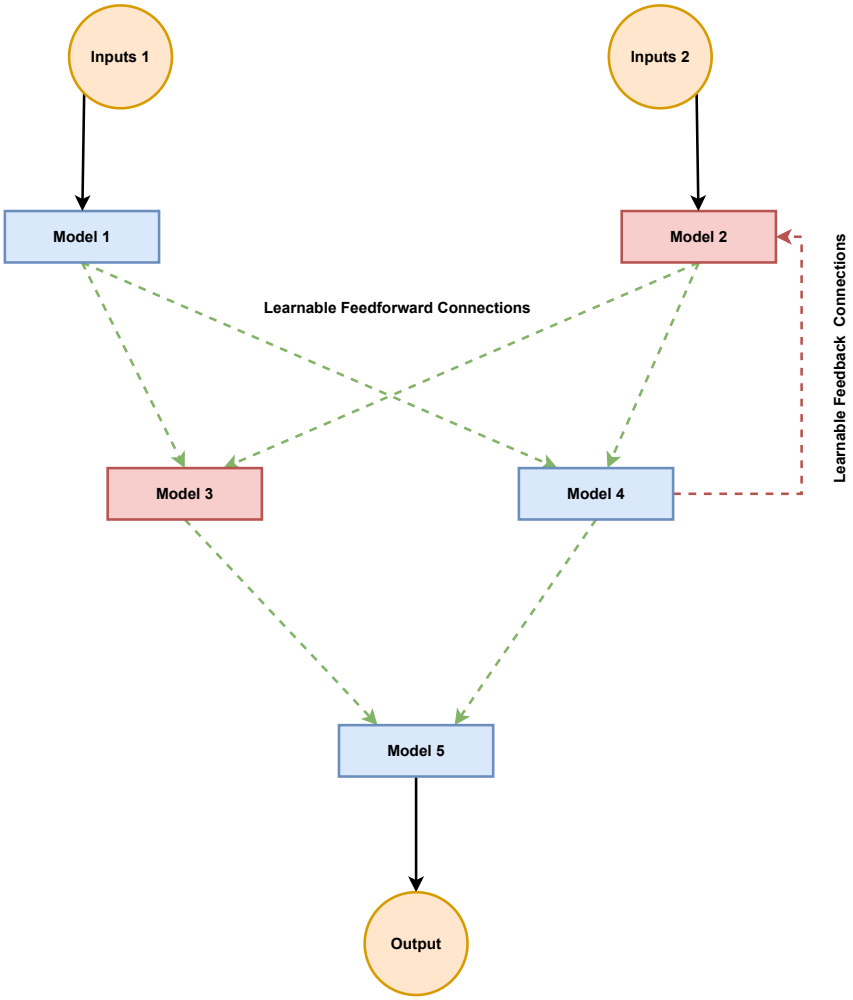


Figure 7.1: An updated compositional architecture for ML-based CPSs with inter-model connections that are learnt and not statically defined. The architecture is also composed of models of multiple types (red and blue boxes) that are combined in a hierarchical manner, as well as feedback loops.

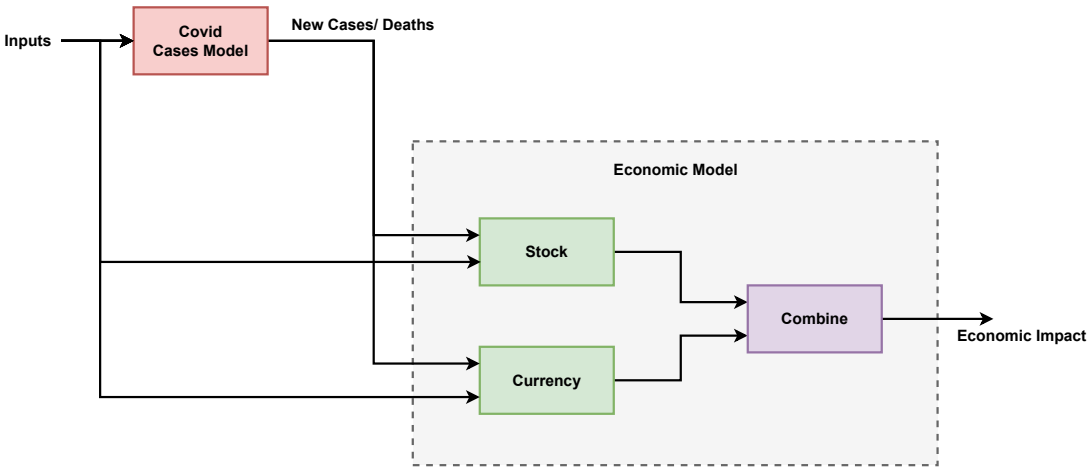


Figure 7.2: Compositional model for Covid-19 Economic Impact Prediction.

- While we focused on manual decomposition of monolithic models, either from a domain expert or a pre-defined policy structure, future work could explore automated methods for learning optimal compositional structures from data. This could include Reinforcement Learning (RL) approaches that discover compositions that optimise for performance, safety, and explainability simultaneously.
- We have only considered small to medium-sized ANNs in our case studies. Future work could explore the use of larger ANNs (e.g. Large Language Models (LLMs)) and their compositional architectures, especially in the context of CPSs that require high-performance and low-latency.

2. **Integration with Formal Temporal Specifications:** Our work establishes the theoretical foundation for compositional policy-based training of ML-based CPSs and their runtime enforcement, but we do not use temporal specifications, which are frequently used to specify safety properties of CPSs. Future work could explore the integration of temporal specifications (such as Linear Temporal Logic (LTL) [175], Computational Tree Logic (CTL) [176] and STL [177]) to specify safety properties of CPSs and use them to guide the training of compositional models. The training will be challenging as a new methodology will have to be developed to handle the temporal specifications, which span multiple time-steps. The runtime enforcement toolchain we used [1] will have to be updated to handle the temporal specifications, especially for STL specifications (see Figure 7.3 for an example modelling flow).

3. **Scalability Analysis:** The scalability (time and memory) of the compositional methodologies described have not been formally tested with larger networks and datasets. Future work could explore the scalability of the compositional methodologies described in this thesis.

4. **Standardisation and Tooling:** As compositional approaches gain adoption, there will be a need for standardised interfaces, tools, and methodologies. Although we have developed a compiler for VHDL and C code, a framework for policy-based training of ML-based CPSs, and a compositional explainability framework, we have not developed the interfaces to make them easily usable by practitioners across different domains. Future work could focus on developing open-source frameworks and tools that make compositional ML accessible to practitioners across different domains. Also, the toolchain will have to be updated to handle the feedback loops in the compositional models and any addition of new types of models.

[1] https://github.com/PRETgroup/easy-rte

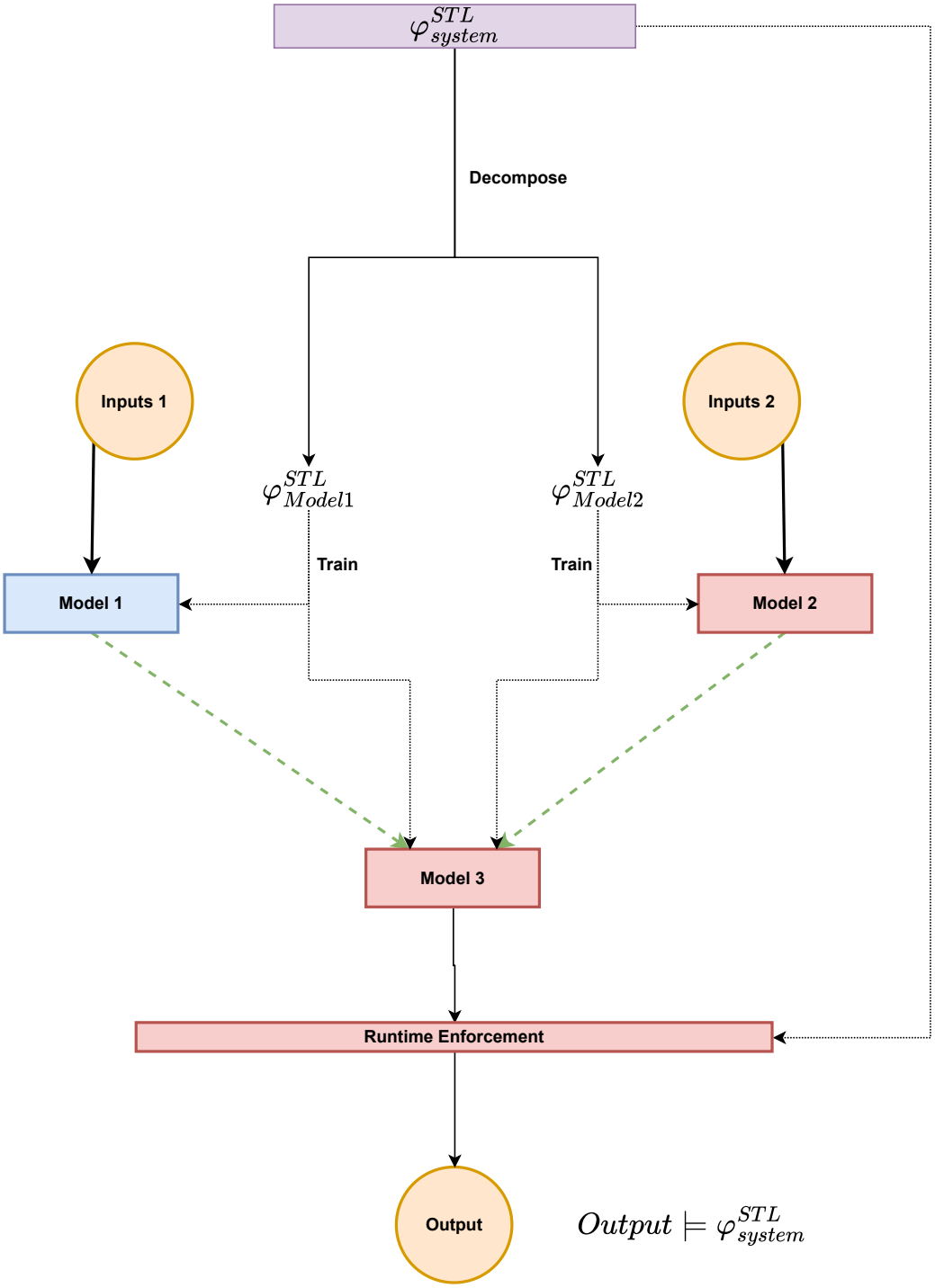


Figure 7.3: Updating the compositional training framework to Signal Temporal Logic (STL) specifications. The system-level specification is decomposed into sub-model specifications and used to guide the training of the compositional models and the runtime enforcement of the compositional models. The decomposition is done so that the sub-model specifications are not contradictory and the system-level specification is satisfied.

5. **Enhancing Compositional Explainability:** Our compositional explainability framework is a novel approach to explainability that combines compositionality with multiple post-hoc explainability techniques. However, it requires the manual analysis of plots and conditionals, which may reduce its utility for non-domain experts. Developing a framework that provides explanations in a more accessible format (e.g. natural language) is an important direction for future work. Also, experimentation with other ML models, explainability techniques, and the inclusion of explanations that comment on causal relationships are important directions for future work.

7.3 Final Remarks

The integration of ML into safety-critical cyber-physical systems represents one of the most significant technological challenges of our time. While ML offers unprecedented capabilities for handling complex, nonlinear relationships in real-world systems, its black-box nature and lack of formal guarantees create fundamental barriers to adoption in safety-critical domains. This thesis demonstrates that compositionality provides a structured solution to these challenges, enabling the development of ML systems that are not only more performant but also more time-analysable, implementable, safer, and trustworthy than their monolithic counterparts.

The compositional framework we have developed represents a significant shift in how we think about ML for safety-critical applications. Rather than treating ML models as black boxes that must be wrapped with safety mechanisms, our approach embeds safety, verification, and explainability into the very structure of the models themselves. This fundamental change in perspective opens new possibilities for deploying ML in domains where reliability and trust are paramount.

As we move toward an increasingly automated and intelligent world, the principles and methodologies developed in this thesis will become essential for ensuring that the benefits of ML can be safely and reliably deployed in applications that directly impact human safety and well-being. While significant challenges remain in extension of the compositional framework, the foundation we have established provides a clear path forward for developing ML systems that can meet the rigorous requirements of safety-critical applications while delivering the performance benefits that make ML so valuable. The future of safe, reliable, and trustworthy AI systems lies in compositionality, and we are closer to realising that future.

Appendix A

Additional Information on Models for Mood Score Prediction

A.1 Conversion of Regression Model Outputs to Classification Model Outputs for Anchors Explanation

The Algorithm 7 serves to convert any real input $r \in [0, 6]$ into a probability distribution over the integer grid $\{0, 1, \ldots, 6\}$. This approach leverages the intuitive concept of "distance to integer" as a foundation for probabilistic assignment, while carefully controlling numerical precision via temperature scaling and softmax stabilization. This yields a flexible mechanism to interpolate between hard rounding (very small T) and uniform uncertainty (large T), all within a rigorously safe and shape-preserving framework. The inspiration for this algorithm comes from [37].

A.2 MLP model hyperparameters

Table A.1: Search values (or limits) for parameters during hyperparameter optimisation of the Depression Dataset for the monolithic models. (N.A. means Not Applicable)

Monolithic			
Hyperparameter	**Lower Limit**	**Upper Limit**	**Values**
Number of layers	2	4	N.A
Neurons in input layer	50	90	N.A
Activation	N.A	N.A	tanh, relu, elu, linear
Batch size	N.A	N.A	4, 6, 8

Table A.2 shows the combination of data preprocessing methodology, the problem type and the metric used to optimise the model hyperparameters. The optimiser for

Algorithm 7 Softmax-over-Distance Probability Mapping

Require: r ▷ scalar or vector with values in $[0, 6]$
Require: $T > 0$ ▷ temperature parameter
Ensure: p ▷ probabilities over $\{0, \ldots, 6\}$
$\mathbf{r} \leftarrow \texttt{as\_array}(r)$
if $\exists\, x \in \mathbf{r} \colon x < 0$ or $x > 6$ **then**
    error "All r values must lie in $[0, 6]$."
end if
reshape $\mathbf{r}$ into a flat vector $\mathbf{r}_{\text{flat}}$ ▷ now of length n
$\mathbf{I} \leftarrow [0, 1, 2, 3, 4, 5, 6]$
▷ Compute pairwise distances:
for $i = 1$ to n **do**
    for $j = 0$ to 6 **do**
        $D_{i,j} \leftarrow \left|\mathbf{r}_{\text{flat}}[i] - j\right|$
    end for
end for
▷ Form logits by negating and scaling distances:
$L_{i,j} \leftarrow -\,D_{i,j}\,/\,T \quad \forall\, i, j$
▷ Apply numerically-stable softmax row-wise:
for $i = 1$ to n **do**
    $m \leftarrow \max_j(L_{i,j})$
    for $j = 0$ to 6 **do**
        $e_{i,j} \leftarrow \exp\big(L_{i,j} - m\big)$
    end for
    $S \leftarrow \sum_{j=0}^{6} e_{i,j}$
    for $j = 0$ to 6 **do**
        $p_{i,j} \leftarrow e_{i,j}/S$
    end for
end for
▷ Reshape output:
if r was scalar **then**
    return $p_{1,*}$ ▷ single length-7 vector
else
    reshape p to shape $(\text{shape}(r), 7)$
    return p
end if

which we obtain the overall best model is also provided. Table A.3 shows the model parameters for the overall best models. It presents the number of neurons in the DNN, the activation used in the layers and the batch size used during the training of the models. The table shows that most models have at least 3 layers (2 hidden layers and one output layer) and use either a ReLU or a Linear activation. However, for participants 15, 18, and 28, the DNN only has one hidden layer. Furthermore, a batch size of six or four samples works best with most participants.

Table A.2: Combinations Yielding the Overall Best Models. *Metric* refers to the metric used to optimise the model hyperparameters, and *Optimiser* refers to the best optimiser (used for hyperparameter optimisation) corresponding to the overall best model. (BA: Balanced Accuracy, F1: F1 score, MAE: Mean Absolute Error, MAPE: Mean Absolute Percentage Error)

Participant	Data Pre-processing	Problem Type	Optimiser	Metric
1	Deletion	Classification	DE	BA
10	Manual Imputation	Classification	Bayesian	F1
12	Deletion	Regression	CMA-DE	MAE
14	Deletion	Regression	PSO	MAE
15	Manual Imputation	Regression	PSO	MAE
18	Manual Imputation	Classification	Bayesian	BA
19	Deletion	Classification	Bayesian	BA
20	Deletion	Classification	DE	BA
21	Manual Imputation	Regression	ES	MAE
23	Manual Imputation	Classification	DE	BA
24	Automatic Imputation	Classification	Bayesian	F1
26	Manual Imputation	Regression	PSO	MAE
28	Deletion	Classification	PSO	F1
29	Deletion	Classification	Bayesian	F1

A.3 Base model hyperparameters

The grid search is implemented in Python using a combination of Python libraries. The Scikit-learn [178, 179] library is used for the modelling and further information on the model parameters reported in the following subsections can be found on their API page.

1. **Adaboost Classifier:** The grid-search parameters are: Estimators: {50, 100, 200}, Learning Rate: {0.5, 1, 2, 10}, Algorithm: {'SAMME', 'SAMME.R'}. The best model parameters for each participant are shown in Table A.4.

Table A.3: Parameters of the overall Best Models. *Neurons* column contains the neurons for each layer (hidden + output layers) in the DNN. (ReLU: Rectified Linear Unit, ELU: Exponential Linear Unit, Tanh: Hyperbolic tangent, Linear: No nonlinear activation)

Participant	Neurons	Activation	Batch Size
1	{38, 7}	ReLU	6
10	{60, 33, 7}	Linear	8
12	{46, 31, 16, 1}	Linear	4
14	{30, 20, 10, 1}	ReLU	4
15	{48, 1}	ReLU	6
18	{52, 7}	Linear	6
19	{35, 21, 7}	Linear	6
20	{45, 32, 19, 7}	ELU	6
21	{50, 25, 1}	Tanh	4
23	{36, 21, 7}	ReLU	8
24	{51, 29, 7}	Linear	6
26	{46, 31, 16, 1}	ReLU	4
28	{60, 7}	ReLU	8
29	{51, 36, 21, 7}	ReLU	4

Table A.4: Parameters of the Adaboost classifier for the best model after grid search.

Participant	Data Preprocessing	Estimators	Learning rate	Algorithm
1	Impute	200	1	SAMME.R
10	Preprocessed	50	0.5	SAMME
12	Deletion	100	0.5	SAMME.R
14	Deletion	200	2	SAMME.R
15	Deletion	200	0.5	SAMME
18	Impute	200	10	SAMME
19	Deletion	50	0.5	SAMME
20	Deletion	200	1	SAMME
21	Preprocessed	50	2	SAMME
23	Impute	200	10	SAMME.R
24	Deletion	50	1	SAMME.R
26	Impute	100	0.5	SAMME
28	Deletion	50	0.5	SAMME
29	Deletion	100	1	SAMME

2. **Adaboost Regressor:** The grid-search parameters are: Estimators: {50, 100, 200}, Learning Rate: {0.5, 1, 2, 10}, Loss: {linear, square, exponential}. The best model parameters for each participant are shown in Table A.5.

Table A.5: Parameters of the Adaboost Regressor for the best model after grid search.

Participant	**Data Preprocessing**	**Estimators**	**Learning rate**	**Loss**
1	Deletion	50	2	exponential
10	Preprocessed	50	0.5	exponential
12	Deletion	100	1	square
14	Deletion	50	0.5	exponential
15	Deletion	200	2	linear
18	Preprocessed	50	2	linear
19	Impute	50	0.5	linear
20	Deletion	200	2	linear
21	Deletion	50	1	square
23	Preprocessed	200	10	exponential
24	Deletion	50	1	square
26	Preprocessed	100	10	exponential
28	Impute	100	10	exponential
29	Deletion	50	1	square

3. **Elasticnet Regressor:** The grid-search parameters are: Alpha: {0.5, 1, 2, 10}, L1-Ratio: {0, 0.5, 1}, Selection: {random, cyclic}. The best model parameters for each participant are shown in Table A.6.

Table A.6: Parameters of the Elasticnet Regressor for the best model after grid search.

Participant	Data Preprocessing	Alpha	L1-Ratio	Selection
1	Impute	2	0	cyclic
10	Impute	0.5	0	cyclic
12	Deletion	0.5	0	random
14	Deletion	0.5	0.5	random
15	Preprocessed	10	0	cyclic
18	Deletion	1	0	random
19	Deletion	1	0	random
20	Deletion	2	0	cyclic
21	Impute	10	0	cyclic
23	Deletion	0.5	0.5	cyclic
24	Deletion	10	0	cyclic
26	Impute	0.5	0	cyclic
28	Deletion	1	0	random
29	Deletion	10	0	cyclic

4. **Gradient Boosting Classifier:** The grid-search parameters are: Loss: {log-loss}, Learning Rate: {0.05, 0.1, 1, 10}, Estimators: {50, 100, 200}, Criterion: {friedman-mse, squared-error}, CCP-Alpha: {0.0, 1, 10}. The best model parameters for each participant are shown in Table A.7.

Table A.7: Parameters of the Gradient Boosting Classifier for the best model after grid search.

Participant	Data Preprocessing	Loss	Learning Rate	Estimators	Criterion	CCP-Alpha
1	Deletion	log-loss	0.05	50	friedman-mse	0.0
10	Impute	log-loss	0.05	50	friedman-mse	0.0
12	Deletion	log-loss	0.05	50	friedman-mse	0.0
14	Preprocessed	log-loss	0.05	50	friedman-mse	1
15	Impute	log-loss	0.05	100	friedman-mse	10
18	Deletion	log-loss	0.05	50	friedman-mse	0.0
19	Deletion	log-loss	0.1	200	friedman-mse	0.0
20	Deletion	log-loss	10	100	squared-error	0.0
21	Deletion	log-loss	0.1	50	squared-error	1
23	Preprocessed	log-loss	0.05	200	squared-error	1
24	Deletion	log-loss	0.1	200	squared-error	1
26	Preprocessed	log-loss	0.05	50	squared-error	10
28	Impute	log-loss	0.05	200	friedman-mse	1
29	Preprocessed	log-loss	0.05	50	squared-error	1

5. **Support Vector Regressor:** The grid-search parameters are: C: {0.1, 1, 5, 10}, Kernel: {linear, poly, rbf, sigmoid}, Degree: {2, 3, 4, 5}, Gamma: {0.1, 1, scale, auto}. The best model parameters for each participant are shown in Table A.8.

Table A.8: Parameters of the Support Vector Regressor for the best model after grid search.

Participant	Data Preprocessing	C	Kernel	Degree	Gamma
1	Preprocessed	0.1	sigmoid	0	auto
10	Impute	1	sigmoid	0	auto
12	Deletion	1	rbf	0	0.1
14	Deletion	5	rbf	0	auto
15	Deletion	0.1	sigmoid	0	auto
18	Preprocessed	5	sigmoid	0	0.1
19	Deletion	0.1	sigmoid	0	0.1
20	Deletion	0.1	poly	4	scale
21	Preprocessed	1	poly	4	scale
23	Impute	0.1	poly	2	auto
24	Impute	5	linear	0	scale
26	Preprocessed	0.1	poly	5	scale
28	Deletion	1	poly	2	scale
29	Deletion	1	poly	2	0.1

6. **Gradient Boosting Regressor:** The grid-search parameters are: Loss: {squared-error, absolute-error, huber, quantile}, Learning Rate: {0.05, 0.1, 1, 10}, Estimators: {50, 100, 200}, Criterion: {friedman-mse, squared-error}, CCP-Alpha: {0.0, 1, 10}. The best model parameters for each participant are shown in Table A.9.

Table A.9: Parameters of the Gradient Boosting Regressor for the best model after grid search.

Participant	Data Preprocessing	Loss	Learning Rate	Estimators	Criterion	CCP-Alpha
1	Preprocessed	absolute-error	0.05	50	squared-error	0.0
10	Preprocessed	squared-error	0.05	50	squared-error	0.0
12	Deletion	absolute-error	0.1	200	friedman-mse	0.0
14	Impute	huber	0.1	200	friedman-mse	0.0
15	Preprocessed	absolute-error	1	50	squared-error	10
18	Preprocessed	absolute-error	1	50	squared-error	1
19	Impute	squared-error	0.05	100	friedman-mse	0.0
20	Deletion	absolute-error	0.05	50	friedman-mse	1
21	Deletion	absolute-error	1	100	squared-error	10
23	Impute	absolute-error	0.1	50	friedman-mse	10
24	Deletion	absolute-error	10	50	friedman-mse	10
26	Preprocessed	absolute-error	1	50	squared-error	10
28	Preprocessed	absolute-error	0.1	50	squared-error	1
29	Deletion	absolute-error	1	50	friedman-mse	0.0

7. **Random Forest Classifier:** The grid-search parameters are: Estimators: {50, 100, 200}, Criterion: {gini, entropy, log-loss}, Max Depth: {None, 5, 10, 20}, Min-Samples-split: {2, 5, 10}, Max Features: {sqrt, log2, None}. The best model parameters for each participant are shown in Table A.10.

Table A.10: Parameters of the Random Forest Classifier for the best model after grid search.

Participant	Data Preprocessing	Estimators	Criterion	Max Depth	Min-Samples-Split	Max Features
1	Preprocessed	200	log-loss	10	10	sqrt
10	Preprocessed	200	log-loss	5	2	None
12	Deletion	100	log-loss	5	2	None
14	Deletion	200	entropy	20	2	sqrt
15	Deletion	50	log-loss	20	5	log2
18	Impute	50	entropy	20	5	None
19	Deletion	50	entropy	5	2	None
20	Deletion	100	log-loss	5	5	sqrt
21	Deletion	100	log-loss	None	2	sqrt
23	Deletion	200	entropy	5	10	log2
24	Deletion	200	log-loss	20	5	None
26	Deletion	100	gini	5	5	log2
28	Deletion	200	gini	5	10	sqrt
29	Deletion	200	gini	20	2	None

8. **Poisson Regressor:** The grid-search parameters are: Alpha: {0.5, 1, 2, 10}, Solver: {lbfgs, newton-cholesky}, Max-Iteration: {100, 200, 500}. The best model parameters for each participant are shown in Table A.11.

Table A.11: Parameters of the Poisson Regressor for the best model after grid search.

Participant	**Data Preprocessing**	**Alpha**	**Solver**	**Max-Iteration**
1	Impute	10	lbfgs	500
10	Impute	2	lbfgs	200
12	Deletion	2	lbfgs	500
14	Impute	1	newton-cholesky	200
15	Deletion	10	lbfgs	500
18	Preprocessed	10	newton-cholesky	100
19	Deletion	2	lbfgs	500
20	Deletion	10	lbfgs	500
21	Deletion	10	lbfgs	200
23	Preprocessed	10	newton-cholesky	500
24	Deletion	10	lbfgs	500
26	Impute	2	lbfgs	500
28	Impute	2	lbfgs	200
29	Deletion	10	newton-cholesky	200

9. **Support Vector Classifier:** The grid-search parameters are: C: {0.1, 1, 5, 10}, Kernel: {linear, poly, rbf, sigmoid}, Degree: {2, 3, 4, 5}, Gamma: {0.1, 1, scale, auto}. The best model parameters for each participant are shown in Table A.12.

Table A.12: Parameters of the Support Vector Classifier for the best model after grid search.

Participant	Data Preprocessing	C	Kernel	Degree	Gamma
1	Preprocessed	5	poly	2	auto
10	Preprocessed	10	rbf	0	0.1
12	Deletion	1	sigmoid	0	0.1
14	Deletion	0.1	linear	0	scale
15	Preprocessed	0.1	rbf	0	auto
18	Impute	0.1	poly	2	scale
19	Deletion	5	linear	0	auto
20	Deletion	1	sigmoid	0	auto
21	Preprocessed	1	poly	4	auto
23	Deletion	0.1	sigmoid	0	scale
24	Impute	10	rbf	0	scale
26	Preprocessed	0.1	rbf	0	auto
28	Preprocessed	1	rbf	0	scale
29	Deletion	5	poly	4	scale

10. **Random Forest Regressor:** The grid-search parameters are: Estimators: {50, 100, 200}, Criterion: {squared-error, absolute-error, friedman-mse, poisson}, Max Depth: {None, 5, 10, 20}, Min-Samples-split: {2, 5, 10}, Max Features: {sqrt, log2, None}. The best model parameters for each participant are shown in Table A.13.

Table A.13: Parameters of the Random Forest Regressor for the best model after grid search.

Participant	Data Preprocessing	Estimators	Criterion	Max Depth	Min-Samples-Split	Max Features
1	Impute	50	absolute-error	5	2	log2
10	Impute	200	poisson	5	10	None
12	Deletion	50	friedman-mse	10	5	log2
14	Deletion	50	absolute-error	10	5	log2
15	Preprocessed	50	squared-error	None	10	log2
18	Deletion	50	absolute-error	5	2	log2
19	Preprocessed	50	poisson	20	2	sqrt
20	Deletion	50	poisson	10	5	sqrt
21	Deletion	50	absolute-error	20	10	log2
23	Deletion	100	absolute-error	5	2	log2
24	Deletion	50	absolute-error	5	10	log2
26	Preprocessed	200	absolute-error	5	2	log2
28	Preprocessed	200	absolute-error	5	10	sqrt
29	Deletion	50	friedman-mse	5	2	sqrt

A.4 Compositional Model Information

A.4.1 Model Features

The following are the input features for each cluster model used in Approach 1.

1. Diet: past_day_fats, past_day_sugars, past_day_caffeine, time_of_day
2. Activity: cumm_step_calorie, cumm_step_distance, cumm_step_count, exercise_calorie, exercise_duration, time_of_day
3. Physiological: heart_rate, ppg_std, MeanBreathingTime, Consistency, time_of_day (excluding deleted columns)
4. Psychological: distracted, anxious, time_of_day
5. Neurocognitive: gw_consistency, gw_efficiency, mf_consistency, mf_efficiency, ls_span, ls_consistency, ls_efficiency, fo_consistency_all, fo_efficiency_all, MeanBreathing-Time_TT, Consistency_TT, LD_RareG_diff, LD_GL_bias, gw_leftDLPFC, mf_leftDLPFC, ls_leftDLPFC, fo_leftDLPFC, gw_dACC, mf_dACC, ls_dACC, fo_dACC, TT_leftDLPFC, TT_dACC, diff_rareLG_leftDLPFC, diff_rareLG_dACC, GLbias_leftDLPFC, GLbias_dACC, time_of_day
6. Sleep: prev_night_sleep, time_of_day (excluding deleted columns)

The following are the input features for each cluster model used in Approach 2.

1. Discrete: anxious, distracted, past_day_fats, past_day_sugars, past_day_caffeine, time_of_day
2. Continuous: MeanBreathingTime, Consistency, cumm_step_calorie, cumm_step_distance, cumm_step_count, exercise_calorie, exercise_duration, heart_rate, ppg_std, prev_night_sleep
3. Neuro: gw_consistency, gw_efficiency, mf_consistency, mf_efficiency, ls_span, ls_consistency, ls_efficiency, fo_consistency_all, fo_efficiency_all, MeanBreathingTime_TT, Consistency_TT, LD_RareG_diff, LD_GL_bias, gw_leftDLPFC, mf_leftDLPFC, ls_leftDLPFC, fo_leftDLPFC, gw_dACC, mf_dACC, ls_dACC, fo_dACC, TT_leftDLPFC, TT_dACC, diff_rareLG_leftDLPFC, diff_rareLG_dACC, GLbias_leftDLPFC, GLbias_dACC

The following are the input features for each cluster model used in Approach 3.

1. Diet: past_day_fats, past_day_sugars, past_day_caffeine
2. EMA (Ecological Momentary Assessment): anxious, distracted, time_of_day
3. Breathing: MeanBreathingTime, Consistency, MeanBreathingTime_TT, Consistency_TT
4. DACC (dorsal Anterior Cingulate Cortex): mf_dACC, ls_dACC, fo_dACC, TT_dACC, gw_dACC, GLbias_dACC, diff_rareLG_dACC

5. DLPFC (Dorsolateral Prefrontal Cortex): mf_leftDLPFC, ls_leftDLPFC, fo_leftDLPFC, TT_leftDLPFC, gw_leftDLPFC, GLbias_leftDLPFC, diff_rareLG_leftDLPFC

6. Efficiency: mf_efficiency, ls_efficiency, fo_efficiency_all, gw_efficiency

7. Consistency: mf_consistency, ls_consistency, fo_consistency_all, gw_consistency

8. LD (Learning/Decision): LD_RareG_diff, LD_GL_bias

A.4.2 Model Hyperparameters and Additional Results Tables

Table A.14: Search values (or limits) for parameters during hyperparameters optimisation for the compositional models. (N.A. means Not Applicable)

Hyperparameter	**Lower Limit**	**Upper Limit**	**Values**
Number of layers	2	4	N.A
Neurons in input layer	30	60	N.A
Activation	N.A	N.A	tanh, relu, elu, linear
Batch size	N.A	N.A	4, 6, 8

Table A.15: Best merge and cluster models for Approach 1. Best models at population levels are Neurocognitive, Diet and Physiological, all in stage 1.

Participant	**Best Merge Model**
ucsd1	Neurocognititive
ucsd10	Psychological
ucsd12	Neurocognititive
ucsd14	Neurocognititive
ucsd15	Neurocognititive
ucsd18	Physiological
ucsd19	Diet
ucsd20	Diet
ucsd21	Physiological
ucsd23	Diet
ucsd24	Sleep
ucsd26	Diet
ucsd28	Neurocognititive
ucsd29	Psychological

Table A.16: Best merge and cluster models for Approach 2. Best models at population level are Neurocognitive (stage 2) and Continuous (stage 1).

Participant	**Best Merge Model**	**Best Cluster Model**
ucsd1	Stage1	Discrete
ucsd10	Neurocognitive	Continuous
ucsd12	Neurocognitive	Continuous
ucsd14	Neurocognitive	Discrete
ucsd15	Stage1	Continuous
ucsd18	Neurocognitive	Discrete
ucsd19	Neurocognitive	Continuous
ucsd20	Neurocognitive	Continuous
ucsd21	Neurocognitive	Discrete
ucsd23	Neurocognitive	Continuous
ucsd24	Neurocognitive	Continuous
ucsd26	Neurocognitive	Discrete
ucsd28	Neurocognitive	Continuous
ucsd29	Neurocognitive	Continuous

Table A.17: Best merge and cluster models for Approach 3.

Participant	**Best Merge Model**	**Best Cluster Model**
ucsd1	Neurocognitive	DAcc
ucsd10	EMA	Ema
ucsd12	EMA	Ema
ucsd14	Smartwatch	Sleep
ucsd15	EMA	Breathing
ucsd18	EMA	Ema
ucsd19	Smartwatch	Heart
ucsd20	Smartwatch	Activity
ucsd21	Smartwatch	Heart
ucsd23	EMA	Ema
ucsd24	EMA	Diet
ucsd26	EMA	Diet
ucsd28	Neurocognitive	Breathing
ucsd29	SMARTWATCH	Sleep

A.5 Correlation Plot

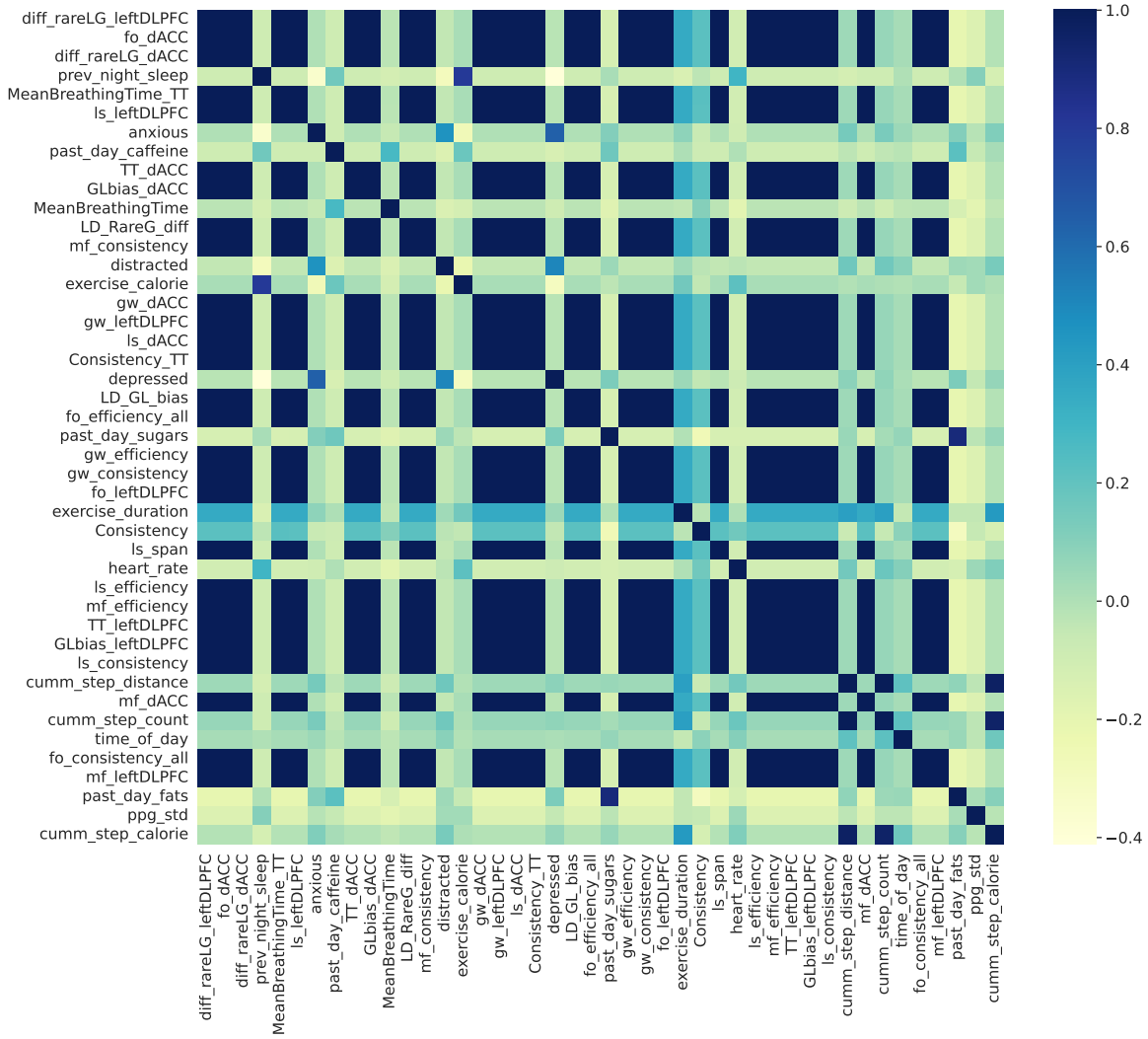


Figure A.1: The figure shows the correlation plot for all the input features and output feature used in the models with each other. The bar on the right shows which colour corresponds to which value of correlation. We use Spearman Rank correlation.

A.6 Additional Plots

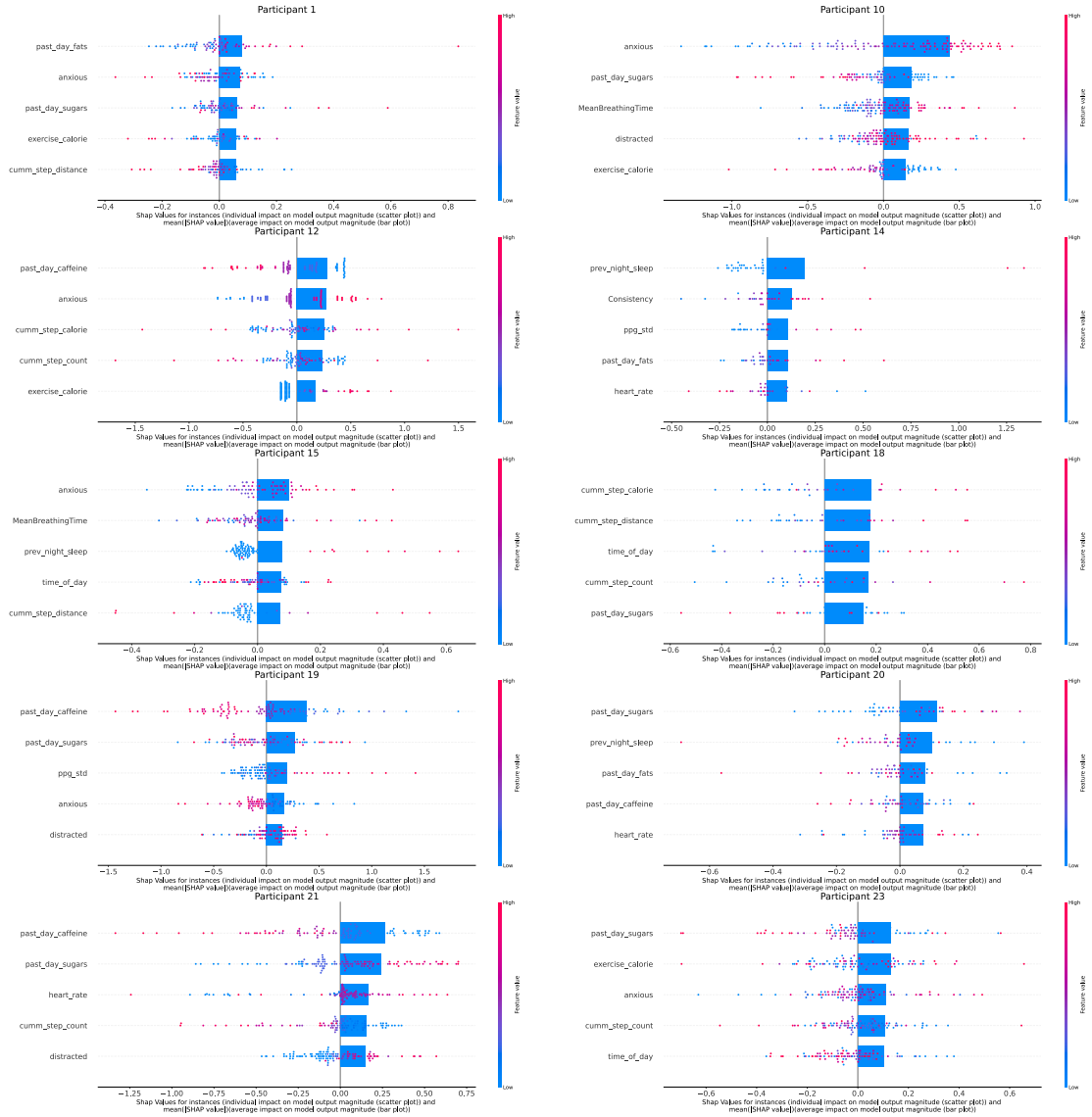


Figure A.2: This figure shows the SHAP value effects for the top-5 features in the overall best models for all participants. The scatter plots depict the SHAP values for individual samples, with the colour of the points denoting their magnitude. The bar plots superimposed on top show the mean of the absolute value of the SHAP values over all data points. Figure continues on next page.

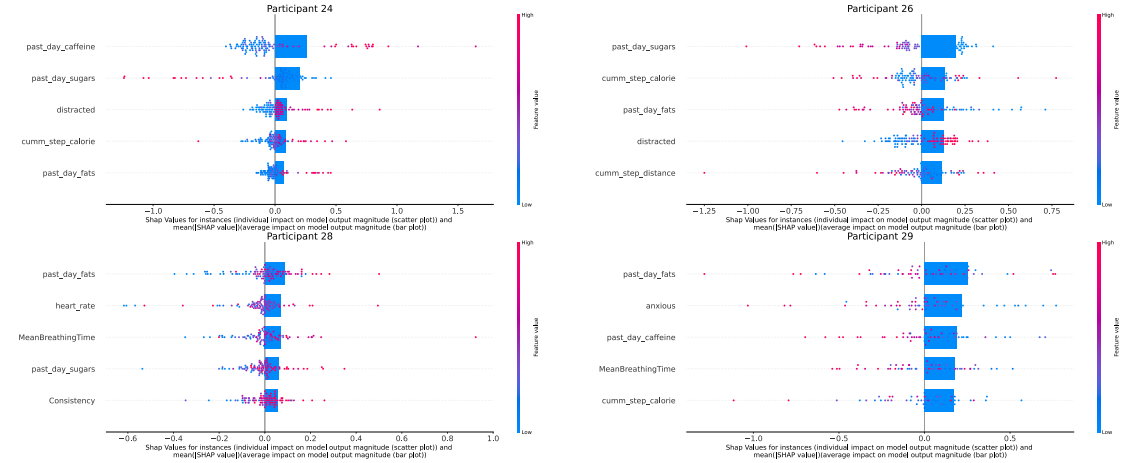


Figure A.3: Continued from previous page. This figure shows the SHAP value effects for the top-5 features in the overall best models for all participants. The scatter plots depict the SHAP values for individual samples, with the colour of the points denoting their magnitude. The bar plots superimposed on top show the mean of the absolute value of the SHAP values over all data points. Figure continues on next page.

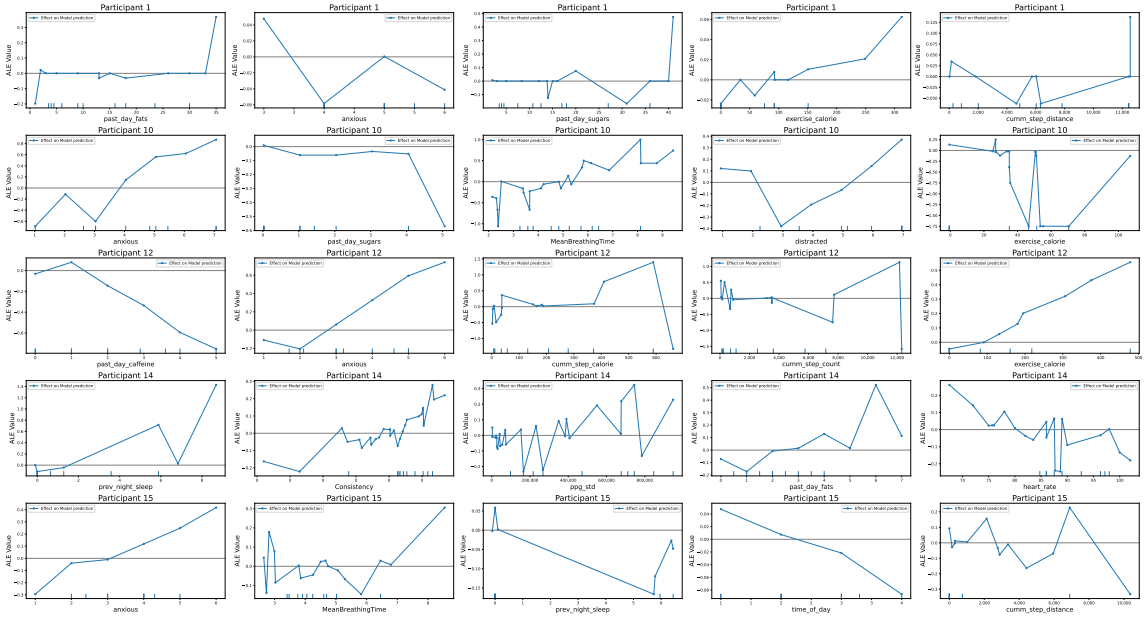


Figure A.4: This figure shows the Accumulated Local Effects (ALE) plots for the top-5 features in the overall best models for all participants. The x-axis contains the feature values, and the y-axis contains the ALE values. The ALE values denote the magnitude of the average effect of a feature value on the model output, i.e., the mood score. Figure continues on next page. Figure continues on next page.

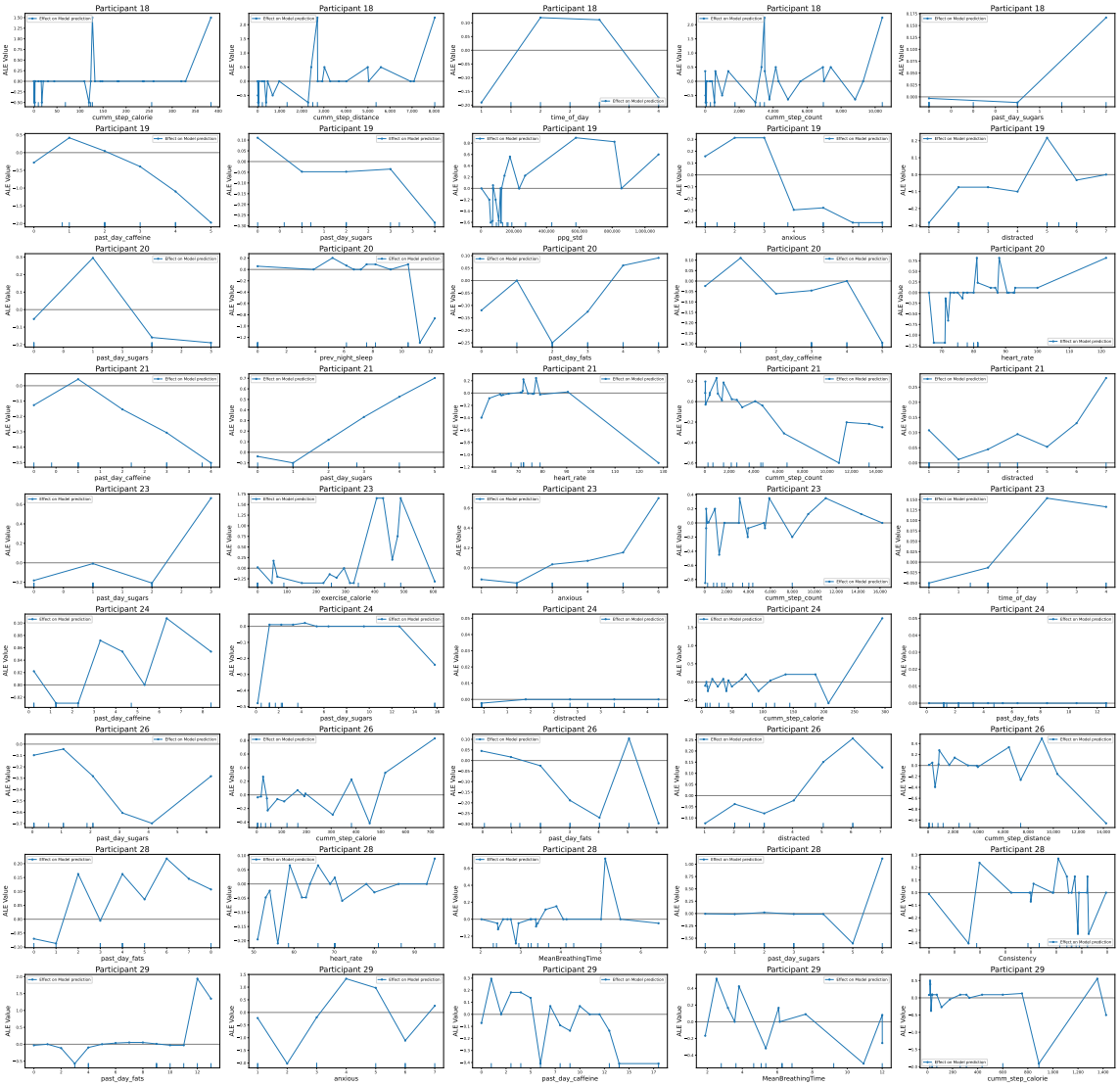


Figure A.5: Continued from previous page. This figure shows the Accumulated Local Effects (ALE) plots for the top-5 features in the overall best models for all participants. The x-axis contains the feature values, and the y-axis contains the ALE values. The ALE values denote the magnitude of the average effect of a feature value on the model output, i.e., the mood score. Figure continues on next page.

Appendix B

Compositional Policy-based training

B.1 Proof of Proposition 1

Informal Proof: The formula φ is a compositional policy, i.e., a well-formed formula constructed inductively from simpler formulas $\{\varphi_1, \ldots, \varphi_n\}$ using logical connectives (e.g., $\wedge$, $\vee$, $\neg$, $\rightarrow$) and quantifiers (e.g., $\forall$, $\exists$). Each sub-formula φ_i corresponds to a learned sub-model M_i, and the Boolean structure of φ is implemented by *Merge* blocks that simulate the appropriate logical operations.

Assume that each M_i approximates the behavior of the intended interpretation of φ_i, i.e., for a significant fraction of inputs $x \in \mathcal{X}_{\text{val}}$, the evaluation $M_i(x)$ satisfies φ_i under interpretation I. Moreover, assume that each *Merge* block implements the logical connective it represents correctly at the functional level (i.e., given Boolean values from sub-models, it outputs the correct result).

Because φ is constructed inductively from its sub-formulas, the satisfaction of φ depends on the satisfaction of its components. The inductive structure of φ ensures that if each constituent formula is satisfied with high probability, and the logical operations are correctly implemented, then φ itself is satisfied with high probability.

Therefore, over the validation set $\mathcal{X}_{\text{val}}$, we can count the number of inputs x for which the overall model output $M_{\text{compositional}}(x)$ satisfies φ under interpretation I. The ratio of such inputs to the total gives the empirical satisfaction level P, i.e.,

$$P = \frac{|\{x \in \mathcal{X}_{\text{val}} \mid I \models \varphi(M_{\text{compositional}}(x))\}|}{|\mathcal{X}_{\text{val}}|}.$$

This completes the informal justification of the proposition. $\square$

B.2 Proof of Proposition 2

Let $\sigma \in \Sigma^*$ denote the word received as input. Then, we let $E_\varphi^{out}(\sigma)$ denote the output of the enforcer.

Let us prove this proposition using induction on the length of the input sequence $\sigma \in \Sigma^*$.

Induction basis. For the input ϵ, all the models and the enforcer will not release any event as output i.e., for $\sigma = \epsilon$, $E_\varphi^{out}(\epsilon) = \epsilon$ and thus the proposition holds trivially for $\sigma = \epsilon$.

Induction step. Assume that for every $\sigma = x_1 \cdots x_k \in \Sigma^*$ of length $k \in \mathbb{N}$, the proposition holds. We now prove that for $\sigma \cdot x_{k+1} \in \Sigma^*$, the proposition holds. We consider two cases based on whether $\sigma \cdot x_{k+1}$ satisfies the property φ.

1. **Case 1:** $\sigma \cdot x_{k+1} \models \varphi$. By transparency of E_φ, $E_\varphi^{out}(\sigma \cdot x_{k+1}) = \sigma \cdot x_{k+1}$. Hence $E_\varphi^{out}(\sigma \cdot x_{k+1}) \models \varphi$. By Definition 6, inter-model consistency holds for all dependent models $M_i \to M_j$. For robustness, for any M_i and any adversarial input σ_{adv}, whenever M_i processes σ_{adv} the enforcer output still satisfies φ because the final sequence is already property-compliant and is passed through unchanged.

2. **Case 2:** $\sigma \cdot x_{k+1} \not\models \varphi$. By soundness of E_φ, the enforcer replaces x_{k+1} with some $x'_{k+1} \in edit_\varphi(\sigma, x_{k+1})$ and releases $E_\varphi^{out}(\sigma \cdot x_{k+1}) = \sigma \cdot x'_{k+1}$. By construction of $edit_\varphi$ (Definition in Section 5.4), there exists an extension σ_{extn} such that $\sigma \cdot x'_{k+1} \cdot \sigma_{\text{extn}} \models \varphi$; for the synthesised enforcer used in this thesis [154], the released prefix satisfies the property, i.e. $E_\varphi^{out}(\sigma \cdot x_{k+1}) \models \varphi$.

   Since $E_\varphi^{out}(\sigma \cdot x_{k+1}) \models \varphi$, Definition 6 implies inter-model consistency: for every M_j that depends on M_i, we have $M_i \sim M_j$. The enforcer is placed after the entire compositional pipeline, so corrections are applied only to the final control output presented to the actuators; internal module interfaces remain structurally unchanged because $x'_{k+1} \in \Sigma$ (same alphabet and valid domains as x_{k+1}). Thus dependent modules still receive compatible inputs and outputs in the sense of Definition 6.

   For robustness, suppose some M_i misbehaves on an adversarial input σ_{adv} so that the composed system would violate φ. The enforcer still produces E_φ^{out} with $E_\varphi^{out} \models \varphi$ by soundness, satisfying the robustness constraint in Definition 6.

In both cases, $E_\varphi^{out}(\sigma \cdot x_{k+1}) \models \varphi$, and the inter-model consistency and robustness constraints hold. This completes the induction step and hence the proof. □

B.3 Simulation set-up

Set-up. The simulation considers a three-lane roadway and operates with a time step (Δt) of 0.1 seconds as we believe this should be fast enough to make accurate decisions. We consider eight simulation durations, with the simulation duration varying linearly between 30 seconds and 480 seconds. The vehicle setup includes an ego vehicle positioned in the centre lane and six other vehicles distributed across adjacent lanes. The ego vehicle starts at the origin with an initial velocity of 20 m/s (desired velocity of 30 m/s) and occupies the centre lane. Surrounding vehicles are initialized with randomized positions and velocities to simulate a realistic driving environment. These vehicles are distributed across all three lanes, including vehicles in the same lane as the ego car and those in opposing and adjacent lanes. The randomized initial positions range from 2 to 100 meters from the ego car, while their velocities range from 5 to 30 m/s. Each vehicle is governed by an Intelligent Driver Model (IDM) for longitudinal control and a lane change model for lateral manoeuvres.

Execution. The execution of all the modules takes place in series every time instant. Each time instant, the car takes in the data from the car sensors and executes the LCD module, the LCE module, and the CF module and their respective enforcers in series with them. Then, the outputs of the enforcers are passed to the controller, which is then fed as set-points to the actuators of the car. This process repeats every time step. The simulation is run in Python, but the enforcers are developed in C using the easyRTE tool [1].

Design-Space Exploration. To explore multiple scenarios, combinations of initial conditions (initial positions, velocities, and accelerations of ego and surrounding cars) and simulation durations are created using a Cartesian product of random seeds and duration intervals. We use ten randomised repetitions of the simulation durations, 80 simulations in total. The eight simulations with different simulation durations are parallelized using Python's multiprocessing library, enabling efficient execution across multiple CPU cores.

As it may take a long time to observe undesired behaviour, we inject faults in the simulation to observe the behaviour of the AV and whether the enforcer takes corrective action or not in the presence of faults. We inject the following faults in the simulation:

[1] https://github.com/PRETgroup/easy-rte

- **Fault 1:** The LCD module outputs a lane change decision to the controller even when $P_{LCD} = 0$ is not met. This is introduced by setting the LCD output to 1 when $P_{LCD} = 0$ at multiples of time step 83, which is chosen randomly.
- **Fault 2:** The car following distance becomes lesser than 2 metres, i.e. the car rollowing property is violated. This is done by setting the gap to 1.99 metres at multiples of time step 89, which is chosen randomly.
- **Fault 3:** The LCE module takes longer than 5 seconds for a lane change, i.e. P_{LCE} is violated. This is done by manually setting the lane change execution time to 5.04 seconds at multiples of time step 97, which is chosen randomly.

B.4 Model Parameters

ANN Models: The ANN models use a four layer architecture with {128, 64, 32, 3} neurons in each layer of the monolithic model and {64, 32, 16, 1} neurons in each layer of the compositional models. We use `tanh` activation in all hidden layers. The output layer for the monolithic model uses a `softmax` activation and the compositional model uses a `sigmoid` activation.

Random Forest Model: The model hyperparameters are set as follows: the number of trees in the forest (`n_estimators`) is 100, with a maximum tree depth (`max_depth`) of 6. The number of features considered for the best split (`max_features`) is set to the square root of the total features. The minimum number of samples required to split an internal node (`min_samples_split`) is 2, while the minimum number of samples required at a leaf node (`min_samples_leaf`) is 1.

Adaboost Model: The model hyperparameters are set as follows: the number of trees in the forest (`n_estimators`) is 100.

XGBoost Model: The XGBoost model was configured with the following hyperparameters: `objective` set to `binary:logistic`, `eval_metric` set to `logloss` and `max_depth` set to 6.

Appendix C

Compilation of ANN Designs to C

C.1 nn2C Compiler

Algorithms 8, 9, 10, and 11 present the high-level pseudocode of the *nn2C* compiler [1]. The tool translates Artificial Neural Network (ANN) designs—single or multi-model—authored in TensorFlow/Keras or PyTorch into synthesizable, fixed-point C code. An intermediate JSON representation (IR) captures both inter-model wiring and intra-model layer connectivity.[2]

Algorithm 8 Compile TensorFlow/PyTorch Models to C

```
1: function NN2C(models, conn, aParts, fname, outDir, verify, simSteps)
2:     iRep ← PARSE(models, conn, aParts)
3:     cFiles ← WRITEC(iRep, fname, outDir, verify, simSteps)
4:     return cFiles
5: end function
```

The compiler proceeds in two main phases:

1. **Model Parsing** (Alg. 9): Builds the IR by extracting architecture, weights and piece-wise linear (PWL) activation tables.

2. **C Code Generation** (Algs. 10-11): Emits C/`.h` sources, a fixed-point arithmetic library, Makefile, and optional SPIN verification harness.

C.2 Interface Algorithm

Algorithm 8 orchestrates the whole flow:

[1] The compiler can be imported in Python from https://test.pypi.org/simple/nn2c==2.1.6

[2] Supported activations: linear, tanh, sigmoid, exponential/softmax, ELU, ReLU in purely feed-forward MLP or CNN topologies.

Algorithm 9 Parse Models to Intermediate Representation

```
1: function PARSE(models, conn, aParts)
2:     iRep ← {}
3:     for model, mName in models do
4:         arch ← GETARCH(model)
5:         weights ← GETWEIGHTS(model)
6:         acts ← PIECEWISEACTS(model, aParts)
7:         layers ← LAYERGRAPH(model)
8:         iRep[mName] ← {"arch": arch, "weights": weights,"activations": acts,"lConn": layers}
9:     end for
10:    iRep["conn"] ← conn
11:    return iRep
12: end function
```

1. **Line 2**: `parse` converts the supplied models into the IR.

2. **Line 3**: `writeC` materialises every artefact required to build and simulate the generated network in C.

3. **Line 4**: The function returns paths to all generated files, enabling downstream integration or packaging.

Algorithm 10 Write C Source Files from Intermediate Representation

```
1: function WRITEC(iRep, fname, outDir, verify, simSteps)
2:     mSrc ← []
3:     actSrc ← []
4:     for mName in iRep excluding "conn" do
5:         mIR ← iRep[mName]
6:         code ← GENERATEC(mIR)
7:         act ← GENACTC(mIR[activations"])
8:         append code to mSrc
9:         append act to actSrc
10:    end for
11:    lib ← GENFIXEDPOINTLIB
12:    mk ← GENMAKEFILE(fname, verify)
13:    if verify then
14:        verif ← GENVERIFICATION(fname, simSteps)
15:    end if
16:    return [mSrc, actSrc, lib, mk, verif]
17: end function
```

The entry function `nn2C` expects:

- `models`: list of models or {*arch, weights*} pairs.

Algorithm 11 Generate C Artefacts

1: **function** GENERATEC($modelIR$) ▷ Emit model-specific `.c/.h` implementing fixed-point inference.
2: **return** $modelCode$
3: **end function**
4: **function** GENACTC($actPieces$) ▷ Emit lookup-table / piece-wise linear activations.
5: **return** $actCode$
6: **end function**
7: **function** GENFIXEDPOINTLIB ▷ Create common arithmetic library (`CREATE_FP`, `FP_MULT`, ...).
8: **return** $libCode$
9: **end function**
10: **function** GENMAKEFILE($projName$, $verify$) ▷ Makefile with build + optional SPIN verification targets.
11: **return** $mkCode$
12: **end function**
13: **function** GENVERIFICATION($projName$, $simSteps$) ▷ Generate `main.c` harness and Promela model for SPIN.
14: **return** $verifFiles$
15: **end function**

- `conn`: list of inter-model connections in textual form (e.g., ‘‘`input[0:16]` → `model_L`’’).
- `aParts`: number of PWL segments for activation approximation (default 32).
- `fname`: project base name; reused for the top-level C file and Make targets.
- `outDir`: output directory for generated sources.
- `verify`, `simSteps`: enable SPIN-based run-time verification and set simulation horizon.

C.3 Model Parsing Algorithm

Algorithm 9 builds the IR as follows:

1. **Line 3**: Initialises an empty container `iRep`.
2. **Lines 4-10**: For each model
   - **Line 5**: `getArch` captures layer type, dimensions, activation.
   - **Line 6**: `getWeights` serialises parameters (dense weights, convolution kernels, biases).

- **Line 7**: `piecewiseActs` generates PWL tables—the slopes and intercepts for each activation segment calculated via the analytic derivative at segment mid-points.
- **Line 8**: `layerGraph` records directed edges between layers.

3. **Lines 11-12**: Stores the per-model description in `iRep`.
4. **Lines 13-14**: Adds global `conn` information for multi-model graphs.

Framework detection is automatic: PyTorch graphs are traversed via `torch.jit.trace` and ONNX-like nodes; Keras models are parsed layer-by-layer. The outcome is a unified, back-end-agnostic IR that feeds the code generator.

C.4 Compositional Model Parsing

When multiple sub-models form a composite ANN, the parser:

1. Extracts each sub-model independently (Lines 4-10).
2. Writes inter-model edges (Line 13) so that later stages can route outputs of one model to inputs of another.
3. Maintains a single fixed-point format across all components, ensuring numerical consistency.

This enables hierarchical designs such as ensembles, split inputs, or late-fusion networks.

C.5 C Code Generation Algorithm

Algorithm 10 transforms the IR into compilable C:

1. **Lines 3-4**: Allocate lists for per-model and activation sources.
2. **Lines 5-11**: For every model in the IR
   - **Line 7**: `generateC` produces a pair of `.c/.h` files that implement inference with 16-bit fractional fixed-point numbers; neurons are time-multiplexed to save area.
   - **Line 8**: `genActC` emits LUT-based or linear-segment kernels for each activation present.

3. **Line 12**: `genFixedPointLib` supplies arithmetic helpers (`CREATE_FP`, `FP_MULT`, `FP_DIV`, `argmax`).

4. **Line 13**: `genMakefile` wires everything into a build script; optional SPIN targets are added when `verify` is true.

5. **Lines 14-15**: If verification is requested, `genVerification` creates a Promela model, harness code, and appropriate assertions (e.g. accuracy thresholds).

Algorithm 11 details the helpers:

- `generateC`: Emits state structs for inputs, hidden and output neurons, plus two procedures—*`Model`*`Init` (zeroing state) and *`Model`*`Run` (one simulation step).
- `genActC`: Converts each PWL table to an `int32_t` fixed-point function with branch-less segments where possible.
- `genMakefile`: Provides rules for library compilation, object generation, linking, and cleaning.
- `genVerification`: Builds a C test-bench reading `test/input.txt` and a Promela script automatically sweeping an input noise range, asserting the desired number of correct predictions.

The generated project folder therefore contains:

- Top-level `main.c` (or `verify/main.c`) calling the model.
- Model-specific `.c/.h` files (*one per sub-model* if compositional).
- Activation kernels (`relu.c`, `tanh.c`, ...).
- Fixed-point arithmetic library (`library.c/h`).
- Optional `result.c/h` for classification post-processing.
- Makefile with build and simulation targets.

The C code is generated using the synchronous approach, as described in Section 4.6. As this code does not require minimisation of hardware resources, we do not use the multicyclic approach. Instead, we use the pipelined layer-by-layer approach, where each layer is executed at a time in a pipeline. For example, if a model has 5 layers, the pipeline will take 5 ticks to produce the first output.

A minimal usage example is shown in Listing C.1.

```
from nn2C.nnParser import ToC
from tensorflow.keras import layers, Model

def build_mlp():
    x_in  = layers.Input(shape=(16,))
    x     = layers.Dense(16, activation='relu')(x_in)
    x     = layers.Dense(8,  activation='relu')(x)
    x_out = layers.Dense(4,  activation='softmax')(x)
    return Model(x_in, x_out)

net1 = build_mlp()
net2 = build_mlp()

ToC(models=[net1, net2],
    model_names=["net_L", "net_R"],
    connections=["input[0:16] -> net_L", "input[6:22] ->
       net_R"],
    classes=[0,1,2],              # for optional result block
    approx_parts=32,              # PWL granularity
    fname="ensemble",             # base project name
    result_block=True,
    verify=True,                  # generate SPIN harness
    simulation_steps=3)
```

Listing C.1: Python invocation of the nn2C compiler

Bibliography

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. The MIT Press, 2016.

[2] X. Yang, P. Roop, H. Pearce, and J. W. Ro, "A compositional approach using Keras for neural networks in real-time systems," in *2020 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2020, pp. 1109–1114.

[3] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer, "Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks," in *Computer Aided Verification*, R. Majumdar and V. Kunčak, Eds. Cham: Springer International Publishing, 2017, pp. 97–117.

[4] R. Sarić, D. Jokic, N. Beganović, L. Gurbeta Pokvic, and A. Badnjevic, "FPGA-based real-time epileptic seizure classification using Artificial Neural Network," *Biomedical Signal Processing and Control*, vol. 62, p. 102106, Sep. 2020.

[5] D. Nguyen, H. Ho, D. Bui, and X. Tran, "An Efficient Hardware Implementation of Artificial Neural Network based on Stochastic Computing," in *2018 5th NAFOSTED Conference on Information and Computer Science (NICS)*, 2018, pp. 237–242.

[6] R. Ivanov, K. Jothimurugan, S. Hsu, S. Vaidya, R. Alur, and O. Bastani, "Compositional Learning and Verification of Neural Network Controllers," *ACM Trans. Embed. Comput. Syst.*, vol. 20, no. 5s, pp. 92:1–92:26, Sep. 2021. [Online]. Available: https://doi.org/10.1145/3477023

[7] C. S. Pasareanu, D. Gopinath, and H. Yu, "Compositional Verification for Autonomous Systems with Deep Learning Components," Oct. 2018, arXiv:1810.08303 [cs]. [Online]. Available: http://arxiv.org/abs/1810.08303

[8] T. Gehr, M. Mirman, D. Drachsler-Cohen, P. Tsankov, S. Chaudhuri, and M. Vechev, "AI2: Safety and Robustness Certification of Neural Networks with

Abstract Interpretation," in *2018 IEEE Symposium on Security and Privacy (SP)*, 2018, pp. 3–18.

[9] C. Molnar, *Interpretable Machine Learning*. [Online]. Available: https://christophm.github.io/interpretable-ml-book/

[10] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html

[11] M. T. Ribeiro, S. Singh, and C. Guestrin, ""Why Should I Trust You?": Explaining the Predictions of Any Classifier," Aug. 2016, arXiv:1602.04938 [cs, stat]. [Online]. Available: http://arxiv.org/abs/1602.04938

[12] R. Alur, *Principles of Cyber-Physical Systems*. MIT Press Ltd, Apr. 2015.

[13] S. Tripakis, "Compositionality in the Science of System Design," *Proceedings of the IEEE*, vol. 104, no. 5, pp. 960–972, 2016.

[14] C. Watanabe, K. Hiramatsu, and K. Kashino, "Modular Representation of Layered Neural Networks," 2017, _eprint: 1703.00168.

[15] R. Pan and H. Rajan, "On Decomposing a Deep Neural Network into Modules," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2020. New York, NY, USA: Association for Computing Machinery, 2020, pp. 889–900, event-place: Virtual Event, USA. [Online]. Available: https://doi.org/10.1145/3368089.3409668

[16] S. Gokulanathan, A. Feldsher, A. Malca, C. Barrett, and G. Katz, "Simplifying Neural Networks using Formal Verification," 2020, _eprint: 1910.12396.

[17] Y. Y. Elboher, J. Gottschlich, and G. Katz, "An Abstraction-Based Framework for Neural Network Verification," in *Computer Aided Verification*, S. K. Lahiri and C. Wang, Eds. Cham: Springer International Publishing, 2020, pp. 43–65.

[18] R. Wilhelm, J. Engblom, A. Ermedahl, N. Holsti, S. Thesing, D. Whalley, G. Bernat, C. Ferdinand, R. Heckmann, T. Mitra, F. Mueller, I. Puaut, P. Puschner, J. Staschulat, and P. Stenström, "The Worst-Case Execution-Time Problem—Overview of Methods and Survey of Tools," *ACM Trans. Embed. Comput. Syst.*, vol. 7, no. 3, May 2008, place: New York, NY,

USA Publisher: Association for Computing Machinery. [Online]. Available: https://doi.org/10.1145/1347375.1347389

[19] S. Himavathi, D. Anitha, and A. Muthuramalingam, "Feedforward Neural Network Implementation in FPGA Using Layer Multiplexing for Effective Resource Utilization," *IEEE Transactions on Neural Networks*, vol. 18, no. 3, pp. 880–888, 2007.

[20] J. R. Quinlan, "Generating production rules from decision trees," in *Proceedings of the 10th International Joint Conference on Artificial Intelligence - Volume 1*, ser. IJCAI'87. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1987, pp. 304—-307.

[21] A. Panda, A. Anand, S. Pinisetty, and P. Roop, "An Explainable and Formal Framework for Hypertension Monitoring Using ECG and PPG," *IEEE Embedded Systems Letters*, vol. 16, no. 4, pp. 405–408, Dec. 2024, conference Name: IEEE Embedded Systems Letters. [Online]. Available: https://ieeexplore.ieee.org/document/10779969

[22] F. B. Schneider, "Enforceable security policies," *ACM Trans. Inf. Syst. Secur.*, vol. 3, no. 1, pp. 30–50, Feb. 2000.

[23] J. Ligatti, L. Bauer, and D. Walker, "Edit automata: enforcement mechanisms for run-time security policies," *Int. J. Inf. Sec.*, vol. 4, no. 1-2, pp. 2–16, 2005. [Online]. Available: https://doi.org/10.1007/s10207-004-0046-8

[24] S. Pinisetty, P. S. Roop, S. Smyth, S. Tripakis, and R. von Hanxleden, "Runtime enforcement of reactive systems using synchronous enforcers," in *Proceedings of the 24th ACM SIGSOFT International SPIN Symposium on Model Checking of Software, Santa Barbara, CA, USA, July 10-14, 2017*, H. Erdogmus and K. Havelund, Eds. ACM, 2017, pp. 80–89. [Online]. Available: https://doi.org/10.1145/3092282.3092291

[25] H. Pearce, S. Pinisetty, P. S. Roop, M. M. Y. Kuo, and A. Ukil, "Smart i/o modules for mitigating cyber-physical attacks on industrial control systems," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 7, pp. 4659–4669, 2020.

[26] H. Byeon, "Advances in Machine Learning and Explainable Artificial Intelligence for Depression Prediction," *International Journal of Advanced Computer Science and Applications*, vol. 14, pp. 520–526, Jul. 2023.

[27] D. W. Apley and J. Zhu, "Visualizing the Effects of Predictor Variables in Black Box Supervised Learning Models," Aug. 2019, arXiv:1612.08468 [stat]. [Online]. Available: http://arxiv.org/abs/1612.08468

[28] M. T. Ribeiro, S. Singh, and C. Guestrin, "Anchors: High-Precision Model-Agnostic Explanations," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, Apr. 2018. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/11491

[29] L. Doyen, G. Frehse, G. J. Pappas, and A. Platzer, "Verification of hybrid systems," in *Handbook of Model Checking*. Cham: Springer International Publishing, 2018, pp. 1047–1110.

[30] W. K. Youn, S. B. Hong, K. R. Oh, and O. S. Ahn, "Software certification of safety-critical avionic systems: Do-178c and its impacts," *IEEE Aerospace and Electronic Systems Magazine*, vol. 30, no. 4, pp. 4–13, 2015.

[31] M. Orshansky and K. Keutzer, "A general probabilistic framework for worst case timing analysis," in *Proceedings 2002 Design Automation Conference (IEEE Cat. No.02CH37324)*, 2002, pp. 556–561.

[32] G. Berry, "The foundations of Esterel." in *Proof, language, and interaction*, 2000, pp. 425–454.

[33] F. Boussinot and R. de Simone, "The sl synchronous language," *IEEE Trans. Softw. Eng.*, vol. 22, no. 4, pp. 256–266, Apr. 1996. [Online]. Available: https://doi.org/10.1109/32.491649

[34] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*. Springer, 2011.

[35] H. Kopetz, *Real-Time Systems: Design Principles for Distributed Embedded Applications*. Springer, 2011.

[36] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[37] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*. The MIT Press, 2012.

[38] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017.

[39] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Y. Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng, "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015. [Online]. Available: https://www.tensorflow.org/

[40] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems 32*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d. Alché-Buc, E. Fox, and R. Garnett, Eds. Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf

[41] P. I. Frazier, "A tutorial on bayesian optimization," 2018. [Online]. Available: https://arxiv.org/abs/1807.02811

[42] D. Simon, *Evolutionary optimization algorithms*. Hoboken, NJ: Wiley-Blackwell, Apr. 2013.

[43] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95 - International Conference on Neural Networks*, vol. 4, 1995, pp. 1942–1948 vol.4.

[44] R. Storn and K. Price, "Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces," *J. of Global Optimization*, vol. 11, no. 4, p. 341–359, Dec. 1997. [Online]. Available: https://doi.org/10.1023/A:1008202821328

[45] M. Emmerich, O. M. Shir, and H. Wang, "Evolution Strategies," in *Handbook of Heuristics*, R. Martí, P. Panos, and M. G. C. Resende, Eds. Cham: Springer International Publishing, 2018, pp. 1–31. [Online]. Available: https://doi.org/10.1007/978-3-319-07153-4_13-1

[46] U. Kamath and J. Liu, "Introduction to Interpretability and Explainability," in *Explainable Artificial Intelligence: An Introduction to Interpretable Machine Learning*, U. Kamath and J. Liu, Eds. Cham: Springer International Publishing, 2021, pp. 1–26. [Online]. Available: https://doi.org/10.1007/978-3-030-83356-5_1

[47] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification And Regression Trees*. Routledge, Oct. 2017.

[48] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *J. Mach. Learn. Res.*, vol. 3, no. null, p. 1157–1182, Mar. 2003.

[49] A. Benveniste, P. Caspi, S. Edwards, N. Halbwachs, P. Le Guernic, and R. de Simone, "The synchronous languages 12 years later," *Proceedings of the IEEE*, vol. 91, no. 1, pp. 64–83, 2003.

[50] G. Nicolescu and P. J. Mosterman, *Model-based design for embedded systems*. CRC Press, Sep. 2018.

[51] W.-P. De Roever, H. Langmaack, and A. Pnueli, Eds., *Compositionality: The significant difference*, ser. Lecture notes in computer science. New York, NY: Springer, Nov. 2006.

[52] J. Eker, J. Janneck, E. Lee, J. Liu, X. Liu, J. Ludvig, S. Neuendorffer, S. Sachs, and Y. Xiong, "Taming heterogeneity - the ptolemy approach," *Proceedings of the IEEE*, vol. 91, no. 1, pp. 127–144, 2003.

[53] A. Alzubaidi, "Rain-aware lane change decision model for autonomous vehicles using deep reinforcement learning," Ph.D. dissertation, MS thesis, Dept. Elect. Eng. Comput. Sci., Khalifa Univ., Abu Dhabi, United Arab Emirates, 2022.

[54] Y. Xing, C. Lv, H. Wang, D. Cao, and E. Velenis, "An ensemble deep learning approach for driver lane change intention inference," *Transportation Research Part C: Emerging Technologies*, vol. 115, p. 102615, 2020. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0968090X19308332

[55] J. Wang, Z. Zhang, and G. Lu, "A Bayesian inference based adaptive lane change prediction model," *Transportation Research Part C: Emerging Technologies*, vol. 132, p. 103363, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0968090X2100365X

[56] X. Huang, M. Kwiatkowska, S. Wang, and M. Wu, "Safety Verification of Deep Neural Networks," in *Computer Aided Verification*, R. Majumdar and V. Kunčak, Eds. Cham: Springer International Publishing, 2017, pp. 3–29.

[57] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2015. [Online]. Available: https://arxiv.org/abs/1412.6572

[58] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla, and F. Herrera, "A unifying view on dataset shift in classification," *Pattern Recognition*, vol. 45, no. 1, pp. 521–530, 2012. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0031320311002901

[59] OpenAI, "Ai-generated image of a dog with slight graininess," https://chat.openai.com, 2025, generated using DALL·E 3 model via ChatGPT on July 30, 2025.

[60] A. Torralba and A. A. Efros, "Unbiased look at dataset bias," in *CVPR 2011*, 2011, pp. 1521–1528.

[61] V. Sze, Y. hsin Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, pp. 2295–2329, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:3273340

[62] E. M. Clarke Jr, O. Grumberg, D. Kroening, D. Peled, and H. Veith, *Model checking*. MIT press, 2018.

[63] T. Mitra, J. Teich, and L. Thiele, "Time-critical systems design: A survey," *IEEE Design & Test*, vol. 35, no. 2, pp. 8–26, 2018, publisher: IEEE.

[64] J. Woodcock, P. G. Larsen, J. Bicarregui, and J. Fitzgerald, "Formal methods: Practice and experience," *ACM computing surveys (CSUR)*, vol. 41, no. 4, p. 19, 2009, publisher: ACM.

[65] R. Ehlers, "Formal Verification of Piece-Wise Linear Feed-Forward Neural Networks," 2017, _eprint: 1705.01320.

[66] T. Dreossi, A. Donzé, and S. A. Seshia, "Compositional Falsification of Cyber-Physical Systems with Machine Learning Components," 2018, _eprint: 1703.00978.

[67] S. A. Seshia, A. Desai, T. Dreossi, D. J. Fremont, S. Ghosh, E. Kim, S. Shivakumar, M. Vazquez-Chanlatte, and X. Yue, "Formal Specification for Deep Neural Networks," in *Automated Technology for Verification and Analysis*, S. K. Lahiri and C. Wang, Eds. Cham: Springer International Publishing, 2018, pp. 20–34.

[68] T. Dreossi, D. J. Fremont, S. Ghosh, E. Kim, H. Ravanbakhsh, M. Vazquez-Chanlatte, and S. Seshia, "VERIFAI: A Toolkit for the Design and Analysis of Artificial Intelligence-Based Systems," *ArXiv*, vol. abs/1902.04245, 2019.

[69] N. Naik and P. Nuzzo, "Robustness Contracts for Scalable Verification of Neural Network-Enabled Cyber-Physical Systems," *Memocode2020*, 2020.

[70] P. Sun, H. Kretzschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, V. Vasudevan, W. Han, J. Ngiam, H. Zhao, A. Timofeev, S. Ettinger, M. Krivokon, A. Gao, A. Joshi, Y. Zhang, J. Shlens, Z. Chen, and D. Anguelov, "Scalability in perception for autonomous driving: Waymo open dataset," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 2446–2454.

[71] M. Bansal, A. Krizhevsky, and A. Ogale, "ChauffeurNet: Learning to drive by imitating the best and synthesizing the worst," in *Robotics: Science and Systems XV*, 2019.

[72] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, "End-to-end autonomous driving: Challenges and frontiers," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 10 164–10 183, 2024.

[73] A. Tampuu, T. Matiisen, M. Semikin, D. Fishman, and N. Muhammad, "A survey of end-to-end driving: Architectures and training methods," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 4, pp. 1364–1384, 2022.

[74] M. Marouf, J. Popovic-Bozovic, and I. Popovic, "FPGA implementation of neural network as processing element in ice detector," in *11th Symposium on Neural Network Applications in Electrical Engineering*, 2012, pp. 81–84.

[75] N. Allen, Y. Raje, J. W. Ro, and P. Roop, "A Compositional Approach for Real-Time Machine Learning," in *Proceedings of the 17th ACM-IEEE International Conference on Formal Methods and Models for System Design*, ser. MEMOCODE '19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: https://doi.org/10.1145/3359986.3361204

[76] R. Sarić, D. Jokic, N. Beganović, L. Gurbeta Pokvic, and A. Badnjevic, "FPGA-based real-time epileptic seizure classification using Artificial Neural Network," *Biomedical Signal Processing and Control*, vol. 62, p. 102106, Sep. 2020.

[77] H. Zairi, M. Kedir Talha, K. Meddah, and S. Ould Slimane, "FPGA-based system for artificial neural network arrhythmia classification," *Neural*

Computing and Applications, vol. 32, no. 8, pp. 4105–4120, 2020. [Online]. Available: www.scopus.com

[78] N. Allen, Y. Raje, J. W. Ro, and P. Roop, "A Compositional Approach for Real-Time Machine Learning," in *Proceedings of the 17th ACM-IEEE International Conference on Formal Methods and Models for System Design*, ser. MEMOCODE '19. New York, NY, USA: Association for Computing Machinery, 2019, event-place: La Jolla, California. [Online]. Available: https://doi.org/10.1145/3359986.3361204

[79] C. Watson, R. Alur, D. Gopinath, R. Mangal, and C. S. Pasareanu, "Scenario-based Compositional Verification of Autonomous Systems with Neural Perception," *arXiv.org*, 2025. [Online]. Available: https://arxiv.org/abs/2504.20942

[80] Y. Zhou and S. Tripakis, "Compositional Inductive Invariant Based Verification of Neural Network Controlled Systems," *NASA Formal Methods*, 2023. [Online]. Available: https://arxiv.org/abs/2312.10842

[81] R. Ivanov, J. Weimer, R. Alur, G. J. Pappas, and I. Lee, "Verisig," in *Proceedings of the 22nd ACM International Conference on Hybrid Systems: Computation and Control*. ACM, apr 16 2019, pp. 169–178. [Online]. Available: http://dx.doi.org/10.1145/3302504.3311806

[82] F. Cai, C. Fan, and S. Bak, "Scalable Surrogate Verification of Image-based Neural Network Control Systems using Composition and Unrolling," *AAAI Conference on Artificial Intelligence*, 2024. [Online]. Available: https://arxiv.org/abs/2405.18554

[83] J. Wang, S. Kalluraya, and Y. Kantaros, "Verified Compositions of Neural Network Controllers for Temporal Logic Control Objectives," in *2022 IEEE 61st Conference on Decision and Control (CDC)*. IEEE, dec 6 2022, pp. 4004–4009. [Online]. Available: http://dx.doi.org/10.1109/CDC51059.2022.9993010

[84] Y. Y. Elboher, J. Gottschlich, and G. Katz, *An Abstraction-Based Framework for Neural Network Verification*. Springer International Publishing, 2020, pp. 43–65. [Online]. Available: http://dx.doi.org/10.1007/978-3-030-53288-8_3

[85] B. Paulsen, J. Wang, and C. Wang, "Reludiff," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. ACM, jun 27 2020, pp. 714–726. [Online]. Available: http://dx.doi.org/10.1145/3377811.3380337

[86] P. Kouvaros and A. Lomuscio, "Towards Scalable Complete Verification of Relu Neural Networks via Dependency-based Branching," in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, 8 2021, pp. 2643–2650. [Online]. Available: http://dx.doi.org/10.24963/ijcai.2021/364

[87] I. Goodfellow, P. McDaniel, and N. Papernot, "Making machine learning robust against adversarial inputs," *Commun. ACM*, vol. 61, no. 7, pp. 56–66, Jun. 2018. [Online]. Available: https://doi.org/10.1145/3134599

[88] S. Pinisetty, Y. Falcone, T. Jéron, H. Marchand, A. Rollet, and O. Nguena Timo, "Runtime enforcement of timed properties revisited," *Form. Methods Syst. Des.*, vol. 45, no. 3, p. 381–422, dec 2014. [Online]. Available: https://doi.org/10.1007/s10703-014-0215-y

[89] D. W. Joyce, A. Kormilitzin, K. A. Smith, and A. Cipriani, "Explainable artificial intelligence for mental health through transparency and interpretability for understandability," *npj Digital Medicine*, vol. 6, no. 1, pp. 1–7, Jan. 2023, number: 1 Publisher: Nature Publishing Group. [Online]. Available: https://www.nature.com/articles/s41746-023-00751-9

[90] A. Barredo Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. Garcia, S. Gil-Lopez, D. Molina, R. Benjamins, R. Chatila, and F. Herrera, "Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Information Fusion*, vol. 58, pp. 82–115, Jun. 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1566253519308103

[91] A. I. Korda, C. Andreou, H. V. Rogg, M. Avram, A. Ruef, C. Davatzikos, N. Koutsouleris, and S. Borgwardt, "Identification of texture MRI brain abnormalities on first-episode psychosis and clinical high-risk subjects using explainable artificial intelligence," *Translational Psychiatry*, vol. 12, no. 1, pp. 1–12, Nov. 2022, number: 1 Publisher: Nature Publishing Group. [Online]. Available: https://www.nature.com/articles/s41398-022-02242-z

[92] K. Schultebraucks, A. Y. Shalev, V. Michopoulos, C. R. Grudzen, S.-M. Shin, J. S. Stevens, J. L. Maples-Keller, T. Jovanovic, G. A. Bonanno, B. O. Rothbaum, C. R. Marmar, C. B. Nemeroff, K. J. Ressler, and I. R. Galatzer-Levy, "A validated predictive algorithm of post-traumatic stress course following emergency department admission after a traumatic stressor," *Nature Medicine*, vol. 26, no. 7, pp. 1084–1088, Jul. 2020,

number: 7 Publisher: Nature Publishing Group. [Online]. Available: https://www.nature.com/articles/s41591-020-0951-z

[93] A. Binder, M. Bockmayr, M. Hägele, S. Wienert, D. Heim, K. Hellweg, M. Ishii, A. Stenzinger, A. Hocke, C. Denkert, K.-R. Müller, and F. Klauschen, "Morphological and molecular breast cancer profiling through explainable machine learning," *Nature Machine Intelligence*, vol. 3, no. 4, pp. 355–366, Apr. 2021, number: 4 Publisher: Nature Publishing Group. [Online]. Available: https://www.nature.com/articles/s42256-021-00303-4

[94] C. Wang, L. Feng, and Y. Qi, "Explainable deep learning predictions for illness risk of mental disorders in Nanjing, China," *Environmental Research*, vol. 202, p. 111740, Nov. 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0013935121010343

[95] T. Zhu, X. Liu, J. Wang, R. Kou, Y. Hu, M. Yuan, C. Yuan, L. Luo, and W. Zhang, "Explainable machine-learning algorithms to differentiate bipolar disorder from major depressive disorder using self-reported symptoms, vital signs, and blood-based markers," *Computer Methods and Programs in Biomedicine*, vol. 240, p. 107723, Oct. 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0169260723003899

[96] M. Squires, X. Tao, S. Elangovan, R. Gururajan, X. Zhou, Y. Li, and U. R. Acharya, "Identifying predictive biomarkers for repetitive transcranial magnetic stimulation response in depression patients with explainability," *Computer Methods and Programs in Biomedicine*, vol. 242, p. 107771, Dec. 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0169260723004376

[97] H. Zogan, I. Razzak, X. Wang, S. Jameel, and G. Xu, "Explainable depression detection with multi-aspect features using a hybrid deep learning model on social media," *World Wide Web*, vol. 25, no. 1, pp. 281–304, Jan. 2022. [Online]. Available: https://doi.org/10.1007/s11280-021-00992-2

[98] R. V. Shah, G. Grennan, M. Zafar-Khan, F. Alim, S. Dey, D. Ramanathan, and J. Mishra, "Personalized machine learning of depressed mood using wearables," *Translational Psychiatry*, vol. 11, no. 1, pp. 1–18, Jun. 2021, number: 1 Publisher: Nature Publishing Group. [Online]. Available: https://www.nature.com/articles/s41398-021-01445-0

[99] Z. Zheng, "Recent developments and research needs in modeling lane changing," *Transportation Research Part B: Methodological*, vol. 60, pp. 16–32, Feb. 2014.

[100] M. Treiber and D. Helbing, "Explanation of observed features of self-organization in traffic flow," 1999. [Online]. Available: https://arxiv.org/abs/cond-mat/9901239

[101] M. Treiber, A. Hennecke, and D. Helbing, "Congested traffic states in empirical observations and microscopic simulations," *Phys. Rev. E*, vol. 62, pp. 1805–1824, Aug 2000. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevE.62.1805

[102] X. Zhang, J. Sun, X. Qi, and J. Sun, "Simultaneous modeling of car-following and lane-changing behaviors using deep learning," *Transportation Research Part C: Emerging Technologies*, vol. 104, pp. 287 – 304, 2019. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0968090X18308003

[103] E. Balal, R. L. Cheu, T. Gyan-Sarkodie, and J. Miramontes, "Analysis of Discretionary Lane Changing Parameters on Freeways," *International Journal of Transportation Science and Technology*, vol. 3, no. 3, pp. 277 – 296, 2014. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S2046043016301149

[104] R. S. Tomar, S. Verma, and G. S. Tomar, "Prediction of Lane Change Trajectories through Neural Network," in *2010 International Conference on Computational Intelligence and Communication Networks*, 2010, pp. 249–253.

[105] D. Xie, Z.-Z. Fang, B. Jia, and Z. He, "A Data-driven Lane-changing Model based on Deep Learning," *Transportation Research Part C Emerging Technologies*, vol. 106, pp. 41–60, Jul. 2019.

[106] X. Liu, J. Liang, and B. Xu, "A Deep Learning Method for Lane Changing Situation Assessment and Decision Making," *IEEE Access*, vol. 7, pp. 133 749–133 759, 2019.

[107] N. C. C. f. M. Health (UK), "The classification of depression and depression rating scales/questionnaires," in *Depression in Adults with a Chronic Physical Health Problem: Treatment and Management*. British Psychological Society (UK), 2010. [Online]. Available: https://www.ncbi.nlm.nih.gov/books/NBK82926/

[108] "Depressive disorder (depression)." [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/depression

[109] B. N. Gaynes, D. Warden, M. H. Trivedi, S. R. Wisniewski, M. Fava, and A. J. Rush, "What did STAR*D teach us? Results from a large-scale, practical, clini-

cal trial for patients with depression," *Psychiatric Services (Washington, D.C.)*, vol. 60, no. 11, pp. 1439–1445, Nov. 2009.

[110] M. H. Trivedi, A. J. Rush, S. R. Wisniewski, A. A. Nierenberg, D. Warden, L. Ritz, G. Norquist, R. H. Howland, B. Lebowitz, P. J. McGrath, K. Shores-Wilson, M. M. Biggs, G. K. Balasubramani, M. Fava, and STAR*D Study Team, "Evaluation of outcomes with citalopram for depression using measurement-based care in STAR*D: implications for clinical practice," *The American Journal of Psychiatry*, vol. 163, no. 1, pp. 28–40, Jan. 2006.

[111] C. E. Carney, J. D. Edinger, M. Kuchibhatla, A. M. Lachowski, O. Bogouslavsky, A. D. Krystal, and C. M. Shapiro, "Cognitive Behavioral Insomnia Therapy for Those With Insomnia and Depression: A Randomized Controlled Clinical Trial," *Sleep*, vol. 40, no. 4, p. zsx019, Apr. 2017.

[112] W. Ramel1, P. R. Goldin, P. E. Carmona, and J. R. McQuaid, "The Effects of Mindfulness Meditation on Cognitive Processes and Affect in Patients with Past Depression," *Cognitive Therapy and Research*, vol. 28, no. 4, pp. 433–455, Aug. 2004. [Online]. Available: https://doi.org/10.1023/B:COTR.0000045557.15923.96

[113] E. Andersson, A. Hovland, B. Kjellman, J. Taube, and E. Martinsen, "[Physical activity is just as good as CBT or drugs for depression]," *Lakartidningen*, vol. 112, p. DP4E, Nov. 2015.

[114] R. S. Opie, A. O'Neil, F. N. Jacka, J. Pizzinga, and C. Itsiopoulos, "A modified Mediterranean dietary intervention for adults with major depression: Dietary protocol and feasibility data from the SMILES trial," *Nutritional Neuroscience*, vol. 21, no. 7, pp. 487–501, Aug. 2018, publisher: Taylor & Francis _eprint: https://doi.org/10.1080/1028415X.2017.1312841. [Online]. Available: https://doi.org/10.1080/1028415X.2017.1312841

[115] N. Parletta, D. Zarnowiecki, J. Cho, A. Wilson, S. Bogomolova, A. Villani, C. Itsiopoulos, T. Niyonsenga, S. Blunden, B. Meyer, L. Segal, B. T. Baune, and K. O'Dea, "A Mediterranean-style dietary intervention supplemented with fish oil improves diet quality and mental health in people with depression: A randomized controlled trial (HELFIMED)," *Nutritional Neuroscience*, vol. 22, no. 7, pp. 474–487, Jul. 2019, publisher: Taylor & Francis _eprint: https://doi.org/10.1080/1028415X.2017.1411320. [Online]. Available: https://doi.org/10.1080/1028415X.2017.1411320

[116] Q.-S. Liu, R. Deng, Y. Fan, K. Li, F. Meng, X. Li, and R. Liu, "Low dose of caffeine enhances the efficacy of antidepressants in major depressive disorder and the underlying neural substrates," *Molecular Nutrition & Food Research*, vol. 61, no. 8, p. 1600910, 2017, _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/mnfr.201600910. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/mnfr.201600910

[117] J. Sarris, A. O'Neil, C. E. Coulson, I. Schweitzer, and M. Berk, "Lifestyle medicine for depression," *BMC psychiatry*, vol. 14, p. 107, Apr. 2014.

[118] D. Highland and G. Zhou, "A review of detection techniques for depression and bipolar disorder," *Smart Health*, vol. 24, p. 100282, Jun. 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2352648322000174

[119] M. K. Ross, T. Tulabandhula, C. C. Bennett, E. Baek, D. Kim, F. Hussain, A. P. Demos, E. Ning, S. A. Langenecker, O. Ajilore, and A. D. Leow, "A Novel Approach to Clustering Accelerometer Data for Application in Passive Predictions of Changes in Depression Severity," *Sensors*, vol. 23, no. 3, p. 1585, Jan. 2023, number: 3 Publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: https://www.mdpi.com/1424-8220/23/3/1585

[120] L. Canzian and M. Musolesi, "Trajectories of depression: unobtrusive monitoring of depressive states by means of smartphone mobility traces analysis," in *Proceedings of the 2015 ACM International Joint Conference on Pervasive and Ubiquitous Computing*, ser. UbiComp '15. New York, NY, USA: Association for Computing Machinery, Sep. 2015, pp. 1293–1304. [Online]. Available: https://doi.org/10.1145/2750858.2805845

[121] A. Dogrucu, A. Perucic, A. Isaro, D. Ball, E. Toto, E. A. Rundensteiner, E. Agu, R. Davis-Martin, and E. Boudreaux, "Moodable: On feasibility of instantaneous depression assessment using machine learning on voice samples with retrospectively harvested smartphone and social media data," *Smart Health*, vol. 17, p. 100118, Jul. 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2352648319300273

[122] X. Xu, P. Chikersal, J. M. Dutcher, Y. S. Sefidgar, W. Seo, M. J. Tumminia, D. K. Villalba, S. Cohen, K. G. Creswell, J. D. Creswell, A. Doryab, P. S. Nurius, E. Riskin, A. K. Dey, and J. Mankoff, "Leveraging Collaborative-Filtering for Personalized Behavior Modeling: A Case Study of Depression Detection among College Students," *Proceedings of the ACM on Interactive, Mobile, Wearable*

and Ubiquitous Technologies, vol. 5, no. 1, pp. 41:1–41:27, Mar. 2021. [Online]. Available: https://dl.acm.org/doi/10.1145/3448107

[123] C. Y. Chiu, H. Y. Lane, J. L. Koh, and A. L. P. Chen, "Multimodal depression detection on instagram considering time interval of posts," *Journal of Intelligent Information Systems*, vol. 56, no. 1, pp. 25–47, Feb. 2021. [Online]. Available: https://doi.org/10.1007/s10844-020-00599-5

[124] A. Nadeem, M. Naveed, M. Islam Satti, H. Afzal, T. Ahmad, and K.-I. Kim, "Depression Detection Based on Hybrid Deep Learning SSCL Framework Using Self-Attention Mechanism: An Application to Social Networking Data," *Sensors*, vol. 22, no. 24, p. 9775, Jan. 2022, number: 24 Publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: https://www.mdpi.com/1424-8220/22/24/9775

[125] R.-E. Mastoras, D. Iakovakis, S. Hadjidimitriou, V. Charisis, S. Kassie, T. Alsaadi, A. Khandoker, and L. J. Hadjileontiadis, "Touchscreen typing pattern analysis for remote detection of the depressive tendency," *Scientific Reports*, vol. 9, no. 1, p. 13414, Sep. 2019, number: 1 Publisher: Nature Publishing Group. [Online]. Available: https://www.nature.com/articles/s41598-019-50002-9

[126] M. Niu, B. Liu, J. Tao, and Q. Li, "A time-frequency channel attention and vectorization network for automatic depression level prediction," *Neurocomputing*, vol. 450, pp. 208–218, Aug. 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925231221005981

[127] A. Ghandeharioun, S. Fedor, L. Sangermano, D. Ionescu, J. Alpert, C. Dale, D. Sontag, and R. Picard, "Objective assessment of depressive symptoms with machine learning and wearable sensors data," in *2017 Seventh International Conference on Affective Computing and Intelligent Interaction (ACII)*, Oct. 2017, pp. 325–332, iSSN: 2156-8111.

[128] J. Choi, S. Lee, S. Kim, D. Kim, and H. Kim, "Depressed Mood Prediction of Elderly People with a Wearable Band," *Sensors*, vol. 22, no. 11, p. 4174, Jan. 2022, number: 11 Publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: https://www.mdpi.com/1424-8220/22/11/4174

[129] D.-K. Nguyen, C.-L. Chan, A.-H. Adams Li, and D.-V. Phan, "Deep Stacked Generalization Ensemble Learning models in early diagnosis of Depression illness from wearable devices data," in *Proceedings of the 5th International Conference on Medical and Health Informatics*, ser. ICMHI '21. New York,

NY, USA: Association for Computing Machinery, Oct. 2021, pp. 7–12. [Online]. Available: https://doi.org/10.1145/3472813.3472815

[130] I. Moshe, Y. Terhorst, K. Opoku Asare, L. B. Sander, D. Ferreira, H. Baumeister, D. C. Mohr, and L. Pulkki-Råback, "Predicting Symptoms of Depression and Anxiety Using Smartphone and Wearable Data," *Frontiers in Psychiatry*, vol. 12, p. 625247, Jan. 2021. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7876288/

[131] B. Little, O. Alshabrawy, D. Stow, I. N. Ferrier, R. McNaney, D. G. Jackson, K. Ladha, C. Ladha, T. Ploetz, J. Bacardit, P. Olivier, P. Gallagher, and J. T. O'Brien, "Deep learning-based automated speech detection as a marker of social functioning in late-life depression," *Psychological Medicine*, vol. 51, no. 9, pp. 1441–1450, Jul. 2021.

[132] T. P. Thakre, H. Kulkarni, K. S. Adams, R. Mischel, R. Hayes, and A. Pandurangi, "Polysomnographic identification of anxiety and depression using deep learning," *Journal of Psychiatric Research*, vol. 150, pp. 54–63, Jun. 2022.

[133] Y. Tazawa, K.-c. Liang, M. Yoshimura, M. Kitazawa, Y. Kaise, A. Takamiya, A. Kishi, T. Horigome, Y. Mitsukura, M. Mimura, and T. Kishimoto, "Evaluating depression with multimodal wristband-type wearable device: screening and assessing patient severity utilizing machine-learning," *Heliyon*, vol. 6, no. 2, p. e03274, Feb. 2020. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7005437/

[134] L. V. Coutts, D. Plans, A. W. Brown, and J. Collomosse, "Deep learning with wearable based heart rate variability for prediction of mental and general health," *Journal of Biomedical Informatics*, vol. 112, p. 103610, Dec. 2020.

[135] S. R. Müller, X. L. Chen, H. Peters, A. Chaintreau, and S. C. Matz, "Depression predictions from GPS-based mobility do not generalize well to large demographically heterogeneous samples," *Scientific Reports*, vol. 11, no. 1, p. 14007, Jul. 2021, number: 1 Publisher: Nature Publishing Group. [Online]. Available: https://www.nature.com/articles/s41598-021-93087-x

[136] R. Belmaker and G. Agam, "Major Depressive Disorder," *New England Journal of Medicine*, vol. 358, no. 1, pp. 55–68, Jan. 2008, publisher: Massachusetts Medical Society _eprint: https://doi.org/10.1056/NEJMra073096. [Online]. Available: https://doi.org/10.1056/NEJMra073096

[137] A. T. Drysdale, L. Grosenick, J. Downar, K. Dunlop, F. Mansouri, Y. Meng, R. N. Fetcho, B. Zebley, D. J. Oathes, A. Etkin, A. F. Schatzberg,

K. Sudheimer, J. Keller, H. S. Mayberg, F. M. Gunning, G. S. Alexopoulos, M. D. Fox, A. Pascual-Leone, H. U. Voss, B. J. Casey, M. J. Dubin, and C. Liston, "Resting-state connectivity biomarkers define neurophysiological subtypes of depression," *Nature Medicine*, vol. 23, no. 1, pp. 28–38, Jan. 2017, number: 1 Publisher: Nature Publishing Group. [Online]. Available: https://www.nature.com/articles/nm.4246

[138] A. Kathan, M. Harrer, L. Küster, A. Triantafyllopoulos, X. He, M. Milling, M. Gerczuk, T. Yan, S. T. Rajamani, E. Heber, I. Grossmann, D. D. Ebert, and B. W. Schuller, "Personalised depression forecasting using mobile sensor data and ecological momentary assessment," *Frontiers in Digital Health*, vol. 4, 2022. [Online]. Available: https://www.frontiersin.org/articles/10.3389/fdgth.2022.964582

[139] N. C. Jacobson and Y. J. Chung, "Passive Sensing of Prediction of Moment-To-Moment Depressed Mood among Undergraduates with Clinical Levels of Depression Sample Using Smartphones," *Sensors (Basel, Switzerland)*, vol. 20, no. 12, p. 3572, Jun. 2020. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7349045/

[140] U. S. D. o. T. F. H. Administration, "Next Generation Simulation (NGSIM) Vehicle Trajectories and Supporting Data [Dataset]," in *Provided by ITS DataHub through Data.transportation.gov*. Accessed 2021-07-09 from http://doi.org/10.21949/1504477, 2016.

[141] R. A. FISHER, "THE USE OF MULTIPLE MEASUREMENTS IN TAXONOMIC PROBLEMS," *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936, _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1469-1809.1936.tb02137.x. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1469-1809.1936.tb02137.x

[142] S. Aeberhard and M. Forina, "Wine," UCI Machine Learning Repository, 1991, DOI: https://doi.org/10.24432/C5PC7J.

[143] B. Efron, T. Hastie, I. Johnstone, and R. Tibshirani, "Least angle regression," *The Annals of Statistics*, vol. 32, no. 2, pp. 407–451, 2004. [Online]. Available: http://www.jstor.org/stable/3448465

[144] S. Chatterjee, N. Allen, N. Patel, and P. Roop, "Exploring compositional neural networks for real-time systems," in *2024 22nd ACM-IEEE International Symposium on Formal Methods and Models for System Design (MEMOCODE)*, 2024, pp. 46–57.

[145] C. Thiemann, M. Treiber, and A. Kesting, "Estimating Acceleration and Lane-Changing Dynamics from Next Generation Simulation Trajectory Data," *Transportation Research Record*, vol. 2088, no. 1, pp. 90–101, 2008, _eprint: https://doi.org/10.3141/2088-10. [Online]. Available: https://doi.org/10.3141/2088-10

[146] Q. Wang, Z. Li, and L. Li, "Investigation of Discretionary Lane-Change Characteristics Using Next-Generation Simulation Data Sets," *Journal of Intelligent Transportation Systems*, vol. 18, Jun. 2014.

[147] J. Zheng, K. Suzuki, and M. Fujita, "Predicting driver's lane-changing decisions using a neural network model," *Simulation Modelling Practice and Theory*, vol. 42, pp. 73 – 83, 2014. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S1569190X13001792

[148] J. Močkus, "On bayesian methods for seeking the extremum," in *Optimization Techniques IFIP Technical Conference Novosibirsk, July 1–7, 1974*, G. I. Marchuk, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1975, pp. 400–404.

[149] J. Kelleher, B. Namee, and A. D'Arcy, *Fundamentals of Machine Learning for Predictive Data Analytics, second edition: Algorithms, Worked Examples, and Case Studies*. MIT Press, 2020. [Online]. Available: https://books.google.co.in/books?id=UM_tDwAAQBAJ

[150] M. Mckay, R. Beckman, and W. Conover, "A Comparison of Three Methods for Selecting Vales of Input Variables in the Analysis of Output From a Computer Code," *Technometrics*, vol. 21, pp. 239–245, May 1979.

[151] D. Arthur and S. Vassilvitskii, "k-means++: the advantages of careful seeding," in *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, ser. SODA '07. USA: Society for Industrial and Applied Mathematics, 2007, p. 1027–1035.

[152] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems 32*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d. Alché-Buc, E. Fox, and R. Garnett, Eds. Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf

[153] G. Plotkin, "A Structural Approach to Operational Semantics," *J. Log. Algebr. Program.*, vol. 60-61, pp. 17–139, Jul. 2004.

[154] S. Pinisetty, P. S. Roop, S. Smyth, N. Allen, S. Tripakis, and R. von Hanxleden, "Runtime enforcement of cyber-physical systems," *ACM Trans. Embed. Comput. Syst.*, vol. 16, no. 5s, pp. 178:1–178:25, 2017. [Online]. Available: https://doi.org/10.1145/3126500

[155] R. Krajewski, J. Bock, L. Kloeker, and L. Eckstein, "The highD Dataset: A Drone Dataset of Naturalistic Vehicle Trajectories on German Highways for Validation of Highly Automated Driving Systems," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 2018, pp. 2118–2125.

[156] N. J. Nilsson, *Artificial Intelligence: A New Synthesis.* San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1998.

[157] R. Fletcher, *Newton-Like Methods.* John Wiley & Sons, Ltd, 2000, ch. 3, pp. 44–79. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/9781118723203.ch3

[158] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD 16. ACM, Aug. 2016, pp. 785–794. [Online]. Available: http://dx.doi.org/10.1145/2939672.2939785

[159] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S002200009791504X

[160] T. K. Ho, "Random decision forests," in *Proceedings of 3rd International Conference on Document Analysis and Recognition*, vol. 1, 1995, pp. 278–282 vol.1.

[161] N. Downs, T. Alderman, S. Bhakta, and T. A. Greenwood, "Implementing a college mental health program - an overview of the first twelve months," *Journal of American college health: J of ACH*, vol. 67, no. 1, pp. 27–31, Jan. 2019.

[162] K. Kroenke, R. L. Spitzer, and J. B. Williams, "The PHQ-9: validity of a brief depression severity measure," *Journal of General Internal Medicine*, vol. 16, no. 9, pp. 606–613, Sep. 2001.

[163] M. Oquendo, B. Halberstam, and J. Mann, "Risk factors for suicidal behavior. The utility and limitations of research instruments," *Standardized Evaluation in Clinical Practice. Review of Psychiatry*, vol. 8, pp. 103–130, Jan. 2003.

[164] P. P. Balasubramani, A. Ojeda, G. Grennan, V. Maric, H. Le, F. Alim, M. Zafar-Khan, J. Diaz-Delgado, S. Silveira, D. Ramanathan, and J. Mishra, "Mapping Cognitive Brain Functions at Scale," *NeuroImage*, vol. 231, p. 117641, May 2021. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8221518/

[165] A. Fernández, S. García, M. Galar, R. C. Prati, B. Krawczyk, and F. Herrera, "Performance Measures," in *Learning from Imbalanced Data Sets*. Cham: Springer International Publishing, 2018, pp. 47–61. [Online]. Available: https://doi.org/10.1007/978-3-319-98074-4_3

[166] S. F. Buck, "A Method of Estimation of Missing Values in Multivariate Data Suitable for use with an Electronic Computer," *Journal of the Royal Statistical Society. Series B (Methodological)*, vol. 22, no. 2, pp. 302–306, 1960, publisher: [Royal Statistical Society, Wiley]. [Online]. Available: https://www.jstor.org/stable/2984099

[167] S. v. Buuren and K. Groothuis-Oudshoorn, "mice: Multivariate Imputation by Chained Equations in R," *Journal of Statistical Software*, vol. 45, pp. 1–67, Dec. 2011. [Online]. Available: https://doi.org/10.18637/jss.v045.i03

[168] O. Troyanskaya, M. Cantor, G. Sherlock, P. Brown, T. Hastie, R. Tibshirani, D. Botstein, and R. B. Altman, "Missing value estimation methods for DNA microarrays," *Bioinformatics (Oxford, England)*, vol. 17, no. 6, pp. 520–525, Jun. 2001.

[169] J. L. Hodges, "The significance probability of the smirnov two-sample test," *Arkiv för Matematik*, vol. 3, no. 5, pp. 469–486, Jan. 1958. [Online]. Available: https://doi.org/10.1007/BF02589501

[170] P. Bruce and A. Bruce, "Statistical Experiments and Significance Testing," in *Practical Statistics for Data Scientists*. O'Reilly Media, Inc., 2017, pp. 87–139. [Online]. Available: https://www.oreilly.com/library/view/practical-statistics-for/9781491952955/

[171] M. Zambrano-Bigiarini, M. Clerc, and R. Rojas, "Standard Particle Swarm Optimisation 2011 at CEC-2013: A baseline for future PSO improvements," in *2013 IEEE Congress on Evolutionary Computation*, Jun. 2013, pp. 2337–2344, iSSN: 1941-0026.

[172] "The Differential Evolution Algorithm," in *Differential Evolution: A Practical Approach to Global Optimization*, ser. Natural Computing Series, K. V. Price, R. M. Storn, and J. A. Lampinen, Eds. Berlin, Heidelberg: Springer, 2005, pp. 37–134. [Online]. Available: https://doi.org/10.1007/3-540-31306-0_2

[173] E. Raponi, H. Wang, M. Bujny, S. Boria, and C. Doerr, "High Dimensional Bayesian Optimization Assisted by Principal Component Analysis," in *Parallel Problem Solving from Nature – PPSN XVI*, ser. Lecture Notes in Computer Science, T. Bäck, M. Preuss, A. Deutz, H. Wang, C. Doerr, M. Emmerich, and H. Trautmann, Eds. Cham: Springer International Publishing, 2020, pp. 169–183.

[174] J. Rapin and O. Teytaud, "Nevergrad - A gradient-free optimization platform," 2018, publication Title: GitHub repository. [Online]. Available: https://GitHub.com/FacebookResearch/Nevergrad

[175] G. De Giacomo and M. Y. Vardi, "Linear temporal logic and linear dynamic logic on finite traces," in *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence*, ser. IJCAI '13. AAAI Press, 2013, p. 854–860.

[176] M. Y. Vardi, "Branching vs. linear time: Final showdown," in *Proceedings of the 7th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, ser. TACAS 2001. Berlin, Heidelberg: Springer-Verlag, 2001, p. 1–22.

[177] A. Donzé, "On signal temporal logic," in *Runtime Verification*, A. Legay and S. Bensalem, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 382–383.

[178] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[179] L. Buitinck, G. Louppe, M. Blondel, F. Pedregosa, A. Mueller, O. Grisel, V. Niculae, P. Prettenhofer, A. Gramfort, J. Grobler, R. Layton, J. VanderPlas, A. Joly, B. Holt, and G. Varoquaux, "API design for machine learning software: experiences from the scikit-learn project," in *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 2013, pp. 108–122.